TS3000 series Robot Controller

Sistema TS3000 SCARA / LINEAR
Sistema TS3100 SCARA / LINEAR

MANUALE DI ISTRUZIONI

MANUALE DEL LINGUAGGIO DEL ROBOT

Avviso

- Assicurarsi che il presente manuale di istruzioni venga consegnato all'utente finale del robot industriale Toshiba Machine.
- Prima di utilizzare il robot industriale, leggere e comprendere completamente il presente manuale.
- Dopo aver letto attentamente il presente manuale, tenerlo a portata di mano per riferimento futuro.

TOSHIBA MACHINE CO., LTD.

Copyright 2012 by Toshiba Machine Co., Ltd.
Tutti i diritti riservati.

Nessuna parte del presente documento può essere riprodotta in qualsiasi forma, senza prima ottenere un permesso scritto da Toshiba Machine Co., Ltd.

Le informazioni contenute nel presente manuale sono soggette a modifiche senza preavviso per apportare miglioramenti.

Prefazione

Questo manuale spiega il linguaggio SCOL del robot, i comandi e le procedure di programmazione che si applicano al controller robot con sistema di serie TS di Toshiba Machine.
SCOL è l'acronimo di "Symbolic Code Language for Robots" (linguaggio a codici simbolici per robot) ed è un linguaggio per robot costituito da vari comandi per controllare il robot. Utilizzando questi comandi, è possibile creare programmi per far eseguire al robot ciò che si vuole.
Questo manuale è indirizzato a coloro che non hanno mai scritto un programma per robot e a coloro che hanno lunga esperienza di programmazione. Tuttavia, questo manuale si occupa solo di linguaggio per robot SCOL. Per una panoramica e il metodo operativo dei controller robot con sistema di serie TS, fare riferimento ai seguenti manuali:

Manuale di avvio
Manuale dell'operatore

Questo manuale è organizzato come segue:

[1] Panoramica del linguaggio del robot
La presente sezione spiega la connessione tra il linguaggio del robot e il movimento del robot e presenta una panoramica dei comandi usati nel linguaggio del robot. Assicurarsi di leggere questa sezione per poter cogliere le fondamenta del linguaggio del robot.

[2] Programmi di scrittura nel linguaggio del robot

Questa sezione descrive varie regole per scrivere un programma con il linguaggio del robot. Assicurarsi di leggere questa sezione prima di iniziare a scrivere il proprio programma.

[3] Spiegazione dei comandi del robot
Questa sezione spiega in dettaglio cosa significa e cosa fa ogni comando. I comandi sono elencati in ordine alfabetico per praticità. Questa sezione risulterà utile quando si scrive il proprio programma.

[4] Esempi di programma
Questa sezione contiene vari esempi di programmi nel linguaggio del robot. Assicurarsi di prendere questa sezione come riferimento quando si scrive il proprio programma.

[5] Programmare suggerimenti e avvertenze

La presente sezione illustra delle considerazioni tempistiche, le cose da non fare e quelle a cui badare quando si scrive un programma. Assicurarsi di leggerla prima di iniziare a lavorare sul proprio programma. Inoltre, assicurarsi di consultare questa sezione nel caso il programma non funzionasse come si era immaginato.

Sommario

Pagina

Sezione 1 Panoramica del linguaggio del robot 1-1
1.1 Movimento del robot 1-1
1.2 Linguaggio del robot 1-3
1.3 Tipo di comandi 1-5

Sezione 2 Programmi di scrittura nel linguaggio del robot 2-1
2.1 Configurazione del programma 2-1
2.1.1 File 2-1
2.1.2 Programma 2-1
2.1.3 Dati di posizione 2-2
2.2 Set di caratteri 2-2
2.3 Identificatori 2-3
2.4 Variabili e costanti 2-4
2.4.1 Dati scalari 2-4
2.4.2 Dati vettoriali 2-7
2.4.3 Variabili di sistema 2-9
2.4.4 Costanti di sistema 2-11
2.5 Espressioni 2-12
2.5.1 Espressioni di calcolo 2-13
2.5.2 Espressioni logiche 2-18
2.6 Etichette 2-18
2.7 Note e commenti 2-19
2.8 Programmi 2-20
2.8.1 Dichiarazione di programma 2-20
2.8.2 Sottoprogrammi 2-21
2.8.3 Libreria 2-23
2.8.4 Elaborazione multitask 2-26
2.8.5 Definizione di variabile globale 2-31
2.8.6 Variabile di tipo array 2-32

Sezione 3 Spiegazione dei comandi del robot 3-1
3.1 Spiegazione dei comandi 3-1
3.2 Spiegazione dei comandi 3-8

Pagina

Sezione 4 Esempi di programma 4-1

Sezione 5 Programmare suggerimenti e avvertenze 5-1
5.1 Tempi di esecuzione del programma 5-1
5.1.1 Tempi del movimento del braccio e dei segnali I/O 5-1
5.1.2 Sincronizzazione del movimento del braccio e dell'esecuzione del programma 5-3
5.1.3 Comando DELAY e comando WAIT 5-5
5.2 Cose da non fare durante la programmazione 5-8
5.2.1 Variabili 5-8
5.3 Cose a cui badare quando si scrive un programma 5-9
5.3.1 Tipo di comandi 5-9
5.3.2 Sistemi di coordinate del robot 5-11
5.3.3 Spostamento rapido 5-19
5.3.4 Configurazione del robot 5-25
5.3.5 Blocchi di dati 5-27
5.3.6 Blocco dei dati globali 5-31
5.3.7 Velocità del movimento del robot 5-33

Appendice A Elenco di comandi A-1

Appendice B Elenco di parole riservate A-6

Appendice C Contenuti del file di libreria (SCOL.LIB) A-7

Appendice D Domini e gamme di funzioni di calcolo A-10

Appendice E Come leggere i simboli A-11

Appendice F Elenco di errori di compilazione A-13

Appendice G Libreria a collegamento dinamico A-23
Appendice G.1 Libreria di pallettizzazione A-23

Appendice H Arresto della pre-lettura eseguito dal linguaggio SCOL del programma A-33

Sezione 1

Panoramica del linguaggio del robot

La presente sezione descrive la connessione tra il linguaggio del robot e il movimento del robot e presenta una panoramica dei comandi usati nel linguaggio del robot.

1.1 Movimento del robot

I robot lavorano al posto delle persone. Ad esempio, supponiamo che qualcuno debba collegare una parte ad un pezzo di lavoro proveniente da un convogliatore. L'operaio prende una parte da un contenitore di parti e collega la parte ad un pezzo di lavoro trasportato da un convogliatore alla sua stazione. Se invece dovessimo installare un robot per fare questo lavoro, avremmo una disposizione simile a quella mostrata nella Figura 1.1.

Nella Fig. 1.1, il robot prende una parte dall'alimentatore di parti e la collega ad un pezzo di lavoro proveniente da un convogliatore. Considerando questo lavoro dal punto di vista del robot, otterremmo uno schema come quello nella Figura 1.2. In questa figura, il robot parte dal Punto B e al Punto A prende una parte. Dopo aver preso la parte, il robot si sposta dal Punto A e torna al Punto B. Dal Punto B, il robot porta la parte al Punto C, che si trova esattamente al di sopra del Punto D, ovvero il luogo in cui la parte viene collegata. Quindi, il robot scende dal Punto C al Punto D e collega la parte al pezzo di lavoro. Una volta terminato il collegamento della parte, il robot ritorna al Punto C e infine al Punto B. Questo termina un singolo ciclo di lavoro.

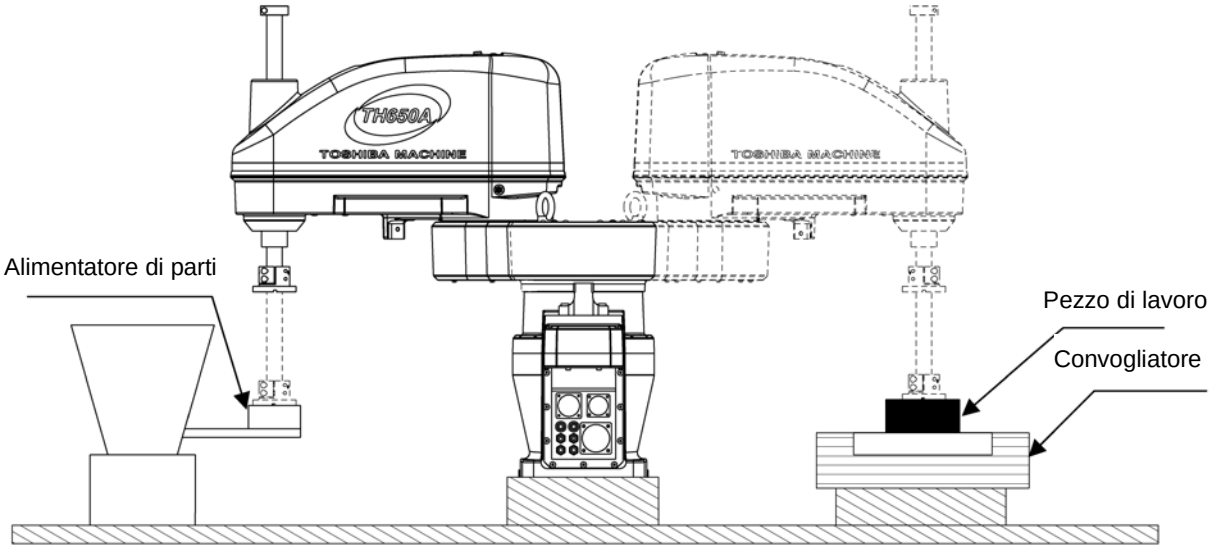


Fig. 1.1 Lavoro di assemblaggio

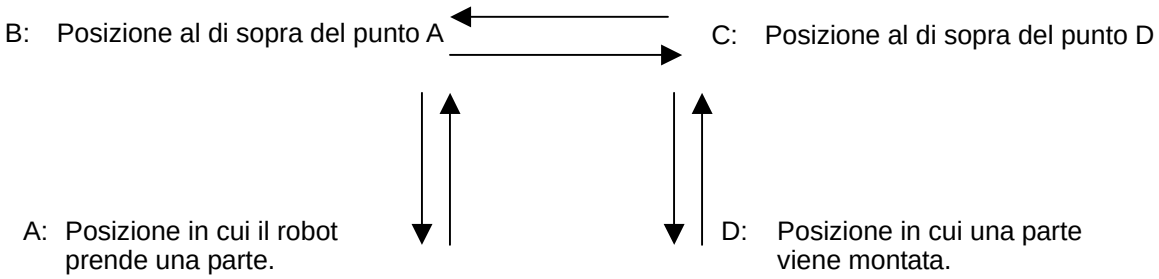


Fig. 1.2 Movimento del robot

1.2 Linguaggio del robot

I robot eseguono il lavoro di assemblaggio e altre operazioni al posto delle persone. Tuttavia, c'è comunque bisogno di qualcuno che programmi il robot relativamente a ciò che deve fare.

I robot eseguiranno solo ciò che si dirà loro di fare ed è importante dir loro esattamente ciò che si vuole che facciano.

Dire a un robot cosa deve fare si dice "programmare" un robot. Far fare a un robot ciò per cui è stato programmato si definisce "apprendimento diretto". Ovviamente questo si riferisce a quelli che vengono chiamati "robot ad apprendimento diretto", che ripetono (o riproducono) i movimenti che gli vengono impartiti durante la programmazione.

Ci sono vari modi per programmare un robot. Uno di questi metodi consiste nel far fare ad un robot un lavoro in ordine. Ad esempio, quando si esegue un lavoro di pittura e saldatura a punti, qualcuno programma un robot eseguendo il lavoro stesso. Il robot esegue il lavoro spostandosi secondo un movimento programmato.

Per eseguire lavori complicati, tuttavia, si deve cambiare il movimento del robot a seconda degli stati delle periferiche. Come si può desumere dall'esempio dell'assemblaggio, si potrebbero presentare dei problemi con il cambio di parti da collegare in risposta ai tipi di pezzi di lavoro trasportati da un convogliatore o con collegamenti e scollegamenti ripetuti delle parti quando avvengono dei collegamenti errati. Il robot ha bisogno di una risposta a circostanze simili che avvengono intorno a sé. È necessario, quindi, fornirgli un metodo per rispondere a varie circostanze.

Il linguaggio utilizzato per scrivere i programmi del movimento di un robot viene definito "Linguaggio del robot". L'attività di un robot espressa nel linguaggio del robot viene definita "Programma" e l'attività svolta per creare un programma si chiama "Programmazione". La serie TS impiega il linguaggio SCOL (acronimo di Symbolic Code Language for robot, linguaggio a codici simbolici per robot) come unico linguaggio del robot. L'esempio del lavoro di assemblaggio descritto nel paragrafo precedente può essere espresso nel linguaggio SCOL come mostrato di seguito.

PROGRAM ASSEMBLY

MOVE B		Spostare il robot al Punto B.
OPEN1		Aprire Mano 1.
MOVE A		Spostare il robot al Punto A.
CLOSE1		Chiudere Mano 1.
DELAY	0.5	Attendere 0,5 secondi prima di prendere la parte.
MOVE B		Spostare il robot al Punto B.
MOVE C		Spostare il robot al Punto C.
MOVE D		Spostare il robot al Punto D.
OPEN1		Aprire Mano 1.
DELAY	0.5	Attendere 0,5 secondi prima di rilasciare la parte.
MOVE C		Spostare il robot al Punto C.
MOVE B		Spostare il robot al Punto B.

END

La parola PROGRAM contrassegna l'inizio di un programma e la parola END la fine. Il nome di questo specifico programma è ASSEMBLY. MOVE A significa spostarsi al Punto A. OPEN1 e CLOSE1 significano, rispettivamente, aprire e chiudere la Mano 1. (Ci sono due mani.) DELAY 0.5 significa non fare nulla per 0,5 secondi. Inoltre, le posizioni dei Punti A, B, C e D sono definiti (programmati) in precedenza guidando fisicamente il robot (in modalità di programmazione) a questi punti.
Quindi possiamo esprimere le attività del robot in SCOL disponendo i comandi di movimento che vengono dati al robot in sequenza per compiere le attività programmate.

1.3 Tipo di comandi

Nella sezione precedente, abbiamo visto come il linguaggio SCOL viene usato per esprimere l'azione del robot. Qui di seguito forniremo alcune informazioni aggiuntive sui comandi SCOL stessi.
Oltre ai comandi come "MOVE A", che realmente muovono il robot, vi sono molti altri comandi che eseguono azioni come inviare segnali all'attrezzatura esterna o indicare al robot di eseguire più volte la stessa operazione. La Tabella 1.1 presenta un elenco di comandi SCOL.
Tutti i comandi SCOL possono essere grosso modo classificati in una di queste sei categorie.

(1) Comandi di controllo del movimento

Questi comandi muovono il robot. In questa categoria vengono inclusi anche i comandi che bloccano temporaneamente il robot, lo riavviano o ne interrompono il movimento. I comandi che realmente muovono il robot vengono chiamati comandi di movimento.

(2) Comandi di controllo del programma

I comandi di controllo del programma controllano l'esecuzione del programma attraverso, ad esempio, l'esecuzione di certe parti del programma secondo segnali esterni o in maniera ripetuta.

(3) Comandi di controllo I/O (ingresso/uscita)

Questi comandi vengono usati per leggere segnali di ingresso o inviare segnali di uscita da e all'attrezzatura esterna. Tra i comandi di controllo I/O vengono inclusi l'ingresso e l'uscita di dati del canale di comunicazione di apertura/chiusura della mano.

(4) Comandi di condizione del movimento

Questi comandi vengono usati per specificare la configurazione e la velocità di vari giunti del robot quando è in movimento.

(5) Comandi calcolatrice

Questi comandi vengono usati per invocare (utilizzare) funzioni matematiche come le funzioni trigonometriche e le radici quadrate.

(6) Comandi di riferimento del movimento

Questi comandi vengono usati per fare riferimento a e verificare il movimento del robot. Ad esempio, questi comandi potrebbero essere usati per determinare che percentuale di una certa azione è stata completata in un dato momento. Essi includono anche comandi per impostare il timer usato durante l'esecuzione del programma e fare riferimento alla modalità operativa del robot. Questi comandi, in combinazione con altri, vengono usati per emettere segnali all'attrezzatura esterna quando il robot ha completato fino al 70% dell'attività assegnata o per eseguire la ramificazione del programma quando il movimento del robot viene riconosciuto come errore se il robot non completa tale movimento in un determinato lasso di tempo.
Combinando questi comandi è possibile programmare attività adatte al robot.

Tabella 1.1 Funzioni del linguaggio SCOL

Tipo	Scopo	Comandi
Comandi di controllo del movimento	(1) Spostano il robot.	MOVE, MOVES, MOVEC, MOVEA, MOVE1, READY MOVEJ
	(2) Blocca temporaneamente il robot.	DELAY
	(3) Muovono la mano del robot.	OPEN1, OPENI1, OPEN2, OPENI2, CLOSE1, CLOSEI1, CLOSE2, CLOSEI2
	(4) Interrompono o riavviano l'operazione.	BREAK, PAUSE, RESUME
Comandi di controllo del programma	(1) Controllano segnali esterni, timer, ecc.	ON ~ DO ~, IGNORE IF ~ THEN ~ ELSE, WAIT, TIMER
	(2) Controllano l'esecuzione del programma.	PROGRAM, END, GOTO, RCYCLE, RETURN, FOR ~ TO ~ STEP ~ NEXT, STOP TASK, KILL, SWITCH, TID, MAXTASK
	(3) Osservazioni e commenti sul programma.	REMARK

Tipo	Scopo	Comandi
Comandi di controllo I/O	(1) Ingresso e uscita di segnali esterni.	DIN, DOUT, PULOUT, RESET, BCDIN, BCDOUT
	(2) Ingresso e uscita di dati di comunicazione.	PRINT, INPUT
Comandi di condizione del movimento	(1) Specificano le condizioni per controllare il movimento del robot.	CONFIG, ACCUR, ACCEL, DECEL, SPEED, PASS, TORQUE, GAIN, ENABLE, SETGAIN, DISABLE, NOWAIT, PAYLOAD, FREELOAD, SWITCH, MOVESYNC
Comandi di pallettizzazione	(1) Carica una libreria.	LOADLIB
	(2) Inizializza un pallet.	INITPLT
	(3) Sposta un pallet in una specifica posizione.	MOVEPLT
Funzioni di calcolo	(1) Esegue calcoli per numeri reali.	SIN, COS, TAN, ASIN, ACOS, ATAN, ATAN2, SQRT, ABS, SGN, INT, REAL, LN, MOD, LOGIO, EXP, AND, OR, NOT
	(2) Esegue calcoli con dati di posizione e di coordinate.	HERE, DEST, POINT, TRANS
	(3) Usa un array.	DIM, AS
Comandi di riferimento del movimento	(1) Verifica il movimento del robot.	MOTION, MOTIONT, REMAIN, REMAINT
	(2) Verifica il movimento del sistema.	MODE, CONT, CYCLE, SEGMENT
	(3) Assegna un sistema di coordinate.	TOOL, BASE, WORK
Altro	(1) Definisce una variabile.	GLOBAL, DATA, END
	(2) Ripristina un valore aggiornato nel file del programma.	RESTORE
	(3) Salva i dati allo spegnimento.	SAVEEND

Sezione 2

Programmi di scrittura nel linguaggio del robot

Nella Sezione 1, abbiamo offerto una descrizione generale di ciò che è il linguaggio di un robot e di come funziona. Ora nella Sezione 2 descriveremo come scrivere un programma nel linguaggio del robot.

2.1 Configurazione del programma

Di seguito presenteremo una descrizione generale della configurazione del programma con il linguaggio SCOL.

2.1.1 File

Per far sì che il robot esegua un'attività, sono necessari un programma scritto nel linguaggio del robot e i dati di posizione utilizzati dal programma. Ovvero, per la serie TS, è necessario disporre di un set corrispondente di un programma (o programmi) e dati di posizione. denominato file. L'esecuzione e la modifica del programma vengono effettuate sulla base del file.

2.1.2 Programma

Un programma è una espressione che adopera il linguaggio del robot di una serie di operazioni eseguite dal robot. Un programma può inoltre essere richiamato e utilizzato da un altro programma. È possibile preparare le operazioni più utilizzare e le operazioni preimpostate come un unico programma e richiamare questi programmi qualora necessario. I programmi richiamati in tal modo sono denominati sottoprogrammi e il programma che richiama il sottoprogramma è denominato programma principale.

Un singolo file può includere vari programmi. Quando si esegue un programma, se non è specificato diversamente, il programma all'inizio del file viene eseguito come programma principale. Quando si richiama un sottoprogramma, esso deve trovarsi nello stesso file del programma principale. Inoltre, anche se in un file sono contenuti vari programmi, tutti questi programmi non vengono eseguiti in ordine. Al contrario, i programmi vengono eseguiti fino alla fine del programma principale come singola unità.

Inoltre, è possibile eseguire vari programmi contemporaneamente utilizzando il comando TASK (esecuzione multitask). Per i dettagli sull'esecuzione multitask, consultare Para. 2.8 “Programmi”. I programmi vengono modificati dal

programmatore portatile utilizzando la funzione di editor di programma del controller. Per i dettagli sull'utilizzo dell'editor, consultare il Manuale dell'operatore.
Per quanto riguarda il robot, il suo compito termina quando si completa il programma principale (ovvero, quando il robot raggiunge l'istruzione finale END del programma principale), e se gli altri programmi non sono stati richiamati fino ad allora non potranno più essere richiamati.
È possibile eseguire vari programmi contemporaneamente utilizzando il comando TASK (esecuzione multitask). Per i dettagli sull'esecuzione multitask, consultare Para. 2.8.
I programmi vengono modificati dal programmatore portatile utilizzando la funzione di editor di programma. Per informazioni sull'utilizzo dell'editor, consultare il "Manuale dell'operatore".

2.1.3 Dati di posizione

I dati di posizione da usare in un programma (o programmi) devono trovarsi nello stesso file del programma (o dei programmi). È possibile accedere (utilizzare) ai dati di posizione presenti in un file da tutti i programmi di quel file. Tuttavia, non è possibile accedere ai dati di posizione presenti in un file da un programma non presente in quel file.
I dati di posizione vengono "trasmessi" al robot utilizzando la funzione di editor di programma del controller. Per informazioni sull'utilizzo dell'editor dei dati, consultare il Manuale dell'operatore.

2.2 Set di caratteri

Il set di caratteri SCOL è costituito da caratteri alfanumerici e dai seguenti simboli speciali.

Caratteri alfanumerici

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
a b c d e f g h i j k l m n o p q r s t u v w x y z
1 2 3 4 5 6 7 8 9 0

Simboli speciali

“ ‘ ( ) + – * / , . < > =
! [ ] ( ) % ^ & ?

Ad eccezione di quasi tutte le lettere minuscole, questi caratteri e simboli possono essere immessi dal programmatore portatile. Quando si esegue un programma, il robot non effettua alcuna distinzione tra lettere maiuscole e minuscole. Per il metodo di lettura dei simboli, consultare "Appendice E".

2.3 Identificatori

Nel linguaggio del robot SCOL vengono utilizzati per esprimere comandi, nomi di programma, nomi di variabile ed etichette (utilizzate per specificare i rami del programma). Gli identificatori devono iniziare con un carattere alfabetico. Non vi è alcun particolare limite di lunghezza, anche se il robot distingue solo i primi dieci caratteri alfanumerici. Non è importante utilizzare lettere maiuscole o minuscole, poiché il robot li considera comunque allo stesso modo. Inoltre, negli identificatori non è possibile utilizzare simboli speciali e spazi.

I caratteri speciali o gli spazi vengono utilizzati per separare gli identificatori.

Esempio:

TOSHIBAROB
Toshibarob
TOHIBAROBOT

I tre identificatori di cui sopra vengono tutti considerato come lo stesso identificatore “TOSHIBAROB”.

Alcuni identificatori sono già stati definiti dal linguaggio SCOL stesso. Essi vengono definiti parole riservate e il programmatore non può utilizzarli per scopi diversi da quelli già definiti. Nell'Appendice B viene mostrato un elenco di parole riservate. Oltre ai comandi SCOL, si troveranno le parole utilizzate nel sistema del computer e un set di parole assegnato per espansione futura.

Non utilizzare identificatori con lo stesso nome per diversi significati. Ad esempio, quando si crea un programma, non utilizzare lo stesso identificatore per il nome del programma e il nome della variabile.

2.4 Variabili e costanti

Non tutti i dati assumono la stessa forma e queste diverse forme di dati vengono denominate tipi di dati. Nel linguaggio SCOL è possibile utilizzare il tipo scalare (tipo intero, tipo numero reale e stringa di caratteri) e il tipo vettoriale (tipo posizione, tipo coordinate e tipo carico). Le variabili sono suddivise in variabile globale e variabile automatica in base al metodo di definizione. Tutti i dati programmati e le variabili definite nell'area tra GLOBAL ed END vengono denominati variabili globali. È possibile fare riferimento e modificare queste variabili da qualsiasi parte del programma. Per tutti i tipi di dati delle variabili globali, può essere dichiarato l'array. Per le descrizioni della variabile globale e dell'array, consultare Para. 2.8.5.

L'area di lavoro del controller viene utilizzata per tutti i dati. Il valore definito viene sostituito per le variabili globali diverse dall'array senza un valore iniziale specifico all'inizio del programma. Se il valore viene inserito per la variabile durante l'esecuzione del programma, viene modificata solo l'area di lavoro. Se il controller è spento, il file di esecuzione viene riselezionato oppure il file viene modificato, l'area di lavoro viene ripristinata in base al valore iniziale della variabile presente nel file e, di conseguenza, si perde il valore modificato. Ciò è valido anche per la modifica dei dati programmati. Se i dati del file devono essere sovrascritti, nel programma deve essere eseguito il comando RESTORE.

2.4.1 Dati scalari

Vi sono tre tipi di dati scalari, ovvero interi, numeri reali e stringhe di caratteri. Le variabili automatiche di tipo scalare possono essere utilizzate solo nel programma in cui sono state dichiarate. Ciò significa che se si utilizza una variabile con lo stesso nome in un altro programma, le due variabili saranno completamente indipendenti e non avranno niente a che vedere tra loro. Pertanto, quando si trasferiscono i dati da un programma all'altro, cercare, se possibile, di ridefinire la variabile come variabile globale di tipo scalare o dichiarare gli argomenti nel programma. Consultare Para. 2.8 "Programmi".

(1) Dati interi

(a) Costanti
SCOL può gestire valori interi ("numeri interi") nella gamma tra –2147483648 e + 2147483647. Quando un numero intero viene utilizzato come costante in un programma, se è positivo, descrivere direttamente il valore; se è negativo, descrivere il valore successivo al simbolo –. Se si immette un valore di 11 o più cifre con il comando INPUT, si verifica un errore.

Esempio:

0
234
–39208
5963

(b) Variabili

Le variabili si distinguono con identificatori e possono rientrare nella gamma tra – 2147483648 e + ~147483647, come in precedenza. Il tipo di dati di una variabile è determinato dal tipo di dati del primo numero assegnato a quella variabile. Ad esempio, se il primo carattere assegnato ad una variabile è un numero intero, tutti gli altri numeri sostituiti in quella variabile diventano interi. Ciò significa che se in seguito si inserisce un numero reale in questa variabile, il controller elimina tutte le cifre decimali e considera ciò che rimane come un numero intero.

La variabile è di due tipi: la variabile globale, che è valida nell'intero programma, e la variabile generale, che è valida in una parte del programma. È possibile modificare la variabile globale da qualsiasi parte del programma.

(c) Valori logici

I valori logici vengono utilizzati nel programma quando si effettuano valutazioni condizionali. Le espressioni e i comandi logici quali DIN (che controllano i segnali di ingresso) ritornano ai valori logici.

Un valore logico potrebbe presentare uno dei due valori: TRUE o FALSE. Internamente, i valori logici vengono considerato come interi, con 1 equivalente a TRUE e 0 a FALSE.

Nota: In senso stretto, 0 è considerato FALSE e tutto il resto TRUE.

(2) Dati reali

Con SCOL, i numero vengono considerati come tipi reali ad eccezione di alcuni casi speciali.

(a) Costanti

SCOL è in grado di gestire numeri reali con un valore assoluto nella gamma di circa $5{,}87\times10^{-39}$ (2^{-127}) e $6{,}80\times10^{38}$ $\{(2^{23}-1)\times2^{106}\}$. Il numero di cifre significative per la mantissa (la mantissa è la parte del numero a destra del punto decimale) è circa 7 in Base 10 (la precisione è 2^{23}).

Il numero di cifre consentite è 9 per il numero intero e 3 per il decimale. Se si immette un valore superiore a queste cifre con il comando INPUT, si verifica un errore.

Quando nel programma si utilizza un numero reale, se è positivo, descrivere direttamente il valore; se è negativo, descrivere il valore successivo al simbolo –.

Quando la parte decimale è 0, è possibile ometterla. Tuttavia, quando si omette il punto decimale, i dati vengono considerati come dati di tipo intero. Inoltre, poiché la parte intera non può essere omessa, anche se il valore assoluto di un valore numerico è inferiore a 1, è necessario designare 0 sulla parte intera.

Esempio:

```
      1234.567
       –28.16
          0.00985
   1234567.
       –369.
```

Come menzionato in precedenza, la precisione del computer è alquanto limitata quando si gestiscono i valori decimali. Di solito, non è un problema se il numero di cifre decimali è accettabile. Pertanto, quando si lavora con il robot, cercare di utilizzare quanto segue come unità minime.

Distanza (dati X, Y e Z)	0,001 mm
Angoli (dati C)	0,001 gradi
Tempo	0,01 sec.
Valori nominali (velocità, coppia, ecc.)	1%
Peso	0,01 kg
Inerzia	0,01 kg.m

(b) Variabili

Le variabili si distinguono con identificatori e rientrano nella stessa gamma elencata in precedenza per le costanti. Il tipo di dati di una variabile è determinato dal tipo di dati del primo numero assegnato a quella variabile. Ad esempio, se il primo carattere assegnato ad una variabile è un numero reale, quella variabile diventano di tipo reale.

(3) Stringhe di caratteri

Le stringhe di caratteri possono gestire solo costanti. Esse vengono espresse racchiudendo uno o più caratteri tra virgolette. Nell'esempio di seguito, la stringa di caratteri è SCOL MESSAGE.

Esempio: "SCOL MESSAGE"

2.4.2 Dati vettoriali

Al contrario dei dati di tipo scalare che contengono un solo elemento di dati, i dati di tipo vettoriale contengono più elementi di dati. In SCOL vi sono tre tipi di dati vettoriali: vettori posizionali, vettori delle coordinate e vettori di carico.
I dati di tipo vettoriale possono essere espressi racchiudendo da uno a cinque elementi tra parentesi { }. Oltre ai vettori posizionali, ai vettori delle coordinate e ai vettori di carico, i dati di tipo vettoriale sono specificati anche per comandi TORQUE e GAIN racchiudendoli tra parentesi { }.
I dati di tipo vettoriale diversi dalla variabile globale di tipo vettoriale, come i dati programmati dall'editor dei dati, vengono memorizzati temporaneamente nell'area di lavoro del controller. I dati non vengono creati nel file. La variabile di tipo vettoriale può essere utilizzata solo nel programma dichiarato. Quindi, anche se la stessa variabile viene utilizzata in un altro programma, il contenuto del primo non concorda con quello del secondo. Quando si trasferiscono i dati da un programma ad un altro, i dati trasferiti devono essere ridefiniti come variabile globale di tipo vettoriale o devono costituire un argomento. Per i dettagli degli argomenti, consultare "2.8.2 Sottoprogrammi".

(1) Dati posizionali

I dati posizionali vengono utilizzati dal robot per descrivere posizioni. I vettori posizionali hanno il seguente formato.

(X, Y, Z, C, T, <configurazione>)

X, Y, Z, C e T sono valori di coordinate rappresentati da numeri reali. Le unità sono in millimetri o gradi.

<Configurazione> contiene un numero intero da 0 a 2 che descrive la configurazione di impostazione del sistema.

0 ... Libero (impostazione del sistema indefinita)
1 ... Sistema sinistrorso
2 ... Sistema destrorso

(2) Dati delle coordinate

I dati delle coordinate vengono utilizzati dal robot per specificare i sistemi di coordinate. I vettori delle coordinate hanno il seguente formato.

(X, Y, Z, C)

X, Y, Z e C sono valori di coordinate rappresentati da numeri reali. Le unità sono in millimetri o gradi.

I vettori delle coordinate consentono di convertire tra diversi sistemi di coordinate come illustrato nella Figura 2.1. Nella figura, viene illustrato un sistema di coordinate originale X, Y e Z. Quindi, con i dati forniti da un vettore delle coordinate (x, y, z, c), il sistema di coordinate originale viene spostato parallelamente lungo i suoi assi delle quantità x, y e z. Ciò forma un nuovo sistema di coordinate centrato di circa 0'. Una volta effettuata questa operazione, si è ruotato il nuovo sistema di coordinate attorno all'asse Z' di una quantità c. In tal modo, si è terminato di orientare il nuovo sistema di coordinate X', Y' e Z'.

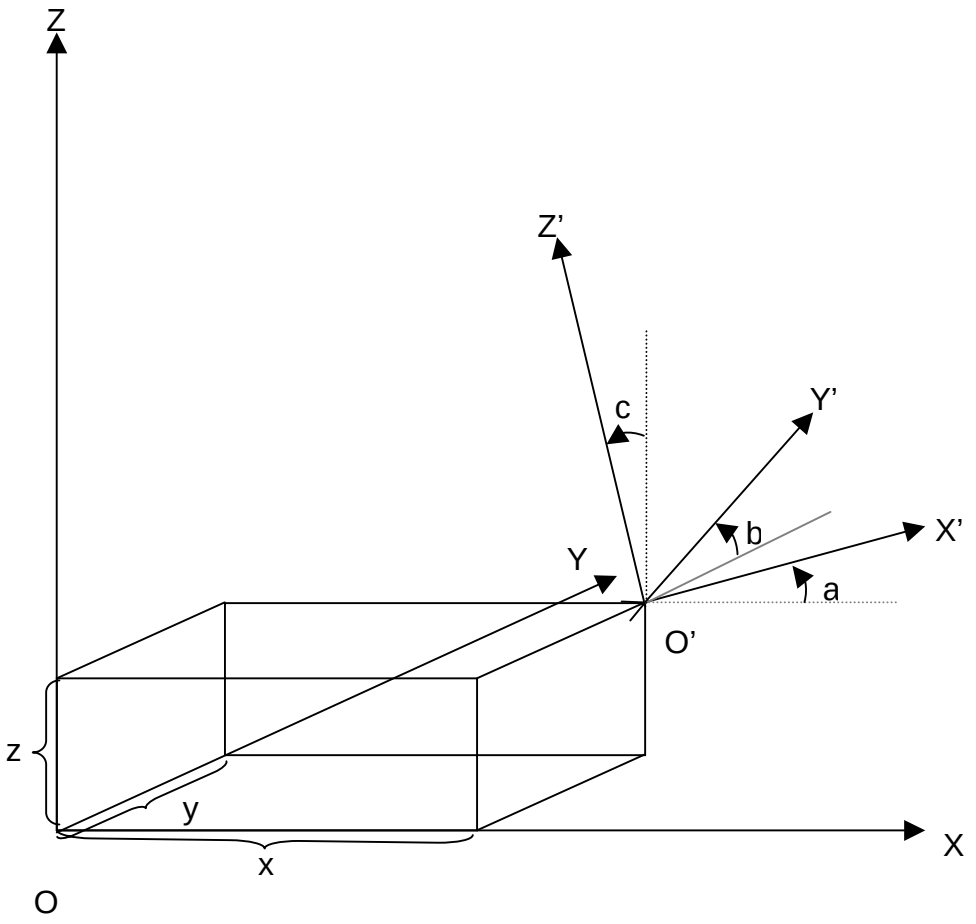


Fig. 2.1 Conversione di coordinate

(3) Dati di carico

I dati di carico vengono utilizzati per definire i carichi fisici che agiscono sull'attuatore finale (mano) del robot. I vettori di carico hanno il seguente formato.

{<peso>, <offset del baricentro>}

<peso> corrisponde al peso del carico che agisce sulla punta della mano del robot. Le unità sono in kg.

<offset del baricentro> è la quantità che rappresenta la distanza tra il baricentro applicato alla punta della mano del robot e il centro della flangia dell'utensile del robot (unità: mm).

2.4.3 Variabili di sistema

Il linguaggio SCOL fornisce variabili speciali che vengono utilizzate nei programmi per specificare e fare riferimento alle condizioni di funzionamento del robot. Tali variabili sono denominate variabili di sistema. Come le altre variabili, è possibile fare riferimento a queste variabili nel programma, modificarne il valore, ecc. Tuttavia, bisogna fare attenzione quando si impostano o sostituiscono i valori nelle variabili di sistema poiché ciò influisce direttamente sulle condizioni di funzionamento del robot.

Nella Tabella 2.1 di seguito viene presentato un elenco delle variabili di sistema.

Tabella 2.1 Elenco delle variabili di sistema

Nome	Descrizione	Valori reali	Valore iniziale	Tipo di dati
CONFIG	Configurazione del robot	0, 1, 2	0	Tipo intero
ACCUR	Precisione di posizionamento	0, 1	1	Tipo intero
ACCEL	Accelerazione (durante l'accelerazione)	0 ~ % max	100	Tipo intero
DECEL	Decelerazione (durante la decelerazione)	0 ~ % max	100	Tipo intero
SPEED	Velocità di movimento	0 ~ % max	100	Tipo intero
PASS	Parametro di spostamento rapido	0 ~ 100%	100	Tipo intero
TORQUE	Coppia massima su ciascun asse	0 ~ % max	300	Tipo vettoriale
GAIN	Guadagno servoazionamento su ciascun asse	0.1	1	Tipo vettoriale
TOOL	Coordinate utensile		0	Tipo di coordinate
BASE	Coordinate di base		0	Tipo di coordinate
WORK	Coordinate di lavoro		0	Tipo di coordinate
TIMER	Timer	0 ~	–	Tipo reale
ERROR	Informazioni di errore	–		Tipo intero
PLAYLOAD	Carico sul robot	0 ~	0	Tipo di carico
SWITCH	Multitask	0, 1	1	Tipo intero
TID	Numero di attività	1 ~	–	Tipo intero
PLCDATAR1 ~ 8	Interfaccia PLC semplificata	0 ~ 65535	0	Tipo intero
PLCDATAW1 ~ 8	Interfaccia PLC semplificata	0 ~ 65535	0	Tipo intero

Nota: I valori massimi sono impostati separatamente per ciascun sistema.

Qualora si modifichi il contenuto di una variabile di sistema correlata al controllo del movimento, tale modifica non avrà effetto fino al movimento successivo; non avrà alcun effetto su un movimento in corso al momento della modifica; tuttavia, utilizzando una struttura WITH, è possibile impostare temporaneamente una variabile di sistema correlata ad un comando di movimento. Ad esempio:

MOVE Al WITH SPEED = 50

Inoltre, si noti che SCOL non controlla se un valore sostituito in una variabile di sistema rientra nella gamma consentita. In tal caso, il sistema imposta un valore in base alle seguenti regole.

- Se si tenta di inserire un valore inferiore al valore minimo consentito, tale valore viene inserito al suo posto.
- Se si tenta di inserire un valore superiore al valore massimo consentito, tale valore viene inserito al suo posto.

Per i dettagli sull'utilizzo delle variabili di sistema, consultare la Sezione 3.

2.4.4 Costanti di sistema

Per rendere più semplice la lettura dei programmi (e il conseguente debug), SCOL fornisce le costanti di sistema illustrate nella Tabella 2.2. Tali nomi possono essere utilizzati nel programma al posto dei numeri al fine di rendere più semplice consultare le proprie operazioni. Tuttavia, assicurarsi di utilizzarli solo nei punti specificati nella colonna Commenti della Tabella 2.2.

Tabella 2.2 Elenco delle costanti di sistema

Nome	Valore	Commenti (punti per l'utilizzo)
FREE LEFTY RIGHTY	0 1 2	Nella variabile di sistema CONFIG Nel comando POINT
COARSE FINE	0 1	Nella variabile di sistema ACCUR
OFF ON	0 1	Nella variabile di sistema GAIN Nel comando SETGAIN
PAI	3.141593	Valore Pi
CONT CYCLE SEGMENT	0 1 2	Nel comando MODE

2.5 Espressioni

Nel presente paragrafo vengono descritte le espressioni fornite da SCOL per la sostituzione, il calcolo e la valutazione.

Nel linguaggio SCOL, le espressioni possono essere utilizzate non solo in modo indipendente per eseguire la sostituzione e il calcolo, ma anche nei comandi. Le espressioni includono espressioni di calcolo in cui si sostituisce il risultato del calcolo in una variabile ed espressioni logiche che determinano i risultati maggiori/minori o veri/falsi.

È possibile utilizzare gli operandi illustrati di seguito. Si noti che il risultato dell'esecuzione di 0/0 è -1, mentre di 0 ^ 0 è 0 piuttosto che un errore, come normalmente ci si aspetterebbe.

Tabella 2.3 Operandi

Tipo	Operando	Funzione	Esempio
Funzioni aritmetiche	^	Elevamento a potenza	A ^ B (A elevato B)
	–	Segno meno	–A
	*, /	Moltiplicazione, divisione	A * B, A / B
	+, –	Addizione, sottrazione	A + B, A – B
	MOD	Resto	A MOD B (resto quando A è diviso per B)
	=	Sostituzione	A = B (inserisce il valore di B in A)
Funzione relazionale	= =	Uguale	A = = B
	< >, > <	Non uguale	A < > B, A > < B
	<	Inferiore a	A < B
	>	Superiore a	A > B
	< =, = <	Inferiore a o uguale	A < = B, A = < B
	> =, = >	Superiore a o uguale	A > = B, A = > B
Operandi logici	AND	Prodotto logico	A AND B
	OR	Somma logica	A OR B
	NOT	Negazione	NOT A
Funzioni	SIN	Seno	SIN (A)
	COS	Coseno	COS (A)
	TAN	Tangente	TAN (A)
	ASIN	Arcoseno	ASIN (A)
	ACOS	Arcocoseno	ACOS (A)

Tipo	Operando	Funzione	Esempio
Funzioni	ATAN	Arcotangente	ATAN (A)
	ATAN2	Arcotangente	ATAN2 (A, B) (arcotangente di A / B)
	SQRT	Radice quadrata	SQRT (A)
	ABS	Valore assoluto	ABS (A)
	SGN	Segno	SGN (A) (estrarre e riportare il segno di A)
	INT	Cambia il numero in un intero.	INT (A)
	REAL	Cambia il numero in un numero reale.	REAL (A)
	LN	Logaritmo naturale	LN (A)
	LOG10	Logaritmo comune	LOG10 (A)
	EXP	Base esponenziale e	EXP (A)

È possibile utilizzare parentesi ( ) piuttosto che espressioni.

2.5.1 Espressioni di calcolo

Nel linguaggio SCOL, i risultati dei calcoli sul lato destro di un segno di uguale sono situati sulla sinistra. Nelle espressioni è possibile utilizzare variabili e costanti.

(1) Ordine della priorità di calcolo

Nel linguaggio SCOL, il calcolo viene effettuato nello stesso ordine di priorità delle normali operazioni di calcolo. Nello specifico, le operazioni vengono eseguite in base alle regole di seguito.

- Se l'espressione contiene parentesi, le operazioni all'interno delle parentesi vengono eseguite per prime.
- Le operazioni vengono eseguite nell'ordine di assegnazione di segni negativi, calcolo di esponenti, moltiplicazione e divisione (*, /) e addizione e sottrazione (+, -).
- Se due operazioni hanno la stessa priorità, le operazioni vengono eseguite da sinistra a destra dell'espressione.

Ad esempio:

$$a = b + c * d / (e – f) – g,$$

L'ordine di calcolo dell'espressione precedente è:

1.	Calcolare e – f.	e–f
2.	Calcolare c * d.	c * d
3.	Dividere c * d per e – f.	(c * d) / (e – f)
4.	Sommare il risultato precedente a b.	b + (c * d) / (e – f)
5.	Sottrarre g dal risultato precedente.	(b + (c * d) / (e –f)) – g

Nella Tabella 2.4 viene illustrato l'ordine della priorità di calcolo per varie operazioni.

Tabella 2.4 Ordine della priorità di calcolo

Priorità	Operazione	Operando	Convenzione di gruppo
Alta	Parentesi	( )	Da sinistra a destra
	Assegnazione di elementi vettoriali	.	Da sinistra a destra
	Assegnazione di segni negativi e negazioni	–, NOT	Da destra a sinistra
	Elevamento a potenza	^	Da sinistra a destra
	Moltiplicazione, divisione, resto	*, /, MOD	Da sinistra a destra
Bassa	Addizione, sottrazione	+, –	Da sinistra a destra
	Confronto	<, >, < =, > =, = <, = >	Da sinistra a destra
	Uguaglianza, disuguaglianza	= =, < >, > <	Da sinistra a destra
	Prodotto logico, somma logica	AND, OR	Da sinistra a destra
	Sostituzione	=	Da destra a sinistra

Nota: Spiegazione della convenzione di gruppo:

Da sinistra a destra 1 + 2 – 3 viene interpretato come (1 + 2) – 3.

Da destra a sinistra NOT–3 viene interpretato come NOT (–3).

(2) Calcolo dei dati di tipo scalare

Nei calcoli è possibile utilizzare i dati di tipo scalare in combinazione con operandi di calcolo. Tuttavia, se anche un solo numero in un'espressione è un numero reale, anche il risultato di tale espressione sarà un numero reale. Inoltre, tutte le seguenti funzioni forniranno un numero reale.

SIN, COS, TAN, ASIN, ACOS, ATAN, ATAN2, SQRT, REAL, LN, LOG10, EXP

Quando la variabile sul lato sinistro dell'equazione è di tipo intero e il risultato del calcolo non è un numero intero, il risultato sarà convertito in un numero intero prima di essere assegnato alla variabile. Non dimenticare, tuttavia, che tutti i punti decimali vengono eliminati quando si converte un numero reale in un numero intero. D'altra parte, quando si esegue la conversione da un numero intero ad un numero reale, il numero di cifre significative è limitato. Quando si desidera conoscere il tipo di dati a disposizione, utilizzare il comando INT o REAL.

Le stringhe di caratteri non possono essere utilizzate nei calcoli. I calcoli possono essere eseguiti tra gli elementi di variabili di tipo vettoriale e i dati scalari. In tal caso, un identificatore di elementi viene aggiunto al termine di una variabile di tipo vettoriale per specificare l'elemento presente nel calcolo. Il valore dell'elemento viene quindi estratto dalla variabile di tipo vettoriale e utilizzato per il calcolo.

Come identificatori di elementi, si possono utilizzare ".X", ".Y", ".Z", ".C" e ".T". Si può inoltre specificare numericamente la posizione dell'elemento con ".1", ".2", ".3", ".4" e ".5".

Esempi:

```
A = POINT1.X/25
GAIN={GAIN.1,GAIN.2,0,0,0}
```

Nota: È possibile utilizzare ciò solo per fornire il valore di un elemento dall'interno di una variabile di tipo vettoriale. Non è possibile modificare il valore dell'elemento stesso.

(3) Calcolo dei dati di tipo vettoriale

È possibile sommare e sottrarre gli elementi corrispondenti di due vettori. Il calcolo è possibile solo tra variabili dello stesso tipo. L'elemento <CONFIG> non è incluso nei calcoli, ma assume piuttosto il valore della variabile sostituito in esso.

Esempio: Dati i seguenti due vettori posizionali e due vettori delle coordinate,

P1: (10, 20, 30, 40, 50, RIGHTY)
P2: (–5, 10, –15, 20, –25, LEFTY)
H1: (100, 50, –50, 0)
H2: (12, 34, 56, 78)

ed eseguendo le seguenti operazioni,

P3 = P1 – P2
H3 = H1 – H2

si ottiene:

P3: (15, 10, 45, 20, 75, RIGHTY)
H3: (88, 16, –106, –78)

Nota: L'elemento <CONFIG> in P3 è indefinito.

(4) Sostituzione nei dati di tipo vettoriale

I seguenti metodi sono disponibili per sostituire (inserire) una costante, una variabile o il risultato di un calcolo in un elemento di dati di tipo vettoriale.

(a) Comandi per convertire una riga di dati di tipo scalare in dati di tipo vettoriale

Un comando POINT e un comando TRANS sono disponibili per convertire righe di dati scalari in dati vettoriali. POINT converte i dati scalari in dati vettoriali posizionali, mentre TRANS converte i dati scalari in dati vettoriali delle coordinate. Se un elemento non è incluso nell'espressione, esso viene considerato 0 all'interno dell'espressione. Per i dettagli sull'uso di questi comandi, consultare la "Sezione 3".

Esempio:

```
P1 = POINT(P2.X, P2.Y, P2.Z + 50, 0, 0)
H1 = H2 + TRANS(100, 100)
```

In genere, le dichiarazioni dei dati di tipo vettoriale vengono effettuate come illustrato di seguito.

Dati posizionali POINT (X, Y, Z, C, T, <CONFIG>)
Dati delle coordinate TRANS (X, Y, Z, C)

X, Y, Z, C e T sono valori di coordinate rappresentati da numeri reali. Le unità sono in millimetri o gradi.

<CONFIG> è l'acronimo di "configurazione" e contiene un numero intero da 0 a 2 che viene utilizzato per descrivere l'impostazione del sistema.

0 ... Libero (impostazione del sistema indefinita)
1 ... Sistema sinistrorso
2 ... Sistema destrorso

Tutti gli elementi omessi vengono considerati "0".

Nota 1: Per conoscere il tipo di dati a disposizione, utilizzare sempre il comando POINT durante la creazione di dati di tipo posizionale e il comando TRANS durante la creazione di dati di tipo coordinate.

Nota 2: Quando in un programma del linguaggio del robot vengono utilizzati dati posizionali non programmati, tali dati vengono provvisoriamente archiviati nella memoria del controller. Quindi, quando si ripristina il programma, i dati posizionali vengono cancellati. I dati posizionali sono validi solo nel programma che utilizza i dati. Pertanto, per utilizzare i dati posizionali in un sottoprogramma, è necessario trasferirli come argomento. Per i dettagli degli argomenti, consultare "2.8.2 Sottoprogrammi".

Nota 3: La sostituzione e il riferimento ai dati di tipo array (tipo di nome variabile [numero indice]) vengono eseguiti allo stesso modo del tipo di dati originali (tipo scalare e tipo vettoriale) dei dati di tipo array.

2.5.2 Espressioni logiche

Con SCOL, le espressioni logiche possono essere utilizzate in combinazione con i comandi IF, WAIT e ON. Inoltre, sono disponibili sei operandi relazionali (<, >, < = (o = <), > = (o = >), < > (o > <) e = =). Le espressioni logiche possono essere combinate anche utilizzando gli operandi logici AND, OR e NOT. Le costanti scalari, le variabili scalari e i risultati dei calcoli possono essere utilizzati come dati nelle espressioni logiche.
Quando si valuta l'equivalenza, utilizzare il segno "= =" e non il segno "=". Quando si confrontano numeri reali, le differenze di 0,001 o meno vengono ignorate.
Le espressioni logiche forniscono un valore intero di 1 se vero e di 0 se falso. Il risultato del calcolo è un numero intero.

Esempi:

1) IF K = =K2 * K3 THEN K = K2
   ON MOTION > = 50 DO DOUT (1,2)

2) Quando si esegue IF J1 THEN GOTO TRUE1 ELSE GOTO FALSE1, se J1 è un numero intero 0 oppure un numero reale con valore assoluto J1 è uguale o inferiore a 0,001, il confronto viene considerato FALSE e il programma si dirama in FALSE1.
   Se J1 è un valore diverso da un numero intero 0 oppure un numero reale con valore assoluto J1 è superiore a 0,001, il confronto viene considerato TRUE e il programma si dirama in TRUE1.

2.6 Etichette

Con il linguaggio SCOL, i rami del programma vengono specificati da etichette posizionate all'inizio della destinazione del ramo. Quando si etichetta un'istruzione come ramo, aggiungere i due punti al termine dell'identificatore.

Quando si dirige il programma da ramificare verso un'altra posizione con il comando GOTO, non aggiungere i due punti al termine dell'etichetta.

La ramificazione del programma può essere eseguita solo con un singolo programma. Non è possibile eseguire la ramificazione da un programma ad un altro. Inoltre, è possibile utilizzare le stesse etichette in diversi programmi, ma non è possibile utilizzare la stessa etichetta in un singolo programma.

Esempi:

```
LOOP1:  MOVE P1
GOTO    LOOP1
```

2.7 Note e commenti

Il linguaggio SCOL consente di aggiungere commenti al programma per renderne più semplice la comprensione (e il debug). È possibile inserire qualsiasi tipo di commento utilizzando il programmatore portatile. Le note e i commenti vengono scritti in uno dei seguenti formati.

(1) Comando REMARK

È possibile scrivere una nota o un commento dopo un comando REMARK. Nel comando REMARK, tutto ciò che viene scritto fino alla fine della riga (fino alla pressione del tasto [EXE]) viene considerato un commento e non viene eseguito come un programma.
Poiché il comando REMARK è indipendente, non può essere scritto dopo un altro comando.

Esempio:

```
REMARK SCOL SAMPLE PROGRAM
```

(2) Virgoletta singola

Il testo scritto dopo una virgoletta singola (') è considerato un commento. Questo metodo consente di inserire commenti dopo altri comandi. Tutto ciò che viene scritto dopo la virgoletta singola (') fino alla fine della riga (fino alla pressione del tasto [EXE]) viene considerato un commento e non viene eseguito come un programma.

Esempio:

```
MOVE P1        'MOVES THE ROBOT TO P1
'*** SCOL COMMENT SAMPLE ***
```

2.8 Programmi

In questo paragrafo vengono descritti i programmi SCOL.

2.8.1 Dichiarazione di programma

La dichiarazione di un programma di combinazione di comandi SCOL è scritta nel seguente formato.

```
PROGRAM <nome del programma>
  Contenuti del programma
END
```

Un programma è costituito da tutto, dall'istruzione PROGRAM all'istruzione END. Scrivere un nome di programma dopo l'istruzione PROGRAM. Il nome del programma viene espresso mediante un identificatore. Mettere il contenuto del programma tra l'istruzione PROGRAM e l'istruzione END.

Esempio:

```
PROGRAM SAMPLE     Nome di programma "SAMPLE"
  REMARK SAMPLE    Commento
  SPEED=20         Impostare la velocità di movimento al 20% della velocità
                   massima.
  MOVE Al          Spostare il robot sulla posizione Al.
  DELAY 0.5        Attendere 0,5 sec.
  MOVE A2          Spostare il robot sulla posizione A2.
  DELAY 0.5        Attendere 0,5 sec.
END                Fine del programma
```

Come illustrato nell'esempio, il corpo del programma è composto da istruzioni costituite da una combinazione di comandi SCOL. Ogni volta che si preme il tasto EXE viene creata una nuova riga. Una singola riga può contenere fino a 130 caratteri. È possibile aggiungere tutti gli spazi desiderati per rendere il programma più ordinato e più semplice da leggere. Si possono scrivere note e commenti utilizzando il simbolo (') in un'istruzione.

Nota: Non vi deve essere alcuno spazio tra i caratteri che formano una parola di un comando e un identificatore.

2.8.2 Sottoprogrammi

È possibile richiamare un sottoprogramma scrivendone semplicemente il nome nel programma principale.

Esempio:

Programma principale

```
PROGRAM MAIN
   REMARK *** SAMPLE 1 ***
   SUB1
END
```

Sottoprogramma

```
PROGRAM SUB1
   REMARK *** SUBPROGRAM NO. 1 ***
   Corpo del sottoprogramma
   RETURN
END
```

Nei sottoprogrammi è necessario inserire un comando RETURN per reinviare il controllo al programma principale. Se si dimentica di scrivere RETURN, SCOL perdonerà l'errore e simulerà la presenza un comando RETURN davanti all'istruzione END.

Prima di trasferire i dati tra i sottoprogrammi e il programma principale, è necessario specificare gli argomenti per il sottoprogramma. Quando si specifica un argomento in un sottoprogramma, l'istruzione del programma deve essere scritta come segue:

```
PROGRAM <nome programma> (<nomi degli argomenti>)
```

L'argomento viene specificato tra parentesi ( ) dopo il nome di programma del sottoprogramma. I valori dell'argomento vengono inseriti nei nomi di variabile specificati nel sottoprogramma. Quando si utilizzano vari argomenti, scrivere virgole (,) per separare i nomi di variabile tra parentesi ( ). Il numero massimo di argomenti è 10.

Nel programma principale, specificare i dati da trasferire quando il sottoprogramma viene specificato tra indicato ( ) dopo il nome del sottoprogramma. I dati specificati ( ) vengono trasferiti nell'ordine specificato nei nomi di variabile del sottoprogramma. I dati dell'argomento del programma principale vengono sostituiti all'interno del sottoprogramma e, quando si modificano i dati, anche i dati corrispondenti alle variabili del programma principale vengono contemporaneamente modificati.

Esempio:

Programma principale

```
PROGRAM MAIN
    REMARK *** SAMPLE 2 ***
    K1 = 15
    K2 = 28
    SUB2(K1, K2, K)
    PRINT K
END
```

Sottoprogramma

```
PROGRAM SUB2(N1, N2, N3)
    REMARK *** SUBPROGRAM NO. 2 ****
    N3 = N1 + N2
    RETURN
END
```

Nell'esempio precedente si sono trasferiti tre argomenti tra il programma principale e il sottoprogramma. In modo specifico, K1 del programma principale è stato trasferito come N1 del sottoprogramma. Allo stesso modo, K2 del programma principale è stato trasferito come N2 del sottoprogramma. Il sottoprogramma aggiunge N1 e N2, inserendo il risultato in una variabile denominata N3. Quando ciò accade, cambia anche il valore di K del programma principale.

Quando si esegue questo programma, il valori relativi a K1=15 e K2=28 vengono aggiunti nel sottoprogramma e il risultato K=43 viene visualizzato sul programmatore portatile mediante il comando PRINT del programma principale.

Si noti che i sottoprogrammi non possono assegnarsi un nome da soli. Inoltre, se si assegna un nome ad un sottoprogramma che si trova in un altro file, il controller non capisce l'istruzione e considererà il nome di quel sottoprogramma come un errore.

Nota 1: Un'espressione, risultato di un'espressione di dati vettoriali come ad esempio dati posizionali ed elemento dati vettoriali, non può essere designata come argomento.

Nota 2: Quando si utilizza una costante come argomento, non può essere sostituita in una variabile in base ad un sottoprogramma.

Nota 3: Per una variabile che è un argomento in un sottoprogramma, è necessario sostituire un valore nella variabile prima di eseguire il sottoprogramma.

2.8.3 Libreria

Il linguaggio SCOL non consente di utilizzare sottoprogrammi che non si trovano nello stesso file del programma principale. Tuttavia, inserendo sottoprogrammi particolarmente utili nel file di libreria, è possibile accedere al sottoprogramma dal programma principale.
Quando si scrive il proprio sottoprogramma da aggiungere al file di libreria, inserire il programma in quel file così come si inserirebbe in qualsiasi altro sottoprogramma. Per informazioni su come inserire un programma in un file, fare riferimento al Manuale dell'operatore.

La libreria è disponibile nei due (2) seguenti tipi.

[1] Libreria di sistema
Questa libreria viene sempre caricata durante l'esecuzione del programma.
Il nome file è “SCOL.LIB” e non può essere modificato, ma i contenuti possono essere aggiunti o modificati qualora necessario.
I comandi OPEN1 e CLOSE1 sono i sottoprogrammi inclusi in questo file di libreria. Il contenuto del file di libreria di norma incluso nel disco di sistema del controller del robot viene illustrato nell'Appendice C.
Quando si crea un nuovo sottoprogramma nella libreria di sistema, aggiungerlo al termine del file di libreria attuale (SCOL.LIB).

[2] Libreria a collegamento dinamico
È il file di libreria che l'utente può caricare qualora necessario.
Il nome del file di libreria è _____.LIB.
La libreria a collegamento dinamico è il programma utente ed è necessario dichiararne il caricamento. È possibile dichiarare il caricamento di un file di libreria nell'area GLOBAL del programma utente nel modo seguente.

```
GLOBAL
     LOADLIB PALLET.LIB    ← Dichiarazione di caricamento libreria.
END
PROGRAM SAMPLE1
     ~ ~ ~ ~ ~ ~
     Omitted
     ~ ~ ~ ~ ~ ~
END
```

È possibile caricare contemporaneamente fino a cinque (5) librerie a collegamento dinamico.

Alcuni comandi SCOL utilizzano questa libreria a collegamento dinamico.

Anche quando si utilizza tale comando, la dichiarazione di caricamento è necessaria con il comando di “LOADLIB + nome file”.

Quando si esegue un programma che utilizza la libreria a collegamento dinamico, il controller crea un file temporaneo denominato “SCOLLIB.TMP (SCOL.LIB + libreria a collegamento dinamico)”. Quando nell'area del programma utente non è disponibile uno spazio sufficiente, il programma potrebbe non essere eseguito.

Attenzione:
Se il nome del programma da eseguire è uguale a quello del sottoprogramma incluso nel file di libreria selezionato, tale sottoprogramma viene eseguito nel funzionamento automatico.

Azionamento della funzione di libreria a collegamento dinamico
Quando si seleziona un programma che richiede la libreria a collegamento dinamico (ovvero, un programma che include il comando LOADLIB), il sistema agisce nel modo seguente.

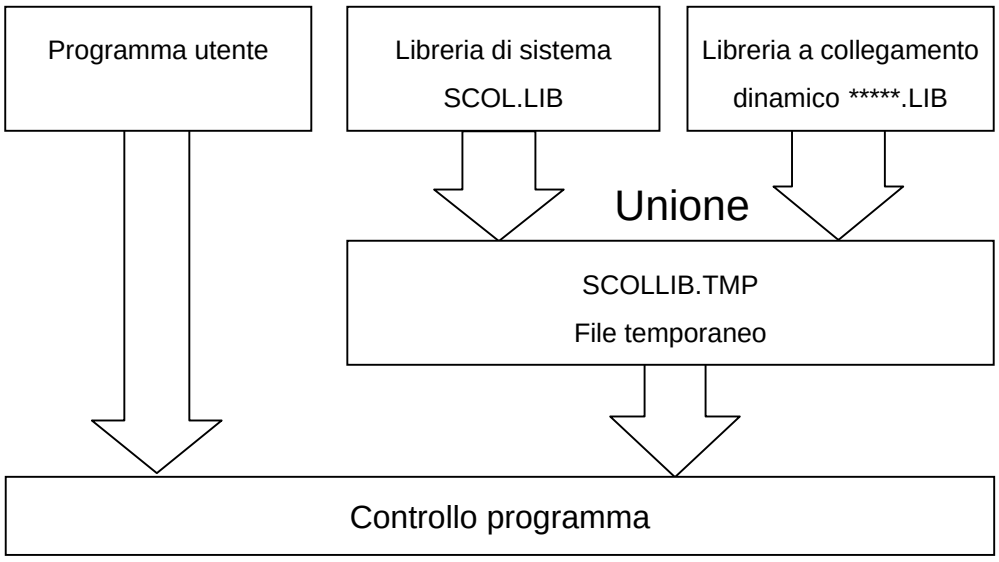
[1] Apertura del programma utente.

[2] Controllo del comando LOADLIB (ovvero, controllo del nome file).

[3] Unione di SCOL.LIB con la libreria a collegamento dinamico per creare SCOLLIB.TMP.

[4] Controllo del programma. A meno che non vi sia un problema, il comando SELECT termina.

[5] Quando il comando SELECT è terminato normalmente, SCOLLIB.TMP viene automaticamente eliminato (in caso di errore, SCOLLIB.TMP non viene eliminato).



In caso di errore nella libreria, il sistema visualizza il messaggio “LINE???:LIB>ERROR–***”.
In “LINE???” viene visualizzato il numero di riga di SCOLLIB.TMP. Controllare il contenuto di SCOLLIB.TMP, quindi modificare il file di libreria.

2.8.4 Elaborazione multitask

In questo paragrafo viene descritto l'utilizzo della funzione multitask del linguaggio SCOL insieme ai relativi comandi e alle variabili di sistema.
L'esecuzione del programma di gestione singola attività e multitask viene descritta nelle Fig. 1 e Fig. 2. Il numero in figura indica l'ordine di esecuzione del programma.
I tempi specifici di passaggio da un programma ad un altro (cambio attività) vengono descritti in seguito.

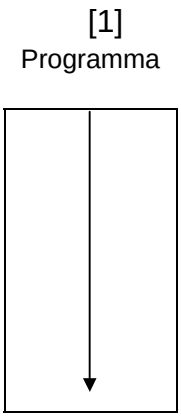


Fig. 1 Gestione singola attività

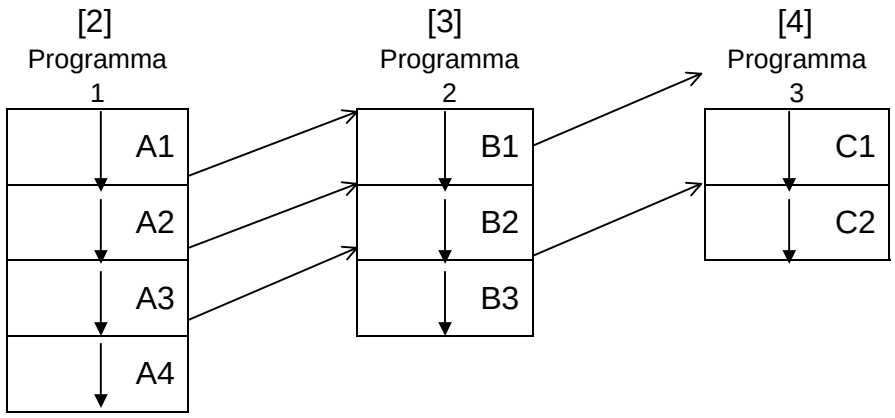


Fig. 2 Gestione multitask

Nella Fig. 1, il programma A viene eseguito continuamente dall'inizio alla fine (gestione singola attività e nessuna chiamata di sottoprogramma).
Un programma che non utilizza alcuno comando multitask viene eseguito nel modo illustrato nella Fig. 1 (nessuna chiamata di sottoprogramma).
L'esecuzione di un programma che utilizza il comando multitask viene illustrato nella Fig. 2.

Come illustrato nella Fig. 2, la gestione multitask viene realizzata passando a vari singoli programmi tramite time-sharing, come se i programmi venissero eseguiti parallelamente.

L'ordine dell'esecuzione del programma viene visualizzato nella seguente tabella.

Ordine	Programma da eseguire	
1	A1	Inizio programma 1
2	B1	Inizio programma 2
3	C1	Inizio programma 3
4	A2	
5	B2	
6	C2	Fine ciclo 1 del programma 3
7	A3	
8	B3	Fine ciclo 1 del programma 2
9	C1	Inizio programma 3
10	A4	Fine ciclo 1 del programma 1
11	B1	Inizio programma 2
12	C2	
13	A1	Inizio programma 1
:	:	

Successivamente, viene descritto l'inizio dell'operazione multitask.
Un programma considerato multitask è il blocco di programma che non contiene argomenti. Con blocco di programma si intende un'area tra il comando PROGRAM e il comando END costituito da istruzioni del linguaggio SCOL. Il sottoprogramma senza argomento può essere considerato come un'attività. L'argomento non può essere tenuto nell'attività.
Per disporre di un programma come attività, utilizzare il comando TASK. Il comando TASK esegue un programma specificato nell'argomento come attività. Se il programma non inizia con il comando TASK, non può essere eseguito come attività. Il blocco di programma (istruzioni tra il comando PROGRAM e il comando END) descritto all'inizio del file di programma è un'eccezione. Anche se non si utilizza il comando TASK, il programma viene eseguito come attività.
Per eseguire il programma 2 come attività nella Fig. 2, è necessario eseguire TASK (“PROG2”) nel programma 1 (il programma 1 viene descritto all'inizio del file e il programma inizia come attività senza il comando TASK).
Per eseguire il programma 3 come attività, è necessario eseguire una nuova attività (“PROG 3”) nell'attività (in questo caso, nel programma 1 o 2) già iniziata.
Se l'attività o il programma iniziati vengono ripristinati oppure l'operazione dell'attività viene richiamata dal linguaggio SCOL, l'attività viene mantenuta attiva.

Viene descritto l'ID di attività (il numero assegnato all'attività).
I numeri caratteristici (ID di attività) vengono assegnati alle attività rispettivamente avviate dal comando TASK. Nell'esempio della Fig. 2, “1” viene assegnato al programma 1, “2” al programma 2 e “3” al programma 3. L'ID di attività inizia in sequenza da 1 e aumenta di un numero ogni volta che inizia l'attività (ogni volta che si esegue l'attività). Se l'attività viene gestita dal linguaggio SCOL, viene utilizzato questo ID di attività.
Per ottenere l'ID di attività, osservare i seguenti esempi.

Esempio: I1 = TASK (“PROG 2”)

“I1” è una variabile desiderata di tipo intero. È possibile ottenere l'ID di attività di PROG 2. Questo comando viene eseguito nel programma 1. In questo esempio, non è possibile fare riferimento all'ID di attività proprio nel programma 2.

Esempio: I2 = TID

“I2” è una variabile desiderata di tipo intero. Se si fa riferimento alla variabile di sistema TID, è possibile acquisire l'ID di attività proprio. Se questo comando viene eseguito nel programma 2, l'ID di attività proprio può apparire nel programma 2 (“2” in questo caso).

Se questo comando viene eseguito nel programma 1, l'ID di attività del programma 1 (“1” in questo caso) viene sostituito da “I2”.
Se altre attività fanno riferimento all'ID di attività diverso dalla propria attività, devono essere definite delle variabili di esempi 1 e 2 come variabile globale.

Viene descritto il cambio di attività.
Come illustrato nella Fig. 2, il sistema esegue il programma 1 ~ 3 tramite time-sharing. Quando ciò accade, i tempi di cambio programma dipendono dalle tre seguenti condizioni.

(1) Quando il cambio di programma viene chiaramente specificato con il comando SWITCH del linguaggio SCOL.
Il comando SWITCH viene utilizzato se l'attività viene cambiata chiaramente con il linguaggio SCOL. Anche se non vengono soddisfatte le condizioni di cambio attività specificate nel sistema mentre si utilizza il comando SWITCH, è possibile cambiare attività.

(2) Quando una nuova attività inizia con il comando TASK del linguaggio SCOL.
Se una nuova attività inizia con il comando TASK, il controllo del programma passa all'attività appena iniziata.

(3) Quando l'attività termina con il comando KILL del linguaggio SCOL.
Se una nuova attività termina con il comando KILL, il controllo del programma passa alla successiva attività.

(4) Quando vengono soddisfatte le condizioni predeterminate specificate nel sistema e il programma viene cambiato dal sistema.

Le condizioni di cambio attività specificate nel sistema sono le seguenti:

(1) Un programma in un'attività viene eseguito per oltre 100 msec.

(2) Quando l'area dei dati per il comando di movimento è piena.
Il comando di movimento può leggere in anticipo fino a quattro dati. Se questa area interna per la lettura precedente è piena, l'attività è cambiata.

(3) Quando si è eseguito un comando necessario per la comunicazione con il dispositivo esterno.
I comandi INPUT, PRINT e RESTORE non vengono eseguiti solo dal programma SCOL attivo. Richiedono del tempo per l'esecuzione della gestione dei file TP e RAM da parte di un operatore. Il programma attivo attende la risposta di fine elaborazione da parte dell'operatore e cambia l'attività.
Per annullare il cambio di attività dal sistema, specificare DISABLE per la variabile di sistema SWITCH.

Nota: Durante l'esecuzione a fasi o quando si è annullato il cambio attività, prosegue solo il programma attualmente eseguito, mentre il programma di un'altra attività già iniziata non viene eseguito (la gestione singola attività diventa operativa).

Precauzioni sulla creazione di un programma multitask

(1) I comandi di movimento possono essere utilizzati solo nell'attività principale. Se si utilizzano in attività secondarie, viene generato un errore.
I comandi di movimento sono MOVEI, MOVEA, MOVE, MOVES, MOVEC, MOVEJ e DELAY.

(2) Poiché ciascuna attività dispone di variabili di sistema TIMER, TID e NOWAIT separate, è possibile impostarle o fare riferimento ad esse in ciascuna attività. Per NOWAIT, è possibile selezionarne l'utilizzo separato o comune tramite il parametro utente [U02].
Si consiglia di utilizzare NOWAIT separatamente e di impostare ENABLE NOWAIT nell'attività secondaria.
Le seguenti variabili di sistema vengono comunemente utilizzate tra le attività. Un valore impostato in un'attività rimane operativo in un'altra attività. Prestare le dovute precauzioni, quindi, quando si impostano varie attività.

CONFIG, ACCUR, ACCEL, DECEL, SPEED, PASS, TORQUE, GAIN, TOOL, BASE, WORK, PAYLOAD, NOWAIT

(3) Quando si esegue un comando per l'attesa della fine di un movimento come illustrato di seguito, mentre il comando di movimento viene attualmente eseguito nell'attività principale, l'attività secondaria attende la fine del comando di movimento attivo nell'attività principale.

1) Esecuzione dei comandi di ingresso/uscita in modalità DISABLE NOWAIT
I comandi di ingresso/uscita includono INPUT, PRINT, DIN, DOUT, BDCIN, BCDOUT e POUT.

2) Esecuzione del comando WAIT MOTION.

(4) Se l'esecuzione di un comando INPUT o PRINT è stata comandata in un'altra attività mentre si è eseguito lo stesso comando in un'attività per un diverso canale di comunicazione, il comando INPUT o PRINT iniziato per primo viene eseguito fino alla fine, mentre l'altra attività deve attendere che il comando venga eseguito.

2.8.5 Definizione di variabile globale

Se viene definita la variabile globale cui si può fare riferimento dall'intero programma, osservare le seguenti regole.

(1) Dichiarazione di variabile globale

Se si utilizza la variabile globale,è necessario definire il tipo e l'identificatore (nome della variabile) da utilizzare.

Questa definizione deve essere eseguita prima della prima istruzione PROGRAM.

Per definire la variabile A di tipo numero reale e la variabile B di tipo intero, la definizione è la seguente:

```
GLOBAL
A = 1.0     (Questo è il valore iniziale della variabile.)
B = 2
END

PROGRAM
:
END
```

(2) Dichiarazione di variabile globale per tipo

Per definire la variabile globale di ciascun tipo, utilizzare i seguenti formati.

Tipo intero:	A = 1	
Tipo numero reale:	B = 1.0	
Tipo array:	DIM D(10) AS INT	Viene definito l'array di dieci elementi di tipo intero. (Nota 1)
	DIM E(10, 3) AS REAL	Viene definito l'array di 10 × 3 elementi di tipo numero reale.
	DIM F(5) AS POINT	Viene definito l'array di cinque elementi di tipo posizionale.

Nel blocco globale tra le parole riservate GLOBAL e END, devono essere descritte solo le variabili di tipo scalare e di tipo array, mentre le variabili di tipo vettoriale devono essere descritte nel blocco di dati riservato tra le parole riservate DATA e END modificate dall'editor dei dati.

(3) Impostazione del valore iniziale della variabile di tipo array
Come per la variabile globale diversa da quella di tipo array, specificare il valore iniziale della variabile di array di tipo scalare con il blocco globale e specificare il valore iniziale della variabile di array di tipo vettoriale con il blocco di dati.

Nota 1: Il valore iniziale della variabile globale di tipo array, che non è impostato chiaramente, è indefinito. La variabile deve essere inizializzata dal programma utente.

2.8.6 Variabile di tipo array

Per le variabili di tipo scalare e vettoriale, ciascun nome rappresenta un (1) dato. Se un (1) nome può significare più dati, tuttavia, la programmazione diventa più facile, e la variabile di tipo array può essere utilizzata a questo scopo.
Per utilizzare la variabile di tipo array, è necessario predeterminare il tipo e il numero di tutti gli elementi con il comando DIM. La variabile di tipo array con lo stesso nome non può avere un tipo con diversi elementi.
Per i dettagli, consultare le descrizioni sul comando DIM.
La variabile dichiarata di tipo array con il comando DIM deve essere descritta nel programma in base al seguente formato.

Nome variabile (<elemento> [. <elemento>] ⋯)

Per il numero di elemento successivo al nome della variabile, è possibile selezionare qualsiasi valore da 1 a un numero specificato con il comando DIM. Tuttavia, se il numero totale di "nome variabile + "(" + elemento + ")" è superiore a dieci (10) caratteri, viene generato un errore di esecuzione.
Quando si esegue il programma SCOL, viene selezionato un (1) dato specifico tra vari dati in base al nome della variabile e al numero di elemento.
Quando si elaborano 25 posizioni (5 × 5) come variabile di array denominata "P" e ciascun asse viene spostato a sua volta nelle rispettive posizioni, il programma è il seguente:

```
GLOBAL
    DIM P(5, 5) AS POINT
END
PROGRAM SAMPLE
FOR I = 1 TO 5
    FOR J = 1 TO 5
        MOVE P(I, J)
```

```
    NEXT J
NEXT I
END
DATA
    POINT P(1,1) = 650, 0, 100, 0, 0 / LEFTY
    POINT P(1,2) = 650, 0, 100, 0, 0 / LEFTY
                  :
                  :
    POINT P(5,5) = 650, 0, 100, 0, 0 / LEFTY
END
```

Non è possibile effettuare l'accesso diretto o la sostituzione di elementi X e Y di una variabile di array di tipo vettoriale. Nell'esempio di cui sopra, il seguente comando non può essere eseguito per variabile P di tipo array come indicato in precedenza.

```
PRINT P(1,1).X, CR
```

Se ciò dovesse accadere, copiare il valore sulla variabile di tipo vettoriale normale, quindi eseguire il comando.

```
PP=P(1,1)
PRINT PP.X
```

Inoltre, se per errore si comando lo spostamento alla posizione P (I, J) con I = 5 e J = 6, viene generato un errore di esecuzione in quanto non è possibile effettuare l'accesso o la sostituzione di un elemento diverso da quello specificato dal comando DIM. Il tipo di indice di array è solo intero. Se si utilizzano i dati di un tipo numero reale e vettoriale, si verifica un errore durante l'esecuzione.

Sezione 3

Spiegazione dei comandi del robot

Qui vengono descritti in dettaglio i significati e le operazioni del comando SCOL. Questi comandi sono elencati in ordine alfabetico.

3.1 Spiegazione dei comandi

I comandi vengono spiegati come segue.

(1) Scopo
In questo paragrafo viene fornita una semplice spiegazione delle operazioni dei comandi.

(2) Formato
In questo paragrafo viene descritto come annotare i comandi. I simboli utilizzati nel paragrafo hanno i seguenti significati:

[ ]	Indica che il contenuto può essere omesso. I comandi sono indicati se necessario.
< >	Indica il contenuto dei dati da descrivere.
{ }	Indica che uno dei dati tra parentesi deve essere selezionato.
...	Indica che un insieme di elementi di dati può essere specificato.

I simboli di cui sopra sono utilizzati a scopo esplicativo e non sono effettivamente scritti nel programma. Se questi simboli devono essere utilizzati in un caso particolare, verrà fornita una spiegazione.

(3) Esempi
In questo paragrafo vengono utilizzati esempi di utilizzo dei comandi.

(4) Analisi e consigli
In questo paragrafo viene fornita un'analisi dei comandi e vengono descritte le avvertenze e le restrizioni per l'uso.

(5) Esempio di programma

In questo paragrafo viene illustrato un breve esempio di programma utilizzando i comandi.

Il significato dei dati in formato di comandi viene illustrato di seguito. Anche un'espressione può essere utilizzata come dati.

<Posizione>	Specifica dati di posizione.
<Asse>	Specifica un asse controllato da giunto. I dati devono essere costituiti da un numero intero da 1 a 5.
<Posizione assoluta>	Specifica la posizione assoluta di ciascun asse. I dati sono in unità di 0,001 mm o 0,001 gradi. Anche le variabili o le espressioni possono essere utilizzate come dati.
<Posizione relativa>	Specifica lo spostamento di ciascun asse in termini di posizione relativa. I dati sono in unità di 0,001 mm o 0,001 gradi. Anche le variabili o le espressioni possono essere utilizzate come dati.
<Tempo>	Specifica il tempo in unità di 0,01 secondi. Anche le variabili o le espressioni possono essere utilizzate.
<Espressione logica>	Specifica un'espressione logica.
<Istruzione>	Specifica un'istruzione da eseguire. Se si tratta di un'istruzione SCOL normale, è possibile specificare tutto ciò che si desidera.
<Condizione monitoraggio>	Con un'istruzione ON, specifica le condizioni da monitorare.
<Etichetta>	Specifica un'etichetta per la ramificazione del programma.
<Commento>	Mostra commenti scritti nel programma.
<Variabile>	Indica una variabile.
<Espressione>	Indica un calcolo. Le singole variabili possono anche essere sostituite da un calcolo.

<Nome segnale>	Specifica il nome di un segnale I/O (ingresso/uscita). Il nome del segnale deve essere assegnato come numero intero. Un valore positivo indica che il segnale è ON, mentre un valore negativo indica che il segnale è OFF. Anche le variabili e le espressioni possono essere utilizzate.
<Peso>	Specifica il peso del carico che agisce sulla mano del robot.
<Offset del baricentro>	Specifica la distanza tra il baricentro del carico applicato alla punta della mano del robot e il centro dell'utensile della mano.
<Configurazione>	Specifica la configurazione del robot con un numero intero. "0" indica una configurazione indefinita (non fissa), "1" una configurazione sinistrorsa e "2" una configurazione destrorsa.
<Interruttore>	Specifica l'interruttore di sistema. Sono disponibili due interruttori di sistema.
	PASS Questo interruttore di sistema specifica lo spostamento rapido.
	NOWAIT Questo interruttore di sistema dirige il segnale I/O da eseguire senza attendere il completamento di un precedente comando di movimento.
<Stato>	Specifica ciò che si deve ripristinare con il comando RESET.

La Tabella 3.1 presenta un elenco di comandi classificati in base allo scopo.

Tabella 3.1 Comandi SCOL

Tipo	Comando	Scopo
Comandi di controllo movimento	BREAK	Sospende immediatamente il movimento.
	CLOSE1, CLOSE2	Chiude la mano una volta completato il movimento.
	CLOSEI1	Chiude la mano.
	CLOSEI2	Chiude la mano.
	DELAY	Si interrompe per un dato periodo di tempo.
	MOVE	Movimento sincrono
	MOVES	Movimento di interpolazione lineare
	MOVEC	Movimento di interpolazione circolare
	MOVEA	Movimento assoluto singolo asse
	MOVEI	Movimento relativo singolo asse
	MOVEJ	Movimento arco
	OPEN1, OPEN2	Apre la mano una volta completato il movimento.
	OPENI1, OPENI2	Apre la mano.
	PAUSE	Sospende un movimento.
	READY	Si sposta all'origine delle coordinate della macchina.
	RESUME	Riavvia un movimento interrotto.
Comandi di controllo programma	FOR ~ TO ~ STEP ~	Ripete il movimento.
	GOTO	Esegue la ramificazione in modo incondizionato.
	GOTO ()	Esegue la ramificazione in base al valore di un'espressione.
	IGNORE	Annulla il monitoraggio.
	IF ~ THEN ~ ELSE ~	Valuta le condizioni.
	NEXT	Ripete il movimento.
	ON ~ DO ~	Registra il monitoraggio delle condizioni.
	PROGRAM	Contrassegna l'inizio del programma.
	RCYCLE	Etichetta per il ripristino ciclo.
	RETURN	Ritorna al programma principale.
	STOP	Arresta il programma.
	WAIT	Attende la creazione di condizioni.
	END	Fine del programma.
	KILL	Arresto dell'attività.
	MAXTASK	Numero massimo di attività.
	REMARK	Commenti
	SWITCH	Cambio attività
	TASK	Inizio attività
	TID	ID attività

Tipo	Comando	Scopo
Comandi di controllo I/O	BCDIN	Immette un segnale BCD.
	BCDOUT	Emette un segnale BCD.
	CR	Emette un codice CR.
	DIN	Legge un segnale di ingresso.
	DOUT	Emette un segnale.
	HEXIN	Legge i segnali con numerazione esadecimale.
	HEXOUT	Emette i segnali con numerazione esadecimale.
	PULOUT	Emette un segnale a impulsi.
	RESET	Ripristina il controller.
	PRINT	Emette dati di comunicazione.
	INPUT	Immette dati di comunicazione.
Comandi di condizione movimento	ACCEL	Specifica l'accelerazione (durante l'accelerazione).
	ACCUR	Specifica la precisione di posizionamento.
	CONFIG	Specifica la configurazione.
	DECEL	Specifica l'accelerazione (durante la decelerazione).
	DISABLE	Spegnimento dell'interruttore di sistema.
	ENABLE	Accensione dell'interruttore di sistema.
	FREELOAD	Annulla i dati di carico.
	GAIN	Guadagno di ciascun asse.
	NOWAIT	Non attendere il completamento del posizionamento relativo al precedente movimento.
	PASS	Parametro di spostamento rapido
	PAYLOAD	Imposta i dati di carico.
	SMOOTH (option)	Movimento uniforme.
	SPEED	Specifica la velocità.
	MOVESYNC	Specifica la modalità di sincronizzazione/desincronizzazione del comando di movimento.
	SWITCH	Proibisce o consente il cambio attività.
	WITH	Specifica le condizioni operative.

Tipo	Comando	Scopo
Comandi calcolatrice	COS	Coseno
	SIN	Seno
	TAN	Tangente
	ABS	Valore assoluto
	ACOS	Arcocoseno
	AND	Prodotto logico
	ASIN	Arcoseno
	ATAN	Arcotangente
	ATAN2	Arcotangente
	DEST	Posizione di destinazione
	EXP	Elevato a potenza e
	HERE	Posizione attuale
	INT	Cambia il numero in un intero.
	LN	Logaritmo naturale
	LOG10	Logaritmo comune
	MOD	Resto
	NOT	Negazione
	OR	Somma logica
	POINT	Crea dati di tipo posizionale.
	REAL	Cambia il numero in un numero reale.
	SGN	Estrae e riporta il segno.
	SQRT	Radice quadrata
	TRANS	Crea dati di tipo coordinate.
Comandi di riferimento del movimento	BASE	Sistema di coordinate di base
	MODE	Modalità operativa del sistema
	MOTION	Quantità di movimento eseguito
	MOTIONT	Tempo impiegato per un movimento
	REMAIN	Quantità di movimento rimanente da eseguire
	REMAINT	Tempo rimanente per un movimento
	TIMER	Timer
	TOOL	Sistema di coordinate utensile
	WORK	Sistema di coordinate lavoro

Tipo	Comando	Scopo
Comandi di definizione dati	DATA	Inizia la definizione dei dati.
	DIM ~ AS	Definizione variabile array
	GLOBAL	Definizione variabile globale
	RESTORE	Salva un valore iniziale della variabile globale su un file.
	SAVEEND	Salva i dati durante lo spegnimento.
Comando di pallettizza-zione	INITPLT	Inizializza un pallet.
	MOVEPLT	Si sposta sulla posizione specificata del pallet.
Funzione latch dati di posizione (opzioni di TS3000)	LATCH	Funzione latch di posizione ON/OFF
	LATCHTRG 1 ~ 8	Direzione bordo rilevata
	LATCHSIG 1 ~ 8	Stato segnale
	LATCHPSN 1 ~ 8	Posizione latch
Costanti di sistema	COARSE	Precisione di posizionamento approssimativa
	COM0, TP	Canale di comunicazione (programmatore portatile)
	COM1	Canale di comunicazione 1
	CONT	Modalità di funzionamento continuo
	CYCLE	Modalità di funzionamento a cicli
	FINE	Precisione di posizionamento ottimale
	OFF	Guadagno di ciascun asse OFF
	ON	Guadagno di ciascun asse ON
	PAI	Pi
	SEGMENT	Modalità di funzionamento a segmenti
PLC semplificato	PLCDATAR 1 ~ 8	Interfaccia PLC semplificata
	PLCDATAW 1 ~ 8	Interfaccia PLC semplificata

Tipo	Comando	Scopo
Simboli matematici	^	Elevamento a potenza
	–	Segno negativo
	*, /	Moltiplicazione e divisione
	+, –	Addizione e sottrazione
	=	Sostituzione
	= =	Uguale
	< >, > <	Non uguale
	<	Inferiore a
	>	Superiore a
	< =, = <	Inferiore a o uguale
	> =, = >	Superiore a o uguale
	‘	Commenti
	.	Designazione di elementi vettoriali

3.2 Spiegazione dei comandi

Nelle seguenti pagine vengono spiegati i comandi SCOL. I comandi sono disposti in ordine alfabetico.

ABS

Scopo

La funzione ABS riporta il valore assoluto di un numero.

Formato

ABS (<espressione>)

Esempi

```
AK = ABS(–20.345)
K  = ABS(K1)
J1 = K – ABS(N – 28.5)
```

Analisi e consigli

Questa funzione riporta il valore assoluto dell'<espressione>.

È possibile utilizzare una costante, una variabile o un risultato di calcolo per il termine <espressione>. Tuttavia, non è possibile utilizzare dati vettoriali.

Questo comando deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    ABSSAMPLE (3, 5, K)
    PRINT TP, K, CR
END
PROGRAM ABSSAMPLE(K1, K2, K)
    K = ABS(K1 – K2)
    RETURN
END
```

Questo programma prende gli argomenti K1 e K2, sottrae K2 da K1, trova il valore assoluto del risultato, lo chiama K e invia di nuovo l'esecuzione del programma al programma principale.

ACCEL

Scopo

Questo comando imposta la frazione dell'accelerazione completa del robot.

Formato

ACCEL = (<espressione>)

Esempi

ACCEL = 80
ACCEL = 0. 8*ACCEL
MOVE A1 WITH ACCEL = 90

Analisi e consigli

ACCEL è una variabile di sistema utilizzata per specificare l'accelerazione del robot quando il robot è in fase di accelerazione.
L'accelerazione è espressa come percentuale dell'accelerazione standard (completa). Nel linguaggio SCOL, l'accelerazione durante l'accelerazione e l'accelerazione durante la decelerazione vengono impostate separatamente. Per impostare l'accelerazione durante la decelerazione, utilizzare i comandi DECEL.

Ciò viene utilizzare per ridurre l'accelerazione, se necessario, quando il robot trasporta un carico pesante. In tal caso, modificare l'accelerazione sia durante l'accelerazione che durante la decelerazione. Per il valore di impostazione dell'accelerazione in base al carico, consultare il "Manuale di installazione e trasporto".

È possibile utilizzare una costante, una variabile o un calcolo per l'<espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'espressione.
Nel controller viene impostato un limite massimo di accelerazione per proteggere il robot. Il robot non deve superare questo limite anche se si inserisce un valore superiore al limite massimo.
Valori di 0 o meno vengono considerati 1.

È possibile visualizzare l'accelerazione attuale durante l'accelerazione visualizzando questa variabile di sistema.

Questo valore iniziale per l'accelerazione è 100%.

Esempio di programma

```
PROGRAM ACCELSAMPL
    FOR K = 1 TO 100
        ACCEL = K
        DECEL = K
        MOVE A1
        MOVE A2
    NEXT K
END
```

Questo programma aumenta l'accelerazione dall'1% al 100% con incrementi dell'1%.

ACCUR

Scopo

Questo comando specifica la precisione di posizionamento del robot.

Formato

ACCUR = (<espressione>)

Esempi

```
ACCUR = 1
N = ACCUR
MOVE A1 WITH ACCUR = COARSE
```

Analisi e consigli

ACCUR è una variabile di sistema utilizzata per specificare la precisione di posizionamento del robot. Una precisione di posizionamento approssimativa è contrassegnata con 0, mentre una precisione di posizionamento ottimale con 1.

Quando la precisione di posizionamento è impostata su approssimativa, il robot esegue il comando successivo prima di completare il posizionamento del robot. È possibile ridurre la cadenza del robot impostando la precisione di posizionamento su COARSE per operazioni in cui non è necessario attendere una precisione di posizionamento elevata.
Le costanti di sistema FINE e COARSE possono essere utilizzate per specificare la precisione di posizionamento. La precisione di posizionamento è impostata su ottimale con ACCUR=FINE e su approssimativa con ACCUR=COARSE.
È possibile utilizzare una costante, una variabile o un calcolo per l'<espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando ACCUR deve essere utilizzato in un'espressione.

Quando si specifica la precisione di posizionamento, tutto ciò che è inferiore a 0 viene considerato 0, mentre tutto ciò che è superiore a 1 viene considerato 1.

È possibile trovare la precisione di posizionamento con la quale il sistema agisce attualmente facendo riferimento ad ACCURA. Un valore ACCUR pari a 0 significa approssimativo, mentre un valore pari a 1 significa ottimale.

L'impostazione iniziale per la precisione di posizionamento è FINE.

Esempio di programma

```
PROGRAM ACCURSMPL
    ACCUR = COARSE
    MOVE A1
    MOVE A2
    MOVE A3   WITH ACCUR = FINE
    MOVE A4
END
```

Il robot si sposta con precisione di posizionamento ottimale solo per il movimento al punto A3. Si sposta con precisione di posizionamento approssimativa per altri movimenti aumentando la durata del ciclo.

ACOS

Scopo

Questa funzione riporta l'arcocoseno di un valore inserito.

Formato

ACOS (<espressione>)

Esempi

K = ACOS (0.577)
J1 = 90 – ACOS (X/L)

Analisi e consigli

Questa funzione riporta l'arcocoseno del valore tra parentesi ( ). Il valore riportato è in unità di gradi.

È possibile inserire una costante, una variabile o un calcolo per l'<espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    ACOS2 (1.0, 2.0, K)
    PRINT TP, K, CR
END
PROGRAM ACOSSAMP (L, X, K)
    K = ACOS (X/L)
    RETURN
END
```

Questo programma prende gli argomenti L e X, divide X per L, prende l'arcocoseno del risultato, lo chiama K e torna al programma principale.

AND

Scopo

AND calcola il prodotto logico delle espressioni.

Formato

<Espressione logica> AND <Espressione logica>

Esempi

IF DIN (1) AND K < = 3 THEN J = 0
WAIT DIN (5) AND TIMER==0

Analisi e consigli

L'istruzione AND viene utilizzata per trovare il prodotto logico di due espressioni logiche. Se entrambe le espressioni logiche sono vere, viene riportato un valore TRUE.

Questo comando deve essere utilizzato in un'espressione logica.

Esempio di programma

```
PROGRAM ANDSAMPLE
    FOR K=1 TO 50
    IF K==50 AND DIN (1) THEN J=1 ELSE J=0
    PRINT TP, J, CR
    NEXT K
END
```

ASIN

Scopo

Questa funzione trova l'arcoseno del valore inserito.

Formato

ASIN ((espressione>)

Esempi

K = ASIN (0.577)
J1 = 90 – ASIN (Y/L)

Analisi e consigli

Questa funzione riporta l'arcoseno del valore tra parentesi ( ). Il valore riportato è in unità di gradi.

È possibile inserire una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'equazione.

Esempio di programma

```
PROGRAM MAIN
    ASIN2 (5.0, 2.0, K)
    PRINT TP, K, CR
END
PROGRAM ASIN2 (L, Y, K)
    K = ASIN (Y/L)
    RETURN
END
```

Questo programma prende gli argomenti L e Y, divide X per L, prende l'arcoseno del risultato, lo chiama K e torna al programma principale.

ATAN/ATAN2

Scopo

Questa funzione riporta l'arcotangente dei valori inseriti.

Formato

ATAN (<espressione>)
ATAN2 (<espressione>, <espressione>)

Esempi

K = ATAN (0.577)
J1 = 90 – ATAN (Y/X)
N = ATAN2 (0.3, 0.5)
D = ATAN2 (100/K, 50/J)
L3 = ABS (180 – ATAN2 (A1, Y, A1, X))

Analisi e consigli

Questa funzione riporta l'arcotangente dei valori tra parentesi ( ).
ATAN2 prende la prima espressione tra parentesi, la divide per la seconda espressione tra parentesi e trova l'arcotangente del risultato.

Per entrambe le funzioni, il valore riportato è in unità di gradi. (Si noti che ATAN2 (0, 0) riporta 0 piuttosto che un errore.)

È possibile inserire una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale. Questo comando deve essere utilizzato in un'equazione.

Esempio di programma

```
PROGRAM MAIN
    ATANSAMPLE (5.0, 3.0, K)
    PRINT TP, K, CR
END
PROGRAM ATANSAMPLE (X, Y, K)
    K = ATAN (Y/X)
    RETURN
END
```

Questo programma prende gli argomenti X e Y, divide Y per X, prende l'arcotangente del risultato, lo chiama K e torna al programma principale.

```
PROGRAM MAIN
   ATAN2SMPL (K)
   PRINT TP, K, CR
END
PROGRAM ATAN2SMPL (K)
   K = ATAN2 (A1. Y, A1. X)
   RETURN
END
```

Questo programma prende gli elementi X e Y dal punto programmato A1, divide Y per X, prende l'arcotangente del risultato, lo chiama K e torna al programma principale.

BASE

Scopo

BASE è una variabile di sistema utilizzata per specificare il sistema di coordinate di base.

Formato

BASE

Esempi

```
BASE = TRANS (0, 0, 0, 0)
BASE1 = BASE
MOVE A1 WITH BASE = BASE + TRANS (,, 100)
```

Analisi e consigli

BASE è una variabile di sistema utilizzata per specificare il sistema di coordinate di base. È possibile gestirla come normali dati di tipo coordinate. Facendo riferimento a BASE, si possono trovare i valori (la posizione) del sistema di coordinate di base attuale.

È possibile designare direttamente i valori delle coordinate di base con uno dei due seguenti metodi:

```
BASE = TRANS (X, Y, Z, C)
BASE = {X, Y, Z, C}
```

Per conoscere il tipo di dati a disposizione, utilizzare sempre il comando TRANS.

La variabile BASE ha il seguente formato:

X, Y, Z, C: X, Y, Z e C sono numeri reali che rappresentano la posizione del sistema di coordinate di base. Le unità sono in millimetri o gradi.

Il sistema di coordinate BASE viene creato "scorrendo" una distanza di X, Y e Z lungo i rispettivi assi del sistema di coordinate WORLD, quindi ruotando il nuovo asse Z di una quantità C.

BASE deve essere utilizzato in un'espressione.

Si noti che se si modificano i sistemi di coordinate di base in un programma, potrebbe verificarsi un disallineamento tra le posizioni programmate e le posizioni in cui il robot si sposta effettivamente.

Esempio di programma

```
PROGRAM BASESAMPLE
    MOVE A1
    MOVE A2
    BASE = BASE + TRANS (,, 200)
    MOVE A1
    MOVE A2
    BASE = TRANS ()
END
```

Il comando BASE sposta il sistema di coordinate BASE di 200 mm lungo l'asse Z e, successivamente, il robot si sposta in un punto inferiore di 200 mm rispetto alla posizione programmata.

BCDIN

Scopo

Questo comando viene utilizzato per leggere i segnali come numerazione BCD (Binary Coded Decimal - Numero decimale codificato in binario).

Formato

BCDIN (<nome segnale>, <lunghezza segnale>)

Esempi

K = BCDIN (1, 2)
J2 = BCDIN (N, N + 2)
GOTO (BCDIN (20, 1)) L1, L2, L3

Analisi e consigli

Il comando causa la lettura di un segnale di ingresso come numerazione BCD. Il segnale inizia dal nome del segnale e continua in un punto corrispondente a 4 volte il valore della lunghezza del segnale specificato. Ad esempio, "K = BCDIN (1, 2)" indica al controller di leggere, come numerazione BCD, un segnale di otto unità che inizia dalla 1ª unità (bit) e prosegue fino all'8ª unità (bit).

Numeri di segnale più elevati corrispondono a bit con una cifra superiore.

I segnali di ingresso sono suddivisi in unità di 4 bit (dal valore più basso a quello più alto (numero segnale)) e convertiti in Base 10.

I segnali vengono codificati come 1 per ON e 0 per OFF.

Esempio:
Nel caso in cui i segnali di ingresso da 1 a 12 indichino lo stato illustrato di seguito, il valore di BCDIN(1,3) è 329.

Numero segnale di ingresso	12	11	10	9	8	7	6	5	4	3	2	1
Stato segnale di ingresso	OFF	OFF	ON	ON	OFF	OFF	ON	OFF	ON	OFF	OFF	ON
Espressione Base 2	0	0	1	1	0	0	1	0	1	0	0	1
Espressione Base 10				3				2				9

È possibile utilizzare costanti, variabili, espressioni o calcoli per <nome segnale> e <lunghezza segnale>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'equazione.

Quando un segnale di ingresso viene codificato in base a 4 bit e se supera 9 (1001 in numerazione binaria), il segnale viene codificato in numerazione binaria. In altre parole, se si immette il segnale “1111” in numerazione binaria, viene letto come “15”.

Esempio di programma

```
PROGRAM BCDINSAMPL
    K = BCDIN(I, 2)
    SPEED = K
    MOVE A1
    MOVE A2
END
```

Questo programma legge un segnale costituito da otto bit e imposta la velocità di movimento in base a quel segnale.

BCDOUT

Scopo	Questo comando presenta un segnale in numerazione BCD ed emette il risultato.
Formato	BCDOUT (<nome segnale>, <lunghezza segnale>, <espressione>)
Esempi	BCDOUT(1, 4, 3) BCDOUT(N, N+4, K)
Analisi e consigli	Questo comando prende il valore dell'espressione e lo trasforma in un segnale BCD denominato <nome segnale> e con una lunghezza di segnale corrispondente a quattro volte il valore di <lunghezza segnale>. Ad esempio, il comando BCDOUT(1, 1, 3) produce un segnale di uscita lungo quattro bit e inizia dalla posizione del segnale 1. Il segnale di uscita esprime il valore 3 come numero binario. Pertanto, i primi due bit (numeri 1 e 2 del segnale di uscita) saranno ON. Numeri di segnale più elevati corrispondono a bit con una cifra superiore. Ciascuna cifra di <espressione> viene suddivisa una per volta e convertita in un codice a 4 bit. Il codice a 4 bit stesso viene creato seguendo l'ordine dal numero inferiore a quello superiore del segnale di uscita. Qualora non vi fosse spazio sufficiente per contenere tutto il codice bit corrispondente all'<espressione> Base 10 (ovvero, nel caso in cui la lunghezza del segnale fosse troppo breve), le cifre in eccesso dell'<espressione> Base 10 verranno ignorate. I segnali vengono considerati 1 per ON e 0 per OFF.

Esempio:
Il comando BCDOUT(1, 3, 952) crea un segnale di uscita simile a quello illustrato di seguito.

Espressione Base 10				9				5				2
Espressione Base 2	1	0	0	1	0	1	0	1	0	0	1	0
Numero segnale di uscita	12	11	10	9	8	7	6	5	4	3	2	1
Stato segnale di uscita	ON	OFF	OFF	ON	OFF	ON	OFF	ON	OFF	OFF	ON	OFF

È possibile utilizzare costanti, variabili, espressioni o calcoli per <nome segnale>, <lunghezza segnale> ed <espressione>.
Tuttavia, non è possibile utilizzare dati di tipo vettoriale.
Se gli stessi segnali vengono emessi consecutivamente, diventa valido l'ultimo segnale emesso.
Fare attenzione che esista una gamma di segnali entro la quale sia possibile garantire la simultaneità.
È possibile garantire la simultaneità di BCDOUT nella gamma di 16 bit da DOUT1, DOUT101, DOUT301, ma non la simultaneità di BCDOUT che superi tali limiti.

Esempio di programma

```
PROGRAM BCDOUTSAMPL
    J = 0, 0
    FOR K = 1 TO 4
    J = 2 ^ (K = 1)
    BCDOUT (1, 1, J)
    TIMER = 0.5
    WAIT TIMER = = 0
    BCDOUT (1, 1, 0)
    NEXT K
END
```

Questo programma emette un numero da 1 a 4 con incrementi di 1 a intervalli di 0,5 secondi.

BREAK

Scopo

Questo comando sospende immediatamente il funzionamento del robot.

Formato

ON <condizione monitoraggio> [{BREAK | PAUSE}] DO <istruzione>

Esempi

ON DIN(l) BREAK DO SUB

Analisi e consigli

Il comando BREAK arresta immediatamente il movimento del robot quando viene soddisfatta la condizione di monitoraggio specificata ed esegue l'istruzione successiva all'istruzione DO. Il robot decelera e arresta il movimento.
Per ulteriori informazioni, fare riferimento al comando ON.

È possibile utilizzare il comando RESUME per riavviare il funzionamento interrotto con il comando BREAK.
Questo comando consente di arrestare il robot e di mettere in atto opportune reazioni in caso di problemi al sistema.

Esempio di programma

```
PROGRAM BREAKSMPL
    REMARK *** MAIN PROGRAM ***
    ON DIN(24) BREAK DO BREAKSUB
    MOVE A1
    MOVE A2
    MOVE A3
    WAIT MOTION >= 100
    IGNORE DIN(24)
END
```

In caso di problemi al sistema e di attivazione (ON) del segnale di ingresso 24, il robot si arresta immediatamente e il controllo passa a BREAKSUB, un sottoprogramma progettato in modo specifico per situazioni simili.

```
PROGRAM BREAKSUB
    REMARK *** SUBROUTINE ***
    WAIT DIN(–24)
    RESUME
END
```

Il sottoprogramma BREAKSUB attende che si disattivi (OFF) il segnale di ingresso 24.

Una volta rimosso il problema, il programma riprende il movimento.

CLOSE1, CLOSE2, CLOSEI1, CLOSEI2

Scopo

Questi comandi chiudono la mano del robot.

Formato

CLOSE1
CLOSE2
CLOSEI1
CLOSEI2

Esempi

CLOSE1
CLOSEI2

Analisi e consigli

Questi comandi vengono utilizzati per chiudere la mano. I numeri 1 e 2 fanno riferimento alla mano 1 e alla mano 2. Questi comandi chiudono la mano modificando lo stato del segnale di uscita che controlla la mano del robot.

Il comando CLOSE dirige il robot a chiudere la mano al termine del movimento in corso.

Il comando CLOSEI dirige il robot a chiudere immediatamente la mano.

Si noti che questi comandi non funzionano se il file SCOL.LIB non è presente nell'unità RAM del controller.

Inoltre, tenere presente che vi è un leggero ritardo tra il momento in cui si esegue il comando CLOSE e quello in cui il robot chiude di fatto la mano.
I comandi corrispondenti OPEN1, OPEN2, OPENI1 e OPENI2 vengono forniti per aprire la mano.

Questi comandi eseguono un programma scritto nella libreria di sistema (SCOL. LIB). I dati di SCOL. LIB devono essere modificati in base alle specifiche della mano del robot.

Esempio di programma

```
PROGRAM CLOSESMPL
    OPENI1
    MOVE A1
    CLOSE1
    DELAY 0.5
    MOVE A2
END
```

Questo programma chiude la mano 1 al termine del movimento del robot verso il punto A1. Il robot attende 0,5 secondi per la chiusura completa della mano una volta eseguito il comando CLOSE.

```
PROGRAM CLOSEISMPL
    ENABLE NOWAIT
    OPENI1
    DELAY 0.5
    MOVE A1
    CLOSET 1
    DELAY 0.5
    MOVE A2
END
```

Qui il robot chiude la mano 1 mentre si sposta verso il punto A1.

COARSE

Scopo

COARSE è una costante di sistema utilizzata per impostare la precisione di posizionamento su approssimativa.

Formato

COARSE

Esempi

ACCUR = COARSE
MOVE A1 WITH ACCUR = COARSE

Analisi e consigli

L'istruzione COARSE imposta la precisione di posizionamento su COARSE.

Come costante di sistema, COARSE ha un valore pari a 0. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 0. Tuttavia, ciò potrebbe rendere il programma estremamente difficile da leggere.

Non è possibile eseguire la sostituzione in costanti di sistema come COARSE.

Per informazioni sulla precisione di posizionamento, fare riferimento al comando ACCUR.

Esempio di programma

```
PROGRAM COARSESMPL
    MOVE A1
    ACCUR = COARSE
    MOVE A2
    MOVE A3
END
```

Questo programma imposta la precisione di posizionamento su approssimativa prima di spostare il robot.

COM0, COM1

Scopo

Questi comandi specificano il canale di comunicazione da utilizzare con un comando PRINT o INPUT.

Formato

PRINT [{COM0 | COM1 | TP},]
{<stringa di caratteri> | <espressione>}][<stringa di caratteri>| <espressione>]]...[, CR]
INPUT [(COM0 | COM1 | TP},]
<variabile> [, <variabile>] ...

Esempi

```
PRINT COM0, "*** INPUT N ***"
PRINT COM1, N, N * 10
INPUT COM1, K
```

Analisi e consigli

Le istruzioni COM vengono utilizzare per designare un canale di comunicazione quando si utilizza un comando PRINT o INPUT.

COM0 è un canale di comunicazione utilizzato esclusivamente dal programmatore portatile.

COM1 corrisponde al canale di comunicazione del connettore del controller COM1.

Se non si specifica un canale di comunicazione per un comando PRINT o INPUT, il controller immette o emette dati al o dal canale di comunicazione utilizzato esclusivamente dal programmatore portatile.

Per l'elaborazione della comunicazione, fare riferimento ai comandi PRINT e INPUT.

Esempio di programma

```
PROGRAM COMSAMPLE
    PRINT COM0, "*** INPUT N ***"
    INPUT COM0.N
    PRINT COM1.N, CR
END
```

Questo programma immette un valore dal programmatore portatile e lo invia al canale di comunicazione 1.

CONFIG

Scopo

Questo comando viene utilizzato per specificare la configurazione del robot.

Formato

CONFIG = <espressione>

Esempi

CONFIG = 1
MOVE A1 WITH CONFIG = RIGHTY

Analisi e consigli

CONFIG è una variabile di sistema utilizzata per esprimere la configurazione del robot. È necessario specificare la configurazione del robot quando le periferiche potrebbero interferire con il movimento del robot.

La configurazione del robot è indefinita a 0, sinistrorsa a 1 e destrorsa a 2. Per specificare la configurazione del sistema, è possibile utilizzare questi numeri o le costanti di sistema FREE, LEFTY e RIGHTY.

Come si può intuire, la configurazione è indefinita a CONFIG = FREE, sinistrorsa a CONFIG = LEFTY e destrorsa a CONFIG = RIGHTY.

La configurazione è inclusa nei dati di posizione trasmessi al robot durante la programmazione. Pertanto, quando CONFIG = FREE, il robot si sposta con la stessa configurazione che aveva al momento della programmazione.

A meno che non vi sia un buon motivo per cambiare, lasciare la configurazione indefinita, ovvero CONFIG = FREE. Questo è il valore iniziale assunto dal robot.

La configurazione del robot potrebbe variare durante l'esecuzione di un comando di movimento.

Quando si esegue l'interpolazione lineare o circolare (con il comando MOVES o MOVEC), la configurazione del robot non può essere modificata e viene generato un errore.

La designazione della configurazione di un robot con coordinate ortogonali viene ignorata.

È possibile utilizzare una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.
Anche se fa riferimento a questa variabile di sistema, la posizione del robot attuale non può essere definita.
È possibile acquisire la posizione del robot attuale con la direzione HERE.
N=HERE.6
Il valore della posizione attuale viene memorizzato in N.
Per i robot tipo SCARA, il valore di HERE.6 è indefinito a 0, sinistrorso a 1 e destrorso a 2. Quando la posizione di partenza del robot non è impostata correttamente, funziona nella posizione in cui si è spostato un robot con la posizione definita durante il cambio di posizione del robot.
Pertanto, quando si definisce la posizione di un robot, il robot si sposta effettivamente alla posizione di lavoro.

Esempio di programma

```
PROGRAM CONFIGSMPL
    CONFIG = RIGHTY
    MOVE A1
    MOVE A2
    MOVE A3 WITH CONFIG = LEFTY
    MOVE A4
END
```

Il robot si sposta con una configurazione sinistrorsa solo quando si sposta verso A3 e con una configurazione destrorsa per altri movimenti.

CONT

Scopo

CONT è una costante di sistema utilizzata per fare riferimento alla modalità operativa del sistema.

Formato

CONT

Esempi

IF MODE < > CONT THEN STOP

Analisi e consigli

CONT viene utilizzato insieme al comando MODE per fare riferimento alla modalità operativa del sistema. Con MODE = = CONT, il sistema funziona in modalità di funzionamento continuo.

Come costante di sistema, CONT ha un valore pari a 0. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 0.
Tuttavia, ciò potrebbe rendere il programma estremamente difficile da leggere.

Non è possibile eseguire la sostituzione in costanti di sistema.

Per informazioni sulle modalità operative, fare riferimento al comando MODE.

Esempio di programma

```
PROGRAM CONTSAMPLE
    IF MODE <>CONT THEN STOP
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Se il sistema cambia dalla modalità di funzionamento continuo, l'esecuzione del programma si arresta e il robot non si sposta.

COS

Scopo

Questa funzione riporta il coseno di un valore inserito.

Formato

COS (<espressione>)

Esempi

K = COS(60)
J1 = 1 – COS(180 – D)

Analisi e consigli

Questa funzione riporta il coseno del valore tra parentesi ( ). I calcoli vengono gestiti in unità di gradi.

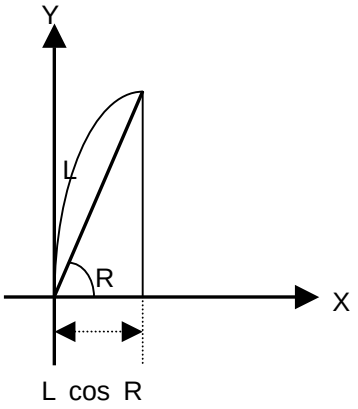
È possibile inserire una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'equazione.

Esempio di programma

```
PROGRAM MAIN
  COSSAMPLE (2, 30, K)
  PRINT TP, X, CR
END
PROGRAM COSSAMPLE (L, R, X)
  LOOP:
    IF R > 180 THEN R = R – 360
    IF R < –180 THEN R = R + 360
    IF R > 180 OR R < –180 THEN GOTO LOOP
    X = L * COS(R)
    RETURN
END
```

Dato (come argomenti) un segmento di linea con una lunghezza L e formando un angolo R con l'asse X, questo programma trova la lunghezza della componente X del segmento di linea e la invia al programma principale come argomento X.



CR

Scopo	Questa funzione emette un codice CR (ritorno a capo) sul canale di comunicazione.
Formato	PRINT [{COM0 | COM1 | TP},] {<stringa di caratteri> | <espressione>} [, {<stringa di caratteri> | <espressione>}] ··· [, CR]
Esempi	PRINT COM1, K, CR PRINT TP, CR
Analisi e consigli	Questa funzione emette un codice CR (ritorno a capo) sul canale di comunicazione. Per il canale di comunicazione, selezionare uno (1) tra COM0, COM1 e TP. COM0 e TP sono canali di comunicazione utilizzati esclusivamente per il programmatore portatile. COM1 corrisponde al canale di comunicazione COM1 del controller. Se non si specifica il canale di comunicazione con il comando PRINT, i dati vengono emessi sul canale di comunicazione esclusivo del programmatore portatile. Quando si specifica CR alla fine del comando PRINT, un codice CR (0DH) viene aggiunto alla parte finale dei dati. Quando i dati vengono emessi su COM0 (TP), vengono visualizzati in una nuova riga.

Esempio di programma

```
PROGRAM COMSAMPLE
    PRINT TP, "*** INPUT N ***", CR
    INPUT TP, N
```

I caratteri vengono visualizzati sul programmatore portatile, quindi vengono spostati su una nuova riga.

```
    PRINT TP, N, CR
END
```

Il valore di "N" viene visualizzato sul programmatore portatile, quindi viene spostato su una nuova riga.

CYCLE

Scopo

CYCLE è una costante di sistema utilizzata per fare riferimento alla modalità operativa del sistema.

Formato

CYCLE

Esempi

IF MODE < > CYCLE THEN GOTO LOOP

Analisi e consigli

CYCLE viene utilizzato insieme al comando MODE per fare riferimento alla modalità operativa del sistema. Con MODE = = CYCLE, il sistema funziona in modalità di funzionamento a cicli.

Come costante di sistema, CYCLE ha un valore pari a 1. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 1. Tuttavia, ciò potrebbe rendere il programma inutilmente difficile da capire.

Non è possibile eseguire la sostituzione in costanti di sistema come CYCLE.

Per informazioni sulle modalità operative, fare riferimento al comando MODE.

Esempio di programma

```
PROGRAM CYCLESMPL
  LOOP:
    MOVE A1
    MOVE A2
    MOVE A3
    IF MODE <> CYCLE THEN GOTO LOOP
END
```

Se il sistema non funziona in modalità di funzionamento a cicli, l'esecuzione del programma continua a tornare all'inizio della sequenza e il robot non cessa di muoversi.

DATA

Scopo	Questa funzione indica l'inizio di un blocco di dati che definisce i dati di posizione e delle coordinate i dati per il punto programmato. Per i dettagli sui blocchi di dati, consultare Para. 2.8.
Formato	DATA
Esempi	DATA
Analisi e consigli	Il blocco di dati viene modificato dall'editor dei dati piuttosto che dall'editor di programma. In caso di errore di formato, per l'editing viene utilizzato l'editor di programma.
Esempio di programma	PROGRAM MAIN MOVE HOME END DATA POINT HOME = 650, 0, 100, 0, 0 /RIGHTY END

DECEL

Scopo

Questo comando imposta la frazione della decelerazione completa del robot.

Formato

DECEL = <espressione>

Esempi

DECEL = 80
DECEL = 0.8 * DECEL
MOVE A1 WITH DECEL = 90

Analisi e consigli

DECEL è una variabile di sistema utilizzata per specificare l'accelerazione del robot quando il robot è in fase di decelerazione. L'accelerazione è espressa come percentuale dell'accelerazione standard (completa).

Nel linguaggio SCOL, l'accelerazione durante l'accelerazione e l'accelerazione durante la decelerazione vengono impostate separatamente. Per impostare l'accelerazione durante l'accelerazione, utilizzare il comando ACCEL.

Ciò viene utilizzare per ridurre l'accelerazione, se necessario, quando il robot trasporta un carico pesante. In tal caso, modificare l'accelerazione sia durante l'accelerazione che durante la decelerazione. Per il valore di impostazione dell'accelerazione in base al carico, consultare il "Manuale di installazione e trasporto".

È possibile utilizzare una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'equazione.

Nel controller viene impostato un limite massimo di decelerazione per proteggere il robot. Il robot non deve superare questo limite anche se si inserisce un valore superiore al limite massimo. Se si inserisce tale valore, il robot funziona al limite massimo.

Valori di 0 o meno vengono considerati 1.

È possibile visualizzare l'accelerazione attuale durante l'accelerazione visualizzando le variabili di sistema.
Il valore iniziale per l'accelerazione è 100%.

Esempio di programma

```
PROGRAM DECELSMPL
  FOR K = 1 TO 100
    DECEL = K
      MOVE A1
      MOVE A2
  NEXT K
END
```

Questo programma aumenta l'accelerazione dall'1% al 100% con incrementi dell'1%.

DELAY

Scopo

Il comando DELAY arresta il movimento del braccio del robot per un determinato periodo di tempo.

Formato

DELAY <tempo>

Esempi

DELAY 0.5
DELAY T * 0.2

Analisi e consigli

Il comando DELAY arresta il movimento del braccio del robot per un determinato periodo di tempo.

La designazione di <tempo> viene specificata in unità di secondi.

Poiché la precisione di esecuzione è limitata, mantenere la designazione del tempo in unità di 0,01 secondi o oltre. È possibile utilizzare una costante, una variabile o un calcolo per la designazione di <tempo>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Quando si effettua l'operazione di arresto del programma mentre il comando DELAY è in fase di esecuzione, una volta trascorso il tempo specificato, il funzionamento automatico viene arrestato. D'altra parte, quando il funzionamento automatico viene annullato con il servocomando OFF o l'operazione di arresto di emergenza mentre il comando DELAY è in fase di esecuzione, il comando DELAY viene eseguito di nuovo quando si riavvia il programma.

DELAY è un comando di controllo del movimento che arresta il movimento del braccio del robot per un determinato periodo di tempo. Tenere presente che il comando DELAY arresta il movimento del robot e non l'esecuzione del programma stesso. Quando si desidera ritardare il programma, utilizzare il comando TIMER o WAIT.

Esempio di programma

```
PROGRAM DELAYSMPL
    MOVE A1
    DELAY 2
    MOVE A2
    DELAY 2
    MOVE A3
    DELAY 2
END
```

Il robot arresta il movimento per 2 secondi al termine di ciascun movimento.

DEST

Scopo

Il comando DEST riporta la destinazione del comando attuale del robot.

Formato

DEST

Esempi

A1 = DEST
X = DEST. X

Analisi e

Il comando DEST viene utilizzato per riferirsi alla destinazione del movimento eseguito in quel momento sul sistema di coordinate universali.

DEST può essere utilizzato come qualsiasi altro dato vettoriale posizionale. Tuttavia, è possibile fare riferimento solo ai valori che contiene e non è possibile modificare i valori stessi.

Se il robot si è arrestato una volta posizionato, DEST riporta tale posizione.

Esempio di programma

```
PROGRAM DESTSAMPLE
    AA = A
    MOVE A
    ON DIN(1) DO AA = DEST
    MOVE A1
    MOVE A2
    MOVE A3
    MOVE A4
    IGNORE DIN(1)
    PRINT "POSITION DATA = ", AA.X, AA.Y, AA.Z
END
```

Questo programma sposta il robot dal punto A1 al punto A4 durante il monitoraggio del segnale di ingresso 1. Se il segnale si attiva (in quel momento) la posizione target viene visualizzata sul programmatore portatile.

DIM …. AS ….

Scopo	Questa funzione definisce una variabile di array.
Formato	DIM <variabile di array> (<numero di elementi>, <numero di elementi>, …) AS <tipo>
Esempi	DIM A (5) AS INT
Analisi e consigli	Questo comando viene utilizzato per definire il tipo e il numero di elementi della variabile di array. La variabile di array può essere definita solo come variabile globale cui si può accedere e che può essere modificata da qualsiasi posizione del programma definito. Il valore di indice dell'array è 1 ~ N. di elementi. Ovvero, in questo esempio, è 1 ~ 5. Se il valore iniziale impostato non rientra nella gamma di indice, si verifica un errore durante l'esecuzione del comando SELECT. Inoltre, l'accesso e la sostituzione al di fuori della gamma di indice provoca un errore durante l'esecuzione del programma. È possibile specificare un totale di cinque (5) tipi: INT (tipo intero), REAL (tipo numero reale), POINT (tipo posizione), TRANS (tipo coordinate) e PAYLOAD (tipo carico). Il valore iniziale di variabili di array di tipo INT e REAL deve essere descritto nel blocco globale, mentre il valore iniziale di variabili di array di tipo POINT, TRANS e PAYLOAD nel blocco di dati.

Esempio di programma

```
GLOBAL
    DIM ICHI (3) AS POINT      Viene dichiarato un array
                               dimensionale di tipo posizione.
END

PROGRAM DIMSAMPLE
    ICHI (1) = P0
    ICHI (2) = POINT (500.0, 0.0, 100.0, 0.0, 0.0, 0)
    ICHI (3) = {500.0, –200.0, 100.0, 0.0, 0.0, 0}
    FOR I = 1 TO 3
       MOVE ICHI (I)
    NEXT I
END
     DATA
     POINT P0 = 500.0, 200.0, 100.0, 0.0, 0.0 / LEFTY
     POINT ICHI (I) = 650.0, 0.0, 100.0, 0.0, 0.0 / LEFTY
END
```

DIN

Scopo

Il comando DIN legge lo stato (ON o OFF) di un segnale (o più segnali) di ingresso.

Formato

DIN (<nome segnale> [,<nome segnale>]...)

Esempi

IF DIN(1) THEN GOTO LOOP
WAIT DIN (1, –2, 3)
ON DIN (J, J + 1, J + 2) DO RETURN

Analisi e consigli

Il comando DIN legge lo stato (ON o OFF) di un simbolo di ingresso. DIN viene utilizzato unitamente ad un comando IF, WAIT o ON per valutare i segnali esterni.

<Nome segnale> specifica il numero di un segnale da leggere nel controller. Un segnale positivo è considerato ON, mentre uno negativo OFF. È possibile specificare fino a 10 nomi di segnale (i nomi di segnale superiori a 10 vengono ignorati).

Quando lo stato di tutti i segnali diventa come specificato, DIN riporta un valore di TRUE (1). Se uno dei segnali non è come specificato, DIN riporta un valore di FALSE (0).

È possibile utilizzare una costante, una variabile o un calcolo per la designazione di <nome segnale>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Esempio di programma

```
PROGRAM DINSAMPLE
    WAIT DIN(1)
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Il robot attende che si attivi il segnale di ingresso 1 prima di iniziare a muoversi.

DISABLE

Scopo

Il comando DISABLE viene utilizzato per disabilitare gli interruttori di sistema.

Formato

DISABLE <interruttore> [, <interruttore>]...

Esempi

DISABLE PASS
DISABLE PASS, NOWAIT

Analisi e consigli

Il comando DISABLE viene utilizzato per disabilitare gli interruttori di sistema correlati al movimento del robot. Vi sono tre interruttori di sistema.

(1) PASS

PASS viene utilizzato per specificare uno spostamento rapido. Lo spostamento rapido è una modalità operativa in cui si comanda al robot di iniziare il movimento successivo prima di completare il posizionamento del movimento precedente. I tempi di commutazione dal movimento attuale a quello successivo vengono specificati con il comando PASS della variabile di sistema.

Lo spostamento rapido consente di ridurre i tempi necessari al robot per spostarsi da un posto all'altro. Per ulteriori informazioni, consultare la Sezione 5.

Un'istruzione DISABLE PASS annulla lo spostamento rapido. L'impostazione iniziale per il controller è DISABLE PASS.

(2) NOWAIT

NOWAIT specifica se il controller deve attendere che il robot termini di posizionarsi prima di inviare (o ricevere) segnali esterni. I tempi di emissione del segnale sono illustrati nella Sezione 5.

L'istruzione DISABLE NOWAIT comanda al controller di attendere che il robot termini di posizionarsi prima di inviare (o ricevere) un segnale esterno. L'impostazione iniziale per il controller è DISABLE NOWAIT.

(3) SWITCH

SWITCH determina se eseguire o meno il cambio attività nella gestione multitask.
Il cambio attività è proibito da DISABLE SWITCH.
Nell'impostazione iniziale, ENABLE SWITCH è operativo.

(4) MOVESYNC

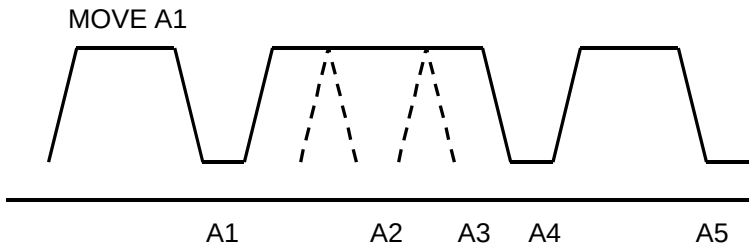
Specifica le modalità sincrona e asincrona del comando di movimento. Nello stato DISABLE MOVESYNC (ovvero, modalità asincrona del comando di movimento), il sistema esegue in anticipo fino ai successivi quattro (4) (max.) comandi e attende che finisca il posizionamento. Se la variabile di sistema PASS è impostata su “ENABLE”, il movimento rapido (passaggio) è consentito. Lo stato iniziale è specificato dal parametro dell'utente [U03].

Per rendere operativo l'interruttore di sistema, utilizzare il comando ENABLE.

Esempio di programma

```
PROGRAM DISABLESPL
    MOVE A1
    PASS = 80
    ENABLE PASS
    MOVE A2
    MOVE A3
    DISABLE PASS
    MOVE A4
    MOVE A5
END
```

Il robot si sposta dal punto A1 al punto A4 con un movimento rapido. Dal punto A4 in poi, il robot si sposta normalmente.



DO

Scopo

In comando DO viene utilizzato unitamente al comando ON per monitorare le condizioni.

Formato

ON <condizione monitoraggio> [{BREAK | PAUSE}] DO <istruzione>

Esempi

ON DIN (1) DO RETURN
ON TIMER DO MOVE A1

Analisi e consigli

Se si soddisfa la <condizione monitoraggio>, viene eseguita l'istruzione successiva al comando DO.
Il monitoraggio della condizione viene eseguito indipendentemente dal movimento eseguito dal robot in quel momento.

Il comando ON viene eseguito parallelamente ai comandi di movimento del robot. Se si utilizza un comando MOTION, MOTIONT, REMAIN o REMAINT come condizione di monitoraggio, viene eseguito il monitoraggio delle condizioni dei successivi comandi di movimento. Se si utilizza TIMER o ERROR come condizione di monitoraggio, le condizioni verranno monitorate indipendentemente dal movimento del robot.

Quando si esegue il monitoraggio di segnali di ingresso con DIN o comandi simili, il tempo di inizio del monitoraggio varia in base all'impostazione dell'interruttore di sistema NOWAIT. Quando è operativa un'istruzione ENABLE NOWAIT, i segnali vengono monitorati indipendentemente dal movimento del robot.

Quando è operativa un'istruzione DISABLE NOWAIT, il monitoraggio del segnale inizia una volta che il robot termina il movimento in corso in quel momento.

L'esecuzione dell'istruzione successiva al comando DO inizia subito dopo l'esecuzione del comando operativo nel momento in cui si è soddisfatta la condizione di monitoraggio. Tuttavia, se si è eseguito un comando WAIT, tale comando WAIT viene immediatamente annullato e il controllo del programma passa all'istruzione successiva al comando DO.

Vi sono tre tipi di tempi di esecuzione specificabili per il robot mentre è in funzione:

BREAK: BREAK arresta immediatamente tutti i movimenti del robot e il controllo passa all'istruzione successiva al comando DO.

PAUSE: L'istruzione successiva al comando DO viene eseguita una volta che termina il movimento in corso in quel momento. Tuttavia, durante il movimento del braccio, l'esecuzione del programma normale prosegue, ad eccezione dei comandi di richiamo del sottoprogramma, di ritorno al programma principale e di movimento. Durante l'esecuzione di questi comandi, l'esecuzione del programma si arresta finché non si arresta il braccio.

Predefiniti: L'impostazione predefinita causa il completamento del movimento in corso mentre si eseguono contemporaneamente le istruzioni successive al comando DO.

Quando l'istruzione successiva al comando DO è un comando di movimento, includere sempre un'istruzione BREAK o PAUSE nella riga di comando DO.

Se si eseguono l'istruzione successiva al comando DO (ovvero, istruzione DO) e il comando di movimento nell'istruzione DO, al termine del movimento del braccio l'esecuzione del programma si riavvia in base alle condizioni precedenti alla condizione per la quale si è soddisfatto il comando ON.

(Se si è interrotto un comando WAIT, l'esecuzione del programma si riavvia dalla posizione in cui si è interrotto tale comando WAIT.) Tuttavia, se si è eseguito un ramo del programma su un'etichetta con l'istruzione successiva al comando DO, l'esecuzione si avvia dall'istruzione recante quella etichetta.

È possibile monitorare contemporaneamente dieci set di condizioni. Inoltre, è possibile specificare un massimo di quattro segnali di ingresso con un singolo comando ON.

Quando diverse condizioni di monitoraggio diventano vere contemporaneamente, viene eseguita l'istruzione DO corrispondente al comando ON con la massima priorità. Questa priorità è determinata dall'ordine in cui i comandi ON si trovano nel programma. Pertanto, il primo comando ON ha la massima priorità. Le istruzioni DO corrispondenti a tutti gli altri comandi ON vengono ignorate.

Il monitoraggio di una condizione specificata da un comando ON viene annullata se l'esecuzione passa ad un'istruzione DO corrispondente ad un altro comando ON. Inoltre, le condizioni non vengono monitorate quando si arresta l'esecuzione del programma a causa di un comando STOP o di un errore.

Quando il timer di sistema viene specificato come condizione di monitoraggio, tale condizione viene verificata solo quando si modificano le condizioni di monitoraggio. Durante il monitoraggio di un segnale esterno, di una condizione di errore o di un comando di riferimento movimento (ad esempio, la quantità di movimento restante da eseguire), il controller verifica lo stato, non la variazione, di quel segnale.

Il comando IGNORE annulla il monitoraggio delle condizioni specificate da un comando ON. Il monitoraggio delle condizioni si arresta inoltre quando si soddisfa una condizione e si esegue un'istruzione successiva ad un comando DO.

[Nota 1]

Al momento, i comandi ON e DO possono essere combinati solo nei modi illustrati di seguito:

ON TIMER DO <istruzione>
Quando il timer diventa 0, eseguire l'istruzione.

ON DIN ( ) DO <istruzione>
Quando lo stato dei segnali di ingresso tra parentesi ( ) diventa come specificato, eseguire l'istruzione. Non è possibile monitorare più di quattro segnali contemporaneamente con una simile istruzione. È possibile specificare fino a quattro segnali di ingresso. I segnali di ingresso superiori a quattro segnali vengono ignorati.

ON MOTION > = <espressione> DO <istruzione>
Eseguire l'istruzione quando la quantità di un movimento da eseguire successivamente a questo comando è superiore al valore specificato. L'unico operando relazionale utilizzabile con MOTION è > =.

ON MOTIONT > = <espressione> DO <istruzione>
Eseguire l'istruzione quando il tempo necessario per un movimento da eseguire successivamente a questo comando è superiore al tempo specificato. L'unico operando relazionale utilizzabile con MOTIONT è > =.

ON REMAIN < = <espressione> DO <istruzione>
Eseguire l'istruzione quando la quantità restante di un movimento da eseguire successivamente a questo comando è inferiore al valore specificato.

L'unico operando relazionale utilizzabile con REMAIN è < =.

ON REMAINT < = <espressione> DO <istruzione>
Eseguire l'istruzione quando il tempo restante necessario per un movimento da eseguire successivamente a questo comando è inferiore al tempo specificato.

L'unico operando relazionale utilizzabile con REMAINT è < =.

[Nota 2]

In un'istruzione successiva all'istruzione DO, non è possibile utilizzare i seguenti comandi correlati al controllo dell'attività.

TASK, KILL, SWITCH

Se tali comandi vengono utilizzati nell'istruzione DO e successivamente, essi non sono operativi. Non è possibile eseguire il monitoraggio della condizione con il comando ON nell'attività secondaria.

[Nota 3]

Se si è arrestato un movimento monitorato nella condizione ON MOTION, ON MOTIONT, ON REMAIN o ON REMAINT o se si è specificato il comando di velocità ridotta durante l'esecuzione di un movimento monitorato, la condizione ON viene annullata.

Esempio di programma

```
PROGRAM MAIN
    DOSAMPLE
    MOVE P
END
PROGRAM DOSAMPLE
    ON DIN(1) PAUSE DO RETURN
    MOVE A1
    MOVE A2
    MOVE A3
      WAIT MOTION >= 100
    IGNORE DIN(1)
    RETURN
END
```

Se il segnale 1 si attiva (ON) mentre si esegue un movimento, il controllo viene reinviato al programma principale al termine di quel movimento.

Precauzioni sulle istruzioni DO:

Per il comando ON ~ DO, vengono registrate le condizioni ON da monitorare e l'istruzione DO che inizia quando sono soddisfatte le condizioni.

```
PRGRAM MAIN
  SIG = 1
  ON DIN (1) DO INPUT SIG
  SUB
  IGNORE DIN(1)
  PRINT SIG
END

PROGRAM SUB
  MOVE P
  WAIT MOTION >= 100
END
```

Nel programma SCOL di cui sopra, se DIN(1) è impostato su ON durante la poligonale su P, non è possibile eseguire l'istruzione DO in quanto la variabile SIG non è definita nel programma SUB e non vi è spazio per il salvataggio della variabile come ingresso con il comando INPUT. In tal caso, la corrispondente istruzione DO può essere eseguita normalmente definendo la variabile SIG come variabile globale.

```
GLOBAL
  SIG = 0
END
PROGRAM MAIN
  SIG = 1
  ON DIN(1) DO INPUT SIG
  SUB
  IGNORE DIN(1)
  PRINT SIG
END
PROGRAM SUB
  MOVE P
  WAIT MOTION >= 100
END
```

Nell'istruzione DO, anche se si stabiliscono le condizioni di cambio attività o si esegue il comando SWITCH, non è possibile cambiare attività. Se si esegue il comando TASK o il comando KILL, si verifica un errore.

DOUT

Scopo

DOUT viene utilizzato per emettere segnali esterni.

Formato

DOUT (<nome segnale> [,<nome segnale>]...)

Esempi

DOUT (1, 2, –3)
DOUT (J, J + 1, J + 2)

Analisi e consigli

DOUT viene utilizzato per emettere segnali esterni.

<Nome segnale> specifica il numero (nome) di un segnale da emettere dal controller.

Un segnale positivo è considerato ON, mentre uno negativo OFF. È possibile specificare fino a dieci nomi di segnale in un comando DOUT.

È possibile utilizzare una costante, una variabile o un calcolo per la designazione di <nome segnale>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Quando lo stesso segnale viene emesso consecutivamente dopo l'esecuzione del comando DO, non si garantisce l'emissione del segnale. Quando si specifica l'emissione di più segnali, non si garantisce la simultaneità dei tempi di emissione del segnale.

DOUT(-4,3,-2,1) Programma del robot in esecuzione dall'intestazione DOUT(-4)
DOUT(3), DOUT(-2), DOUT(1) Risoluzione ed esecuzione con il programma del robot
In altre parole, cambia e il segnale diventa un valore per lo scopo con "1010"→"0010"→"0011"→"0001"→"0101" al numero - il grado di intervallo di centinaia di ms, che termina quando si esegue il programma del robot denominato DOUT(-4,3,-2,1) dietro DOUT(4,-3,2,-1).

Esempio di programma

```
PROGRAM DOUTSAMPLE
    FOR K = 1 TO 16
    DOUT(K)
    TIMER = 0.5
    WAIT TIMER == 0
    DOUT(–K)
    NEXT K
END
```

Questo programma invia (avvia) i segnali di uscita nell'ordine da 1 a 16 e in intervalli di 0,5 secondi.

ELSE

Scopo

L'istruzione ELSE viene utilizzata unitamente alle strutture IF ~ THEN per valutare le condizioni.

Formato

IF <espressione logica> THEN <istruzione> [ELSE <istruzione>]

Esempi

IF DIN (1) THEN K = K + 1 ELSE K = 0

Analisi e consigli

ELSE viene utilizzato in un'istruzione IF per specificare un'istruzione da eseguire se no si soddisfa la condizione IF.

Un'istruzione ELSE non è obbligatoria in una struttura IF. Se non si soddisfa una condizione IF e non vi è alcuna istruzione ELSE, l'esecuzione del programma passa al prossimo comando successivo al comando IF.

L'<istruzione> successiva alle istruzioni THEN e ELSE non può contenere PROGRAM, END, IF, FOR, NEXT o WAIT. Per ulteriori informazioni sulla valutazione delle condizioni, fare riferimento al comando IF.

Esempio di programma

```
PROGRAM ELSESAMPLE
    IF DIN (1) THEN SPEED = 100 ELSE SPEED = 50
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Se il segnale di ingresso 1 è ON, il robot agisce alla massima velocità (100%). Se è OFF, il robot agisce a mezza velocità (50%).

ENABLE

Scopo

Il comando ENABLE viene utilizzato per abilitare gli interruttori di sistema.

Formato

ENABLE <interruttore> [, <interruttore>]...

Esempi

ENABLE PASS
ENABLE PASS, NOWAIT

Analisi e consigli

Il comando ENABLE viene utilizzato per abilitare gli interruttori di sistema correlati al movimento del robot. Vi sono quattro (4) interruttori di sistema.

(1) PASS

PASS viene utilizzato per specificare uno spostamento rapido. Lo spostamento rapido è una modalità operativa in cui si comanda al robot di iniziare il movimento successivo prima di completare il movimento precedente. I tempi di commutazione dal movimento attuale a quello successivo vengono specificati con il comando PASS.

Lo spostamento rapido consente di ridurre i tempi necessari al robot per spostarsi da un posto all'altro. Per ulteriori informazioni, consultare la Sezione 5.

Una istruzione ENABLE PASS specifica uno spostamento rapido. L'impostazione iniziale per il controller è DISABLE PASS.

(2) NOWAIT

NOWAIT specifica se il controller deve attendere che il robot termini di posizionarsi prima di inviare (o ricevere) segnali esterni. I tempi di emissione del segnale sono illustrati nella Sezione 5.

L'istruzione ENABLE NOWAIT comanda al controller di inviare (o ricevere) segnali esterni senza attendere che il robot termini di posizionarsi. L'impostazione iniziale per il controller è DISABLE NOWAIT.

(3) SWITCH

SWITCH determina se eseguire o meno il cambio attività nella gestione multitask.
Il cambio attività è proibito da DISABLE SWITCH.
Nell'impostazione iniziale, ENABLE SWITCH è operativo.

(4) MOVESYNC

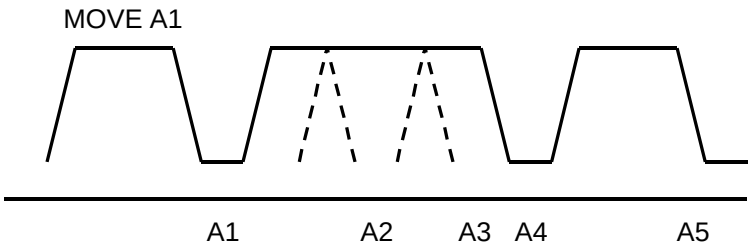
Specifica le modalità sincrona e asincrona del comando di movimento.
Nello stato ENABLE MOVESYNC (ovvero, modalità sincrona del comando di movimento), il sistema esegue in anticipo fino al successivo comando e attende che finisca il posizionamento. Lo stato iniziale è specificato dal parametro dell'utente [U03].
Nella modalità MOVESYNC, il movimento rapido (passaggio) non è consentito indipendentemente dallo stato della variabile di sistema PASS. Se la variabile di sistema PASS è impostata su “DISABLE” nel programma SCOL, il movimento rapido (passaggio) è consentito.

Per disabilitare l'interruttore di sistema, utilizzare il comando DISABLE.

Esempio di programma

```
PROGRAM ENABLESMPL
    MOVE A1
    PASS = 80
    ENABLE PASS
    MOVE A2
    MOVE A3
    DISABLE PASS
    MOVE A4
    MOVE A5
END
```

Questo programma comanda al robot di spostarsi dal punto A1 al punto A4 con un movimento rapido e dal punto A3 al punto A5 senza movimento rapido.



END

Scopo	L'istruzione END contrassegna la fine di un programma.
Formato	END
Esempi	END
Analisi e consigli	L'istruzione END contrassegna la fine di un programma. Quando funziona in modalità di funzionamento a cicli, il programma si arresta con l'istruzione END dell'attività principale generata automaticamente all'inizio del programma. In modalità di funzionamento continuo, l'esecuzione del programma viene riportata all'inizio del programma e il programma si ripete. Quando si esegue un programma come sottoprogramma, è necessario disporre di un comando RETURN nella riga sopra l'istruzione END. Anche se si dimentica di inserire un comando RETURN, il controllo viene reinviato al programma principale quando si trova l'istruzione END. PROGRAM contrassegna l'inizio di un programma, mentre END la fine. Il programma in sé è inserito tra le due istruzioni. Se non si dispone di un'istruzione END, viene visualizzato un messaggio di errore. Dopo l'esecuzione dell'istruzione END nel programma principale, i valori delle variabili interne vengono cancellate.

Esempio di programma

```
PROGRAM ENDSAMPLE
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Da PROGRAM a END, questo programma viene eseguito come singolo programma.

EXP

Scopo

La funzione EXP riporta l'elevamento a potenza e di un numero.

Formato

EXP (<espressione>)

Esempi

K = EXP (3.5)
J1 = N * EXP (L – K)

Analisi e consigli

Questa funzione viene utilizzata per calcolare l'elevamento a potenza e di un'<espressione> (e = 2,71828...)

È possibile utilizzare una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale. Questo comando deve essere utilizzato in un'equazione.

Esempio di programma

```
PROGRAM MAIN
    EXPSAMPLE (5, K)
    PRINT TP, K, CR
END
PROGRAM EXPSAMPLE (N, K)
    K = EXP (N)
    RETURN
END
```

Questo sottoprogramma prende un argomento N, trova l'elevamento a potenza e di quel dato argomento, chiama il risultato K e reinvia il controllo al programma principale.

FINE

Scopo

FINE è una costante di sistema utilizzata per impostare la precisione di posizionamento su ottimale.

Formato

FINE

Esempi

ACCUR = FINE
MOVE A1 WITH ACCUR = FINE

Analisi e consigli

L'istruzione FINE viene utilizzata con il comando ACCUR per impostare la precisione di posizionamento su FINE. La costante di sistema ha un valore pari a 1. Tale valore può essere utilizzato come costante 1 nell'espressione. Tuttavia, non utilizzarlo in quanto rende il programma inutilmente complicato.

Non è possibile eseguire la sostituzione in costanti di sistema. Per informazioni sulla precisione di posizionamento, fare riferimento al comando ACCUR.

Esempio di programma

```
PROGRAM FINESAMPLE
    ACCUR = FINE
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Questo programma imposta la precisione di posizionamento su FINE prima di spostare il robot.

FOR

Scopo

FOR comanda la ripetizione di una parte del programma.

Formato

FOR <variabile> = <espressione 1> TO <espressione 2> [STEP <espressione 3>)

• • • •

NEXT [<variabile>)

Esempi

FOR K = 1 TO 4

• • • •

NEXT K

FOR N = K1 TO K1 + K2 STEP K3

• • • •

NEXT N

Analisi e consigli

FOR comanda la ripetizione di una parte del programma.
Il blocco di programma tra il comando FOR e il comando NEXT viene eseguito ripetutamente. Il blocco continua a ripetersi finché non si soddisfa la condizione specificata per l'istruzione FOR.

Quando si esegue un'istruzione FOR, il valore di <espressione 1> viene sostituito nella <variabile>. Quando si esegue l'istruzione NEXT, il valore di <espressione 3> specificato dall'istruzione STEP viene aggiunto nella <variabile>.

Se a questo punto il valore della <variabile> diventa superiore a quello dell'<espressione 2>, l'esecuzione del programma passa all'istruzione successiva al comando NEXT. Se <variabile> non è superiore ad <espressione 2>, l'esecuzione del programma si ramifica (ritorna) all'istruzione successiva all'istruzione FOR.

I valori di <espressione 1>, <espressione 2> ed <espressione 3> utilizzati per la struttura FOR sono quelli operativi al momento della prima esecuzione dell'istruzione FOR. Pertanto, anche se si modificano questi valori mentre si esegue la sequenza, il numero di volte in cui si ripete la sequenza non cambia.

La <variabile> deve essere utilizzata esclusivamente per controllare il numero di volte in cui si ripete la sequenza. Pertanto, non modificare il valore della <variabile> mentre si esegue la sequenza.

Se il valore di <espressione 3> è pari a 1, si possono omettere le istruzioni successive all'istruzione STEP.

È possibile utilizzare una costante, una variabile o un calcolo per <espressione 1>, <espressione 2> o <espressione 3>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Per la corrispondente istruzione NEXT, è necessario specificare la variabile specificata dall'istruzione FOR.

Se non si specifica <variabile> nell'istruzione NEXT, viene eseguita una sequenza tra la più vicina istruzione FOR (eseguita alla fine) e l'istruzione NEXT.

Quando si utilizza un altro comando FOR (annidato) per una sequenza FOR – NEXT, il numero di livelli annidati deve essere 32 o meno.
Quando il numero di livelli annidati è superiore a 32, si verifica un errore.

Nota 1: Una sequenza FOR termina con il comando NEXT. Pertanto, qualunque cosa accada, la sequenza viene eseguita almeno una volta.

Nota 2: È possibile utilizzare numeri reali per <variabile>, <espressione 1>, <espressione 2> o <espressione 3>. Tuttavia, poiché non vi è una certa imprecisione nella gestione di numeri reali, utilizzare numeri interi quando si comanda il numero di ripetizioni all'istruzione FOR. Inoltre, i valori sostituiti in <variabile>, <espressione 1>, <espressione 2> o (espressione 3> vengono convertiti in tipi i dati quando si esegue la sequenza. I tipi di dati vengono convertiti come illustrato di seguito.

(1) Se il tipo di dati <variabile> è indefinito quando si esegue un'istruzione FOR:
Il tipo di dati di una variabile è indefinito se l'identificatore utilizzato per la <variabile> viene visualizzato per la prima volta nel programma nel comando FOR. In tal caso, tutti i dati vengono convertiti ed elaborati come tipo di dati di <espressione 1>.

(2) Se il tipo di dati <variabile> è stato definito prima di eseguire un'istruzione FOR:
Se l'identificatore (utilizzato per la variabile) è stato utilizzato in precedenza nel programma, il tipo di dati della <variabile> sarà uguale a quello dei dati inseriti precedentemente in quella <variabile>.

Esempio:

```
PROGRAM SAMPLE
    J1 = 5
    FOR J1 = 0.1 TO 0.9 STEP 0.01
        MOVE A1
        MOVE A2
    NEXT J1
END
```

Quando si esegue il programma di cui sopra, il tipo di dati di J1 è definito come tipo intero prima di eseguire il comando FOR. Pertanto, tutte le variabili utilizzate per il comando FOR vengono convertite in tipo intero. Quindi, il controller interpreta l'istruzione FOR precedente come:
FOR J1 = 0 TO 0 STEP 0
Di conseguenza, la sequenza FOR viene eseguita una sola volta.

Nota 3: Quando l'istruzione FOR viene utilizzata nel modo seguente, si verifica un errore.

(1)
```
FOR J1 = ...
FOR J2 = ...
 ...
NEXT J1
```

Qui, manca un'istruzione NEXT corrispondente all'istruzione FOR J2.

(2)
```
FOR K = ...
...
IF DIN(1) THEN GOTO L1
...
NEXT K
...
L1:
```

Non è consentito un comando ramo dall'interno all'esterno della sequenza o viceversa.

Esempio di programma

```
PROGRAM FORSAMPLE
    FOR K = 1 TO 100
        MOVE A1
        MOVE A2
    NEXT K
END
```

Il robot esegue 100 spostamenti avanti e indietro tra A1 e A2.

FREE

Scopo

FREE è una costante di sistema utilizzata per rendere la configurazione del robot indefinita.

Formato

FREE

Esempi

CONFIG = FREE
MOVE A1 WITH CONFIG = FREE

Analisi e consigli

La costante di sistema FREE viene utilizzata unitamente al comando CONFIG per specificare la configurazione del robot come indefinita.

FREE ha un valore pari a 0. Se lo si desidera, si può utilizzare FREE nel programma come costante con valore pari a 0. Tuttavia, non utilizzarlo in quanto rende il programma difficile da leggere.

Non è possibile eseguire la modifica (sostituzione) in costanti di sistema, compreso FREE.
Per informazioni sulla configurazione del robot, vedere il "comando CONFIG".

Esempio di programma

```
PROGRAM FREESAMPLE
    CONFIG = FREE
    MOVE A1
    MOVE A2
END
```

Questo programma rende la configurazione del robot indefinita prima di iniziare a spostare il robot.

FREELOAD

Scopo	Il comando FREELOAD azzera i dati di carico che agiscono sulla fine della mano del robot.
Formato	FREELOAD
Esempi	FREELOAD
Analisi e consigli	Il comando FREELOAD azzera i dati di carico che agiscono sulla fine della mano del robot. Affinché il robot funzioni in modo efficace con vari carichi, il linguaggio SCOL consente di impostare i dati di carico che agiscono sulla fine della mano del robot. È importante che il robot conosca queste informazioni in modo che non si danneggi oscillando in modo eccessivamente rapido. I carichi che agiscono sulla mano del robot vengono impostati con la variabile di sistema PAYLOAD. Il controller utilizza questi valori per calcolare le costanti di comando per l'accelerazione e la decelerazione del robot adeguate al carico. I dati di carico sono costituiti dai valori relativi al peso e al momento di inerzia del carico. Quando si esegue un comando FREELOAD, tutti i dati di carico diventano 0. Ciò è valido per i dati di carico relativi al peso e all'inerzia. Questo comando non funziona se non si dispone del file SCOL.LIB nell'unità RAM del controller.

Esempio di programma

```
PROGRAM FREELOADSL
    PAYLOAD=HAND
    MOVE A1
    CLOSE1
    DELAY 0.5
    MOVE A2 WITH PAYLOAD=HAND + MOTOR
    OPEN1
    DELAY 0.5
    MOVE A3
    FREELOAD
    MOVE A2
    MOVE A3
END
```

Questo programma azzera i dati di carico prima di spostare il robot.

GAIN

Scopo

Il comando GAIN viene utilizzato per specificare se il guadagno (per il servocomando) è ON o OFF per ciascun asse.

Formato

GAIN = {<espressione>, <espressione>, <espressione>, <espressione>, <espressione>}

Esempi

GAIN = {0, 0, 1, 0, 0}
MOVE A1 WITH GAIN = {,, ON}

Analisi e consigli

Il comando GAIN viene utilizzato per specificare se il guadagno (per il servocomando) di ciascun asse deve essere ON o OFF.

Se GAIN viene specificato come OFF, il servocomando di quel determinato asse si arresta la prossima volta che si esegue un comando di movimento. Gli assi dei quali si è arrestato il servocomando sono in stato di servocomando libero (in cui non viene eseguito il controllo del posizionamento) e non possono essere spostati liberamente da forze esterne.

Non vengono eseguiti controlli di posizionamento per gli assi il cui guadagno è OFF. Inoltre, se si esegue un comando di movimento per uno di questi assi, l'asse non si sposta.

Questo comando viene utilizzato, ad esempio, quando si sta montando una parte in un foro di un pezzo di lavoro.

Disattivando tutti i guadagni ad eccezione di quello per l'asse Z (che deve essere attivo per poter muovere la mano su e giù), tutti gli assi (ad eccezione dell'asse Z) sono liberi (ovvero non bloccati) e la mano del robot può muoversi liberamente in direzione orizzontale. Ciò consente al robot di muoversi orizzontalmente insieme al pezzo di lavoro e di montare la parte nel foro, anche se il pezzo di lavoro e la parte non sono allineati in modo preciso.

Il comando GAIN viene considerato come una variabile di sistema dotata di valori fissi (che corrispondono ai cinque assi). Il programmatore specifica quali guadagni devono essere attivati e disattivati per ciascun asse all'interno delle parentesi { } che seguono i comandi GAIN.

Le specifiche sono separate da virgole con la prima specifica corrispondente all'asse 1, la seconda all'asse 2, e così via. "0" indica che il guadagno deve essere OFF e "1" che il segnale deve essere ON. Se le specifiche sono abbreviate (ad esempio, {,, ON}), il controller considera OFF tutti i guadagni non specificati.

È possibile utilizzare costanti, variabili o calcoli per i termini (espressione>. È inoltre possibile utilizzare le costanti di sistema ON e OFF (ON imposta l'attivazione del guadagno, mentre OFF la disattivazione). Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

È sempre necessario sincronizzare l'esecuzione di un comando GAIN con il movimento del robot. Ovvero, assicurarsi che il comando GAIN venga eseguito al termine del precedente comando di movimento (come illustrato nell'esempio di programma).

Se si esegue un comando GAIN quando il robot è ancora in movimento, il robot potrebbe spostarsi in modo incorretto. Per ulteriori informazioni su come impostare i guadagni in sincronia con il movimento del robot, vedere il "comando SETGAIN".

Se si completa il funzionamento automatico e si commuta alla modalità manuale, le impostazioni GAIN non cambiano. Per attivare i guadagni disattivati in modalità manuale, premere i corrispondenti tasti guida di quei determinati assi mentre si è in modalità di guida.

Quando un valore di guadagno selezionato non è superiore a 0 per ON e non è inferiore a 1 per OFF, 0 e 1 vengono considerati da specificare.

Esempio di programma

```
PROGRAM GAINSAMPLE
    MOVE A1
    WAIT MOTION > = 100
    GAIN = {OFF, OFF, ON, OFF, OFF}
    MOVE A2
    OPEN1
    DELAY 0.5
    MOVE A1
    WAIT MOTION > = 100
    GAIN = {ON, ON, ON, ON, ON}
    READY
END
```

Questo programma disattiva tutti i guadagni ad eccezione di quello per l'asse Z (asse 3) prima che il robot si sposti sul punto A2.

```
PROGRAM GMOVE
    G = GAIN
    GAIN = {,, ON}
    MOVE P1
    WAIT MOTION > = 100
    GAIN = G
    MOVE P2
END
```

Questo programma disattiva i guadagni su tutti gli assi ad eccezione dell'asse 3 (asse Z) e, successivamente, ripristina lo stato originale dei guadagni dopo il movimento.

GLOBAL

Scopo

Questa funzione specifica l'area della variabile globale.

Formato

GLOBAL

Esempi

```
GLOBAL
   A = 10
END
```

Analisi e consigli

Viene definita la variabile globale cui è possibile riferirsi e che può essere sostituita da qualsiasi parte del programma. Per identificare il tipo di una variabile diversa dall'array, è necessaria l'impostazione di un valore iniziale nel formato di sostituzione.

Poiché la variabile di array definisce il tipo e il numero di elementi con il comando DIM e imposta il valore iniziale separatamente, alcune potrebbero non avere un valore iniziale.

È possibile utilizzare i seguenti tipi per l'area della variabile globale.

Tipo intero, tipo numero reale, tipo posizione, tipo coordinate e tipo carico.

Combinazione delle precedenti variabili con l'array.

Specificare la definizione della variabile di tipo intero e di tipo numero reale ed i valori iniziali degli array di tipo intero e di tipo numero reale in blocchi globali, mentre la definizione della variabile di tipo posizione, di tipo coordinate e di tipo carico ed i valori iniziali degli array di tipo posizione, di tipo coordinate e di tipo carico in blocchi di dati.

Esempio di programma

```
GLOBAL
    A = 1
END

PROGRAM TEST
    A = A + 1
    PRINT “A=”, A, CR
END
```

GOTO

Scopo

Il comando GOTO indica che l'esecuzione del programma deve essere ramificata sulla posizione contrassegnata dall'etichetta del comando GOTO.

Formato

GOTO <etichetta>

Esempi

IF DIN(1) THEN GOTO L1
GOTO LOOP
GOTO RESTART

Analisi e consigli

Il comando GOTO indica che l'esecuzione del programma deve essere ramificata sulla posizione contrassegnata dall'etichetta del comando GOTO.

Le posizioni di ramificazione per il comando GOTO sono limitate alle istruzioni nello stesso programma. Se non vi è alcuna posizione nel programma corrispondente all'etichetta GOTO, si verifica un errore. Inoltre, se vi sono varie istruzioni con la stessa etichetta nello stesso programma, il controller non sa dove andare e si verifica un errore.

Per etichettare una posizione di ramificazione, inserire il nome dell'etichetta (identificatore) all'inizio delle istruzioni da eseguire. Assicurarsi di aggiungere i due punti al termine dell'identificatore.

Esempio di programma

```
PROGRAM GOTOSAMPLE
    MOVE A1
    LOOP:
        MOVE A2
        MOVE A3
    GOTO LOOP
END
```

Quando il programma viene eseguito alla fine, esso torna all'etichetta LOOP.

GOTO ( )

Scopo

Questo comando provoca l'esecuzione del robot da ramificare in base al valore dell'espressione tra parentesi.

Formato

GOTO (<espressione>) <etichetta> [, <etichetta>]….

Esempi

GOTO (K) LABEL1, LABEL2, LABEL3
GOTO (N1 – N2) L1, L2, L3, L4, L5

Analisi e consigli

Il comando GOTO ( ) provoca l'esecuzione del programma da ramificare in base al valore dell'espressione tra parentesi ( ).

Quando il valore tra parentesi (ovvero, l'<espressione>) è 1, il programma si ramifica sull'<etichetta> più a sinistra. Quando il valore tra parentesi è 2, il programma si ramifica sulla seconda <etichetta> più a sinistra, e così via. È possibile specificare fino a 10 etichette. Le etichette superiori a 10 vengono ignorate.

Se il valore tra parentesi è superiore al numero di etichette oppure se il valore è zero o meno, l'esecuzione del programma procede all'istruzione successiva all'istruzione GOTO. Se il valore tra parentesi è un numero reale, tutti i punti decimali vengono eliminati e ciò che rimane viene considerato un numero intero.

È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Le posizioni di ramificazione per il comando GOTO ( ) sono limitate alle istruzioni nello stesso programma. Se non vi è alcuna posizione nel programma corrispondente all'etichetta GOTO ( ) specificata, si verifica un errore. Inoltre, se vi sono varie istruzioni con la stessa etichetta nello stesso programma, il controller non sa dove andare e si verifica un errore.

Per etichettare una posizione di ramificazione, inserire il nome dell'etichetta (identificatore) all'inizio delle istruzioni da eseguire. Assicurarsi di aggiungere i due punti al termine dell'identificatore.

Esempio di programma

```
PROGRAM MAIN
    INPUT N
    GOTOSAMPL2 (N)
END
PROGRAM GOTOSAMPL2 (N)
    GOTO (N) L1, L2, L3
    RETURN
    L1: MOVE A1
    RETURN
    L2: MOVE A2
    RETURN
    L3: MOVE A3
    RETURN
END
```

Se l'argomento N è 1, 2 o 3, il robot si sposta rispettivamente al punto A1, A2 o A3. Se N non è 1, 2 o 3, l'esecuzione del programma viene reinviata al programma principale senza spostare il robot.

HERE

Scopo

L'istruzione HERE riporta la posizione attuale del robot.

Formato

HERE

Esempi

MOVE HERE
A1 = HERE
X = HERE. X

Analisi e consigli

Il comando HERE riporta la posizione attuale del robot nel sistema di coordinate universali.
HERE può essere gestito come qualsiasi dato di tipo posizionale con l'eccezione che si può fare riferimento solo ai valori contenuti all'interno e non si possono modificare i valori stessi.
Quando si esegue un comando HERE con il robot in movimento, viene riportata la posizione comandata HERE al momento dell'esecuzione del comando HERE.

Nota: La posizione riportata con il comando HERE è quella comandata al robot. Mentre il robot è in movimento, la posizione attuale effettiva del robot presenta un ritardo rispetto alla posizione comandata.
È possibile acquisire la posizione del robot attuale con il comando HERE.

N=HERE.6

Il valore della posizione attuale viene memorizzato in N.
Con un robot SCARA, il valore di HERE.6 è 0 per Libero (indefinito), 1 per Sistema sinistrorso o 2 Sistema destrorso.

HEXIN

Scopo

Questa funzione legge i segnali di ingresso con numerazione esadecimale.

Formato

HEXIN (<nome segnale>, <lunghezza segnale>)

Esempi

K = HEXIN (1, 2)
J2 = HEXIN (N, N+2)
GOTO (HEXIN (20, 2)) L1, L2, L3

Analisi e consigli

Questa funzione legge i segnali di ingresso di <lunghezza segnale> con numerazione esadecimale, ad iniziare con il segnale specificato con <nome segnale>.
Ad esempio, quando K = HEXIN (1, 2), lo stato di due (2) segnali di ingresso 1 e 2 viene letto come codice HEX.
È possibile specificare la lunghezza del segnale nell'intervallo di 1 ~ 32. Man mano che aumenta il numero di segnale di ingresso, corrisponde ad un bit di ordine superiore. Ciascun segnale viene codificato come segue.
ON = 1, OFF = 0

Esempio: Quando lo stato dei segnali di ingresso 1 ~ 12 corrisponde a quanto illustrato nella tabella di seguito, il valore di "HEXIN (1, 12)" è 809 (ovvero, 329 con numerazione esadecimale).

Numero segnale di ingresso	12	11	10	9	8	7	6	5	4	3	2	1
Stato segnale di ingresso	OFF	OFF	ON	ON	OFF	OFF	ON	OFF	ON	OFF	OFF	ON
Espressione binaria	0	0	1	1	0	0	1	0	1	0	0	1
Espressione esadecimale				3				2				9

È possibile utilizzare una costante, una variabile, un'espressione e un calcolo per <nome segnale> e <lunghezza segnale>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale. Questo comando viene utilizzato nell'espressione.

Esempio di programma

```
PROGRAM HEXINSAMPL
    K = HEXIN (1, 7)
    SPEED = K
    MOVE A1
    MOVE A2
END
```

Lo stato dei segnali di ingresso 1 ~ 7 viene codificato, la velocità di movimento viene impostata in base a tale valore, quindi il robot si sposta alla velocità impostata.

HEXOUT

Scopo

Questa funzione emette segnali codificandoli con numerazione esadecimale.

Formato

HEXOUT (<nome segnale>, <lunghezza segnale>, <espressione>)

Esempi

HEXOUT (1, 4, 3)
HEXOUT (N, N+4, K)

Analisi e consigli

Questa funzione codifica il valore di <espressione> con numerazione esadecimale ed emette le cifre specificate da <lunghezza segnale> sul numero di segnali come specificato da <lunghezza segnale>, ad iniziare con il segnale specificato con <nome segnale>.
Ad esempio, quando si comanda HEXOUT (1, 4, 3), il valore di "3" codificato con numerazione esadecimale viene emesso ai quattro segnali (1 ~ 4) e i segnali di uscita 1 e 2 diventano ON.
Man mano che aumenta il numero di segnale di uscita, corrisponde ad un bit di ordine superiore. È possibile specificare la lunghezza del segnale nell'intervallo di 1 ~ 32. Se il valore di <espressione> è superiore al valore specificato da <lunghezza segnale>, la cifra o le cifre di ordine superiore vengono ignorate.
Man mano che aumenta il numero di segnale di entrata, corrisponde ad un bit di ordine superiore.
Ciascun segnale viene codificato come segue.
ON = 1, OFF = 0

Esempio: Quando si comanda HEXOUT (1, 12, 952), i segnali di uscita sono come illustrato di seguito.

Espressione decimale	952											
Espressione esadecimale	3				B				8			
Espressione binaria	0	0	1	1	1	1	0	1	1	0	0	0
Numero segnale di uscita	12	11	10	9	8	7	6	5	4	3	2	1
Stato segnale di uscita	OFF	OFF	ON	ON	ON	ON	OFF	ON	ON	OFF	OFF	OFF

È possibile utilizzare una costante, una variabile, un'espressione e un calcolo per <nome segnale>, <lunghezza segnale> ed <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale. Se gli stessi segnali vengono emessi consecutivamente, diventa valido l'ultimo segnale emesso.
Fare attenzione che esista una gamma di segnali entro la quale sia possibile garantire la simultaneità.
È possibile garantire la simultaneità di HEXOUT nella gamma di 16 bit da DOUT1, DOUT101, DOUT301, ma non la simultaneità di HEXOUT che superi tali limiti.

Esempio di programma

```
PROGRAM HEXOUTSMPL
    FOR K = 1 TO 4
    J = 2 ^ (K–1)
    HEXOUT (1, 4, J)
    TIMER = 0.5
    WAIT TIMER == 0
    HEXOUT (1, 4, 0)
    NEXT K
END
```

I segnali di uscita 1 ~ 4 vengono emessi a turno in intervalli di 0,5 secondi.

IF

Scopo	L'istruzione IF viene utilizzata per valutare le condizioni.
Formato	IF <espressione logica) THEN <istruzione> [ELSE <istruzione>]
Esempi	IF DIN (1) THEN K = K + 1 ELSE K = 0
Analisi e consigli	Se vengono soddisfatte le condizioni dell'<espressione logica> successiva a IF, viene eseguita l'<istruzione> successiva a THEN. Se non vengono soddisfatte le condizioni, viene eseguita l'istruzione successiva a ELSE. Un'istruzione ELSE non è obbligatoria in una struttura IF. Se non viene soddisfatta la condizione IF e non vi è alcuna istruzione ELSE, l'esecuzione del programma passa alla fase successiva al comando IF. L'<istruzione> successiva all'istruzione THEN o ELSE potrebbe non contenere PROGRAM, END, IF, FOR, NEXT o WAIT. Le strutture IF ~ THEN ~ ELSE vengono considerate come singola unità e, per questo motivo, devono trovarsi sulla stessa riga (ciò significa che su quella riga si deve scrivere tutto ciò che segue l'istruzione ELSE).

Esempio di programma

```
PROGRAM IFSAMPLE
    IF DIN (1) THEN K = 1 ELSE K = 0
    MOVE A1
    MOVE A2
    MOVE A3
    PRINT TP, K, CR
END
```

Se il segnale di ingresso 1 è ON, K equivale a 1.
Se il segnale di ingresso 1 è OFF, K equivale a 0.

IGNORE

Scopo	Il comando IGNORE viene utilizzato per annullare il monitoraggio di una condizione specificata da un comando ON precedente.
Formato	IGNORE <condizione monitoraggio>
Esempi	IGNORE DIN (1) IGNORE TIMER
Analisi e consigli	Il comando IGNORE viene utilizzato per annullare il monitoraggio di una condizione specificata da un comando ON precedente. Nella specifica <condizione monitoraggio>, utilizzare la stessa espressione logica adoperata per la corrispondente istruzione ON. Tuttavia, l'istruzione della condizione ON che può essere ignorata deve essere specificata prima della riga di comando IGNORE. A questo punto, la stessa condizione ON non deve essere specificata su due (2) o più righe. Quando si esegue l'istruzione IGNORE, cessa il monitoraggio della condizione. Per ulteriori informazioni sul monitoraggio della condizione, fare riferimento al comando ON.

Esempio di programma

```
PROGRAM MAIN
    IGNORESMPL
    MOVE P
    MOVE P2
END
PROGRAM IGNORESMPL
    ON DIN (1) PAUSE DO RETURN
    MOVE A1
    MOVE A2
    WAIT MOTION > = 100
    IGNORE DIN (1)
    RETURN
END
```

Se il segnale di ingresso 1 è attivo, l'esecuzione viene reinviata al programma principale al termine del movimento in corso in quel momento. Il monitoraggio del segnale di ingresso 1 cessa al termine del movimento dal punto A1 al punto A2.

INITPLT

Scopo

Inizializza un pallet.

Formato

INITPLT (<numero pallet>, <i>, <j>, <k>)

Esempi

INITPLT (1, 5, 4, 3)

Analisi e consigli

Questa funzione viene utilizzata per inizializzare il pallet per eseguire il comando di pallettizzazione.

Numero pallet : Numero assegnato al pallet, che inizia con il numero "1" (ovvero, un numero intero superiore a "1").

i : Numero di elementi dal punto iniziale del pallet al punto I (ovvero, un numero intero superiore a "1").

j : Numero di elementi dal punto iniziale del pallet al punto J (ovvero, un numero intero superiore a "1").

k : Numero di elementi dal punto iniziale del pallet al punto K (ovvero, un numero intero superiore a "1").

Se il valore di i, j o k è zero (0) o meno, il programma si arresta con un messaggio di errore recante **“ERR!! ELEMENT IS TOO SMALL”** visualizzato sul display del programmatore portatile.

Il comando INITPLT è disponibile nella libreria a collegamento dinamico. Quando si esegue questo comando, è necessario dichiarare la compilazione libreria e la variabile globale nell'area GLOBAL.

Per ulteriori informazioni, consultare l'Appendice G–1.

Esempio di programma

```
GLOBAL
   LOADLIB PALLET.LIB          Dichiarazione compilazione
                               libreria.
   DIM PLTP (1, 7) AS POINT    Dichiarazione variabile globale.
END

PROGRAM SAMPLE
   INITPLT (1, 3, 4, 2)        Inizializzazione pallet su "3 × 4 ×
                               2" con punti di programmazione
                               PLTP (1, 1) ~ PLTP (1, 4).
   MOVEPLT (1, 1, 0, 0, 0, 0)  Spostamento su pallet N. 1,
                               elemento N. 1.
END
```

INPUT

Scopo	Il comando INPUT legge i dati da un canale di comunicazione specificato.
Formato	INPUT[{COM0 | COM1 | TP},] <variabile> [, <variabile>]…
Esempi	INPUT K1, K2, K3 INPUT COM1, K
Analisi e consigli	Il comando INPUT viene utilizzato per leggere i dati da un canale di comunicazione. Questi dati possono essere numeri reali o interi. Specificare un (1) canale di comunicazione tra COM0, COM1 e TP. COM0 e TP sono canali utilizzati esclusivamente dal programmatore portatile. COM1 corrisponde al canale di comunicazione COM1 del controller. Se non si specifica un canale di comunicazione nell'istruzione INPUT, i dati vengono letti dal canale di comunicazione del programmatore portatile. Quando si esegue un comando INPUT, il programma attende che i dati vengano letti dal canale di comunicazione. I dati letti vengono posizionati nelle variabili assegnate. Se vi sono più dati che variabili, i dati in eccesso vengono ignorati. Se vi sono meno dati che variabili, il programma attende l'immissione dei dati restanti. Quando si immettono dati dal programmatore portatile, separare i numeri reali con virgole. Al termine dell'immissione di numeri, premere il tasto EXE.

Quando i dati non vengono immessi dal programmatore portatile, vengono elaborati al termine della trasmissione. Per informazioni sulla comunicazione dei dati, fare riferimento al Manuale di comunicazione.

Una volta arrestato il movimento del braccio, questo comando non viene eseguito.

Esempio di programma

```
PROGRAM INPUTSMPL
PRINT COM0, "*** INPUT N1, N2, N3 ***"
INPUT COM0, N1, N2, N3
PRINT (N1 + N2 + N3)/3, CR
END
```

Questo programma legge tre valori (N1, N2 e N3) dal programmatore portatile, trova la media e la visualizza sul programmatore portatile.

INT

Scopo

Il comando INT cambia un valore numerico in un numero intero.

Formato

INT (<espressione>)

Esempi

AK = INT (–20.345)
N = INT (K)
J1 = K – INT (N – 28.5)

Analisi e consigli

Il comando INT converte il numero o il risultato di un calcolo tra parentesi ( ) in un numero intero.
Il comando INT elimina semplicemente i numeri reali sulla destra del punto decimale e li converte in un numero intero.

Questo comando viene utilizzato quando si desidera specificare il tipo di dati di una variabile come tipo intero.

È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando INT deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    INTSAMPLE (2, 30, K)
    PRINT TP, K, CR
END
PROGRAM INTSAMPL (L, R, K)
    K = INT (L * COS (R))
    RETURN
END
```

Questo programma prende gli argomenti L e R, trova il valore di (L * COS (R)), elimina i punti decimali e riporta il risultato come argomento K sul programma principale.

KILL

Scopo	Questa funzione determina la gestione multitask.
Formato	KILL (<espressione>)
Esempi	KILL (TASKID)
Analisi e consigli	Il comando KILL termina l'attività che presenta il numero di attività specificato dal risultato del calcolo dell'espressione tra parentesi ( ). Se si specifica l'ID di attività (regolato su 1) dell'attività principale generata automaticamente all'inizio del programma o l'ID di attività non esistente, l'operazione NOP è operativa. L'attività terminata dal comando KILL inizia con il comando TASK. Quando ciò accade, all'attività viene assegnato un nuovo numero. È possibile utilizzare una costante, una variabile o un'espressione aritmetica per <espressione>. Non è possibile utilizzare dati di tipo vettoriale. L'attività per la quale si è specificato l'arresto deve essere eliminata dal sistema nel momento in cui viene eseguita. Ovvero, se la variabile di sistema SWITCH è DISABLE, non avviene il cambio attività e non è possibile arrestare altre attività. Per eseguire il comando KILL, impostare la variabile di sistema SWITCH su DISABLE. Questo comando non è valido durante l'esecuzione della fase.

Esempio di programma

```
GLOBAL
    MAXTASK=2
    K=0
END

PROGRAM MAIN
    TID1=0
    LOOP:
    IF DIN(1) AND TID1==0 THEN TID1=TASK(“SUB1”)
    IF DIN(–1) AND TID1<>0 THEN KILL(TID1)ELSE GOTO
     LOOP1
    TID1=0
    LOOP1:
    MOVEA 1, –90
    MOVEA  1,90
    GOTO LOOP
END
PROGRAM SUB1
    K=K+1
    PRINT K,CR
END
```

Un'attività viene creata quando il segnale di ingresso 1 è attivato e viene cancellata quando lo stesso segnale è disattivato.

LATCH (opzione di TS3000)

Scopo	Questa funzione specifica ON/OFF della funzione latch di posizione che utilizza i segnali esclusivi della porta di ingresso.
Formato	LATCH
Esempi	DISABLE LATCH ENABLE LATCH
Analisi e consigli	Questa funzione specifica se monitorare o meno i segnali esclusivi della porta di ingresso per eseguire il latch di posizione. Per accendere e spegnere l'interruttore di sistema, utilizzare rispettivamente i comandi ENABLE e DISABLE. Quando si specifica il comando ENABLE LATCH, la funzione latch di posizione che utilizza i segnali esclusivi della porta di ingresso diventa operativa. Quando si specifica il comando DISABLE LATCH, la stessa funzione diventa non operativa. Nello stato iniziale, il comando DISABLE LATCH diventa operativo. Se si specifica il rilevamento bordo iniziale del segnale esclusivo della porta di ingresso e si utilizza il comando ENABLE LATCH con il segnale esclusivo della porta su ON, non si garantisce il funzionamento della funzione latch di posizione. Lo stesso è applicabile anche se si specifica il rilevamento del bordo di caduta del segnale esclusivo della porta di ingresso e si utilizza il comando ENABLE LATCH con il segnale esclusivo della porta su OFF. Quando ciò accade, tuttavia, non viene generato alcun errore e l'elaborazione continua. Solo una volta confermato lo stato del segnale esclusivo della porta di ingresso con il comando DIN, è possibile programmare il comando ENABLE LATCH. I segnali esclusivi della porta di ingresso vengono assegnati alle posizioni da 53 a 56.

Inoltre, dopo aver confermato lo stato di LATCHSIG1 e 2, acquisire la posizione di latch.
Quando si utilizza questa funzione, è necessario utilizzare una scheda esclusiva. Se si esegue il comando ENABLE LATCH mentre il sistema non è dotato di una scheda di estensione, si verifica un errore.

Esempio di programma

```
PROGRAM LATCHSMP
    DISABLE NOWAIT
    MOVE A0
    LATCHTRG1 = 1
    IF DIN (49) THEN GOTO FINI
    ENABLE LATCH
    MOVES A1
    WAIT MOTION >= 100
    IF LATCHSIG1 == 1 THEN LP = LATCHPSN1
        ELSE LP = HERE
    DISABLE LATCH
    FINI:
    MOVE LP
END
```

LATCHTRG1 ~ 8 (opzione di TS3000)

Scopo

Questa funzione specifica la direzione bordo rilevata della funzione latch di posizione che utilizza i segnali esclusivi della porta di ingresso.

Formato

LATCHTRG1 = {0 | 1}
LATCHTRG3 = {0 | 1}

Esempi

LATCHTRG1 = 1
A = LATCHTRG8

Analisi e consigli

Si tratta di variabili di sistema per specificare la direzione bordo rilevata che serve come trigger del latch di posizione, che utilizza i segnali esclusivi della porta di ingresso.
LATCHTRG1 ~ 8 specifica la direzione bordo rilevata delle porte di ingresso esclusive.
Quando si specifica "0", il segnale cade (ON → OFF). Allo stesso modo, quando si specifica "1", il segnale inizia (OFF → ON).
Non è possibile specificare un valore numerico diverso dai numeri interi per LATCHTRG1 ~ 8. Se si specifica un valore diverso da "0", il sistema lo considera "1".
Quando si fa riferimento a questa variabile di sistema, è possibile riferirsi alla direzione bordo attuale rilevata.
L'impostazione predefinita della direzione bordo rilevata è "1" (OFF → ON).
Se si modifica la direzione bordo rilevata durante l'esecuzione del comando ENABLE LATCH, non si garantisce l'operatività della funzione latch di posizione. Quando ciò accade, tuttavia, non viene generato alcun errore e l'elaborazione continua.
Per identificare il funzionamento programmato, eseguire il comando DISABLE LATCH, quindi modificare la direzione bordo rilevata.
Quando si utilizza questa funzione, è necessario utilizzare una scheda esclusiva. Tuttavia, se nel sistema si specifica LATCHTRG1 ~ 8 senza una scheda di estensione, il funzionamento non viene influenzato per niente.

Esempio di programma

```
PROGRAM LATCHSMP
    DISABLE NOWAIT
    MOVE A0
    LATCHTRG1 = 1
    IF DIN (49) THEN GOTO FINI
    ENABLE LATCH
    MOVES A1
    WAIT MOTION >= 100
    IF LATCHSIG1 == 1 THEN P1 = LATCHPSN1
        ELSE P1 = HERE
    DISABLE LATCH

    LATCHTRG2 = 0
    IF DIN (–50) THEN GOTO FINI
    ENABLE LATCH
    MOVES A2
    WAIT MOTION >= 100
    IF LATCHSIG2 == 1 THEN P2 = LATCHPSN2
        ELSE P2 = HERE
    DISABLE LATCH

    FINI:
    MOVE A0
END
```

LATCHSIG1 ~ 8 (opzione di TS3000)

Scopo

Questa funzione fa riferimento allo stato del latch di posizione che utilizza i segnali esclusivi della porta di ingresso.

Formato

LATCHSIG1
LATCHSIG5

Esempi

```
A = LATCHSIG1
IF LATCHSIG2 ==0 THEN GOTO ERR
```

Analisi e consigli

Questa funzione fa riferimento allo stato del latch di posizione che utilizza i segnali esclusivi della porta di ingresso.
LATCHSIG1 ~ 8 fa riferimento allo stato del latch di posizione delle porte di ingresso esclusive. Quando si esegue il latch di posizione, il sistema riporta "1". In caso contrario, riporta "0".
Durante l'esecuzione del comando DISABLE LATCH, il sistema riporta "0" indipendentemente dallo stato del segnale esclusivo.
Inoltre, se si disattiva il segnale esclusivo durante il rilevamento bordo all'inizio del segnale, lo stato di LATCHSIG1 ~ 8 diventa "0". Allo stesso modo, se si attiva il segnale esclusivo durante il rilevamento bordo alla caduta del segnale, lo stato di LATCHSIG1 ~ 8 diventa "0".
È possibile solo fare riferimento LATCHSIGN1 ~ 8, ma non può essere sostituito. Inoltre, non è possibile utilizzare LATCHSIGN1 ~ 8 come condizione ON.
Per riferirsi al segnale in condizione ON, è necessario fare riferimento a 49 ~ 56 mediante il comando DIN.
Quando si utilizza questa funzione, è necessario utilizzare una scheda I/O esclusiva. Tuttavia, se si fa riferimento a LATCHSIG1 ~ 8 nel sistema senza scheda I/O di estensione, il sistema riporta sempre "0".

Nota: Lo stato di latch identificato da LATCHSIG1 ~ 8 corrisponde sempre allo stato attuale del robot. Il movimento effettivo del robot viene ritardato a causa dell'elaborazione del programma SCOL.
Si consiglia di ottenere lo stato di latch e la posizione di latch solo dopo l'esecuzione di "WAIT MOTION >= 100".

Esempio di programma

```
PROGRAM LATCHSMP
    DISABLE NOWAIT
    MOVE A0
    LATCHTRG1 = 1
    IF DIN (49) THEN GOTO ERR
    ENABLE LATCH
    MOVES A1
    WAIT MOTION >= 100
    IF LATCHSIG1 == 0 THEN GOTO ERR
    LP = LATCHPSN1
    DISABLE LATCH
    GOTO FINI
    ERR:
    PRINT "LATCH ERROR", CR
    LP = HERE
    FINI:
    MOVE LP
END
```

LATCHPSN1 ~ 8 (opzione di TS3000)

Scopo

Questa funzione ottiene la posizione di latch mediante la funzione latch di posizione.

Formato

LATCHPSN1
LATCHPSN2

Esempi

P1 = LATCHPSN1
AX = LATCHPSN2. X

Analisi e consigli

Questa funzione ottiene una posizione nel sistema di coordinate universali quando si rileva il bordo del segnale esclusivo della porta di ingresso.
LATCHPSN1 ~ 8 ottiene la posizione di bordo delle porte di ingresso esclusive 1 ~ 8 nel sistema di coordinate universali.
Specificare la direzione bordo rilevata di LATCHTRG1 ~ 8.
L'elaborazione di rilevamento bordo diventa operativa in modalità ENABLE LATCH e non operativa in modalità DISABLE LATCH.
È possibile riferirsi a LATCHPSN1 ~ 8 quando lo stato di LATCHSIG1 ~ 8 corrispondente è "1". Anche se lo stato di LATCHSIG1 ~ 8 è "0", è possibile riferirsi a LATCHPSN1 ~ 8, ma non si garantisce il valore.
Quando si utilizza questa funzione, è necessario utilizzare una scheda esclusiva. Tuttavia, se si fa riferimento a LATCHPSN1 ~ 8 nel sistema senza scheda di estensione, viene indicata l'origine del sistema di coordinate universali.

Nota: Lo stato di latch identificato da LATCHSIG1 ~ 8 corrisponde sempre allo stato attuale del robot. Il movimento effettivo del robot viene ritardato a causa dell'elaborazione del programma SCOL.
Si consiglia di confermare lo stato di latch e la posizione di latch solo dopo l'esecuzione di "WAIT MOTION >= 100".

Esempio di programma

```
PROGRAM LATCHSMP
    DISABLE NOWAIT
    MOVE A0
    LATCHTRG1 = 1
    IF DIN (49) THEN GOTO ERR
    ENABLE LATCH
    MOVES A1
    WAIT MOTION >= 100
    IF LATCHSIG1 == 0 THEN GOTO ERR
    LP = LATCHPSN1
    DISABLE LATCH
    GOTO FINI
    ERR:
    PRINT "LATCH ERROR", CR
    LP = HERE
    FINI:
    MOVE LP
END
```

LEFTY

Scopo

LEFTY è una costante di sistema utilizzata per modificare la configurazione del robot su un sistema sinistrorso.

Formato

LEFTY

Esempi

CONFIG = LEFTY
MOVE A1 WITH CONFIG = LEFTY

Analisi e consigli

LEFTY viene utilizzato unitamente a CONFIG per impostare la configurazione del robot su un sistema sinistrorso.

Come costante di sistema, LEFTY ha un valore pari a 1. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 1.
Tuttavia, ciò potrebbe rendere il programma inutilmente complicato.

Non è possibile eseguire la sostituzione in costanti di sistema.

Per i robot a coordinate cartesiane, la designazione della configurazione del robot viene ignorata.

Per informazioni sulla configurazione del robot, fare riferimento al comando CONFIG.

Esempio di programma

```
PROGRAM LEFTYSMPL
    CONFIG = LEFTY
    MOVE A1
    MOVE A2
END
```

Questo programma imposta la configurazione del robot su un sistema sinistrorso prima di spostare il robot sulla sua posizione.

LN

Scopo

Questa funzione calcola il logaritmo normale di un numero.

Formato

LN (<espressione>)

Esempi

K = LN (100)
J1 = 1 – N (50 – D)

Analisi e consigli

Il comando LN riporta il logaritmo normale del numero tra parentesi ( ). Tuttavia, il risultato di LN (0) viene riportato come 0 (quando in realtà è indefinito e si prevede che venda riportato come errore).

È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando LN deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN PROGRAM LNSMPL (N, K)
    LNSMPLE (3, K)
    PRINT TP, K, CR
END
PROGRAM LNSMPL (N,K)
    K = 10 ^ N
    K = LN (K)
    RETURN
END
```

Questo programma prende il valore di un logaritmo della costante dato dall'argomento N, converte questo valore in un logaritmo naturale e reinvia il risultato al programma principale come argomento K.

LOADLIB

Scopo

Questa funzione legge una libreria a collegamento dinamico.

Formato

LOADLIB <nome file>

Esempi

LOADLIB PALLET.LIB

Analisi e consigli

Assicurarsi di dichiarare il comando LOADLIB nell'area GLOBAL. Il nome del file di libreria ********.LIB.

È possibile leggere fino a cinque (5) librerie nello stesso programma.

Per la libreria dinamica, consultare Para. 2.8.3 e Appendice G.

Esempio di programma

```
GLOBAL
    LOADLIB PALLET.LIB      Dichiarazione compilazione
libreria
END
PROGRAM MAIN
    INPUT N
    GOTOSAMPL2 (N)
END

PROGRAM GOTOSAMPL2 (N)   Esempio di file di libreria
    GOTO (N) L1, L2, L3
    RETURN
    L1:
    MOVE A1
    RETURN
    L2:
    MOVE A2
    RETURN
    L3:
    MOVE A3
    RETURN
END
```

LOG10

Scopo

Questa funzione calcola il logaritmo comune di un numero.

Formato

LOG10 (<espressione>)

Esempi

K = LOG10 (100)
J1 = 1 – LOG10 (50 – D)

Analisi e consigli

Il comando LOG10 riporta il logaritmo comune del numero tra parentesi ( ). Tuttavia, il risultato di LOG10 (0) viene riportato come numero indefinito (quando in realtà si prevede che venda riportato come errore).

È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando LOG10 deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    LOG10SAMPLE (3, K)
    PRINT TP, K, CR
END
PROGRAM LOG10SMPL (N,K)
    K= EXP (N)
    K= LOG10 (K)
    RETURN
END
```

Questo programma prende il valore di un logaritmo naturale dato dall'argomento N, converte questo valore in un logaritmo comune e reinvia il risultato al programma principale come argomento K.

MAXTASK

Scopo	Questa funzione specifica il numero massimo di attività eseguibili contemporaneamente nel programma contenente la funzione multitask.
Formato	MAXTASK
Esempi	MAXTASK = 4
Analisi e consigli	Questa variabile può essere utilizzata nel blocco di dati globale. Di solito, specificare il valore di "N. di comandi TASK + 1" per questa variabile. Il valore massimo è quattro (4). Se viene utilizzato un numero plurimo di attività, diventa operativo solo l'ultimo valore. Se non si utilizza la funzione multitask, questa variabile non viene utilizzata. Quando ciò accade, viene mantenuto il valore predefinito 1 e solo l'area di lavoro dell'attività principale. L'area di lavoro del controller viene assegnata a ciascuna attività, suddivisa in parti uguali da questa variabile. Se si specifica un grande valore, l'area di lavoro che può essere utilizzata da una (1) attività si riduce e non è possibile eseguire un programma di grandi dimensioni.

Esempio di programma

```
GLOBAL
  A=0
  MAXTASK=2
END
PROGRAM MAIN
  ID=0
  ID=TASK ("SUB")
  LOOP:
  IF DIN(1) THEN A=1 ELSE A=0
  GOTO LOOP
END
PROGRAM SUB
  ENABLE NOWAIT
  IF A==0 THEN PRINT "A", A, CR
END
```

Se si utilizza una (1) attività secondaria, viene impostato "2".

Viene creata una sequenza nell'istruzione GOTO per evitare di richiamare ripetutamente il comando dell'attività.

MOD

Scopo

La funzione MOD riporta il resto di una divisione.

Formato

<espressione> MOD <espressione>

Esempi

N = K MOD 3
J = K + (L MOD M)

Analisi e consigli

La funzione MOD prende l'<espressione> a sinistra, la divide per l'<espressione> a destra e reinvia il resto.

È possibile utilizzare costanti, variabili o equazioni per l'<espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando MOD deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    MODSAMPLE (5.0, 3.0, K)
    PRINT TP, K, CR
END
PROGRAM MODSAMPLE (N1, N2, K)
    K = N1 MOD N2
    RETURN
END
```

Questo programma prende l'argomento N2, il programma prende l'argomento N1, lo divide e reinvia il risultato all'argomento K.

MODE

Scopo

MODE viene utilizzato per fare riferimento alla modalità operativa del sistema.

Formato

MODE

Esempi

IF MODE < > CONT THEN STOP

Analisi e consigli

MODE viene utilizzato per fare riferimento alla modalità operativa del sistema.

Se il valore MODE è 0, il sistema è in modalità di funzionamento continuo. Se è 1, il sistema è in modalità di funzionamento a cicli, mentre se è 2, il sistema è in modalità di funzionamento a segmenti.

Quando si fa riferimento alla modalità operativa del sistema, è possibile utilizzare le costanti di sistema CONT, CYCLE e SEGMENT. Come si può prevedere, MODE = = CONT porta il sistema in modalità di funzionamento continuo, MODE = = CYCLE porta il sistema in modalità di funzionamento a cicli e MODE = = SEGMENT porta il sistema in modalità di funzionamento a segmenti.

È possibile utilizzare il comando di monitoraggio MODE MOTION per specificare il funzionamento a segmenti.

Esempio di programma

```
PROGRAM MODESAMPLE
    MOVE A1
    MOVE A2
    IF MODE <> CONT THEN STOP
    MOVE A3
END
```

Questo programma arresta l'esecuzione stessa se il sistema passa alla modalità di funzionamento continuo.

MOTION

Scopo

L'istruzione MOTION viene utilizzata per fare riferimento alla quantità di movimento completata.

Formato

MOTION

Esempi

K = MOTION
ON MOTION > – 50 DO DOUT (1)

Analisi e consigli

L'istruzione MOTION può essere utilizzata per consultare la percentuale di movimento del robot completata.

La "quantità di movimento" è definita come la percentuale di movimento completata dal robot rispetto alla distanza totale che tale movimento deve percorrere. I calcoli della quantità di movimento vengono effettuati per l'asse che presenta la maggiore distanza da percorrere.

La quantità di movimento viene riportata come numero reale. Combinando l'istruzione MOTION con un comando ON, il robot invia segnali mentre il movimento è ancora in corso. Questa istruzione deve essere utilizzata in un'espressione.

Nota: La quantità di movimento cui si fa riferimento con questo comando corrisponde alla posizione comandata al robot. Mentre il robot è in movimento, la posizione attuale del robot presenta un ritardo rispetto alla posizione comandata.
Prestare attenzione in quanto non è possibile utilizzare == per l'operatore comparativo.

Con ENABLE PASS durante l'utilizzo, fare attenzione in quanto, a seconda dell'utilizzo, viene eseguita una sequenza infinita. Poiché la formazione della traiettoria del percorso attende che P2 si verifichi da P1, il seguente esempio diventa una sequenza infinita.
Sostituendola con la frase WAIT, è possibile evitare una sequenza infinita.

```
ENABLE PASS                                  ENABLE PASS
PASS=50                                      PASS=50
MOVE P1                          →           MOVE P1
LOOP1:                                       WAIT MOTION >= 95
IF MOTION < 95 THEN GOTO LOOP1               MOVE P2
MOVE P2
```

Esempio di programma

```
PROGRAM MOTIONSMPL
    ENABLE NOWAIT
    ON MOTION > = 50 DO DOUT (1)
    MOVE A1
    ON MOTION > = 80 DO DOUT (2)
    MOVE A2
END
```

Quando la mano del robot è al 50% del percorso verso A1, viene emesso il segnale 1. Quando la mano del robot è all'80% del percorso verso A2, viene emesso il segnale 2.

MOTIONT

Scopo

L'istruzione MOTIONT viene utilizzata per fare riferimento alla quantità di tempo trascorso dall'inizio di un movimento.

Formato

MOTIONT

Esempi

K = MOTIONT
ON MOTIONT > = 5 DO DOUT (1)

Analisi e consigli

L'istruzione MOTIONT può essere utilizzata per consultare il tempo trascorso dall'inizio di un determinato movimento.

Il tempo di esecuzione viene fornito come numero reale in unità di secondi. Il tempo di esecuzione diventa 0 quando il robot termina il posizionamento finale di quel determinato movimento.

Combinando l'istruzione MOTIONT con un comando ON, il robot invia segnali mentre il movimento è ancora in corso. Quando questa istruzione verifica il tempo di percorso di un movimento del robot nel periodo di tempo specificato, può gestire l'errore. Il comando MOTIONT deve essere utilizzato in un'espressione.

Prestare attenzione in quanto non è possibile utilizzare == per l'operatore comparativo.

Nota: La quantità di movimento cui si fa riferimento con il comando MOTION corrisponde alla posizione comandata al robot. Mentre il robot è in movimento, la posizione attuale del robot presenta un ritardo rispetto alla posizione comandata.
Prestare attenzione in quanto non è possibile utilizzare == per l'operatore comparativo.
Con ENABLE PASS durante l'utilizzo, fare attenzione in quanto, a seconda dell'utilizzo, viene eseguita una sequenza infinita.

Poiché la formazione della traiettoria del percorso attende che P2 si verifichi da P1, il seguente esempio diventa una sequenza infinita. Sostituendola con la frase WAIT, è possibile evitare una sequenza infinita.

```
ENABLE PASS                                  ENABLE PASS
PASS=50                                      PASS=50
MOVE P1                          →           MOVE P1
LOOP1:                                       WAIT MOTION >= 5
IF MOTION < 5 THEN GOTO LOOP1                MOVE P2
MOVE P2
```

Esempio di programma

```
PROGRAM MOTIONTSMPL
    ENABLE NOWAIT
    ON MOTIONT > = 10 DO DOUT (1)
    MOVE A1
    MOVE A2
END
```

Se il robot impiega più di 10 secondi a completare un movimento verso A1, il segnale 1 viene emesso immediatamente.

MOVE

Scopo

Il comando MOVE sposta il robot in una posizione specifica.

Formato

MOVE <posizione> [clausola WITH]

Esempi

MOVE A1
MOVE A1 WITH SPEED = 50

Analisi e consigli

Il comando MOVE sposta il robot in una posizione specifica con movimento sincrono.

Tutti i giunti del robot iniziano e arrestano il movimento contemporaneamente. Il controller regola di conseguenza le velocità dei giunti rispetto al giunto più lento. Ciò viene denominato movimento sincrono (o, a volte, interpolazione angolare del giunto).

È possibile utilizzare un vettore posizionale per <posizione>. Inoltre, è possibile specificare direttamente i valori delle coordinate per <posizione> in uno dei due modi illustrati di seguito.

Non è possibile utilizzare dati di tipo coordinate o di tipo carico per <posizione>.

MOVE POINT (X, Y, Z, C, T, <configurazione>)
MOVE {X, Y, Z, C, T} WITH CONFIG = <configurazione>

(Qualora possibile, si può tentare di utilizzare il comando POINT per conoscere il tipo di dati che si sta gestendo.)

Per entrambi i metodi illustrati in precedenza:

X, Y, Z, C, T: I valori delle coordinate X, Y, Z, C e T vengono specificati con numeri reali (in unità di millimetri o gradi).

<Configurazione>: La configurazione del robot viene specificata con un valore intero di 0, 1 o 2.
(0 = indefinita (FREE); 1 = sistema sinistrorsa; 2 = sistema destrorsa)

È possibile utilizzare una costante, una variabile o un calcolo per ciascun singolo elemento. Tuttavia, non è possibile utilizzare dati di tipo vettoriale per un elemento. Tutti i valori inferiori a 0 immessi come <configurazione> vengono considerati 0, mentre tutti i valori superiori a 2 vengono considerati 2.
Gli elementi individuali dei dati vengono omessi, ma tutti questi elementi vengono considerati 0. Ad esempio, le due istruzioni di seguito significano lo stesso:

MOVE POINT (100, 100, 0, 0, 0, RIGTHY)
MOVE POINT (100, 100)

(Tutto da Z a <configurazione> viene considerato 0.)

Quando si immettono dati di posizione dal programmatore portatile, vengono registrati anche i dati del sistema di coordinate del lavoro specificati al momento della programmazione. Quando si esegue un comando di movimento, il sistema di coordinate del lavoro passa a quanto specificato al momento della programmazione dei dati di posizione. Si noti, tuttavia, che le coordinate di base e dell'utensile rimangono come erano prima dell'esecuzione del comando.

Quando si specificano direttamente i valori delle coordinate di <posizione> (a volte anche per creare o gestire i dati di posizione con comandi quali DEST, HERE e POINT), i movimenti vengono eseguiti con il sistema di coordinate del lavoro operativo prima dell'esecuzione del comando.

Il controller capisce condizioni di movimento quali velocità ed accelerazione, utilizzando i valori della variabile di sistema operativa in quel momento. Per modificare una condizione di movimento per un'operazione, utilizzare un comando WITH per specificare quella condizione. Per ulteriori informazioni, fare riferimento al comando WITH.

Esempio di programma

```
PROGRAM MOVESAMPLE
    MOVE A1
    MOVE A2
END
```

Questo programma sposta il robot al punto A1 con movimento sincrono.

MOVEA

Scopo

MOVEA sposta un determinato giunto del robot in una posizione specifica.

Formato

MOVEA <asse>, <posizione assoluta> [clausola WITH]

Esempi

MOVEA 1, 60
MOVEA 3, 0 WITH GAIN = {,, ON}

Analisi e consigli

Il comando MOVEA sposta un determinato giunto del robot in una posizione specifica. Tale movimento viene denominato "movimento assoluto singolo asse".

La designazione <asse> contiene un numero intero da 1 a 5 e specifica il giunto del robot da spostare. Tutti gli altri assi non vengono spostati.

La designazione <posizione assoluta> specifica la destinazione di quel movimento relativo all'origine dell'asse. Per i giunti rotanti, <posizione assoluta> è in termini di gradi. Per i giunti lineari (azionamento diretto), <posizione assoluta> è in termini di millimetri. Se si specifica una <posizione assoluta> che non rientra nella gamma di quel giunto, il robot si sposta alla posizione precedente alla fine di quel limite. È possibile utilizzare costanti, variabili o calcoli per le designazioni <asse> e <posizione assoluta>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Se si utilizza un numero diverso da 1 a 5 per la specifica dell'<asse> oppure si indica un asse non presente nel robot, il robot non si sposta.

Il controller capisce condizioni di movimento quali velocità ed accelerazione, utilizzando i valori della variabile di sistema operativa in quel momento. Per modificare una condizione di movimento per un'operazione, utilizzare il comando WITH per specificare quella condizione. Per ulteriori informazioni, fare riferimento al comando WITH.

Esempio di programma

```
PROGRAM MOVEASAMPL
    MOVEA 1, 0
    MOVEA 2, 90
END
```

Questo programma sposta l'asse 1 alla sua posizione di 0 gradi.

MOVEC

Scopo	MOVEC sposta la mano del robot su una posizione specifica tramite una determinata posizione transitoria in interpolazione circolare.
Formato	MOVEC <posizione transitoria> <posizione> [clausola WITH]
Esempi	MOVEC A1 A2 MOVEC A1 A2 WITH SPEED = 10
Analisi e consigli	La punta della mano robot viene spostata in un percorso circolare collegato tra posizione attuale, <posizione transitoria> e <posizione>. La punta della mano robot viene spostata in direzione dalla posizione attuale a <posizione> con una velocità angolare costante. Specificare i dati di tipo posizione su <posizione transitoria> e <posizione>. Come i comandi MOVE e MOVES, il comando POINT consente di specificare direttamente un valore delle coordinate. Non è possibile utilizzare dati di tipo coordinate o di tipo carico per <posizione transitoria> e <posizione>. [Sistema di coordinate lavoro] Nel sistema di coordinate del lavoro, la punta della mano del robot viene spostata come programmato per <posizione transitoria> e <posizione>. Quando il sistema di coordinate del lavoro viene specificato con la clausola WITH, la punta della mano viene spostata nel sistema di coordinate del lavoro. Quando viene specificato direttamente un valore delle coordinate per <posizione transitoria> e <posizione>, la punta della mano viene spostata nel sistema di coordinate del lavoro quando si esegue il comando.

[Clausola WITH]
Quando si sposta il robot, condizioni di movimento quali velocità ed accelerazione vengono determinate in base ai valori di impostazione delle variabili di sistema in quel momento.
Utilizzare la clausola WITH per modificare le condizioni di un movimento.
Per ulteriori informazioni, fare riferimento al comando WITH.
[Limite di ripristino dopo l'annullamento dell'interpolazione circolare]
Una volta annullato un movimento di interpolazione circolare tramite "ON ~ BREAK ~ DO ~", interruzione di alimentazione, arresto di emergenza o guasto, quando si riprende l'esecuzione senza ripristinare il programma (quando un movimento viene annullato con "ON ~ BREAK ~ DO ~" e ripreso con il comando RESUME), il comando annullato viene nuovamente eseguito. In quel momento, il movimento di interpolazione circolare funziona come movimento lineare di interpolazione su <posizione>.

[Limite di rapporto di posizione di tre punti]
Quando tre o due posizioni di tre punti che formano un arco (posizione attuale, <posizione transitoria> e <posizione>) sono uguali o simili, la punta della mano del robot può essere spostata lungo un arco differente da quello previsto.
Quando un movimento di interpolazione circolare viene utilizzato durante uno spostamento rapido, il percorso del robot deve essere collegato alla tangente della circolare. Quando l'angolo diventa acuto in un giunto di interpolazione circolare dello spostamento rapido, potrebbe essere applicata un'accelerazione improvvisa al robot.

[Offset dell'utensile]
Quando la mano viene spostata linearmente o in interpolazione circolare con un utensile in offset mentre si modifica l'orientamento dell'utensile, se l'offset dell'utensile non è impostato correttamente, potrebbe non ottenersi il movimento specificato.

L'offset dell'utensile viene utilizzato nella programmazione delle posizioni. Prima di programmare le posizioni, è necessario verificare se l'offset dell'utensile è impostato correttamente.

(Per il metodo di selezione del sistema di coordinate dell'utensile, consultare "6.7.6 Selezione delle coordinate dell'utensile" del "Manuale dell'operatore".)
Quando si utilizza un utensile in offset, prima di programmare le posizioni è necessario impostare l'offset dell'utensile. Inoltre, all'inizio del programma del linguaggio del robot, specificare in modo sicuro con il comando TOOL il corretto offset dell'utensile.

Esempio:
```
PROGRAM MAIN
TOOL=TOOL1    Successivamente, utilizzare
              "TOOL1" come offset dell'utensile.
   ...
END
```

[Aggiunta del 5º asse (opzionale)]
Quando si aggiunge il 5º asse (opzionale), prestare attenzione in quanto non è possibile che il 5º asse esegua un'interpolazione ad arco. Gli altri assi possono eseguire un'interpolazione ad arco.

Esempio di programma

```
PROGRAM MOVECSAMPL
    MOVES A1
    MOVEC A2 A3
END
```

Sposta la mano da A1 a A2 in interpolazione ad arco.

MOVEI

Scopo

MOVEI sposta un determinato giunto del robot di una quantità specifica dalla posizione attuale.

Formato

MOVEI <asse>, <posizione relativa) [clausola WITH]

Esempi

MOVEI 1, 60
MOVEI 3, 10 WITH SPEED = 50

Analisi e consigli

Il comando MOVEI sposta un determinato giunto del robot di una quantità specifica dalla posizione attuale. Tale movimento viene denominato "movimento relativo singolo asse".

La designazione <asse> contiene un numero intero da 1 a 5 e specifica il giunto del robot da spostare. Tutti gli altri assi non vengono spostati.

La designazione <posizione relativa> specifica la quantità di quel movimento relativo alla posizione attuale di quel giunto. Per i giunti rotanti, <posizione relativa> è in termini di gradi. Per i giunti lineari (azionamento diretto), <posizione relativa> è in termini di millimetri. Se si specifica una <posizione relativa> che non rientra nella gamma di quel giunto, il robot si sposta alla posizione precedente alla fine di quel limite.

È possibile utilizzare costanti, variabili o calcoli per le designazioni <asse> e <posizione relativa>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Se si utilizza un numero diverso da 1 a 5 per la specifica dell'<asse> oppure si indica un asse non presente nel robot, il robot non si sposta.

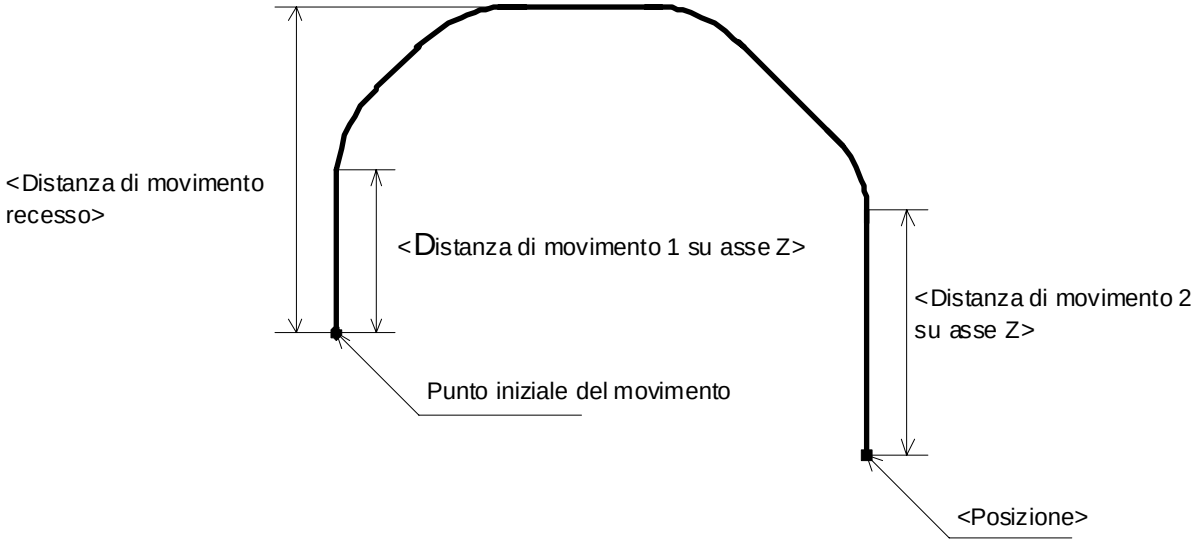
Il controller capisce condizioni di movimento quali velocità ed accelerazione, utilizzando i valori della variabile di sistema operativa in quel momento. Per modificare una condizione di movimento per un'operazione, utilizzare il comando WITH per specificare quella condizione. Per ulteriori informazioni, fare riferimento al comando WITH.

Esempio di programma

```
PROGRAM MOVEISAMPL
    MOVEI 1, 30      Questo programma sposta l'asse 1 a 30
                     gradi dalla posizione attuale.
    MOVEI 2, 30      Questo programma sposta l'asse 2 a 30
                     gradi dalla posizione attuale.
    MOVEI 3, 30      Questo programma sposta l'asse 3 a 30
                     gradi dalla posizione attuale.
    MOVEI 4, 30      Questo programma sposta l'asse 4 a 30
                     gradi dalla posizione attuale.
END
```

MOVEJ

Scopo	Sposta il robot sulla posizione specifica lungo un arco.
Formato	MOVEJ <posizione> <definizione di arco>
Esempi	MOVEJ A1 AC WITH SPEED=30 MOVEJ A1 {50.0, 20.0, 30.0}
Analisi e consigli	Questa funzione viene utilizzata per spostare il robot sulla posizione specifica lungo un arco. Come il comando MOVE, il robot si sposta con il comando PTP (da punto a punto). Ovvero, in un movimento orizzontale nel movimento ad arco, il robot non si sposta lungo una linea retta. <Posizione> specifica la posizione target finale nel movimento MOVEJ. Per <posizione> è possibile utilizzare i dati di posizione. <Definizione di arco> definisce il profilo del movimento ad arco con un valore. I dati di posizione vengono utilizzati per <definizione di arco>. Per <definizione di arco> sono validi solo tre (3) elementi (i valori degli elementi 4 ~ 6 vengono ignorati). <definizione di arco> = {<distanza di movimento recesso>, <distanza di movimento 1 su asse Z>, <distanza di movimento 2 su asse Z>} Non è possibile utilizzare i dati delle coordinate e i dati di carico per <posizione> e <definizione di arco>.



<Distanza di movimento recesso> significa la distanza dal punto iniziale del movimento alla posizione più elevata nella direzione dell'asse Z in unità di "mm". Il valore di <distanza di movimento recesso> deve essere un numero reale superiore a 0,0. Se si specifica un valore negativo per <distanza di movimento recesso>, si verifica un errore.

<Distanza di movimento 1 su asse Z> indica la distanza di movimento dell'asse 3 solo in direzione verso l'alto in unità di "mm". Il valore di <distanza di movimento 1 su asse Z> deve essere un numero reale superiore a 0,0. Se si specifica un valore negativo per <distanza di movimento 1 su asse Z>, si verifica un errore.
Se <distanza di movimento 1 su asse Z> è superiore alla distanza di movimento in direzione verso l'alto, il sistema lo interpreta come una designazione della distanza di movimento in direzione verso l'alto.

<Distanza di movimento 2 su asse Z> indica la distanza di movimento dell'asse 3 solo in direzione verso il basso in unità di "mm". Il valore di <distanza di movimento 2 su asse Z> deve essere un numero reale superiore a 0,0. Se si specifica un valore negativo per <distanza di movimento 2 su asse Z>, si verifica un errore.

Se <distanza di movimento 2 su asse Z> è superiore alla distanza di movimento in direzione verso il basso, il sistema lo interpreta come una designazione della distanza di movimento in direzione verso il basso.

È inoltre possibile designare direttamente i valori di <posizione> e <definizione di arco> nel modo seguente.

MOVEJ POINT(X,Y,Z,C,T, <configurazione>) POINT (Z1,Z2,Z3)

MOVEJ {X,Y,Z,C,T} {Z1,Z2,Z3} WITH CONFIG=<configurazione>

(Per identificare il tipo di dati, utilizzare il comando POINT.)

X, Y, Z, C, T	:	Specificare un numero reale per ciascuna coordinata di X, Y, Z, C, T. (Unità: mm, gradi)
<Configurazione>	:	Specificare la configurazione del robot con un numero intero di 0 ~ 2. (0: indefinita, 1: sinistrorsa, 2: destrorsa)
Z1, Z2, Z3	:	Specificare un numero reale per definire il profilo dell'arco. (Unità: mm)

Il comando MOVEJ compone i movimenti nelle direzioni verso l'alto, orizzontale e verso il basso e non traccia un arco. Inoltre, per dare priorità alla posizione più elevata nella direzione dell'asse Z, i movimenti nelle direzioni verso l'alto e verso il basso non vengono composti.
Se la distanza di movimento nella direzione verso l'alto o verso il basso sono inferiore al movimento in direzione orizzontale, la distanza di movimento su asse Z potrebbe essere superiore al valore specificato.

Se la configurazione sul punto iniziale del movimento del comando MOVEJ differisce dalla configurazione target, la configurazione varia nel movimento orizzontale.

Il valore della distanza già percorsa, mostrato con il comando MOTION al momento dell'esecuzione del comando MOVEJ, corrisponde alla percentuale del tempo trascorso rispetto al tempo di movimento totale del comando MOVEJ. Allo stesso

modo, il valore della distanza da percorrere, mostrato con il comando REMAIN, corrisponde alla percentuale del tempo restante rispetto al tempo di movimento totale del comando MOVEJ.

Quando il comando MOVEJ viene interrotto con BREAK, vengono mantenute le distanza da percorrere nelle direzioni verso l'alto, verso l'alto e dell'asse Z in ciascuna direzione e in posizione target.

Per riprendere il funzionamento, il comando MOVEJ viene creato di nuovo in base a questi dati. Pertanto, se si sposta il robot con guida manuale, ecc. durante l'interruzione effettuata con BREAK, non si garantisce il passaggio di metà corsa.

Il comando MOVEJ non è in grado di consentire uno spostamento rapido nell'intervallo con un altro comando di movimento (compreso il comando MOVEJ).

Esempio di programma

```
PROGRAM MOVEJSAMPL
    P1 = POINT (300.0, 350.0, 50.0, 0.0, 0.0)
    P2 = POINT (300.0, –350.0, 100.0, 0.0, 0.0)
    ARCH = POINT (100.0, 40.0, 50.0)
    MOVE P1
    MOVEJ P2 ARCH
END
```

MOVEPLT

Scopo	Sposta il robot sulla posizione specifica sul pallet.
Formato	MOVEPLT (<numero pallet>, <numero elemento> X, Y, Z, C)
Esempi	MOVEPLT (1, 10, 0, 0, 50, 0)
Analisi e consigli	Questa funzione viene utilizzata per spostare il robot alla posizione specificata dal numero pallet e dal numero elemento e include offset X, Y, Z, C. Non è possibile omettere il valore di offset. Prima di eseguire il comando MOVEPLT, **è necessario inizializzare un pallet** adeguato tramite il comando INITPLT. Il comando MOVEPLT è disponibile nella libreria a collegamento dinamico. Quando si esegue questo comando, è necessario dichiarare la compilazione libreria e la variabile globale nell'area GLOBAL. Se il numero elemento è zero (0) o meno oppure superiore al numero massimo di elementi, il programma si arresta con il seguente messaggio di errore visualizzato sul display del programmatore portatile. Con numero elemento < 1: “ERR!! ELEMENT NO. IS TOO SMALL” Con numero elemento > i × j × k di INITPLT: “ERR!! ELEMENT NO. IS TOO LARGE” Per ulteriori informazioni, consultare l'Appendice G–1.

Esempio di programma

```
GLOBAL
  LOADLIB PALLET.LIB          Dichiarazione compilazione
                              libreria.
  DIM PLTP (1, 7) AS POINT    Dichiarazione variabile globale.
END

PROGRAM SAMPLE
  INITPLT (1, 3, 4, 2)        Inizializzazione pallet su "3× 4×2"
                              con punti programmati PLTP (1,
                              1) ~ PLTP (1, 4).
  MOVEPLT (1, 1, 0, 0, 50, 0) Spostamento sulla posizione di
                              50 mm di pallet 1, elemento N. 1.
  OPEN1
  MOVEPLT (1, 1, 0, 0, 0, 0)  Spostamento su pallet 1,
                              elemento N. 1.
  CLOSE1
  MOVEPLT (1, 1, 0, 0, 50, 0) Spostamento sulla posizione di
                              50 mm di pallet 1, elemento N. 1.
END
```

MOVES

Scopo

Il comando MOVES sposta il robot con interpolazione lineare in una posizione specifica.

Formato

MOVES <posizione> [clausola WITH]

Esempi

MOVES A1
MOVES A1 WITH GAIN = {,, ON}

Analisi e consigli

Il comando MOVES sposta il robot in una posizione specifica lungo un percorso determinato con interpolazione lineare.

Questo comando causa lo spostamento del robot dalla posizione attuale ad una posizione specifica lungo una linea retta che unisce le due posizioni. Questo tipo di movimento viene denominato movimento interpolato lineare. In questo movimento, la velocità lineare della mano del robot rimane costante (tranne che durante accelerazione e decelerazione).

È possibile utilizzare un vettore posizionale per <posizione>. Inoltre, è possibile specificare direttamente i valori delle coordinate per <posizione> in uno dei due modi illustrati di seguito.

Non è possibile utilizzare dati di tipo coordinate o di tipo carico per <posizione>.

MOVES POINT (X, Y, Z, C, T, <configurazione>)
MOVES (X, Y, Z, C, T) WITH CONFIG = <configurazione>

(Qualora possibile, si può tentare di utilizzare il comando POINT per conoscere il tipo di dati che si sta gestendo.)

X, Y, Z, C, T : I valori delle coordinate X, Y, Z, C e T vengono specificati con numeri reali (in unità di millimetri o gradi).

<configurazione>: La configurazione del robot viene specificata con un valore intero di 0, 1 o 2.

(0 indefinito (FREE); 1 = sistema sinistrorso; 2 = sistema destrorso)

È possibile utilizzare una costante, una variabile o un calcolo per ciascun singolo elemento. Tuttavia, non è possibile utilizzare dati di tipo vettoriale per un elemento. Tutti i valori inferiori a 0 immessi come <configurazione> vengono considerati 0, mentre tutti i valori superiori a 2 vengono considerati 2.

Gli elementi individuali dei dati vengono omessi, ma tutti questi elementi vengono considerati 0. Ad esempio, le due istruzioni di seguito significano lo stesso:

MOVES POINT (100, 100, 0, 0, 0, 0)

Quando si immettono dati di posizione con il programmatore portatile, vengono registrati anche i dati del sistema di coordinate del lavoro operativi in quel momento. Quando si esegue un comando di movimento, il sistema di coordinate del lavoro attuale passa a quanto specificato al momento della programmazione dei dati di posizione. Si noti, tuttavia, che le coordinate di base e dell'utensile rimangono come erano prima dell'esecuzione del comando.

Quando si specificano direttamente i valori di <posizione> (a volte anche per creare o gestire i dati di posizione con comandi quali DEST, HERE e POINT), i movimenti vengono eseguiti con il sistema di coordinate del lavoro operativo prima dell'esecuzione del comando.

Il controller capisce condizioni di movimento quali velocità ed accelerazione, utilizzando i valori della variabile di sistema operativa in quel momento. Per modificare una condizione di movimento per un'operazione, utilizzare il comando WITH per specificare quella condizione. Per ulteriori informazioni, fare riferimento al comando WITH.

[Aggiunta del 5º asse (opzionale)]
Quando si aggiunge il 5º asse (opzionale), prestare attenzione in quanto non è possibile che il 5º asse esegua un'interpolazione su linea retta. Gli altri assi possono eseguire un'interpolazione su linea retta.

Esempio di programma

```
PROGRAM MOVESSAMPL
    MOVES A1
    MOVES A2
END
```

Questo programma sposta il robot al punto A2 con movimento lineare.

MOVESYNC

Scopo	Specifica le modalità sincrona e asincrona del comando di movimento.
Formato	MOVESYNC
Esempi	DISABLE MOVESYNC
Analisi e consigli	Assumere un movimento durante ENABLE NOWAIT nel programma SCOL in cui i comandi di movimento multipli e i comandi di ingresso/uscita segnale si allineano alternativamente. Nello stato ENABLE MOVESYNC (modalità sincrona del comando di movimento), il sistema esegue fino al successivo comando di movimento e attende il completamento del posizionamento. Pertanto, l'ingresso/uscita del secondo segnale vengono eseguiti subito dopo l'inizio del secondo comando di movimento e l'ingresso/uscita del terzo segnale vengono eseguiti subito dopo l'inizio del terzo comando di movimento. Tuttavia, in questa modalità, il sistema non entra nello stato della variabile di sistema PASS e non è possibile eseguire lo spostamento rapido. Nello stato DISABLE MOVESYNC (modalità asincrona del comando di movimento), il sistema esegue in anticipo fino ai successivi quattro comandi di movimento (massimo) e attende il completamento del posizionamento. Pertanto, i comandi di ingresso/uscita del secondo e del successivo segnale potrebbero essere eseguiti durante il primo movimento. Abilitando la variabile di sistema PASS, il movimento di passaggio diventa possibile. Il valore di questa variabile di sistema quando si attua il programma SCOL viene impostato dal parametro dell'utente [U03].

Esempio di programma

```
PROGRAM MAIN
  ENABLE NOWAIT
  ENABLE MOVESYNC
  MOVEA 1, 90
  DOUT (1)
  MOVEA 1, – 90
  DOUT (2)
  DISABLE MOVESYNC
  MOVEA 2, 90
  DOUT (3)
  MOVEA 2, –90
  DOUT (4)
END
```

D1 si attiva quando l'asse 1 si sposta alla posizione +90°.

D2 si attiva quando l'asse 1 si sposta alla posizione –90°.

D3 si attiva quando l'asse 2 si sposta alla posizione +90°.

D4 si attiva quando l'asse 2 si sposta alla posizione +90° (non sulla posizione – 90°a).

NEXT

Scopo	NEXT viene utilizzato unitamente all'istruzione FOR per comandare la ripetizione di una parte del programma per un determinato numero di volte.
Formato	NEXT [<variabile>]
Esempi	NEXT K
Analisi e consigli	L'istruzione NEXT viene utilizzata con l'istruzione FOR per comandare la ripetizione di una parte del programma. La parte del programma da ripetere è contenuta in un blocco che inizia con il comando FOR e termina con il comando NEXT. Il blocco continua a ripetersi finché non si soddisfa la condizione specificata per l'istruzione FOR. Specificare la variabile specificata dalla corrispondente istruzione FOR per <variabile>. Se non si specifica <variabile> nell'istruzione NEXT, viene eseguita una sequenza tra la più vicina istruzione FOR (eseguita alla fine) e l'istruzione NEXT. Per le condizioni di ripetizione del programma, fare riferimento al comando FOR.

Esempio di programma

```
PROGRAM NEXTSAMPLE
    FOR K = 1 TO 100
        MOVE A1
        MOVE A2
    NEXT K
END
```

Il robot ripete per 100 volte l'operazione avanti e indietro tra A1 e A2.

NOT

Scopo	NOT inverte la valutazione di un'espressione logica.
Formato	NOT <espressione logica>
Esempi	IF NOT DIN (1) THEN STOP
Analisi e consigli	NOT inverte la valutazione di un'espressione logica. L'istruzione NOT deve essere utilizzata in un'espressione.
Esempio di programma	PROGRAM NOTSAMPLE IF NOT DIN (1) THEN DOUT(1) END Se il segnale di ingresso 1 è OFF, il segnale di uscita 1 è ON.

NOWAIT

Scopo

NOWAIT è un interruttore di sistema che comanda al controller di continuare ad elaborare segnali I/O senza attendere che il robot termini di posizionarsi.

Formato

NOWAIT

Esempi

DISABLE NOWAIT
ENABLE NOWAIT

Analisi e consigli

NOWAIT è una costante di sistema utilizzata per indicare al controller di non attendere che il robot termini di posizionarsi prima di elaborare segnali I/O (ingresso/uscita).

I tempi di emissione del segnale sono descritti in dettaglio nella Sezione 5.

I comandi ENABLE e DISABLE vengono utilizzati per accendere e spegnere gli interruttori di sistema (come NOWAIT).

ENABLE NOWAIT indica al controller di non attendere che il robot termini di posizionarsi prima di inviare (o ricevere) segnali. DISABLE NOWAIT indica al controller di attendere che il robot termini di posizionarsi prima di inviare (o ricevere) segnali.

L'impostazione iniziale per il sistema è DISABLE NOWAIT.

Esempio di programma

```
PROGRAM NOWAITSMPL
    ENABLE NOWAIT
    MOVE A1
    DOUT (1)
    MOVE A2
    DOUT (2)
    MOVE A3
END
```

Qui, il controller invia segnali esterni senza attendere che il robot termini di posizionarsi.

OFF

Scopo

OFF è una costante di sistema utilizzata per specificare gli assi in cui il guadagno (servocomando) deve essere OFF.

Formato

OFF

Esempi

GAIN = {OFF, OFF, ON, OFF, OFF}
MOVE A1 WITH GAIN = {,, OFF}

Analisi e consigli

OFF è una costante di sistema utilizzata unitamente a GAIN o SETGAIN per specificare la disattivazione del guadagno (servocomando) di un asse specifico.
Se GAIN viene specificato come OFF, il servocomando di quel determinato asse si arresta la prossima volta che si esegue un comando di movimento. Gli assi dei quali si è arrestato il servocomando sono in "stato di servocomando libero" (in cui non viene eseguito il controllo del posizionamento).
Come costante di sistema, OFF ha un valore pari a 0. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 0. Tuttavia, ciò potrebbe rendere il programma inutilmente difficile da capire.
Non è possibile eseguire la sostituzione in costanti di sistema.
Per informazioni sui guadagni, fare riferimento al comando GAIN.

Esempio di programma

```
PROGRAM OFFSAMPLE
    MOVE A1
    WAIT MOTION > = 100
    GAIN = {OFF, OFF, ON, OFF, OFF}
    MOVE A2
    OPEN 1
    DELAY 0.5
    MOVE A1
    WAIT MOTION >= 100
    GAIN = {ON, ON, ON, ON, ON}
    READY
END
```

Questo programma disattiva tutti i guadagni ad eccezione di quello per l'asse Z (asse 3) prima che il robot si sposti sul punto A2.

ON

Scopo	ON è una costante di sistema utilizzata per specificare gli assi in cui il guadagno (servocomando) deve essere ON.
Formato	ON
Esempi	GAIN = {OFF, OFF, ON, OFF, OFF} MOVE A1 WITH GAIN = {,, ON}
Analisi e consigli	ON è una costante di sistema utilizzata unitamente a GAIN o SETGAIN per specificare l'attivazione del guadagno (servocomando) di un asse specifico. Se GAIN viene specificato come ON, il servocomando di quel determinato asse si avvia la prossima volta che si esegue un comando di movimento. Gli assi dei quali si è arrestato il servocomando sono in "stato di servocomando libero" (in cui non viene eseguito il controllo del posizionamento). Come costante di sistema, ON ha un valore pari a 1. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 1. Tuttavia, ciò potrebbe rendere il programma inutilmente difficile da capire. Non è possibile eseguire la sostituzione in costanti di sistema. Per informazioni sui guadagni, fare riferimento al comando GAIN.

Esempio di programma

```
PROGRAM ONSAMPLE
    MOVE A1
    WAIT MOTION >= 100
    GAIN = {OFF, OFF, ON, OFF, OFF}
    MOVE A2
    OPEN 1
    DELAY 0.5
    MOVE A1
    WAIT MOTION >= 100
    GAIN = {ON, ON, ON, ON, ON}
    READY
END
```

Questo programma disattiva tutti i guadagni ad eccezione di quello per l'asse Z (asse 3) prima di spostare il robot sul punto A2. Quindi, una volta completati tutti i movimenti, il programma ripristina i guadagni di tutti gli assi.

ON

Scopo

ON viene utilizzato per il monitoraggio delle condizioni (per la designazione ON/OFF del guadagno, consultare le descrizioni su [ON] di cui sopra).

Formato

ON <condizione monitoraggio> [{BREAK | PAUSE}] DO <istruzione>

Esempi

ON DIN (1) DO RETURN
ON TIMER DO MOVE A1

Analisi e consigli

Se si soddisfa la <condizione monitoraggio>, viene eseguita l'istruzione successiva al comando DO.

Il monitoraggio della condizione viene eseguito indipendentemente dal movimento eseguito dal robot in quel momento.

Il comando ON viene eseguito parallelamente ai comandi di movimento del robot. Se si utilizza un comando MOTION, MOTIONT, REMAIN o REMAINT come condizione di monitoraggio, viene eseguito il monitoraggio delle condizioni dei successivi comandi di movimento. Se si utilizza TIMER come condizione di monitoraggio, le condizioni verranno monitorate indipendentemente dal movimento del robot.

Quando si esegue il monitoraggio di segnali di ingresso con DIN o comandi simili, il tempo di inizio del monitoraggio varia in base all'impostazione dell'interruttore di sistema NOWAIT. Quando è operativa un'istruzione ENABLE NOWAIT, i segnali vengono monitorati indipendentemente dal movimento del robot.

Quando è operativa un'istruzione DISABLE NOWAIT, il monitoraggio del segnale inizia una volta che il robot termina il movimento in corso in quel momento.

L'esecuzione dell'istruzione successiva al comando DO inizia subito dopo l'esecuzione del comando operativo nel momento in cui si è soddisfatta la condizione di monitoraggio. Tuttavia, se si è eseguito un comando WAIT nel momento in cui si è soddisfatta la condizione di monitoraggio, tale comando WAIT viene immediatamente annullato e il controllo del programma passa all'istruzione successiva al comando DO.

Vi sono tre tipi di tempi di esecuzione specificabili per il robot mentre è in funzione:

BREAK : BREAK arresta immediatamente tutti i movimenti del robot e il controllo passa all'istruzione successiva al comando DO.

PAUSE: L'istruzione successiva al comando DO viene eseguita una volta che termina il movimento in corso in quel momento. Tuttavia, durante il movimento del braccio, l'esecuzione del programma normale prosegue, ad eccezione dei comandi di richiamo del sottoprogramma, di ritorno al programma principale e di movimento. Durante l'esecuzione di questi comandi, l'esecuzione del programma si arresta finché non si arresta il braccio.

Predefiniti: L'impostazione predefinita causa il completamento del movimento in corso mentre si eseguono contemporaneamente le istruzioni successive al comando DO. Quando l'istruzione successiva al comando DO è un comando di movimento, includere sempre un'istruzione BREAK o PAUSE nella riga di comando ON.

Se si eseguono l'istruzione successiva al comando DO (ovvero, istruzione DO) e il comando di movimento nell'istruzione DO, al termine del movimento del braccio l'esecuzione del programma si riavvia in base alle condizioni precedenti alla condizione per la quale si è soddisfatto il comando ON.

(Se si è interrotto un comando WAIT, l'esecuzione del programma si riavvia dall'inizio di quel comando WAIT, ovvero il comando WAIT viene eseguito di nuovo.) Tuttavia, se si è eseguito un ramo del programma su un'etichetta con l'istruzione successiva al comando DO, l'esecuzione si avvia dall'istruzione recante quella etichetta.

È possibile monitorare contemporaneamente dieci set di condizioni. Inoltre, è possibile specificare un massimo di quattro segnali di ingresso con un singolo comando ON.

Quando diverse condizioni di monitoraggio diventano vere contemporaneamente, viene eseguita l'istruzione DO corrispondente al comando ON con la massima priorità. Questa priorità è determinata dall'ordine in cui i comandi ON si trovano nel programma. Pertanto, il primo comando ON ha la massima priorità. Le istruzioni DO corrispondenti a tutti gli altri comandi ON vengono ignorate.

Il monitoraggio di una condizione specificata da un comando ON viene annullata se l'esecuzione passa ad un'istruzione DO corrispondente ad un altro comando ON. Inoltre, le condizioni non vengono monitorate quando si arresta l'esecuzione del programma a causa di un comando STOP o di un errore.

Se un sottoprogramma viene specificato con un'istruzione successiva a DO, è possibile eseguire due o più processi descritti nel sottoprogramma quando si soddisfa la condizione. Se si utilizza un'istruzione ON nell'esecuzione del programma come istruzione successiva a DO, il monitoraggio della condizione diventa valido solo quando si torna al sottoprogramma.

Quando il timer di sistema viene specificato come condizione di monitoraggio, tale condizione viene verificata solo quando si modifica lo stato del timer.

Durante il monitoraggio di un segnale esterno, di una condizione di errore o di un comando di riferimento movimento (ad esempio, la quantità di movimento restante da eseguire), il controller verifica lo stato, non la variazione, di quel segnale.

Il comando IGNORE annulla il monitoraggio delle condizioni specificate da un comando ON. Il monitoraggio delle condizioni si arresta inoltre quando si soddisfa una condizione e si esegue un'istruzione successiva ad un comando DO.

Nota 1: Al momento, le combinazioni dei comandi ON e DO possono essere utilizzate solo nei modi illustrati di seguito:

- ON TIMER DO <istruzione>
  Quando il timer diventa 0, eseguire l'istruzione.
- ON DIN ( ) DO <istruzione>
  Quando lo stato dei segnali di ingresso tra parentesi ( ) diventa come specificato, eseguire l'istruzione. Non è possibile monitorare più di quattro segnali contemporaneamente con una simile istruzione. Quando vengono specificati quattro o più punti, i punti extra superiori a quattro vengono ignorati.
- ON MOTION > = <espressione> DO <istruzione>
  Eseguire l'istruzione quando la quantità di un movimento da eseguire successivamente a questo comando è superiore al valore specificato. L'unico operando relazionale utilizzabile con MOTION è > =.
- ON MOTIONT > = <espressione> DO <istruzione>
  Eseguire l'istruzione quando il tempo necessario per un movimento da eseguire successivamente a questo comando è superiore al valore specificato. L'unico operando relazionale utilizzabile con MOTIONT è > =.
- ON REMAIN < = <espressione> DO <istruzione>
  Eseguire l'istruzione quando la quantità restante di un movimento da eseguire successivamente a questo comando è inferiore al valore specificato. L'unico operando relazionale utilizzabile con REMAIN è < =.

- ON REMAINT <= <espressione> DO <istruzione>
  Eseguire l'istruzione quando il tempo restante di un movimento da eseguire successivamente a questo comando è inferiore al valore specificato.
  L'unico operando relazionale utilizzabile con REMAINT è < =.

Nota 2: Non è possibile utilizzare il seguente comando correlato al controllo dell'attività nell'area successiva all'istruzione DO.
TASK, KILL, SWITCH

Se si utilizzano questi comandi dopo l'istruzione DO, non vengono eseguiti. Non è possibile eseguire il monitoraggio delle condizioni specificate con il comando ON dell'attività secondaria.

Nota 3: Se si è arrestato un movimento monitorato nella condizione ON MOTION, ON MOTIONT, ON REMAIN o ON REMAINT o se si è specificato il comando di velocità ridotta durante l'esecuzione di un movimento monitorato, la condizione ON viene annullata.

Esempio di programma

```
PROGRAM MAIN
    DOSAMPLE
    MOVE P
END
PROGRAM ONSAMPLE
    ON DIN (1) PAUSE DO RETURN
    MOVE A1
    MOVE A2
    MOVE A3
    WAIT MOTION > = 100
    IGNORE DIN (1)
    RETURN
END
```

Se il segnale 1 si attiva (ON) mentre si esegue un movimento, il controllo viene reinviato al programma principale al termine di quel movimento.

Precauzioni sulle istruzioni DO:

Per il comando ON ~ DO, vengono registrate le condizioni ON da monitorare e l'istruzione DO che inizia quando sono soddisfatte le condizioni.

```
PRGRAM MAIN
  SIG = 1
  ON DIN (1) DO INPUT SIG
  SUB
  IGNORE DIN(1)
  PRINT SIG
END
PROGRAM SUB
  MOVE P
  WAIT MOTION >= 100
END
```

Nel programma SCOL di cui sopra, se DIN(1) è impostato su ON durante la poligonale su P, non è possibile eseguire l'istruzione DO in quanto la variabile SIG non è definita nel programma SUB e non vi è spazio per il salvataggio della variabile come ingresso con il comando INPUT. In tal caso, la corrispondente istruzione DO può essere eseguita normalmente definendo la variabile SIG come variabile globale.

```
GLOBAL
  SIG = 0
END
PROGRAM MAIN
  SIG = 1
  ON DIN(1) DO INPUT SIG
  SUB
  IGNORE DIN(1)
  PRINT SIG
END
PROGRAM SUB
  MOVE P
  WAIT MOTION >= 100
END
```

Nell'istruzione DO, anche se si stabiliscono le condizioni di cambio attività o si esegue il comando SWITCH, non è possibile cambiare attività. Se si esegue il comando TASK o il comando KILL, si verifica un errore.

ONGAIN

Scopo	Attiva il guadagno di ciascun asse (servocomando).
Formato	ONGAIN (<intero>, <intero>, <intero>, <intero>, <intero>)
Esempi	ONGAIN (0, 0, 1, 0, 0)
Analisi e consigli	Il guadagno di ciascun asse (servocomando) del robot viene attivato. Per i dettagli sul guadagno, consultare le descrizioni del comando GAIN. Per disattivare il guadagno, utilizzare il comando OFFGAIN. Per ciascun giunto degli assi da 1 a 5, specificare i guadagni delimitando i valori dei cinque assi con una virgola. I valori vengono specificati con 1 o 0. Quando si specifica 1, il guadagno dell'asse è attivato. Quando si specifica 0, lo stato dell'asse rimane invariato. Lo stato di attivazione/disattivazione del guadagno viene modificato al termine del completamento del movimento attuale. Questo comando può essere eseguito solo quando il file SCOL.LIB è presente nell'unità RAM del controller. In questo comando, non è possibile utilizzare le costanti di sistema ON e OFF.

Esempio di programma

```
PROGRAM ONGAINSMPL
    MOVE A1
    OFFGAIN (1,1,0,1,1)
    MOVE A2
    ONGAIN (1,1,1,1,1)
    MOVE A3
END
```

Una volta spostato il robot su A1, ad eccezione dell'asse 3, i guadagni vengono disattivati. Una volta spostato il robot su A2, i guadagni di tutti gli assi vengono attivati.

OFFGAIN

Scopo	Disattiva il guadagno di ciascun asse (servocomando).
Formato	OFFGAIN (<intero>, <intero>, <intero> ,<intero>, <intero>)
Esempi	OFFGAIN (1, 1, 0, 1, 1)
Analisi e consigli	Il guadagno di ciascun asse (servocomando) del robot viene disattivato. Per i dettagli sul guadagno, consultare le descrizioni del comando GAIN. Per attivare il guadagno, utilizzare il comando ONGAIN. Per ciascun giunto degli assi da 1 a 5, specificare i guadagni delimitando i valori dei cinque assi con una virgola. I valori vengono specificati con 1 o 0. Quando si specifica 1, il guadagno dell'asse è disattivato. Quando si specifica 0, lo stato dell'asse rimane invariato. Lo stato di attivazione/disattivazione del guadagno viene modificato al termine del completamento del movimento attuale. Questo comando può essere eseguito solo quando il file SCOL.LIB è presente nell'unità RAM del controller. In questo comando, non è possibile utilizzare le costanti di sistema ON e OFF.

Esempio di programma

```
PROGRAM OFFGAINSMPL
    MOVE A1
    OFFGAIN (1,1,0,1,1)
    MOVE A2
    ONGAIN (1,1,1,1,1)
    MOVE A3
END
```

Una volta spostato il robot su A1, ad eccezione dell'asse 3, i guadagni vengono disattivati. Una volta spostato il robot su A2, i guadagni di tutti gli assi vengono attivati.

OPEN1, OPEN2, OPENI1, OPENI2

Scopo

Questi comandi aprono la mano del robot.

Formato

OPEN1
OPEN2
OPENI1
OPENI2

Esempi

OPEN1
OPENI2

Analisi e consigli

Questi comandi vengono utilizzati per aprire la mano. I numeri 1 e 2 fanno riferimento alla mano 1 e alla mano 2.

Questi comandi aprono la mano modificando lo stato del segnale di uscita che controlla la mano del robot.

Il comando OPEN dirige il robot ad aprire la mano al termine del movimento in corso.

Il comando OPENI dirige il robot ad aprire immediatamente la mano.

Si noti che questi comandi non funzionano se il file SCOL.LIB non è presente nell'unità RAM del controller.

Inoltre, tenere presente che vi è un leggero ritardo tra il momento in cui si emette il comando OPEN e quello in cui il robot apre di fatto la mano.

I comandi corrispondenti CLOSE1, CLOSE2, CLOSEI1 e CLOSEI2 vengono forniti per chiudere la mano.

Questi comandi eseguono un programma scritto nella libreria di sistema (SCOL. LIB). I dati di SCOL. LIB devono essere modificati in base alle specifiche della mano del robot.

Esempio di programma

```
PROGRAM OPENSAMPLE
    CLOSEI1
    MOVE A1
    OPEN1
    DELAY 0.5
    MOVE A2
END
```

Questo programma apre la mano al termine del movimento del robot verso il punto A1. Il robot attende 0,5 secondi per l'apertura completa della mano una volta eseguito il comando OPEN1.

```
PROGRAM OPENISMPL
    ENABLE NOWAIT
    OPENI 1
    DELAY 2
    MOVE A1
    CLOSEI 1
    DELAY 2
    MOVE A2
END
```

Qui il robot apre la mano 1 mentre si sposta verso il punto A1.

OR

Scopo

OR viene utilizzato per trovare la somma logica di due espressioni logiche.

Formato

<espressione logica> OR <espressione logica>

Esempi

IF DIN (1) OR K < = 3 THEN J = 0
WAIT DIN (5) OR TIMER==0

Analisi e consigli

L'istruzione OR calcola la somma logica delle espressioni a destra e a sinistra. Se una delle due istruzioni è vera, OR riporta TRUE.

L'istruzione OR deve essere utilizzata in un'espressione.

Esempio di programma

```
PROGRAM ORSAMPLE
    FOR K=1 TO 50
    IF DIN (1) OR K==50 THEN J=1 ELSE J=0
    PRINT TP, J, CR
    NEXT K
END
```

PAI

Scopo

PAI (scritto di norma "~" o "pi greco") è una costante di sistema con valore pari a 3,14159.

Formato

PAI

Esempi

R = D*PAI/180
D = N*PAI*2

Analisi e consigli

PAI è una costante di sistema con valore che inizia con 3,14159.... È possibile utilizzarlo per rappresentare "pi greco" quando si calcola la lunghezza di un arco, l'area di un cerchio, ecc.

Non è possibile eseguire la sostituzione in costanti di sistema che includono PAI.

Esempio di programma

```
PROGRAM MAIN
   PAISAMPLE (5, D)
   PRINT TP, D, CR
END
PROGRAM PAISAMPLE (R, D)
   D = R/PAI*180
   RETURN
END
```

Questo sottoprogramma converte il valore dell'argomento da radianti a gradi e reinvia il risultato al programma principale come argomento D.

PASS

Scopo

PASS è un interruttore di sistema utilizzato per specificare uno spostamento rapido (per l'impostazione dei parametri di spostamento rapido, consultare la pagina successiva).

Formato

PASS

Esempi

DISABLE PASS ENABLE PASS

Analisi e consigli

PASS è un interruttore di sistema utilizzato per richiamare uno spostamento rapido

Lo spostamento rapido è una modalità operativa in cui si comanda al robot di iniziare il movimento successivo prima di completare il posizionamento del movimento precedente. I tempi di commutazione dal movimento attuale a quello successivo vengono specificati con il comando PASS della variabile di sistema. Lo spostamento rapido consente di ridurre i tempi necessari al robot per spostarsi da un posto all'altro. Per ulteriori informazioni, consultare la Sezione 5.

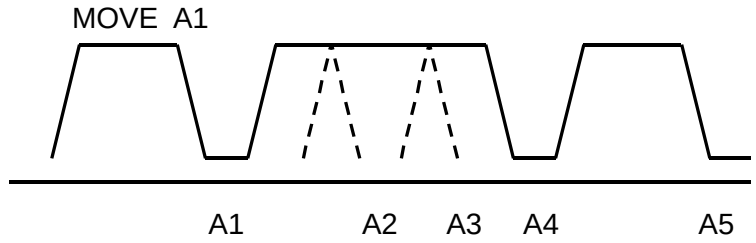
I comandi ENABLE e DISABLE vengono utilizzati per accendere e spegnere gli interruttori di sistema come PASS. Un'istruzione ENABLE PASS attiva lo spostamento rapido, mentre DISABLE PASS lo annulla.

L'impostazione iniziale per il controller è DISABLE PASS.

Esempio di programma

```
PROGRAM PASSSAMPLE
    MOVE A1
    PASS = 80
    ENABLE PASS
    MOVE A2
    MOVE A3
    DISABLE PASS
    MOVE A4
    MOVE A5
END
```

Questo programma sposta il robot con spostamento rapido da A1 ad A4 e, per lo spostamento dal punto A4 in poi, annulla lo spostamento rapido.



PASS

Scopo

PASS è una variabile di sistema utilizzata per impostare parametri di spostamento rapido (per l'impostazione dei parametri di spostamento rapido, consultare la pagina precedente).

Formato

PASS = <espressione>

Esempi

PASS = 80
PASS = PASS*0.8

Analisi e consigli

PASS è una variabile di sistema utilizzata per specificare parametri di spostamento rapido.

Lo spostamento rapido è una modalità operativa in cui si comanda al robot di iniziare il movimento successivo prima di completare il posizionamento del movimento precedente. I tempi di commutazione dal movimento attuale a quello successivo vengono specificati con il comando PASS della variabile di sistema. Il parametro di spostamento rapido viene espresso come percentuale di movimento completato da un robot rispetto all'intero movimento. Quando il movimento del robot supera quella percentuale, il movimento eseguito in quel momento e il movimento successivo si sovrappongono.

La quantità di spostamento si riferisce ad una posizione cui è diretto il robot, in cui si inizia il movimento anche se il robot effettivo non può muoversi a causa delle interferenze tra il robot ed il controller.

Per PASS è possibile specificare un valore intero da 50 a 100. I numeri inferiori a 50 vengono considerati come 50%, mentre quelli superiori a 100 vengono considerati come 100%.

La designazione <espressione> può contenere una costante, una variabile o un calcolo. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Quando si fa riferimento alla variabile di sistema PASS, è possibile riferirsi al parametro dello spostamento rapido attuale.

Lo spostamento rapido consente di ridurre i tempi necessari al robot per spostarsi da un posto all'altro. Per ulteriori informazioni, consultare la Sezione 5.

I comandi ENABLE e DISABLE vengono utilizzati per accendere e spegnere gli interruttori di sistema come PASS. Un'istruzione ENABLE PASS attiva lo spostamento rapido, mentre un'istruzione DISABLE PASS lo annulla.

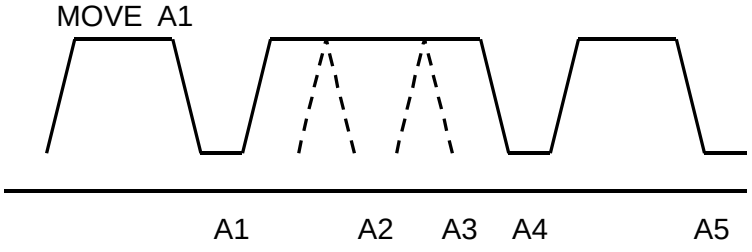
L'impostazione iniziale per il controller è DISABLE PASS.

Se si specificano i comandi WAIT e I/O mentre si esegue un comando di movimento prima di iniziare l'esecuzione del movimento PASS, tale movimento PASS potrebbe non essere eseguito.

Esempio di programma

```
PROGRAM PASSSAMPLE
    MOVE A1
    PASS = 80
    ENABLE PASS
    MOVE A2
    MOVE A3
    DISABLE PASS
    MOVE A4
    MOVE A5
END
```

Questo programma sposta il robot con spostamento rapido da A1 ad A4 e, per lo spostamento dal punto A4 in poi, annulla lo spostamento rapido.



PAUSE

Scopo

PAUSE viene utilizzato per comandare al controller di attendere che il robot termini un movimento.

Formato

ON <condizione monitoraggio> [{BREAK | PAUSE}] DO <istruzione>

Esempi

ON DIN (1) PAUSE DO SUB

Analisi e consigli

Quando si stabiliscono le condizioni di monitoraggio specificate dall'istruzione ON, PAUSE comanda al controller di attendere che il robot termini il movimento in corso prima di eseguire l'istruzione DO. Per i dettagli, fare riferimento al comando "ON".

Esempio di programma

```
PROGRAM PAUSESMPL
    ENABLE NOWAIT
    REMARK *** MAIN PROGRAM ***
    ON DIN (24) PAUSE DO STOP
    MOVE A1
    MOVE A2
    MOVE A3
    WAIT MOTION >= 100
    IGNORE DIN (24)
END
```

Qui, in caso di problemi di sistema e se il segnale di ingresso 24 si attiva (ON), il robot arresta il movimento al termine del movimento in corso in quel momento.

PAYLOAD

Scopo

PAYLOAD è una variabile di sistema utilizzata per impostare i dati per i carichi che agiscono sulla mano del robot.

Formato

PAYLOAD = {<peso>, <offset del baricentro>}

Esempi

PAYLOAD = {10, 10}
MOVE A1 WITH PAYLOAD = MOTOR

Analisi e consigli

PAYLOAD è una variabile di sistema utilizzata per impostare i dati per i carichi che agiscono sulla mano del robot.

Affinché il robot funzioni in modo efficace con vari carichi, il linguaggio SCOL consente di impostare i dati di carico che descrivono il peso e l'inerzia sulla fine della mano del robot.

I carichi che agiscono sulla mano del robot vengono impostati con la variabile di sistema PAYLOAD. Il controller utilizza questi valori per calcolare le costanti di comando per l'accelerazione e la decelerazione del robot adeguate al carico.

I dati di carico sono costituiti dai valori relativi al peso e al momento di inerzia del carico.

La specifica <perso> indica il peso del carico applicato alla punta della mano del robot in chilogrammi. L'<offset del baricentro> indica la distanza tra il baricentro del carico applicato alla punta della mano del robot e il centro dell'utensile della mano del robot in unità di millimetri.

È possibile utilizzare costanti, variabili e calcoli per le designazioni <peso> e <offset del baricentro>. Inoltre, è possibile utilizzare dati di tipo carico per la specifica {<peso>, <offset del baricentro>}.

I dati di tipo carico vengono impostati come illustrato di seguito:

Formato dati di tipo carico:
<variabile> = {<peso>, <offset del baricentro>}

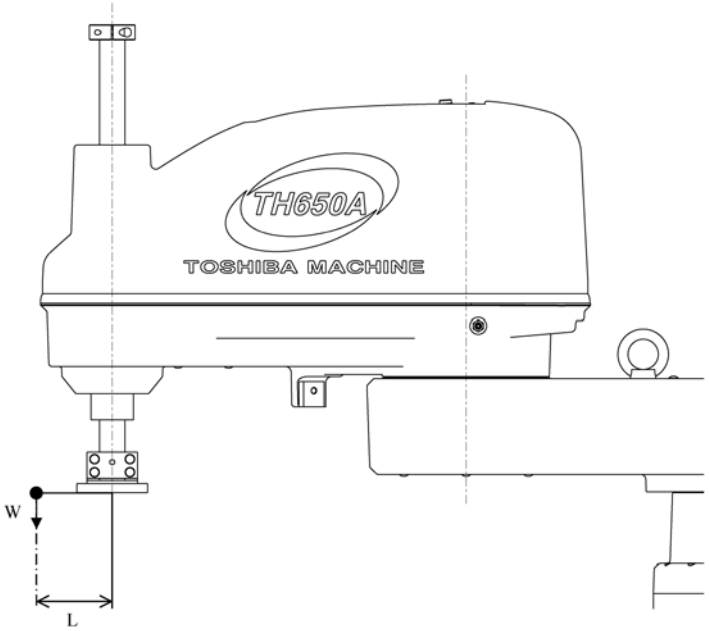
Esempio: MOTOR = {5, 10}
WORKA = HAND + MOTOR

Il comando FREELOAD è disponibile per impostare i dati di carico su zero.

Esempio di programma

```
PROGRAM PAYLOAD
    PAYLOAD = HAND
    MOVE A1
    CLOSE1
    DELAY 0.5
    MOVE A2 WITH PAYLOAD = HAND + MOTOR
    OPEN1
    DELAY 0.5
    MOVE A3
    FREELOAD
    MOVE A2
    MOVE A3
END
```

HAND viene assegnato come dato di carico prima che il robot si sposti al punto A1. Il robot afferra qualcosa al punto A1 e la somma di HAND + MOTOR viene assegnata come nuovo dato di carico.



I dati di carico vengono immessi in termini di:
Peso W [unità: kg]
Offset del baricentro L [unità: mm].

POINT

Scopo

POINT crea dati di tipo posizionale.

Formato

POINT (<espressione>, <espressione>, <espressione>, <espressione>, <espressione>, <configurazione>)

Esempi

A = POINT (100, 100, 0, 0, 0, 0)
MOVE POINT (100, 100)

Analisi e consigli

Il comando POINT crea dati di tipo posizionale.

Da sinistra a destra, le designazioni <espressione> corrispondono agli elementi X, Y, Z, C e T. Questi elementi sono specificati in unità di millimetri o gradi.
L'elemento <configurazione> deve contenere un numero intero da 0 a 2 che specifichi la configurazione del robot. La configurazione del robot è indefinita (libera) a 0, sinistrorsa a 1 e destrorsa a 2. Per specificare la configurazione del sistema, è possibile utilizzare questi numeri o le costanti di sistema FREE, LEFTY e RIGHTY. Come si può intuire, la configurazione è indefinita a CONFIG = FREE, sinistrorsa a CONFIG = LEFTY e destrorsa a CONFIG = RIGHTY. Tutti i valori inferiori a 0 immessi come <configurazione> vengono considerati 0, mentre tutti i valori superiori a 2 vengono considerati 2.

È possibile utilizzare costanti, variabili o calcoli per i termini <espressione> e <configurazione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale. Inoltre, tutti i termini <espressione> o <configurazione> omessi vengono considerati 0.
Questo comando deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM POINTSMPL
MOVE POINT (100, 100)
END
```

Questo comando sposta il robot alla posizione X = 500, Y = 500, Z = 0, A = 0, B = 0, C = 0 e T = 0.

PLCDATAR1~8 (opzione di TS3000)

Scopo	Sono variabili di sistema per la ricezione di dati dal PLC semplice integrato nel robot.
Formato	PLCDATAR1
Esempi	A = PLCDATAR1 B = PLCDATAR5
Analisi e consigli	PLCDATAR1 ~ 8 sono variabili di sistema di sola lettura. È possibile leggere i valori impostati nel PLC semplice (la funzione PLC semplice è opzionale). I valori che queste variabili di sistema possono ricevere sono 0 ~ 65535 (non è possibile utilizzare un valore negativo o un punto decimale).
Esempio di programma	PROGRAM PLCDIN A=PLCDATAR1 B=PLCDATAR2 END

PLCDATAW1~8 (opzione di TS3000)

Scopo

Sono variabili di sistema per la scrittura di dati sul PLC semplice integrato nel robot.

Formato

PLCDATAW1 = <espressione>

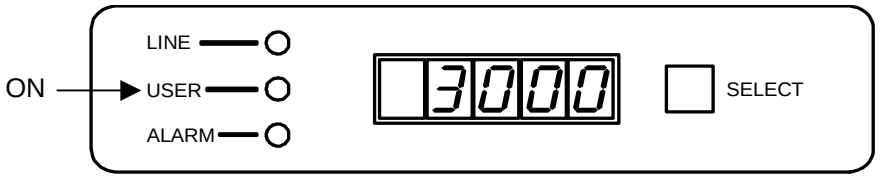
Esempi

PLCDATAW1 = 1
PLCDATAW5 = A+B

Analisi e consigli

PLCDATAW1 ~ 8 sono variabili di sistema di sola scrittura. È possibile trasferire i valori sul PLC semplice (la funzione PLC semplice è opzionale).
I valori che queste variabili di sistema possono trasferire sono 0 ~ 65535 (non è possibile utilizzare un valore negativo o un punto decimale).

Se non si seleziona la funzione PLC semplice opzionale, è valido solo PLCDATAW1 e viene visualizzato un valore scritto sul display a 7 segmenti (modalità USER) del pannello anteriore del controller.



Se si seleziona la funzione PLC semplice opzionale, è possibile utilizzare i valori di PLCDATAW1 ~ 8 nella sequenza dei programmi a discrezione dell'utente.

Esempio di programma

```
PROGRAM PLCDIN
   A=10
   PLCDATAW1=1
   PLCDATAW2=A
END
```

PRINT

Scopo

Il comando PRINT emette i dati su un canale di comunicazione specificato.

Formato

PRINT [{COM0 | COM1 | TP},] {<stringa di caratteri> | <espressione>}[, {<stringa di caratteri> | <espressione>}] ... [, CR]

Esempi

PRINT "X = ", A1. X
PRINT COM1, K, CR

Analisi e consigli

Il comando PRINT viene utilizzato per emettere i dati su un canale di comunicazione.

Specificare un (1) canale di comunicazione tra COM0, COM1 e TP. COM0 e TP sono canali utilizzati esclusivamente dal programmatore portatile. COM1 corrisponde al canale di comunicazione COM1 del controller.

Se non si specifica un canale di comunicazione nell'istruzione PRINT, i dati vengono emessi sul canale di comunicazione del programmatore portatile.

Quando si esegue un comando PRINT, i dati vengono emessi sul canale di comunicazione specificato.
I dati contenuti in una <stringa di caratteri> vengono emessi come sono.
I dati inclusi in un'<espressione> vengono emessi in blocchi solidi con una lunghezza fissa di 12 caratteri allineati a destra. Se l'espressione ha un valore reale, l'emissione è costituita da un numero reale con un massimo di quattro numeri interi e un massimo di tre cifre decimali per un totale massimo di otto spazi (contando il punto decimane).

Uno spazio di un carattere viene fornito davanti al numero per il simbolo più o meno, anche se il simbolo + viene omesso. I numeri vengono portati a destra nello spazio dei 12 caratteri e gli spazi non utilizzati vengono lasciati vuoti.

Tutti i dati sono in codice ASCII. Se si scrive CR (ritorno a capo) alla fine del comando PRINT, l'emissione da un successivo comando PRINT viene visualizzata sulla riga seguente.

Se si emettono dati sul programmatore portatile, tali dati vengono visualizzati sul programmatore portatile stesso.

Per informazioni sulla comunicazione dei dati, fare riferimento al Manuale di comunicazione.

Una volta arrestato il movimento del braccio, questo comando non può essere eseguito.

Esempio di programma

```
PROGRAM PRINTSMPL
    PRINT COM0, "*** INPUT N1, N2, N3 ***"
    INPUT COM0, N1, N2, N3
    PRINT (N1 + N2 + N3)/3, CR
END
```

Questo programma legge tre valori (N1, N2 e N3) dal programmatore portatile, trova la media e la visualizza sul programmatore portatile.

PROGRAM

Scopo

La parola PROGRAM viene utilizzata per contrassegnare l'inizio di un programma.

Formato

PROGRAM <nome programma> [(<nome variabile>, ...)]

Esempi

PROGRAM SAMPLE
PROGRAM MAIN

Analisi e consigli

PROGRAM SUB1 (N1, N2, N3)
PROGRAM viene utilizzato per contrassegnare l'inizio di un programma.

Il nome del programma viene designato da un identificatore nella specifica <nome programma>.

Il testo del programma in sé è inserito tra le istruzioni PROGRAM e END.

Quando si designa un sottoprogramma, è necessario specificare un argomento tra parentesi, qualora richiesto.
Per i dettagli sui sottoprogrammi e sugli argomenti, consultare "2.8 Programmi".

Esempio di programma

```
PROGRAM ENDSAMPLE
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Tutto quanto compreso dall'istruzione PROGRAM all'istruzione END viene eseguito come singolo programma.

PULOUT

Scopo

Il comando PULOUT indica al controller di emettere un segnale esterno come impulso.

Formato

PULOUT (<nome segnale> [,<nome segnale>)...)

Esempi

PULOUT (1, 2, –3)
PULOUT (J, J+1, J+2)

Analisi e consigli

Il comando PULOUT indica l'invio di un segnale esterno come impulsi di 0,2 secondi.

<Nome segnale> deve contenere il numero del segnale da emettere. Se il simbolo di <nome segnale> è positivo, il segnale viene modulato come OFF ~ ON ~ OFF. Se il simbolo di <nome segnale> è negativo, il segnale viene modulato come ON ~ OFF ~ ON. Se il segnale è ON e il simbolo di <nome segnale> è positivo, il segnale non viene modulato. Allo stesso modo, se il segnale è OFF e il simbolo di <nome segnale> è negativo, il segnale non viene modulato.

È possibile effettuare fino a dieci designazioni di <nome segnale> con un comando PULOUT.

È possibile utilizzare costanti, variabili o calcoli per <nome segnale>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Utilizzando un'istruzione ENABLE NOWAIT, è possibile emettere segnali a impulsi parallelamente (contemporaneamente) a tali operazioni come movimento del robot ed elaborazione di altri segnali di uscita senza impulsi.

Se è operativa un'istruzione DISABLE NOWAIT, l'elaborazione di un comando successivo al comando PULOUT non inizia finché non viene completamente emesso il segnale a impulsi.

Quando lo stesso segnale viene emesso consecutivamente dopo l'esecuzione del comando PULOUT mentre è operativa un'istruzione ENABLE NOWAIT, non si garantisce l'emissione di impulsi.

Esempio di programma

```
PROGRAM PULOUTSMPL
    DISABLE NOWAIT
    FOR K = 1 TO 16
    PULOUT (K)
    NEXT K
END
```

I segnali di uscita da 1 a 16 si attivano (ON) uno dopo l'altro in intervalli di 0,2 secondi.

RCYCLE

Scopo	RCYCLE è un'etichetta utilizzata per il ripristino cicli.
Formato	RCYCLE
Esempi	RCYCLE
Analisi e consigli	L'etichetta RCYCLE viene utilizzata per avviare l'esecuzione del programma principale dalla prima fase solo sul primo ciclo e dalla fase con l'etichetta RCYCLE sul secondo ciclo e sui cicli successivi. È possibile programmare il comando da eseguire una sola volta tra la prima fase e la fase con l'etichetta RCYCLE, ad esempio, l'inizializzazione del contatore crescente per ciclo. Con questa operazione, è possibile prelevare un pezzo di lavoro dal pallet (depallettizzazione) anche se il robot si è arrestato durante l'operazione di prelievo. Quando si utilizza il comando RCYCLE, assicurarsi di osservare le seguenti precauzioni. È possibile utilizzare il comando RCYCLE solo nel programma principale. Assicurarsi di programmare in modo che venga eseguita almeno una (1) riga con l'etichetta “RCYCLE:”. Questa funzione non può essere utilizzata per il programma multitask. Durante il ripristino dei cicli si verifica un errore. Per il comando ON ~ DO, questa funzione non può essere utilizzata durante l'esecuzione dell'istruzione DO. Durante il ripristino dei cicli si verifica un errore. Per ulteriori informazioni sul ripristino dei cicli, fare riferimento al Manuale dell'operatore.

Esempio di programma

```
PROGRAM RCYCLESMPL
    COUNT = 0
    RCYCLE:
    MOVE A1
    MOVE A2
    COUNT = COUNT + 1
    IF COUNT < = 10 THEN GOTO RCYCLE
END
```

Qui, anche se l'esecuzione viene interrotta a metà, è possibile procedere dal punto in cui si è interrotta se viene mantenuto il valore del contatore COUNT.

READY

Scopo

Il comando READY riporta il robot alla sua origine meccanica.

Formato

READY

Esempi

READY

Analisi e consigli

Il comando READY sposta ciascun asse del robot alla sua origine meccanica.

Questo comando non funziona se non si dispone del file SCOL.LIB nell'unità RAM del controller.

Nel robot a rotazione orizzontale, gli assi vengono spostati nel seguente ordine: asse 3, asse 4, asse 2, asse 1 e asse 5. Il punto zero della macchina dell'asse 3 si avvicina al limite inferiore di movimento.

Esempio di programma

```
PROGRAM READYSMPL
    READY
END
```

Questo programma riporta il robot alla sua origine meccanica.

REAL

Scopo

Il comando REAL cambia un valore numerico in un numero reale.

Formato

REAL (<espressione>)

Esempi

AK = REAL (–20)
N = REAL (K)
J1 = K – REAL (N – 28)

Analisi e consigli

Il comando REAL converte il numero o il risultato di un calcolo tra parentesi ( ) in un numero reale.

Questo comando viene utilizzato quando si desidera specificare il tipo di dati di una variabile come numero reale (e non intero).

È possibile utilizzare costanti, variabili o equazioni per l'<espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando REAL deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM REALSAMPL
    K=REAL (0.12345)
    PRINT TP, K, CR
END
```

Questo programma dichiara il tipo di dati della variabile K come reale.

REMAIN

Scopo

L'istruzione REMAIN viene utilizzata per fare riferimento alla quantità di movimento restante da completare.

Formato

REMAIN

Esempi

K = REMAIN
ON REMAIN < = 50 DO DOUT (1)

Analisi e consigli

L'istruzione REMAIN può essere utilizzata per consultare la percentuale restante di movimento del robot da completare.

La "quantità di movimento restante" è definita come la percentuale di movimento non ancora completata dal robot rispetto alla distanza totale che tale movimento deve percorrere. I calcoli della quantità di movimento restante vengono effettuati per l'asse che presenta la maggiore distanza da percorrere. REMAIN riporta un numero reale.
Combinando l'istruzione REMAIN con un comando ON, il robot invia segnali mentre il movimento è ancora in corso.

Questa istruzione deve essere utilizzata in un'espressione.

Nota: La quantità di movimento cui si fa riferimento con il comando REMAIN corrisponde alla posizione comandata al robot. Mentre il robot è in movimento, la posizione attuale del robot presenta un ritardo rispetto alla posizione comandata.
Prestare attenzione in quanto non è possibile utilizzare == per l'operatore comparativo.

Poiché la formazione della traiettoria del percorso attende che P2 si verifichi da P1, il seguente esempio diventa una sequenza infinita. Sostituendola con la frase WAIT, è possibile evitare una sequenza infinita.

```
ENABLE PASS                                    ENABLE PASS
   PASS=50                                        PASS=50
   MOVE P1                          →             MOVE P1
LOOP1:                                         WAIT REMAIN < 5
   IF REMAIN > 5 THEN GOTO LOOP1                  MOVE P2
   MOVE P2
```

Esempio di programma

```
PROGRAM REMAINSMPL
    ENABLE NOWAIT
    ON REMAIN < = 50 DO DOUT (1)
    MOVE A1
    ON REMAIN < = 20 DO DOUT (2)
    MOVE A2
END
```

Quando la mano del robot è al 50% del percorso verso A1, viene emesso il segnale 1. Quando la mano del robot è all'80% del percorso verso A2 (o, in altre parole, quando rimane il 20% del movimento da completare), viene emesso il segnale 2.

REMAINT

Scopo

L'istruzione REMAINT viene utilizzata per fare riferimento alla quantità di tempo restante prima di completare un movimento.

Formato

REMAINT

Esempi

K = REMAINT
ON REMAINT < = 1 DO DOUT (1)

Analisi e consigli

L'istruzione REMAINT può essere utilizzata per consultare il tempo restante prima di completare un determinato movimento.

Il tempo restante viene fornito come numero reale in unità di secondi. Il tempo restante diventa 0 quando il robot termina il posizionamento finale di quel determinato movimento.

Combinando l'istruzione REMAINT con un comando ON, il robot invia segnali mentre il movimento è ancora in corso. Questa istruzione deve essere utilizzata in un'espressione.

Prestare attenzione in quanto non è possibile utilizzare == per l'operatore comparativo.

Poiché la formazione della traiettoria del percorso attende che P2 si verifichi da P1, il seguente esempio diventa una sequenza infinita. Sostituendola con la frase WAIT, è possibile evitare una sequenza infinita.

```
ENABLE PASS                                          ENABLE PASS
   PASS=50                                              PASS=50
   MOVE P1                              →               MOVE P1
LOOP1:                                               WAIT REMAINT < 1
   IF REMAINT > 1 THEN GOTO LOOP1                       MOVE P2
   MOVE P2
```

Esempio di programma

```
PROGRAM REMAINTSMPL
    ENABLE NOWAIT
    ON REMAINT < = 1 DO DOUT (1)
    MOVE A1
    MOVE A2
END
```

Il segnale 1 viene emesso un secondo prima che il robot raggiunga il punto A1.

REMARK

Scopo

L'istruzione REMARK viene utilizzata per contrassegnare commenti.

Formato

REMARK [<commento>]

Esempi

REMARK *** SCOL SAMPLE ***

Analisi e consigli

I commenti vengono utilizzati nel programma per agevolare la lettura e la comprensione.

Le istruzioni REMARK stesse vengono interpretate come commenti.

I commenti non vengono eseguiti.
Il simbolo (') ha lo stesso significato dell'istruzione REMARK. Quando si scrive un commento successivo ad altri comandi, si utilizza questo simbolo ('). Tutti i caratteri successivi al simbolo (') vengono interpretati come commenti.

Esempio di programma

```
PROGRAM REMARKSMPL
    REMARK *** SAMPLE PROGRAM ***
    MOVE A1    'MOVES TO A1
END
```

“MOVES TO A1” viene programmato come commento.

RESET

Scopo

Il comando RESET viene utilizzato per ripristinare determinate condizioni del controller, quale lo stato dei segnali di uscita.

Formato

RESET <stato> [, <stato>]

Esempi

RESET DOUT
RESET RESUME

Analisi e consigli

Il comando RESET viene utilizzato per ripristinare lo stato di alcune condizioni, quali i segnali di uscita del controller. È possibile utilizzare le due seguenti istruzioni.

(1) DOUT
DOUT disattiva (OFF) tutti i segnali di uscita dell'utente.

(2) RESUME
RESUME ripristina il movimento del robot sospeso da un comando BREAK in una struttura ON. Una volta effettuato il ripristino, il movimento interrotto dal comando BREAK non può essere ripreso dal comando RESUME.

Esempio di programma

```
PROGRAM RESETSMPL
    RESET DOUT
END
```

Questo comando disattiva (OFF) tutti i segnali di uscita dell'utente.

RESTORE

Scopo	Questo comando aggiorna il valore iniziale della variabile globale.
Formato	RESTORE (“<variabile>”)
Esempi	RESTORE (“I”)
Analisi e consigli	La modifica, ecc. dei dati di posizione programmati viene ripristinata sul file. La variabile che può essere specificata è l'unica variabile globale diversa dall'array priva di valore iniziale. Se vengono ripristinate altre variabili, si verifica un errore durante la selezione.
Esempio di programma	GLOBAL A = 0 END PROGRAM STORETEST A = A + 1 RESTORE (“A”) PRINT "A=", A, CR END

RESUME

Scopo	Il comando RESUME riavvia il movimento del robot interrotto da un comando BREAK.
Formato	RESUME
Esempi	RESUME
Analisi e consigli	Il comando RESUME viene utilizzato per riavviare il movimento del robot sospeso da un comando BREAK in una struttura ON. Il movimento del robot viene riavviato (ripreso) dalla posizione in cui è stato sospeso. Pertanto, se si riavvia in modalità di interpolazione circolare, il percorso seguito dal robot potrebbe variare di molto a seconda del rapporto tra la posizione attuale, i punti di interpolazione e la destinazione. Quando vengono eseguiti più comandi BREAK (e sono ancora operativi), è possibile riprendere solo il movimento interrotto dall'ultimo comando BREAK. Se si esegue un'istruzione RESET RESUME, il movimento sospeso viene ripristinato. Il movimento sospeso viene inoltre ripristinato se si esegue il comando PAUSE con il comando ON. Questo comando è operativo solo nell'istruzione DO. Se viene eseguito in un'istruzione diversa da DO, questo comando viene ignorato.

Esempio di programma

```
PROGRAM RESUMESMPL
    ENABLE NOWAIT
    REMARK *** MAIN PROGRAM ***
    ON DIN (24) BREAK DO BREAKSUB
    MOVE A1
    MOVE A2
    MOVE A3
    WAIT MOTION > = 100
    IGNORE DIN (24)
END
```

In caso di problemi al sistema e di attivazione (ON) del segnale di ingresso 24, il controller arresta immediatamente il movimento del robot e l'esecuzione del programma passa al sottoprogramma BREAKSUB (illustrato di seguito).

```
PROGRAM BREAKSUB
    REMARK *** SUBROUTINE ***
    WAIT DIN (–24)
    RESUME
END
```

Il sottoprogramma BREAKSUB attende che si disattivi (OFF) il segnale di ingresso 24. Quando si risolve l'errore, viene ripreso il movimento interrotto.

RETURN

Scopo

L'istruzione RETURN comanda all'esecuzione del programma di tornare al programma principale da un sottoprogramma.

Formato

RETURN

Esempi

RETURN

Analisi e consigli

L'istruzione RETURN viene utilizzata per tornare al programma principale da un sottoprogramma.

Anche se si dimentica di includere un'istruzione RETURN nel sottoprogramma, il controller riporta il controllo al programma principale tramite l'esecuzione dell'istruzione END.

Se si inserisce un'istruzione RETURN nel programma principale, si verifica un errore.

Esempio di programma

```
PROGRAM MAIN
    RETURNSAMPLE (5, K)
    PRINT TP, K, CR
END
PROGRAM RETURNSMPL (N, K)
    K = N * N
    RETURN
END
```

Questo programma prende l'argomento N, lo moltiplica per se stesso e reinvia il risultato al programma principale come argomento K.

RIGHTY

Scopo

RIGHTY è una costante di sistema utilizzata per modificare la configurazione del robot su un sistema destrorso.

Formato

RIGHTY

Esempi

CONFIG = RIGHTY
MOVE A1 WITH CONFIG = RIGHTY

Analisi e consigli

RIGHTY viene utilizzato unitamente a CONFIG per impostare la configurazione del robot su un sistema destrorso.

Come costante di sistema, RIGHTY ha un valore pari a 2. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 2. Tuttavia, ciò potrebbe rendere il programma inutilmente complicato.

Non è possibile eseguire la sostituzione in costanti di sistema che includono RIGHTY.

Per i robot a coordinate cartesiane, la designazione della configurazione del robot viene ignorata.

Per informazioni sulla configurazione del robot, vedere il comando CONFIG.

Esempio di programma

```
PROGRAM RIGHTYSMPL
    CONFIG = RIGHTY
    MOVE A1
    MOVE A2
END
```

Questo programma imposta la configurazione del robot su un sistema destrorso prima di spostare il robot sulla sua posizione.

SAVEEND

Scopo

Specificare la variabile globale salvata nella memoria di bac kup quando si arresta il robot.

Formato

SAVEEND:

Esempi

SAVEEND:

Analisi e consigli

Quando si arresta il robot, la variabile specificata viene salvata nella memoria di backup.

Definire le variabili che devono essere salvate tra “GLOBA L” e "SAVEEND:".

I dati salvati vengo ripristinati la prossima volta che si accende il controller.

Le variabili possono essere salvate fino a 10 kByte.

La variabile globale specificata viene salvata dalle seguenti condizioni di arresto.

- STOP ciclo
- Interruzione alimentazione
- BREAK
- STOP emergenza
- Servocomando OFF

Inoltre, i dati salvati vengono cancellati in base ai seguent i tempi di funzionamento.

- Selezione programma
- Ripristino programma
- Quando si spegne la principale fonte di alimentazione durante il movimento (nello stato di “esecuzione”).

< Avviso>

- **Per salvare i dati occorrono circa 100 msec/1 kbyte.**

- Spegnere la fonte di alimentazione dopo un secondo dall'arresto del robot, se i dati salvati sono di grandi dimensioni.

- Vengono salvate anche le variabili di sistema (tuttavia, le variabili TIMER e TID diverse dal programma principale vengono escluse).

- Per quanto concerne il tipo di variabile che può essere descritto in un blocco globale, vi è un limite indicato nel manuale "Capitolo del linguaggio" e "2.8.5 Definizione della variabile globale". La variabile che non può essere descritta in un blocco globale con questo limite non può essere salvata da questa funzione.

Esempio di programma

```
GLOBAL
    I=0          “I” viene salvato durante “STOP”.
SAVEEND:
    J=0          “J” non viene salvato durante “”STOP”.
END

PROGRAM MAIN
    FOR K=1 TO 10
        I=I+1
        J=J+1
    NEXT
    PRINT “I=",I,“J=",J,CR
END
```

Elaborazione ripristino flusso dati con alimentazione OFF - ON

1. Avviare il programma "MAIN" in modalità di funzionamento a cicli.
2. "I=10,J=10" viene visualizzato sul programmatore portatile.
3. Il programma si arresta con il comando “END”.
   ⇒"I=10" viene salvato nella memoria di backup.
4. Spegnere → sul controller.
5. Avviare il programma "MAIN".
6. "I=20,J=10" viene visualizzato sul programmatore portatile.

SEGMENT

Scopo

SEGMENT è una costante di sistema utilizzata per fare riferimento alla modalità operativa del sistema.

Formato

SEGMENT

Esempi

IF MODE = = SEGMENT THEN RETURN

Analisi e consigli

SEGMENT viene utilizzato insieme al comando MODE per fare riferimento alla modalità operativa del sistema. Con MODE = = SEGMENT, il sistema funziona in modalità di funzionamento a segmenti.

Come costante di sistema, SEGMENT ha un valore pari a 2. Se lo si desidera, si può utilizzare nel programma come costante con valore pari a 2. Tuttavia, ciò potrebbe rendere il programma inutilmente difficile da capire.

Non è possibile eseguire la sostituzione in costanti di sistema.

È possibile utilizzare il comando di monitoraggio MODE MOTION per specificare il funzionamento a segmenti.

Per informazioni sulle modalità operative, fare riferimento al comando MODE.

Esempio di programma

```
PROGRAM MAIN
    SEGMENTSAMPLE
END
PROGRAM SEGMENTSPL
    IF MODE = = SEGMENT THEN RETURN
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Se viene impostata la modalità di funzionamento a segmenti, l'esecuzione del programma torna al programma principale senza effettuare l'esecuzione.

SETGAIN

Scopo

Il comando SETGAIN viene utilizzato per specificare se il guadagno (per il servocomando) deve essere ON o OFF per ciascun asse.

Formato

SETGAIN (<intero>, <intero>, <intero>, <intero>, <intero>)

Esempi

SETGAIN (0, 0, 1, 0, 0)

Analisi e consigli

Il comando SETGAIN viene utilizzato per specificare se il guadagno (per il servocomando) di ciascun asse deve essere ON o OFF.

Il comando SETGAIN diventa operativo al completamento del movimento del robot eseguito in quel momento.
Per ulteriori informazioni sui guadagni, fare riferimento al comando "GAIN".
Il comando SETGAIN non funziona se non si dispone del file SCOL.LIB nell'unità RAM del controller. Con questo comando, non è possibile attivare/disattivare una costante di sistema.

Due comandi ONGAIN e OFFGAIN vengono forniti nel file della libreria per attivare e disattivare il guadagno. Utilizzare questi due comandi per attivare e disattivare il guadagno.

Esempio di programma

```
PROGRAM SETGAINSMPL
    MOVE A1
    SETGAIN (0, 0, 1, 0, 0)
    MOVE A2
    SETGAIN (1, 1, 1, 1, 1)
    MOVE A3
END
```

Questo programma disattiva tutti i guadagni ad eccezione di quello dell'asse 3 quando il robot raggiunge il punto A1. Quindi, il programma attiva tutti i guadagni quando raggiunge il punto A2.

SGN

Scopo

La funzione SGN riporta il segno di un valore numerico.

Formato

SGN (<expression>)

Esempi

S = SGN (–20.345)
N = ABS (K) * SGN (L)

Analisi e consigli

La funzione SGN riposta il segno dell'<espressione> tra parentesi.

SGN riporta 1 se l'<espressione> è positiva, –1 se negativa e 0 se 0.

È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Il comando SGN deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    SGNSAMPLE (5, 3, K)
    PRINT TP, K, CR
END
PROGRAM SGNSAMPLE (K1, K2, K)
    K = SGN (K1 – K2)
    RETURN
END
```

Questo sottoprogramma prende due argomenti K1 e K2 e, come argomento K, riporta 1 se K1 è superiore a K2, 0 se è uguale e –1 se è inferiore al programma principale.

SIN

Scopo

Questa funzione riporta il seno di un valore inserito.

Formato

SIN (<expression>)

Esempi

K = SIN (60)
J1 = 1 – SIN (180 – D)

Analisi e consigli

Questa funzione riporta il seno del valore tra parentesi ( ). I calcoli vengono gestiti in unità di gradi.

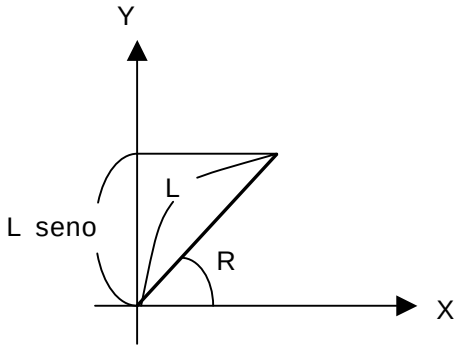
È possibile inserire una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale.

Questo comando deve essere utilizzato in un'equazione.

Esempio di programma

```
PROGRAM MAIN
    SINSAMPLE (2.0, 60.0, Y)
    PRINT TP, Y, CR
END
PROGRAM SINSAMPLE (L, R, Y)
    LOOP:
        IF R > 180 THEN R = R – 360
        IF R < –180 THEN R = R + 360
        IF R > 180 OR R < –180 THEN GOTO LOOP
        Y = L * SIN (R)
        RETURN
END
```

Dato (come argomenti) un segmento di linea con una lunghezza L e formando un angolo R con l'asse X, questo programma trova la lunghezza della componente Y del segmento di linea L e la invia al programma principale come argomento Y.



SMOOTH

Scopo

Questa funzione indica un movimento uniforme.

(Per l'impostazione del parametro di movimento uniforme, consultare le descrizioni di seguito.)

Formato

SMOOTH

Esempi

DISABLE SMOOTH
ENABLE SMOOTH

Analisi e consigli

È possibile specificare il movimento uniforme.
Il comando di movimento specificato come SMOOTH consente al robot di spostarsi sulla posizione target senza decelerare. Con il successivo comando di movimento, il robot inizia a spostarsi senza accelerare indipendentemente dalla presenza o assenza di SMOOTH.

Per accendere e spegnere l'interruttore del sistema, utilizzare i comandi ENABLE e DISABLE.

Con il comando ENABLE SMOOTH, si avvia il movimento uniforme.
Con il comando DISABLE SMOOTH, si annulla il movimento uniforme.

L'impostazione iniziale è DISABLE SMOOTH.

La funzione uniforme è operativa solo per i comandi di interpolazione (MOVES, MOVEC).

Durante la modalità ENABLE SMOOTH, COARSE viene automaticamente selezionato per la precisione di posizionamento.

Se la velocità durante il collegamento uniforme non raggiunge la velocità specificata per quel comando di movimento, il robot accelera (o decelera) alla velocità specificata alla massima accelerazione.

Se si specifica il comando di movimento uniforme mentre la distanza di spostamento dell'asse C del comando di movimento uniforme non è sufficiente rispetto alle distanze di spostamento X, Y e Z, si verifica un errore.

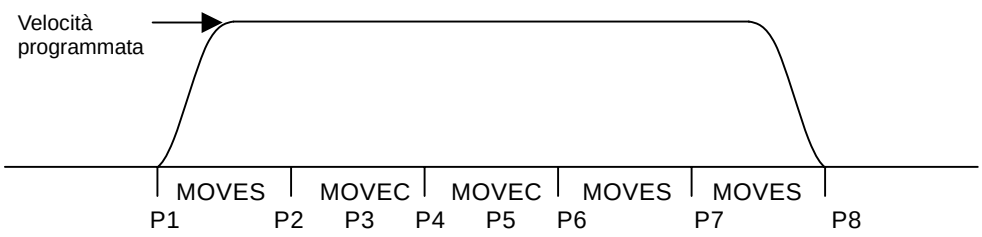
Quando si specifica come SMOOTH il comando di movimento che include lo spostamento dell'asse T, si verifica un errore.

Se il comando di movimento DISABLE SMOOTH non presenta una distanza di spostamento sufficiente per decelerazione e arresto, non si garantisce il controllo della velocità durante decelerazione e arresto.
La funzione PASS non può essere utilizzata unitamente alla funzione SMOOTH. Inoltre, il movimento PASS non può collegato al movimento SMOOTH (consultare l'esempio di programmi SMPL04 e SMPL05).

Esempio di programma

```
PROGRAM SMPL01
  MOVE   P01
  ENABLE SMOOTH
  MOVES P02
  MOVEC P03 P04
  MOVEC P05 P06
  MOVES P07
  DISABLE SMOOTH
  MOVES P08
END
```

Collegando i punti P02, P04, P06 e P07 con movimento uniforme, il robot decelera e si arresta al punto P08.



(Esempio errore 1)

```
PROGRAM SMPL02
  PASS=50
  ENABLE SMOOTH
  MOVES P01
  MOVES P02
  MOVES P03
  ENABLE PASS      ← Viene generato un allarme.
                     2–039 “Comando PASS vietato”
  MOVES P04
  DISABLE SMOOTH
  MOVES P05
  MOVES P06
  MOVES P07
  DISABLE PASS
  MOVES P08
END
```

Se si specifica ENABLE PASS nella modalità ENABLE SMOOTH, si verifica un allarme e il robot arresta il movimento.

(Esempio errore 2)

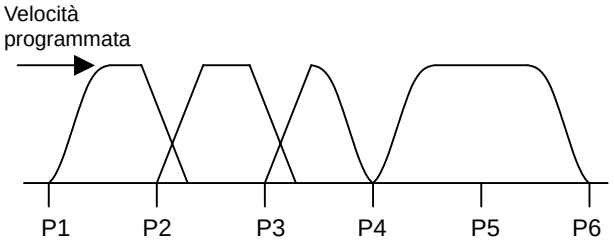
```
PROGRAM SMPL03
  PASS=50
  ENABLE PASS
  MOVES P01
  MOVES P02
  MOVES P03
  ENABLE SMOOTH    ← Viene generato un allarme.
                     2–040 “Comando SMOOTH
                     vietato”
  MOVES P04
  DISABLE PASS
  MOVES P05
  MOVES P06
  MOVES P07
  DISABLE SMOOTH
```

```
   MOVES P08
END
```

Se si specifica ENABLE SMOOTH nella modalità ENABLE SMOOTH, si verifica un allarme e il robot arresta il movimento.

(Cambio da PASS a SMOOTH)

```
PROGRAM SMPL04
 PASS=50
 MOVE P01
 ENABLE PASS
 MOVES P02
 MOVES P03
 DISABLE PASS
 ENABLE SMOOTH
 MOVES P04
 MOVES P05
 DISABLE SMOOTH
 MOVES P06
END
```

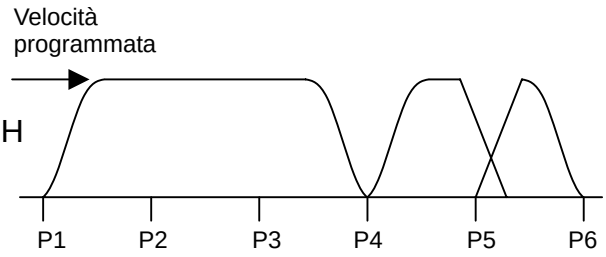


Quando il robot rallenta e si arresta durante DISABLE PASS, la designazione SMOOTH per il punto P04 diventa non valida.

Ovvero, i punti P02 e P03 vengono collegati con spostamento rapido, il punto P04 con decelerazione e arresto e il punto P05 con movimento uniforme. Al punto P06, il robot rallenta e si arresta. Quando ciò si verifica, viene generato l'allarme 1–018 "Collegamento uniforme non valido" al punto P04.

(Cambio da SMOOTH a PASS)

```
PROGRAM SMPL05
 PASS=50
 MOVE P01
 ENABLE SMOOTH
 MOVES P02
 MOVES P03
 DISABLE SMOOTH
 ENABLE PASS
 MOVES P04
 MOVES P05
 DISABLE PASS
 MOVES P06
END
```



Quando il robot rallenta e si arresta durante DISABLE SMOOTH, la designazione PASS per il punto P04 diventa non valida.

Ovvero, i punti P02 e P03 vengono collegati con spostamento uniforme, il punto P04 con decelerazione e arresto e il punto P05 con movimento rapido. Al punto P06, il robot rallenta e si arresta. Quando ciò si verifica, viene generato l'allarme 1–017 "Collegamento passaggio non valido" al punto P4.

Attenzione: La vita utile della macchina può essere influenzata da alcune condizioni operative di questa funzione. Si prega di consultarci prima dell'uso.

SPEED

Scopo	SPEED è una variabile di sistema utilizzata per specificare la velocità di movimento del robot.
Formato	SPEED = <espressione>
Esempi	SPEED = 50 MOVE A1 WITH SPEED = SPEED * 0.8
Analisi e consigli	SPEED è una variabile di sistema utilizzata per specificare la velocità di movimento del robot. Viene espressa in termini di percentuale della velocità massima (consentita dal controller). SPEED viene specificato con un numero intero positivo. Quando si specifica un valore numerico pari a 0 o meno, la specifica viene considerata 1. Quando si specifica un valore di 100 o più, la velocità di movimento è soppressa alla massima velocità indicata nel sistema. È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale. Facendo riferimento a questa variabile di sistema, è possibile trovare l'impostazione di velocità operativa in quel momento. L'impostazione iniziale per SPEED è 100%.

Esempio di programma

```
PROGRAM SPEEDSMPL
    SPEED = 50
    MOVE A1
    MOVE A2
    MOVE A3 WITH SPEED = 100
    MOVE A4
END
```

Qui, il robot si sposta su tutti i punti al 50% della piena velocità ad eccezione del punto A3, in cui il robot si sposta a piena velocità.

SQRT

Scopo

La funzione SQRT riporta la radice quadrata di un dato numero.

Formato

SQRT (<espressione>)
J1 = SQRT (L1 ^ 2 + L2 ^ 2)

Esempi

K = SQRT (60)
J1 = 1 – SIN (180 – D)

Analisi e consigli

La funzione SQRT riporta la radice quadrata del valore tra parentesi ( ).

È possibile utilizzare costanti, variabili o equazioni per il termine <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Quando il valore di <espressione> è negativo, si verifica un errore.

Questa funzione deve essere utilizzata in un'espressione.

Esempio di programma

```
PROGRAM MAIN
    SQRTSAMPLE (3, 5, L)
    PRINT TP, L, CR
END
PROGRAM SQRTSAMPLE (X, Y, L)
    L = SQRT (X ^ 2 + Y ^ 2)
    RETURN
END
```

Questo sottoprogramma prende gli argomenti X e Y (la lunghezza dei due lati perpendicolari di un triangolo rettangolo), trova la lunghezza dell'ipotenusa di quel triangolo e riporta quella lunghezza come argomento L sul programma principale.

STEP

Scopo

STEP viene utilizzato unitamente al comando FOR per specificare come ripetere una sequenza.

Formato

FOR <variabile> = <espressione 1> TO <espressione 2> [STEP <espressione 3>]

Esempi

FOR K = 1 TO 4 STEP 2
FOR N = K1 TO K1 + K2 STEP K3

Analisi e consigli

L'istruzione STEP viene utilizzata in strutture FOR ~ TO per comandare la ripetizione di una parte del programma per un determinato numero di volte.

La parte del programma da ripetere è contenuta in un blocco che inizia con il comando FOR e termina con il comando NEXT. Il blocco continua a ripetersi finché non si soddisfa la condizione specificata per l'istruzione FOR.

Quando si esegue un'istruzione FOR, il valore di <espressione 1> viene sostituito nella <variabile>. Quando si esegue l'istruzione NEXT, il valore di <espressione 3> specificato dall'istruzione STEP viene aggiunto nella <variabile>. Se a questo punto il valore della <variabile> diventa superiore a quello dell'<espressione 2>, l'esecuzione del programma passa all'istruzione successiva al comando NEXT. Se <variabile> non è superiore ad <espressione 2>, l'esecuzione del programma si ramifica (ritorna) all'istruzione successiva all'istruzione FOR.

Il valore di <espressione 3> alla prima sequenza viene mantenuto operativo fino all'ultima sequenza. Pertanto, anche se si modifica questo valore mentre si esegue la sequenza, il numero di volte in cui si ripete la sequenza non cambia.

Se il valore di <espressione 3> è pari a 1, si può omettere l'istruzione STEP (e tutto ciò che viene dopo) dalla struttura ON – DO.

È possibile utilizzare una costante, una variabile o un calcolo per <espressione 3>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Per ulteriori informazioni sulla "sequenza" del programma, fare riferimento al comando FOR.

Esempio di programma

```
PROGRAM STEPSAMPLE
    FOR K = 1 TO 100 STEP 2
        MOVE A1 WITH SPEED   K
        MOVE A2
    NEXT K
END
```

Questo programma sposta il robot cinquanta volte avanti e indietro tra i punti A1 e A2. In ogni spostamento, il robot aumenta la velocità del 2%.

STOP

Scopo

Il comando STOP viene utilizzato per arrestare l'esecuzione del programma.

Formato

STOP

Esempi

STOP

Analisi e consigli

Il programma arresta l'esecuzione in presenza di un comando STOP indipendentemente dalla modalità operativa del sistema in quel momento.

Non vi è alcun modo di riavviare un programma arrestato. è necessario, pertanto, riavviare il programma dall'inizio.

Quando si riavvia il programma, il movimento del robot viene ripristinato dalla fase successiva.

Esempio di programma

```
PROGRAM STOPSAMPLE
    ENABLE NOWAIT
    ON DIN (10) DO STOP
    MOVE A1
    MOVE A2
    MOVE A3
END
```

Questo programma arresta la sua esecuzione se si attiva (ON) il segnale di ingresso 10 con il robot in movimento.

SWITCH

Scopo

Questo comando cambia compulsivamente attività nella gestione multitask.

Formato

SWITCH

Esempi

SWITCH

Analisi e consigli

Se la singola attività è operativa o la variabile di sistema “SWITCH” è impostata su “DISABLE” e il comando STEP è operativo, questo comando non è valido.

Esempio di programma

```
GLOBAL
  MAXTASK=2
END
PROGRAM MAIN
  ID = 0
  ID = TASK("SUB1")
  LOOP:
  IF DIN(1) THEN SWITCH
  PRINT " TASK1 ",CR
  GOTO LOOP
END
PROGRAM SUB1
  ENABLE NOWAIT
  LOOP1:
  IF DIN(1) THEN SWITCH
  PRINT " TASK2",CR
  GOTO LOOP1
END
```

Quando le due (2) attività utilizzano lo stesso I/O, se DIN(1) è OFF, entrambe le attività occupano I/O. Quando DIN(1) è impostato su ON, l'attività viene cambiata compulsivamente per evitare l'occupazione unilaterale di I/O.

SWITCH

Scopo

Questo comando proibisce o consente il cambio attività.

Formato

SWITCH

Esempi

ENABLE SWITCH
DISABLE SWITCH

Analisi e consigli

Quando la variabile di sistema passa a DISABLE, anche se si esegue il comando SWITCH o si soddisfano le condizioni di cambio attività predeterminate nel sistema, questa attività non viene cambiata. Il valore iniziale di questa variabile di sistema è ENABLE.

Esempio di programma

```
GLOBAL
  MAXTASK=2
END
PROGRAM MAIN
  ID = 0
  ID = TASK("SUB1")
  LOOP:
  IF DIN(1) THEN DISABLE SWITCH
  ELSE ENABLE SWITCH
  MOVEA 1,90
  MOVEA 1,–90
  GOTO LOOP
END
PROGRAM SUB1
  ENABLE NOWAIT
  DOUT (–1,–2)
  TIMER=1
  WAIT TIMER==0
```

All'inizio della sequenza dell'attività principale, il cambio attività ENABLE o DISABLE viene selezionato con l'ingresso di DIN (1). Se viene disabilitato il cambio attività, l'attività secondaria non viene eseguita e DOUT rimane invariato.

```
  DOUT (1)
  TIMER=1
  WAIT TIMER==0
  DOUT (2)
  TIMER=1
  WAIT TIMER==0
END
```

TAN

Scopo

Questa funzione riporta la tangente del valore inserito.

Formato

TAN (<espressione>)

Esempi

K = TAN (60)
J1 = 1 – TAN (180 – D)

Analisi e consigli

Questa funzione riporta la tangente del valore tra parentesi ( ).

I calcoli vengono gestiti in unità di gradi.

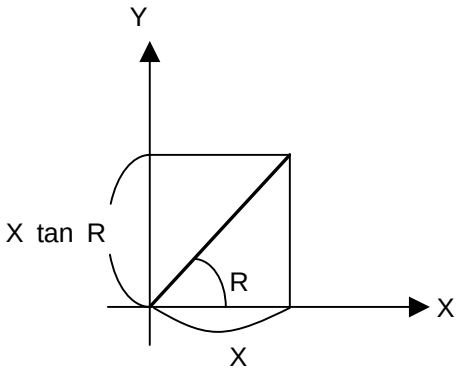
È possibile inserire una costante, una variabile o un calcolo per il termine <espressione>. Tuttavia, non è possibile inserire dati di tipo vettoriale.

Questa funzione deve essere utilizzata in un'equazione.

Esempio di programma

```
PROGRAM MAIN
    TANSAMPLE (2, 60, Y)
    PRINT TP, Y, CR
END
PROGRAM TANSAMPLE (X, R, Y)
    LOOP:
        IF R > 180 THEN R = R – 360
        IF R < – 180 THEN R = R + 360
        IF R > 180 OR R < – 180 THEN GOTO LOOP
        Y = X * TAN (R)
        RETURN
END
```

Questo programma prende la lunghezza della componente X del segmento e l'angolo formato da R con l'asse X (come argomento R). Il programma trova, quindi, la lunghezza della componente Y del segmento e riporta il risultato come argomento Y sul programma principale.



TASK

Scopo

Questo comando esegue il multitask.

Formato

TASK (“<nome programma>”)

Esempi

ID = TASK (“SUB”)

Analisi e consigli

Un programma tra parentesi inizia come attività.
Il valore riportato è il numero caratteristico dell'attività secondaria iniziata (ID di attività). L'ID di attività viene utilizzato come argomento durante l'arresto dell'attività.

Quando si utilizza il comando I/O nell'attività secondaria, dichiarare ENABLE NOWAIT nell'attività secondaria.

Esempio di programma

```
GLOBAL
    ID=0
    MAXTASK=2
END
PROGRAM MAIN
    IF ID ==0 THEN ID= TASK (“SUB”)
    LOOP:
    MOVEA 1,90
    MOVEA 1,–90
END
PROGRAM SUB
    ENABLE NOWAIT
    WAIT DIN (1)
    DOUT (1)
    WAIT DIN (–1)
    DOUT (–1)
END
```

Il sottoprogramma inizia come attività.
Il segnale, asincrono rispetto al movimento nell'attività principale, viene emesso in risposta all'ingresso del segnale.

THEN

Scopo

L'istruzione THEN viene utilizzata unitamente ad un'istruzione IF per la valutazione delle condizioni.

Formato

IF <espressione logica> THEN <istruzione> [ELSE <istruzione>]

Esempi

IF DIN (1) THEN K = K + 1 ELSE K = 0

Analisi e consigli

Se vengono soddisfatte le condizioni dell'<espressione logica> successiva a IF, viene eseguita l'<istruzione> successiva a THEN. Se non vengono soddisfatte le condizioni, viene eseguita l'istruzione successiva a ELSE.

Un'istruzione ELSE non è obbligatoria in una struttura IF ~ THEN. Se non si soddisfa una condizione IF e non vi è alcuna istruzione ELSE, viene eseguito il comando successivo all'istruzione IF.

L'<istruzione> successiva all'istruzione THEN o ELSE potrebbe non contenere PROGRAM, END, IF, FOR, NEXT o WAIT.

Per ulteriori informazioni sulla valutazione delle condizioni, fare riferimento al comando IF.

Esempio di programma

```
PROGRAM THENSAMPLE
    IF DIN (1) THEN K = 1 ELSE K = 0
    MOVE A1
    MOVE A2
    MOVE A3
    PRINT "K=", K, CR
END
```

Se il segnale di ingresso 1 è ON, K equivale a 1.
Se il segnale di ingresso 1 è OFF, K equivale a 0.

TID

Scopo

Questo comando si riferisce all'ID di attività (numero) della propria attività.

Formato

TID

Esempi

MAINID = TID

Analisi e consigli

Questa variabile di sistema è caratteristica di ciascuna attività avviata con il comando TASK.

Non è possibile eseguire la scrittura di questa variabile.

Esempio di programma

```
GLOBAL
    MAXTASK=2
END
PROGRAM MAIN
    TASK(“SUB”)
    LOOP:
    SWITCH
    PRINT “MAINID=”,TID,CR
    GOTO LOOP
END
PROGRAM SUB
    ENABLE NOWAIT
    SWITCH
    PRINT “SUBID=”,TID,CR
END
```

TID viene espresso nell'attività principale e in quella secondaria. I valori differiscono.

TIMER

Scopo

La variabile di sistema TIMER è un timer che può essere modificato dal sistema.

Formato

TIMER

Esempi

```
TIMER = 20
WAIT TIMER = = 0
```

Analisi e consigli

TIMER è un timer utilizzabile all'interno dei programmi SCOL. La variabile di sistema TIMER può essere impostata in unità di secondi. Quando si specifica un valore di 0 o meno, il timer non funziona correttamente.

Il valore della variabile di sistema TIMER esegue il conto alla rovescia nel momento in cui viene impostato. Quando arriva a 0, non è più possibile eseguire il conto alla rovescia.

Facendo riferimento a TIMER nel programma, si può osservare il tempo restante in quel momento nel programma.

Esempio di programma

```
PROGRAM TIMERSMPL
    TIMER = 5
    WAIT TIMER = = 0
    PRINT TIMER, CR
END
```

Il programma attende che il valore impostato diventi 0 una volta impostato su 5 secondi.

```
PROGRAM TIMERSMPL2
    FOR J = 1 TO 1000
        PRINT "*", CR
        TIMER = 1
        WAIT TIMER = = 0
    NEXT J
END
```

Questo programma visualizza un * sul programmatore portatile ogni secondo per 1000 secondi.

TO

Scopo

TO viene utilizzato unitamente al comando FOR per specificare la ripetizione di una parte del programma *un* determinato numero di volte.

Formato

FOR <variabile> = <espressione 1> TO <espressione 2> [STEP <espressione 3>]

Esempi

FOR K = 1 TO 4
FOR N = K1 TO K1 + K2 STEP K3

Analisi e consigli

L'istruzione TO viene utilizzata in strutture FOR TO per comandare la ripetizione di una parte del programma per un determinato numero di volte.

La parte del programma da ripetere è contenuta in un blocco che inizia con il comando FOR e termina con il comando NEXT. Il blocco continua a ripetersi finché non si soddisfa la condizione specificata per l'istruzione FOR.

Quando si esegue un'istruzione FOR, il valore di <espressione 1> viene sostituito nella <variabile>. Quando si esegue l'istruzione NEXT, il valore di <espressione 3> specificato dall'istruzione STEP viene aggiunto nella <variabile>. Se a questo punto il valore della <variabile> diventa superiore a quello dell'<espressione 2>, l'esecuzione del programma passa all'istruzione successiva al comando NEXT. Se <variabile> non è superiore ad <espressione 2>, l'esecuzione del programma si ramifica all'istruzione successiva all'istruzione FOR.

I valori di <espressione 1>, <espressione 2> ed <espressione 3> utilizzati per la struttura FOR sono quelli operativi al momento della prima esecuzione dell'istruzione FOR. Pertanto, anche se si modificano questi valori mentre si esegue la sequenza, il numero di volte in cui si ripete la sequenza non cambia.

È possibile utilizzare costanti, variabili e calcoli per <espressione 1> ed <espressione 2>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Per ulteriori informazioni sulla "sequenza" del programma, fare riferimento al comando FOR.

Esempio di programma

```
PROGRAM TOSAMPLE
    FOR K = 1 TO 100
        MOVE A1
        MOVE A2
    NEXT K
END
```

Questo programma sposta il robot 100 volte avanti e indietro tra i punti A1 e A2.

TOOL

Scopo

TOOL è una variabile di sistema utilizzata per specificare il sistema di coordinate dell'utensile.

Formato

TOOL

Esempi

TOOL = TRANS (0, 0, 0, 0)
TOOL1 = TOOL
MOVE A1 WITH TOOL = TOOL + TRANS (,, 100)

Analisi e consigli

TOOL è una variabile di sistema utilizzata per specificare il sistema di coordinate dell'utensile.
È possibile gestire la variabile di sistema TOOL come normali dati di tipo coordinate.
Facendo riferimento a TOOL, si possono trovare i valori (la posizione) del sistema di coordinate dell'utensile attuale.

È possibile designare direttamente i valori dell'offset dell'utensile con uno dei due seguenti metodi:

TOOL = TRANS (X, Y, Z, C)
TOOL = {X, Y, Z, C}
(Per conoscere il tipo di dati a disposizione, utilizzare sempre il comando TRANS.)

X, Y, Z, C: X, Y, Z e C sono valori di coordinate rappresentati da numeri reali (unità: mm o gradi).

Il sistema di coordinate TOOL viene creato "scorrendo" una distanza di X, Y e Z lungo i rispettivi assi del sistema di coordinate dell'interfaccia meccanica, quindi ruotando il nuovo asse Z di una quantità C.

TOOL deve essere utilizzato in un'espressione.

Si noti che se si modifica l'offset dell'utensile in un programma, potrebbe verificarsi un disallineamento tra le posizioni programmate e il movimento del robot.

Esempio di programma

```
PROGRAM TOOLSAMPLE
    MOVE A1
    MOVE A2
    TOOL = TOOL + TRANS (,, 500)
    MOVE A1
    MOVE A2
    TOOL = TRANS ( )
END
```

Questo programma sposta il sistema di coordinate dell'utensile di 500 mm lungo l'asse Z e, successivamente, il robot si sposta in un punto superiore di 500 mm rispetto alla posizione programmata.

TORQUE

Scopo

Il comando TORQUE viene utilizzato per specificare il valore limite di coppia per ciascun asse.

Formato

TORQUE = {<espressione>, <espressione>, <espressione>, <espressione>, <espressione>}

Esempi

TORQUE = {300, 300, 100, 300, 300}
MOVE A1 WITH TORQUE = {T, T, T, T, T}

Analisi e consigli

Il comando TORQUE viene utilizzato per specificare il valore limite di coppia per ciascun asse.

TORQUE è una variabile di sistema con cinque elementi di dati corrispondenti ai cinque assi. Il comando TORQUE specifica i valori limite di coppia per i cinque assi tra parentesi { } successive all'istruzione TORQUE. Tali specifiche di dati sono separate da virgole con la prima specifica corrispondente all'asse 1, la seconda all'asse 2, e così via.

Il valore del controllo di coppia viene espresso come percentuale della coppia nominale del corrispondente motore. Quando non vi è alcun limite di coppia, un motore può erogare fino al 100% della sua coppia nominale. Se i valori del controllo di coppia sono abbreviate, il controller considera 0 tutte le coppie non specificate.

Quando si specifica un valore inferiore a 100, viene ridotto un livello di riconoscimento di errore uscita e blocco motore.

È possibile utilizzare costanti, variabili o calcoli per i termini <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale.

Quando si specifica il valore limite della coppia non superiore a 0, il valore viene considerato 0.

Si noti che se l'impostazione della coppia è eccessivamente bassa, il robot non è in grado di muoversi correttamente e si verifica un errore.

Esempio di programma

```
PROGRAM TORQUESMPL
    MOVE A1
    TORQUE = {100, 100, 100, 100, 100}
    MOVE A2
    OPEN
    DELAY 0.5
    MOVE A1
    TORQUE = {200, 200, 200, 200, 200}
    READY
END
```

Questo programma limita la coppia dell'asse Z (asse 3) al 100% della coppia nominale del motore quando il robot si avvicina al punto A2.

TP

Scopo

Il comando TP viene utilizzato con i comandi PRINT e INPUT per specificare il programmatore portatile come canale di comunicazione.

Formato

PRINT [{COM0 | COM1 | TP},]
{<stringa di caratteri> | <espressione>} [,{<stringa di caratteri> | <espressione>}] ... [,CR]

INPUT [{COM0 | COM1 | TP},]
<variabile> [, <variabile>] …..

Esempi

```
PRINT TP, "*** INPUT N ***", CR
PRINT TP, N, N*10, CR
INPUT TP, K
```

Analisi e consigli

Il comando TP viene utilizzato con i comandi PRINT e INPUT per specificare il programmatore portatile come canale di comunicazione.

Quando si utilizza TP per specificare un canale di comunicazione, il controller immette ed emette dati con il programmatore portatile sul canale di comunicazione (COM0) assegnato al programmatore portatile.

Se non si specifica un canale di comunicazione per un comando PRINT o INPUT, il controller emette (o immette) dati al (o dal) programmatore portatile.

Per ulteriori informazioni sull'elaborazione della comunicazione, fare riferimento ai comandi PRINT e INPUT.

Esempio di programma

```
PROGRAM COMSAMPLE
    PRINT TP, "*** INPUT N ***", CR
    INPUT TP, N
    PRINT TP, N, CR
END
```

Questo programma immette un numero dal programmatore portatile e lo emette di nuovo sul programmatore portatile.

TRANS

Scopo

Il comando TRANS viene utilizzato per creare dati di tipo coordinate.

Formato

TRANS (<espressione>, <espressione>, <espressione>, <espressione>)

Esempi

A = TRANS (100, 100, 0, 0)
WORK = WORK + TRANS (100, 100)

Analisi e consigli

Il comando TRANS viene utilizzato per creare variabili di tipo coordinate.

Gli elementi <espressione) contengono, da sinistra a destra, i valori dei dati delle coordinate per X, Y, Z e C. Gli elementi X, Y e Z sono in unità di millimetri e l'elemento C in unità di gradi.

È possibile utilizzare costanti, variabili o calcoli per i termini <espressione>. Tuttavia, non è possibile utilizzare dati di tipo vettoriale. Tutti i termini <espressione> omessi vengono considerati 0.

Questo comando deve essere utilizzato in un'espressione.

Esempio di programma

```
PROGRAM TRANSSMPL
    MOVE A1
    MOVE A2
    WORK=TRANS (,, 300)
    MOVE A1 WITH WORK=WORK1
    MOVE A2 WITH WORK=WORK1
END
```

Imposta i valori del sistema di coordinate del lavoro su Z=300, X=0, Y=0,A=0, B=0, C=0.

WAIT

Scopo

Questa funzione attende che venga stabilita la condizione.

Formato

WAIT <espressione logica>

Esempi

WAIT DIN (1)
WAIT TIMER == 0
WAIT MOTION >= 100

Analisi e consigli

Questa funzione ritarda l'esecuzione del programma finché non si stabilisce la condizione di <espressione logica>.

La condizione viene monitorata indipendentemente dal movimento del robot in corso.

Esempio di programma

```
PROGRAM WAITSAMPLE
    WAIT DIN (1)      Attende che il segnale di ingresso 1 sia ON.
    MOVE A1
    MOVE A2
    MOVE A3
END
```

WITH

Scopo

L'istruzione WITH viene utilizzata per aggiungere un'istruzione condizionale ad un comando di movimento.

Formato

WITH <istruzione> [, <istruzione>]...

Esempi

MOVE A1 WITH SPEED = 50
MOVE A1 WITH TOOL = TOOL1, PASS = 80

Analisi e consigli

L'istruzione WITH viene utilizzata per specificare le condizioni di movimento corrispondenti a singoli comandi di movimento.

Le condizioni di movimento, quali velocità, accelerazione e decelerazione, sono determinate in base ai valori impostati delle variabili di sistema necessarie per il movimento del robot. Per modificare le condizioni di movimento per un singolo movimento, la corrispondente condizione di movimento viene specificata mediante la clausola WITH. La condizione di movimento specificata dalla clausola WITH diventa valida solo per il comando di movimento in cui si è specificata la clausola WITH. I valori delle variabili di sistema rimangono invariati dopo e prima dell'esecuzione del comando di movimento.

È possibile specificare le seguenti condizioni di movimento nel termine <istruzione>. Per specificare più condizioni simili, separare i termini con virgole.

Condizione di movimento	Nome variabile di sistema
Configurazione del robot	CONFIG
Precisione di posizionamento	ACCUR
Accelerazione (durante l'accelerazione)	ACCEL
Decelerazione (durante la decelerazione)	DECEL
Velocità	SPEED
Parametro di spostamento rapido	PASS
Coppia max. per ciascun asse	TORQUE
Guadagno servoazionamento per ciascun asse	GAIN
Sistema di coordinate utensile	TOOL
Sistema di coordinate di base	BASE
Sistema di coordinate lavoro	WORK
Carico del robot	PAYLOAD

Esempio di programma

```
PROGRAM WITHSAMPLE
SPEED = 50
MOVE A1
MOVE A2 WITH SPEED = 100
MOVE A3
MOVE A4
END
```

Questo programma si sposta su tutti i punti a mezza velocità (50%) ad eccezione del punto A2, in cui il robot si sposta a piena velocità (100%).

WORK

Scopo

WORK è una variabile di sistema utilizzata per specificare il sistema di coordinate del lavoro.

Formato

WORK

Esempi

```
WORK    TRANS (0, 0, 0, 0)
WORK1   WORK
MOVE A1 WITH WORK = WORK + TRANS (,, 100)
```

Analisi e consigli

WORK è una variabile di sistema utilizzata per specificare il sistema di coordinate del lavoro. È possibile gestirla come normali dati di tipo coordinate. Facendo riferimento a WORK, si possono trovare i valori (la posizione) del sistema di coordinate del lavoro attuale.

È possibile designare direttamente i valori delle coordinate del lavoro con uno dei due seguenti metodi:

WORK = TRANS (X, Y, Z, C)
WORK = {X, Y, Z, C}
(Per conoscere il tipo di dati a disposizione, utilizzare sempre il comando TRANS.)

X, Y, Z, C: X, Y, Z e C sono valori di coordinate rappresentati da numeri reali (unità: mm o gradi).

Il sistema di coordinate WORK viene creato "scorrendo" una distanza di X, Y e Z lungo i rispettivi assi del sistema di coordinate WORLD, quindi ruotando il nuovo asse Z di una quantità C.

WORK deve essere utilizzato in un'espressione.

Quando si immettono dati di posizione con il programmatore portatile, viene immesso anche il sistema di coordinate del lavoro operativo in quel momento. Successivamente, quando si esegue un comando di movimento utilizzando i dati di posizione, il sistema di coordinate del lavoro passa automaticamente a quanto specificato al momento della registrazione dei dati di posizione. Per spostare il robot in termini di due (o più) sistemi di coordinate del lavoro (ad esempio, il sistema di coordinate del lavoro operativo durante la programmazione e un altro sistema di coordinate), è possibile:

(1) Utilizzare l'istruzione WITH per specificare il sistema di coordinate del lavoro operativo per quel movimento;
o
(2) Modificare i dati stessi delle coordinate del lavoro.

Si noti che se si modificano i sistemi delle coordinate del lavoro in un programma, potrebbe verificarsi un disallineamento tra le posizioni programmate e il movimento del robot.

Esempio di programma

```
PROGRAM WORKSAMPLE
    MOVE A1
    MOVE A2
    WORK1 = WORK + TRANS (,, 300)
    MOVE A1 WITH WORK = WORK1
    MOVE A2 WITH WORK = WORK1
END
```

Il comando WORK sposta il sistema di coordinate del lavoro di 300 mm lungo l'asse Z e, successivamente, il robot si sposta in un punto superiore di 300 mm rispetto alla posizione programmata.

Sezione 4

Esempi di programma

In questa sezione illustreremo vari esempi di programmazione utilizzando il linguaggio SCOL. Quando si applicano questi programmi ad un robot reale, assicurarsi di modificare i programmi a seconda delle condizioni operative del robot in questione, tenendo in considerazione, ad esempio, l'ambiente di lavoro e il raggio di movimento.

(1) Programma per riportare il robot alle sue origini meccaniche
Questo programma riporta il robot alle sue origini meccaniche (punti zero per ogni asse). Utilizzando un movimento assoluto ad albero singolo, il robot si azzera a partire dall'asse Z (Asse 3), operando quindi dalla punta alla base.

```
PROGRAM ORIGIN
    MOVEA 3, 0
    MOVEA 5, 0
    MOVEA 4, 0
    MOVEA 2, 0
    MOVEA 1, 0
END
```

(2) Programma di riscaldamento
Questo programma viene usato per riscaldare il robot prima di iniziare il lavoro. Il robot si avvia lentamente e diventa via via più veloce.

```
PROGRAM WARMINGUP
  FOR K = 1 TO 100
    SPEED = K
    MOVEA 3, 100
    MOVEA 3, 0
    MOVEA 5, 50
    MOVEA 5, 0
    MOVEA 4, 50
    MOVEA 4, 0
    MOVEA 2, 50
    MOVEA 2, 0
```

```
    MOVEA 1, 50
    MOVEA 1, 0
  NEXT K
END
```

(3) Movimento del robot

Questo programma fa sì che il robot si sposti dalla posizione A1 alla posizione A2. La velocità del movimento del robot è impostata al 20% della velocità massima.

(a) Quando si usa il movimento PTP (comando MOVE):

```
PROGRAM MOVEA1A2
  SPEED=20
  MOVE A1
  MOVE A2
END
```

(b) Quando si usa il movimento lineare interpolato (comando MOVES):

```
PROGRAM MOVESA1A2
  SPEED=20
  MOVES A1
  MOVES A2
END
```

(4) Segnali I/O

In questo programma l'accensione dei segnali di ingresso da 1 a 4 determina l'accensione dei segnali di uscita da 1 a 4 e lo spegnimento dei segnali di ingresso da 1 a 4 determina lo spegnimento dei segnali di uscita da 1 a 4.

```
PROGRAM SAMPLE
  IF DIN(1) THEN DOUT(1) ELSE DOUT(–1)
  IF DIN(2) THEN DOUT(2) ELSE DOUT(–2)
  IF DIN(3) THEN DOUT(3) ELSE DOUT(–3)
  IF DIN(4) THEN DOUT(4) ELSE DOUT(–4)
  END
```

(5) Blocco

Questo programma interrompe il movimento del robot quando il segnale di ingresso 1 è spento nelle posizioni tra la A1 e la A4. La velocità del movimento del robot è impostata al 20% della velocità massima.

(a) Quando si usa il comando WAIT:

```
PROGRAM SAMPLE
   SPEED=20
   WAIT DIN(1)
   MOVE A1
   WAIT DIN(1)
   MOVE A2
   WAIT DIN(1)
   MOVE A3
   WAIT DIN(1)
   MOVE A4
END
```

(b) Quando il movimento del robot viene interrotto con il comando ON e attende fino a quando il segnale di ingresso 1 non sia spento:

```
PROGRAM SAMPLE
   SPEED=20
   ENABLE NOWAIT
   ON DIN(–1) BREAK DO SUB
   WAIT DIN(1)
   MOVE A1
   MOVE A2
   MOVE A3
   MOVE A4
END
PROGRAM SUB
   WAIT DIN(1)
   RESUME
   ON DIN(–1) BREAK DO SUB
END
```

(6) Presa e posizionamento

Questo programma serve a prendere un pezzo di lavoro dalla posizione A1 e metterlo nella posizione A2. La mano viene aperta/chiusa con il segnale di uscita 201. La velocità del movimento del robot è impostata al 20% della velocità massima.

(a) Quando le posizioni al di sopra delle posizioni A1 e A2 vengono programmate come la A3 e la A4, rispettivamente:

```
PROGRAM SAMPLE
    SPEED=20
    DOUT(–201)
    MOVE A3
    MOVE A1
    DOUT(201)
    DELAY 0.5
    MOVE A3
    MOVE A4
    MOVE A2
    DOUT(–201)
    DELAY 0.5
    MOVE A4
END
```

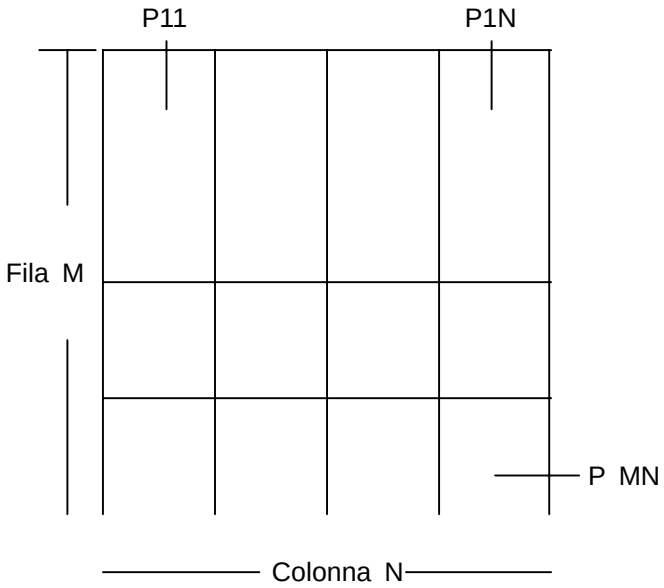
(b) Quando il robot viene spostato su una posizione al di sopra di quella programmata nel formato di "MOVE A1+POINT(0,0,20)".

```
PROGRAM SAMPLE
    SPEED=20
    DOUT(–201)
    MOVE A1+POINT(0,0,20)
    MOVE A1
    DOUT(201)
    DELAY 0.5
    MOVE A1+POINT(0,0,20)
    MOVE A2+POINT(0,0,20)
    MOVE A2
    DOUT(–201)
    DELAY 0.5
```

```
    MOVE A2+POINT(0,0,20)
END
```

(c) Quando si esegue uno spostamento rapido:

```
PROGRAM SAMPLE
    SPEED=20
    DOUT(–201)
    ENABLE PASS
    PASS=80
    MOVE A1+POINT(0,0,50)
    DISABLE PASS
    MOVE A1
    DOUT(201)
    DELAY 0.5
    ENABLE PASS
    MOVE A1+POINT(0,0,50)
    MOVE A2+POINT(0,0,50)
    DISABLE PASS
    MOVE A2
    DOUT(–201)
    DELAY 0.5
    ENABLE PASS
    MOVE A2+POINT(0,0,50)
END
```



(7) Pallettizzazione

Consideriamo un programma che serva a posizionare delle parti su un pallet come mostrato nella figura.

Dimensioni del pallet:

File M x colonne N

Posizioni programmate:

P11: Posizione della colonna 1 e della fila 1 del pallet
P1N: Posizione della fila 1 e della colonna N del pallet
PMN: Posizione della fila M e della colonna N del pallet
PP: Posizione di scarico della parte
PO: Posizione di standby

Designazione dei segnali I/O:

DI1: Standby del pallet (Questo segnale si accende quando il pallet è in posizione operativa.)

DI2: Standby della parte (Questo segnale si accende quando una parte può essere presa.)

DO1: Chiusura della mano (Questo segnale si accende quando la mano viene chiusa.)

DO2: Completamento di un'operazione con il pallet (Questo segnale si accende quando tutte le parti sono posizionate sul pallet. Quando si rimuove il pallet il segnale di standby del pallet si spegne.)

```
PROGRAM PALLET
   MOVE P0
   RESET DOUT
   M=10                          Specifica il numero di righe e file.
   N=15
   RX=(P1N.X–P11.X) /(N–1)       Calcola il numero di spostamenti per
                                 elemento.
   RY=(P1N.Y–P11.Y)/(N–1)
   LX=(PMN.X–P1N.X)/(M–1)
   LY=(PMN.Y–P1N.Y)/(M–1)
   PASS=80
   ENABLE PASS
   ENABLE NOWAIT
   L=0
   LOOPL:                        Ripete l'operazione per ogni riga.
      R=0
   LOOPR:                        Ripete l'operazione per ogni fila.
      MOVE PP+POINT(0,0,50)
      WAIT DIN(2)
      MOVE PP
      WAIT MOTION>=l00
      DOUT(1)                    Scarica la parte.
      DELAY 0.5
      MOVE PP+POINT(0,0,50)
      MOVE P11+POINT(R*RX+L*LX,R*RY+L*LY, 100)
      WAIT DIN(1)
      MOVE P11+POINT(R*RX+L*LX,R*RY+L*LY)
      WAIT MOTION>=100
      DOUT(–1)                   Posiziona la parte.
      DELAY 0.5
      MOVE P11+POINT(R*RX+L*LX,R*RY+L*LY, 100)
   R = R + 1
      IF R<=N–1 THEN GOTO LOOPR
      L = L + 1
      IF L<=M–1 THEN GOTO LOOPL
      WAIT MOTION>=l00
      DOUT(2)
```

```
        WAIT DIN(–2)
        MOVE P0
END
```

(8) Creazione di un programma per monitorare un errore di inserimento
Di seguito viene descritto un programma per rilevare un errore di inserimento di una parte e per gestire un errore.

A1: Posizione per inserire la parte

ACCUR=COARSE	Imposta la precisione di posizione su approssimativa. [1]
MOVE A1+POINT(0, 0, 50)	Sposta la mano ad una posizione 50 mm al di sopra del punto di inserimento della parte.
SETGAIN(0, 0, 1, 0, 0)	Spegne i servocomandi tranne per l'asse Z. [2]
TORQUE={300, 300, 50, 300, 300}	Limita la coppia dell'asse Z al 50%. [3]
MOVE A1	Inserisce tutte le parti.
SETGAIN (0, 0, 0, 0, 0) MOVE HERE	Spegne il servocomando per tutti gli assi. [4]
SETGAIN (0, 0, 1, 0, 0) MOVE HERE	Accende il servocomando per l'asse Z. [5]
HS=HERE.Z–A1.Z IF HS<5 THEN GOTO OK	Quando la differenza di altezza tra la posizione attuale e quella inserita è minore di 5 mm, si determina che la posizione inserita è normale. [6]
SETGAIN(1, 1, 1, 1, 1) DELAY 0.1 TORQUE={300, 300, 300, 300, 300} DELAY 0.1 MOVE A1+POINT(0, 0, 50)	Accende il servocomando per tutti gli assi, riporta la coppia al 300% e sposta la mano ad una posizione 50 mm al di sopra del punto di inserimento della parte.

Processo nel caso in cui le parti siano state inserite in modo anormale.

OK:

```
SETGAIN(1, 1, 1, 1, 1)
DELAY 0.1
TORQUE={300, 300, 300, 300, 300}
DELAY 0.1
MOVE A1+POINT(0, 0, 50)
```

Accende il servocomando per tutti gli assi, riporta la coppia al 300% e sposta la mano ad una posizione 50 mm al di sopra del punto di inserimento della parte.

Processo nel caso in cui le parti siano state inserite normalmente.

Commenti per la descrizione del programma:

[1] A meno che la precisione di posizione non sia impostata su approssimativa, il sistema del robot potrebbe rilevare un errore e, di conseguenza, determinare l'interruzione del funzionamento automatico.

[2] Quando si spegne il servocomando tranne per l'asse Z, il robot può essere liberamente spostato sul piano X-Y. Quindi, una volta che il pezzo di lavoro o la parte è stata smussata, può essere inserita lungo la superficie smussata.

[3] Con il comando TORQUE, la coppia del motore è limitata. Quando avviene un errore di inserimento, la forza applicata al lavoro e alla mano viene ridotta. (Normalmente la coppia è impostata al 300% del valore nominale.) Riducendo la coppia a meno del 100%, il livello di rilevamento dell'errore del controller sarà inferiore. Quindi, quando si vogliono monitorare errori di inserimento, è necessario impostare la coppia a meno del 100%. Quando si diminuisce eccessivamente il valore di impostazione della coppia, dal momento che non si riesce ad ottenere la coppia necessaria per spostare il robot, si verifica un errore.

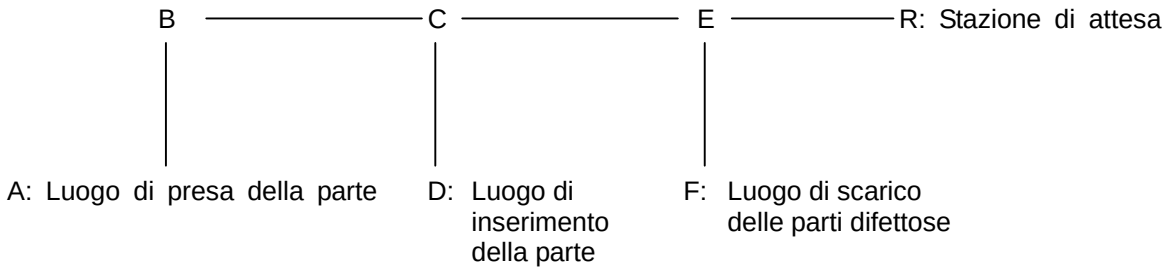
[4] La posizione che viene letta con il comando HERE è la posizione comandata al robot. Per leggere la posizione del robot di un dato momento con il comando HERE, è necessario spegnere il servocomando e poi accenderlo nuovamente. Quindi, con il comando GAIN, vengono spenti i servocomandi per tutti gli assi.

[5] Il servocomando che era stato spento in ↓ viene acceso.

[6] La differenza di altezza tra la posizione attuale e quella di inserimento dovrebbe essere specificata ad un valore adeguato.

(9) Esempio del programma di spostamento rapido (PASS)

Il seguente esempio di programmazione utilizza spostamenti rapidi per eseguire operazioni di "presa e posizionamento".



Il robot prenderà una parte dal luogo di presa della parte (A), la sposterà lungo i punti B e C e proverà a inserire la parte nel luogo di inserimento della parte (D). Se la parte è difettosa (es. non può essere inserita), il robot si muove lungo i punti C ed E e posiziona la parte nel luogo di scarico delle parti difettose (F).

Lo spostamento rapido viene utilizzato in tutto il programma.

In questo programma vengono utilizzati i seguenti segnali:

Segnali di ingresso:

DI1: Completamento della preparazione della presa
Passa ad ON quando viene completata la preparazione della parte da prendere.

DI2: Completamento della preparazione dell'inserimento
Passa ad ON quando viene completata la preparazione della posizione di inserimento della parte.

DI3: Parte difettosa
Passa ad ON quando si rileva che la parte presa è difettosa.

DI4: Completamento della preparazione dello scarico della parte difettosa
Passa ad ON quando viene completata la preparazione della posizione di scarico della parte.

Segnali di uscita:

DO1: Completamento della presa della parte
Passa ad ON quando viene completata la presa della parte e si può iniziare la preparazione della parte successiva.

DO2: Completamento dell'inserimento della parte
Passa ad ON quando viene completato l'inserimento della parte e si può iniziare la preparazione per l'inserimento della parte successiva.

DO3: Completamento dello scarico della parte difettosa
Passa ad ON quando viene completato lo scarico della parte difettosa e si può iniziare la preparazione per lo scarico della parte difettosa successiva.

Il programma d'esempio è presentato qui di seguito:

```
PROGRAM PICKPLACE

*   PICK AND PLACE SAMPLE PROGRAM *

 INPUT:                    Impostazioni iniziali
     OPEN1
     DOUT (1, 2, 3)        Presa della parte, inserimento della parte
     ENABLE NOWAIT         Completamento dell'eliminazione della parte
                           difettosa
     SPEED = 80
     PASS = 80
     ENABLE PASS
     MOVE B                Spostare alla posizione al di sopra del luogo di
                           presa della parte

 PICKUP:                   Sezione per prendere la parte
     DOUT (–1)             Iniziare a prendere la parte
     WAIT DIN (1)          Attendere fino al completamento delle
                           preparazioni per la presa
     MOVE A                Abbassare fino ad A
     WAIT MOTION > = 100
     CLOSE1                Chiudere la mano
     DELAY 0. 3
     MOVE B                Salire fino a B

 PLACE:                    Sezione per posizionare la parte
     MOVE C                Spostare alla posizione al di sopra del luogo di
                           inserimento della parte
     DOUT (1, –2)          Presa della parte completa – inizio inserimento
                           della parte
     WAIT DIN (2)          Attendere fino al completamento delle
                           preparazioni per l'inserimento
```

```
        MOVE D                          Abbassare fino a D
        WAIT MOTION > = 100
        IF DIN (3) THEN GOTO ERR        Valutazione della parte come difettosa
        OPEN1                           Aprire la mano
        DELAY 0. 3
        MOVE C                          Salire fino a C
        MOVE B                          Salire fino alla posizione al di sopra del luogo di
                                        presa della parte
        DOUT (2)                        Completamento dell'inserimento della parte
        IF MODE = = CYCLE THEN GOTO CYCLEEND
                                        Controllo della modalità
        GOTO PICKUP
    ERR:
        MOVE C                          Sezione per gestire la parte difettosa
                                        Spostare alla posizione al di sopra del luogo di
                                        inserimento della parte
        MOVE E                          Spostare alla posizione al di sopra del luogo di
                                        scarico della parte difettosa
        DOUT (–3)                       Inizio dello scarico della parte difettosa
        WAIT DIN (4)                    Attendere fino al completamento delle
                                        preparazioni per lo scarico
        MOVE F                          Abbassare fino a F
        WAIT MOTION > = 100
        OPEN1                           Aprire la mano
        DELAY 0. 3
        MOVE E                          Salire fino a E
        MOVE B                          Spostare alla posizione al di sopra del luogo di
                                        presa della parte
        DOUT (3)                        Completamento dello scarico della parte
                                        difettosa
        IF MODE = = CYCLE THEN GOTO CYCLEEND
                                        Controllo della modalità
        GOTO PICKUP
    CYCLEEND:                           Elaborazione per la fine del ciclo
        MOVE R                          Spostarsi alla stazione di attesa
END
```

Sezione 5

Programmare suggerimenti e avvertenze

La presente sezione illustra delle considerazioni tempistiche, le cose da non fare e quelle a cui badare quando si scrive un programma.

5.1 Tempi di esecuzione del programma

I programmi del robot vengono eseguiti una riga alla volta a partire dall'alto. Normalmente, quando un robot esegue un comando di movimento, il comando successivo non viene eseguito fino quando non venga completato il posizionamento finale per quel movimento. Tuttavia, quando si gestiscono segnali I/O e altre operazioni non direttamente relazionate al movimento del robot, ciò determina una perdita di tempo (attendere che si completi il movimento).

Al fine di utilizzare efficientemente i tempi e velocizzare il robot il più possibile, il linguaggio SCOL permette di immettere/emettere segnali e di elaborare comunicazioni mentre il robot è in movimento.

5.1.1 Tempi del movimento del braccio e dei segnali I/O

Quando si immette o si emette un segnale, le istruzioni del programma del robot scritte nel linguaggio del robot specificano se attendere o no che il braccio completi il movimento.

L'interruttore NOWAIT del sistema viene usato per dire al controller se attendere che il robot si fermi (finisca di posizionarsi) prima di immettere o emettere segnali.

ENABLE NOWAIT	Questo dice al controller di non attendere che il braccio finisca di posizionarsi prima di immettere o emettere segnali.
DISABLE NOWAIT	Questo dice al controller di attendere che il braccio finisca di posizionarsi prima di immettere o emettere segnali.

L'impostazione iniziale (predefinita) è DISABLE NOWAIT.

Di seguito viene illustrato un esempio di programma. Utilizzando questo programma, descriveremo i tempi dei segnali I/O relativi all'elaborazione di comandi e al movimento del braccio.

Esempio: Immaginiamo di dover eseguire i seguenti comandi.

```
PROGRAM SAMPLE
  MOVE P1
  DOUT (1)
  MOVE P2
  DOUT (2)
  MOVE P3
  DOUT (3)
  MOVE P4
  DOUT (4)
  MOVE P5
  DOUT (5)
END
```

(1) Tempi quando è attivo DISABLE NOWAIT

Elaborazione dei comandi	MOVE P1	Attesa della fine del movimento	DOUT(1)	MOVE P2	Attesa della fine del movimento	DOUT(2)	MOVE P3	Attesa della fine del movimento	DOUT(3)	MOVE P4	Attesa della fine del movimento	DOUT(4)	MOVE P5	Attesa della fine del movimento
Movimento del braccio	Si sposta su P1		Si sposta su P2			Si sposta su P3			Si sposta su P4			Si sposta su P5		

Come si può vedere, i segnali vengono emessi dopo che il braccio abbia terminato il movimento.

Nota: Quando si specifica ACCURE=COARSE, dal momento che il comando successivo viene eseguito prima che l'operazione di posizionamento sia terminata, è possibile che si emetta un segnale prima che il movimento del robot sia completamente finito.

(2) Tempi quando è attivo ENABLE NOWAIT

Elaborazione dei comandi	MOVE P1	DOUT(1)	MOVE P2	DOUT(2)	MOVE P3	DOUT(3)	MOVE P4	DOUT(4)	Attesa della fine del movimento	MOVE P5	DOUT(5)				
Movimento del braccio	Si sposta su P1									Si sposta su P2		Si sposta su P3		Si sposta su P4	Si sposta su P5

Il movimento del braccio dettato dal comando precedente finisce e comincia quello del comando successivo. Tuttavia, anche se il braccio è in movimento, viene eseguita l'elaborazione dell'uscita del segnale successivo a quel comando di movimento, il che è definito "pre-lettura del comando di movimento". Il presente controller robot legge in anticipo fino a quattro (4) comandi di movimento. (Mentre il braccio si sposta sul punto P1, il controller legge in anticipo i comandi di movimento fino al punto P4 e attende il completamento del movimento prima che il braccio esegua il comando di movimento al punto P5.)

5.1.2 Sincronizzazione del movimento del braccio e dell'esecuzione del programma

Nel paragrafo precedente abbiamo spiegato che il movimento del braccio viene eseguito parallelamente all'immissione/emissione del segnale. Per essere più precisi, possono essere eseguiti al contempo il movimento del braccio e i comandi SCOL diversi da quello del movimento del braccio.

Nel normale programma SCOL, quindi, l'elaborazione del programma viene evidentemente eseguita prima del movimento del braccio, perché è necessaria una speciale procedura (ovvero il comando "WAIT MOTION >= 100") per sincronizzare il movimento del braccio con l'esecuzione del programma. Questo comando non si completerà fino a quando il braccio non si sia posizionato. Descrivere "WAIT MOTION >= 100" prima del programma da eseguire in sincronizzazione.

Di seguito viene illustrato un esempio di programma. Utilizzando questo programma, descriveremo i tempi dei segnali I/O relativi all'elaborazione di comandi e al movimento del braccio.

Esempio: Immaginiamo di dover eseguire i seguenti comandi.

```
PROGRAM SAMPLE
  MOVE P1
  K = 1
  DOUT (1)
  K = K + 1
  WAIT MOTION >= 100
  K = K – 1
  DOUT (2)
END
```

(1) Tempi quando è attivo DISABLE NOWAIT

Elaborazione dei comandi	MOVE P1	K=1	Attesa della fine del movimento	DOUT(1)	K=K+1	WAIT MOTION>=100	K=K–1	DOUT(2)
Movimento del braccio	Si sposta su P1							

Il comando DOUT attende che il movimento del braccio termini.

(2) Tempi quando è attivo ENABLE NOWAIT

Elaborazione dei comandi	MOVE P1	K=1	DOUT(1)	K=K+1	WAIT MOTION>=100	K=K–1	DOUT(2)			
Movimento del braccio	Si sposta su P1									

Il comando “WAIT MOTION >=100” attende che il movimento del braccio termini.

Nota: Oltre all'immissione ed emissione di segnali, l'elaborazione di comandi diversi da quelli qui sotto elencati continua a prescindere dal movimento del robot.

DIN BCDIN INPUT PLCINPUT
DOUT BCDOUT PULOUT PRINT PLCPRINT

5.1.3 Comando DELAY e comando WAIT

Vi sono due modi per arrestare il movimento del braccio per un tempo specifico durante l'esecuzione del programma. Un metodo utilizza il comando DELAY e l'altro il comando WAIT in combinazione con il comando TIMER. Quando si scrive un programma, si dovrebbe ricordare che i tempi di esecuzione dei due sono differenti.

(1) Comando DELAY

Il comando DELAY è un tipo di comando di controllo del movimento e arresta il movimento del braccio per un periodo di tempo prestabilito. Tuttavia, dal momento che il programma del linguaggio SCOL viene eseguito in sincronizzazione con il movimento del braccio, il programma viene eseguito anche durante l'esecuzione del comando DELAY.

Esempio: Immaginiamo di dover eseguire i seguenti comandi.

```
PROGRAM SAMPLE
  ENABLE NOWAIT
  DELAY 2.0
  K = 10
  DOUT (1)
  K = K – 1
  MOVE P1
  DOUT (–1)
END
```

Elaborazione dei comandi	ENABLE NOWAIT	DELAY 2.0	K=10	DOUT(1)	K=K–1	MOVE P1	DOUT(–1)	
Movimento del braccio	Il braccio si blocca per 2,0 secondi							Si sposta su P1

Dal momento che il programma continua ad essere eseguito anche dopo che il comando DELAY mette in pausa il braccio del robot, il Segnale 1 sarà emesso mentre il braccio è bloccato.

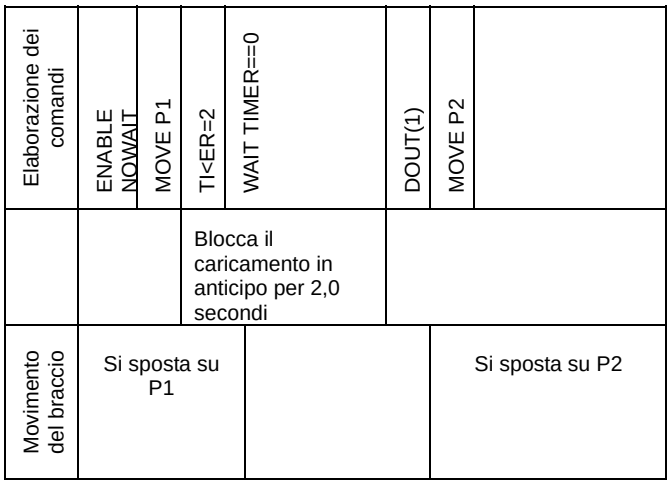
Nel caso si volesse ritardare anche l'esecuzione del programma, si dovrà inserire un comando “WAIT MOTION >= 100” tra il comando DELAY e il comando DOUT.

(2) Comando WAIT

Il comando WAIT può essere usato con il comando TIMER per ritardare l'esecuzione del programma per uno specifico periodo di tempo. Tuttavia, come con il comando DELAY descritto precedentemente, il programma viene elaborato parallelamente al movimento del robot e bisogna prestare attenzione ai tempi di esecuzione.

Esempio: Immaginiamo di dover eseguire i seguenti comandi.

```
PROGRAM SAMPLE
    ENABLE NOWAIT
    MOVE P1
    TIMER = 2
    WAIT TIMER = = 0
    DOUT (1)
    MOVE P2
END
```



Il ritardo nell'esecuzione del programma specificato dal comando WAIT termina mentre il robot è ancora in movimento. Quando è necessario che resti in attesa anche il robot, inserire un'istruzione WAIT MOTION come mostrato di seguito:

```
MOVE P1
WAIT MOTION >= 100
TIMER = 2
WAIT TIMER = = 0
```



5.2 Cose da non fare durante la programmazione

Questo paragrafo illustra le restrizioni e i divieti in vigore quando si scrivono programmi. Fare riferimento alle descrizioni dei comandi nella Sezione 3 per maggiori informazioni sui singoli comandi.

5.2.1 Variabili

(1) In riferimento a variabili indefinite

Il tipo di dati di una variabile è definito per la prima volta quando qualcosa in essa viene sostituito. Quindi, non bisogna fare riferimento alle variabili che appaiono per la prima volta nel programma. Se si fa riferimento ad una variabile che non è stata precedentemente usata (non ha subito sostituzioni), i valori e il tipo di dati della variabile divengono indefiniti. L'utente sarà dispiaciuto quando proverà ad effettuare il debug del proprio programma.

Esempio:

```
PROGRAM SAMPLE
    IF K < > 0 THEN GOTO RESTART
    K = 1
    A1 = A
RESTART:
    FOR N = 1 TO 3
        MOVE A1
        A1 = A1 + POINT (100, 100)
    NEXT N
END
```

In questo esempio, quando il programma viene eseguito per la prima volta, la variabile K viene inclusa nell'istruzione IF. Tuttavia, la variabile K non è mai stata usata (non ha mai subito sostituzioni) in precedenza, quindi diventa indefinita. Poi, l'1 viene sostituito in K e quindi la variabile K prende il valore 1.

5.3 Cose a cui badare quando si scrive un programma

Questo paragrafo illustra le cose a cui badare e presenta una panoramica di alcuni comandi SCOL che possono risultare utili durante la scrittura di un programma.

5.3.1 Tipo di comandi

Una classificazione funzionale del linguaggio SCOL è stata presentata nella Sezione 1. Qui descriviamo il linguaggio SCOL in termini di elaborazione interna.

I comandi SCOL possono essere suddivisi in comandi di base, funzioni e variabili d sistema. I comandi di base costituiscono il nucleo del linguaggio SCOL e vengono eseguiti insieme ai parametri che rispondono ai comandi. Le funzioni vengono fornite per comodità per agevolare l'utilizzo del linguaggio SCOL. Le variabili di sistema vengono usate per fare diretto riferimento a (e cambiare) elementi come velocità, sistemi di coordinate, ecc. Possono essere gestite come qualunque altra variabile.

Questi tre tipi di comandi sono descritti più dettagliatamente qui di seguito.

(1) Comandi di base

I comandi di base forniti dal linguaggio SCOL sono mostrati di seguito. A differenza delle funzioni elencate di seguito nel Para. (2), è possibile non usare nessuno dei seguenti comandi all'interno di una espressione (matematica).

MOVE	MOVES	MOVEC	MOVEA
MOVEI	MOVEJ	DELAY	BREAK
PAUSE	RESUME	ON ~ DO	IGNORE
GOTO	RETURN	WAIT	STOP
IF ~ THEN ~	ELSE	FOR ~ NEXT	
PRINT	INPUT	ENABLE	DISABLE
PROGRAM	END	RESET	REMARK
DIM ~ AS	TASK	KILL	SWITCH
GLOBAL	MAXTASK		
RESTORE	MOVESYNC	SAVEEND	RCYCLE

(2) Funzioni

Al contrario dei comandi di base, le funzioni hanno la caratteristica di poter trasferire argomenti avanti e indietro proprio come i sottoprogrammi. Possono essere specificati fino a dieci argomenti.

Le funzioni possono essere suddivise in funzioni integrate e funzioni della libreria di sistema (che sono così chiamate perché sono tenute in un file in una libreria di sistema). Nel linguaggio SCOL, la libreria di sistema viene chiamata SCOL.LIB e a meno che questa non sia nell'unità RAM del sistema, non sarà possibile utilizzare nessuna delle funzioni della libreria di sistema.

SCOL fornisce le seguenti funzioni integrate.

MOTION	MOTIONT	REMAIN	REMAINT
HERE	DEST	POINT	TRANS
DIN	DOUT	PULOUT	BCDIN
BCDOUT	SIN	COS	TAN
ASIN	ACOS	ATAN	ATAN2
SQRT	ABS	SGN	INT
REAL	LN	LOG10	EXP
MODE			

Le funzioni integrate possono essere usate nelle espressioni (matematiche) eccetto con i seguenti comandi di segnali I/O: DOUT, PULOUT e BCDOUT.

SCOL fornisce le seguenti funzioni della libreria di sistema (che sono contenute nella libreria di sistema).

OPEN1	CLOSE1	OPENI1	CLOSEI1
OPEN2	CLOSE2	OPENI2	CLOSEI2
READY	ONGAIN	OFFGAIN	FREELOAD
SETGAIN			

Se si crea un sottoprogramma e gli si attribuisce lo stesso nome di una funzione contenuta nella libreria di sistema, questo nuovo sottoprogramma dovrebbe essere eseguito come priorità.

(3) Variabili di sistema

Le variabili di sistema vengono usate per cambiare le condizioni del sistema e possono essere gestite come qualunque altra variabile.
SCOL fornisce le seguenti variabili di sistema.

CONFIG	ACCUR	ACCEL	DECEL
SPEED	PASS	TORQUE	GAIN
TOOL	BASE	WORK	TIMER
ERROR	PAYLOAD	TID	

5.3.2 Sistemi di coordinate del robot

Questo paragrafo illustra i sistemi di coordinate del robot gestite dal linguaggio SCOL.

(1) Sistemi di coordinate del robot

Il robot presenta cinque tipi di sistemi di coordinare, ovvero il sistema di coordinate universali, il sistema di coordinate di base, il sistema di coordinate del lavoro, il sistema di coordinate dell'interfaccia meccanica e il sistema di coordinate dell'utensile. Qui di seguito viene presentata una breve descrizione di questi sistemi:

(a) Sistema di coordinate universali (sistema assoluto di coordinate)

Il sistema di coordinate universali è un sistema singolo utilizzato per descrivere l'orientamento del sito di lavoro che circonda il robot. Questo sistema di coordinate può essere definito in modo indipendente dal robot.

Se questo sistema di coordinate è impostato sulla posizione iniziale del robot, il sistema di coordinate universali e quello di coordinate di base sono gli stessi.

(b) Sistema di coordinate di base (sistema meccanico di coordinate)

Il sistema di coordinate di base è il sistema usato dal robot stesso. Il luogo delle coordinate di base viene determinato dal tipo di robot e l'origine del sistema corrisponde sempre alle origini meccaniche del robot.

(c) Sistema di coordinate lavoro

Il sistema di coordinate del lavoro è definito in base al pezzo di lavoro gestito dal robot.

(d) Sistema di coordinate dell'interfaccia meccanica
Il sistema di coordinate dell'interfaccia meccanica è definito in base all'attuatore finale collegato al robot. Questo sistema di coordinate si sposterà in base al movimento del robot.

(e) Sistema di coordinate utensile
Il sistema di coordinate dell'utensile è definito in base all'estremità di un utensile collegato al robot. Questo sistema di coordinate si sposterà in base al movimento del robot.

Questo sistema di coordinate svolge il proprio ruolo nella guida del robot, nell'impartirgli le posizioni e nel funzionamento dello stesso. Durante la guida del robot è possibile specificare i sistemi di coordinate universali, del lavoro e dell'utensile. Il robot si sposta lungo il sistema di coordinate specificato. Tuttavia, non c'è bisogno di preoccuparsi del sistema di coordinate da scegliere durante la programmazione o il funzionamento del robot. In condizioni normali di utilizzo, il robot si sposterà verso i punti che gli sono stati impartiti durante la programmazione.

Attenzione: Il sistema di coordinate e gli assi supplementari

L'asse 5 (asse T) del robot SCARA e l'asse 4 (asse C) e l'asse 5 (asse T) del robot a coordinate cartesiane sono gli assi supplementari sui quali il sistema di coordinate non inciderà. I valori sono gli stessi in qualunque sistema di coordinate selezionato.

La Fig. 5.1 mostra il sistema di coordinate del robot.

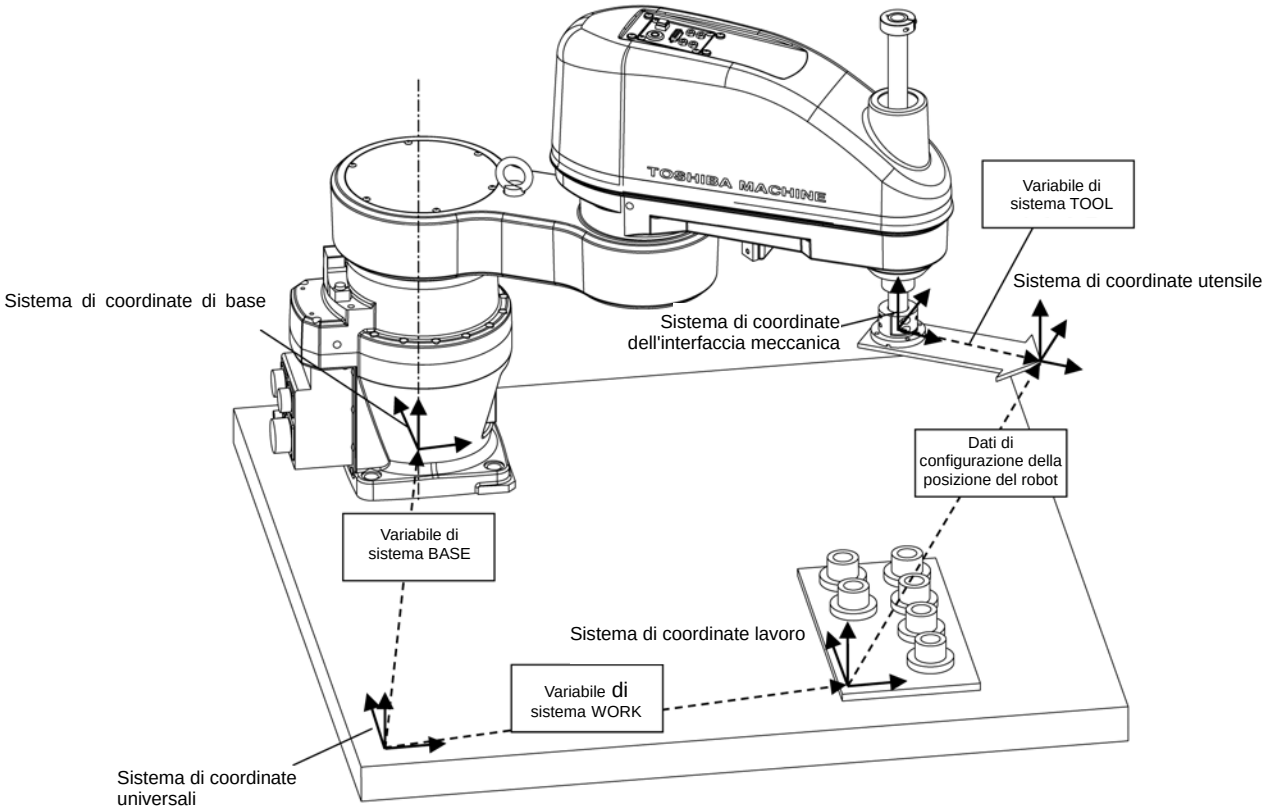


Fig. 5.1 Sistema di coordinate del robot

(2) Sistemi di coordinate e variabili di sistema

I sistemi delle coordinate di base, del lavoro e dell'utensile potrebbero contenere sistemi di coordinate multiple, ognuno dei quali può essere selezionato in base alle attività del robot. È possibile specificare ciascun sistema di coordinate anche nel programma del linguaggio SCOL. Questi sistemi di coordinate possono anche essere specificati (definiti) all'interno di un programma utilizzando le variabili di sistema BASE, WORK e TOOL. Queste variabili di sistema possono essere utilizzate proprio come qualunque altra variabile di sistema di tipo coordinate.

Qui di seguito viene descritto il significato di ciascuna di queste variabili di sistema. Per maggiori informazioni sulla programmazione dei sistemi di coordinate sul robot o sulla selezione di tale sistema di coordinate, fare riferimento al Manuale dell'operatore.

(a) Variabile di sistema BASE

La variabile di sistema BASE definisce l'origine del sistema delle coordinate di base in termini di sistema di coordinate universali. Si usa quando è necessario basare il movimento del robot sul sistema di coordinate del sito di lavoro (sistema di coordinate universali) o quando è necessario guidare il robot secondo coordinate universali (quali opposte alle coordinate di base).

Con un tale sistema, il robot può essere reinstallato in un luogo a parte e rimesso in azione semplicemente dopo aver ridefinito il luogo del sistema delle coordinate di base.

Nel caso in cui il valore della variabile di sistema BASE fosse impostato su 0, il sistema delle coordinate universali e quello delle coordinate di base coinciderebbero. (La variabile di sistema BASE presenta dati di tipo coordinate e contiene quattro elementi, che devono essere impostati tutti su 0 al fine di impostare la variabile di sistema BASE stessa su 0.) Se non si presenta nessuna particolare necessità di specificare un sistema di coordinate universali, si dovrebbe impostare BASE su 0. Inoltre, bisogna prestare attenzione a non cambiare accidentalmente il sistema delle coordinate di base dopo che è stato impostato.

(b) Variabile di sistema WORK

La variabile di sistema WORK definisce l'origine del sistema di coordinate del lavoro in termini di sistema di coordinate universali. Nel caso in cui WORK avesse valore di 0, il sistema delle coordinate universali e quello delle coordinate del lavoro coinciderebbero. (La variabile di sistema WORK presenta dati di tipo coordinate e contiene quattro elementi.)

Quando si gestiscono più pezzi di lavoro, è possibile specificare più sistemi di coordinate del lavoro per poter esser al corrente di dove sia ogni pezzo di lavoro.

Inoltre, la variabile di sistema WORK è utile quando è necessario guidare il robot in relazione al pezzo di lavoro (piuttosto che, ad esempio, la base del robot). Se non si presenta nessuna particolare necessità di specificare un sistema di coordinate del lavoro, impostare la variabile di sistema WORK su 0.

(c) Variabile di sistema TOOL

La variabile di sistema TOOL definisce l'origine del sistema di coordinate dell'utensile in termini di sistema di coordinate dell'interfaccia meccanica. Nel

caso in cui TOOL avesse valore 0, il sistema di coordinate dell'utensile e quello di coordinate dell'interfaccia meccanica coinciderebbero. (La variabile di sistema TOOL presenta dati di tipo coordinate e contiene quattro elementi.) Quando si gestiscono più utensili, è possibile specificare più sistemi di coordinate dell'utensile per poter esser al corrente di dove sia ogni utensile. Inoltre, la variabile di sistema WORK è utile quando è necessario guidare il robot in relazione all'utensile (piuttosto che, ad esempio, la base del robot).

Bisogna prestare attenzione quando si cambia una qualunque di queste variabili di sistema nel proprio programma perché cambierà anche il sistema di coordinate secondo le quali il robot si sposta.

(3) Programmazione di dati e sistemi di coordinate
Quando si programma una posizione sul robot, il robot registra anche la posizione della punta dell'utensile relativa al sistema di coordinate del lavoro. Inoltre, il robot ricorderà anche il sistema di coordinate del lavoro valido in quel momento.

Quando si esegue un programma utilizzando questi dati, il robot si sposta sulla posizione definita dai dati di posizione per quel punto. Tuttavia, se è stato cambiato il sistema di coordinate del lavoro stesso (e quindi non coincide con il sistema di coordinate del lavoro valido al momento della programmazione), gli stessi dati di posizione definiranno un punto differente (dal momento che la cornice di riferimento è differente). Quindi, il robot non si sposterà sul punto programmato, ma su un altro punto.

(4) Cambiare i dati dei sistemi di coordinate nel programma
Qui di seguito viene presentata una semplice spiegazione su come cambiare i dati dei sistemi di coordinate nel programma. È raccomandabile cambiare tali dati solo nel caso in cui vi sia una buona ragione per farlo.

(a) Cambiare il sistema di coordinate di base
Non è necessario cambiare il sistema di coordinate di base dal programma.

(b) Cambiare il sistema di coordinate del lavoro
Quando si programma una posizione sul robot, il robot ricorderà anche il sistema di coordinate del lavoro valido in quel momento. Quando un comando di movimento comunica al robot di spostarsi su quel punto, il sistema di coordinate del lavoro attuale (in vigore nel momento in cui è stato riscontrato il comando nel programma) tornerà automaticamente al precedente sistema

di coordinate del lavoro (in vigore quando è stato programmato il punto sul robot).

Esempio:
Abbiamo a disposizione tre sistemi diversi di coordinate del lavoro, ovvero WORK1, WORK2 e WORK3. Per ogni sistema di coordinate, è stato programmato un solo punto sul robot, ovvero c'è il punto A1 definito in base al sistema di coordinate del lavoro WORK1, il punto A2 definito in base al sistema di coordinate del lavoro WORK2 e il punto A3 definito in base al sistema di coordinate del lavoro WORK3. Quando si esegue il seguente programma, il sistema di coordinate del lavoro cambia come di seguito spiegato.

PROGRAM SAMPLE
MOVE A1: Il sistema di lavoro in vigore con questo comando sarà WORK1.
MOVE A2: Il sistema di lavoro in vigore con questo comando sarà WORK2.
MOVE A3: Il sistema di lavoro in vigore con questo comando sarà WORK3.
END

Quando si gestiscono più pezzi di lavoro, è possibile eseguire la stessa operazione più volte su pezzi di lavoro diversi cambiando il sistema di coordinate del lavoro. Vi sono tre modi per farlo.

1) Cambiare il sistema di coordinate del lavoro stesso
La stessa operazione può essere eseguita su più pezzi di lavoro cambiando i valori del sistema di coordinate del lavoro stesso dal programma.

Esempio:
Ci sono tre punti (A1, A2 e A3) definiti in base al sistema di coordinate del lavoro WORK1. Si vuole ripetere questa operazione, ma questa volta in base al sistema di coordinate del lavoro WORK2.

```
PROGRAM SAMPLE
    DUMMYWORK = WORK1
    WORK1 = WORK2
    MOVE A1
    MOVE A2
    MOVE A3
     WORK1 = DUMMYWORK
END
```

I movimenti verso i punti A1, A2 e A3 vengono eseguiti in base a WORK1 proprio come prima. L'unica differenza è che il valore di WORK1 stesso è stato precedentemente cambiato a quello di WORK2. Quindi, il robot si muove nella cornice di riferimento di WORK2.
La variabile DUMMYWORK viene usata per mantenere il valore dell'originale WORK1. In caso contrario, il valore del WORK1 originale si perderebbe per sempre quando si inserisce il valore di WORK2 in WORK1. Quando il robot smette di muoversi, viene ripristinato il valore originale di WORK1.

2) Cambiare il sistema di coordinate del lavoro con un'istruzione WITH
È possibile cambiare diversi sistemi di coordinate del lavoro utilizzando istruzioni WITH.

Esempio:
Ci sono tre punti (A1, A2 e A3) definiti in base al sistema di coordinate del lavoro WORK1. Si vuole ripetere questa operazione, ma questa volta in base al sistema di coordinate del lavoro WORK2.

```
PROGRAM SAMPLE
    MOVE A2
    WORK2=WORK+TRANS(, , 20)
    MOVE A1 WITH WORK = WORK2
    MOVE A2 WITH WORK = WORK2
    MOVE A3 WITH WORK = WORK2
END
```

Utilizzando le istruzioni WITH in questa maniera, è possibile specificare sistemi di coordinate del lavoro diversi da quello utilizzato durante la programmazione.

3) Metodo generale
Come metodo generale, consigliamo di usare una combinazione dei due metodi sopra descritti. Nello specifico, il programma dovrebbe utilizzare variabili temporanee per definire il sistema di coordinate del lavoro. Ogni volta che si cambia il pezzo di lavoro, si dovrebbe usare una variabile temporanea corrispondente per cambiare il sistema di coordinate del lavoro.

Esempio:
Ci sono tre punti (A1, A2 e A3) definiti in base al sistema di coordinate del lavoro WORK1. Si vuole ripetere questa operazione, ma questa volta in base al sistema di coordinate del lavoro WORK2.

```
PROGRAM SAMPLE
   WORK2=WORK+TRANS(, , 20)
   DUMMYWORK = WORK2
   MOVE A1 WITH WORK = DUMMYWORK
   MOVE A2 WITH WORK = DUMMYWORK
   MOVE A3 WITH WORK = DUMMYWORK
END
```

(c) Cambiare il sistema di coordinate dell'utensile

Cambiando il sistema di coordinate dell'utensile, è possibile gestire operazioni nelle quali gli utensili vengono cambiati più volte man mano che il lavoro procede.

Il robot usa sempre il sistema di coordinate dell'utensile attuale al fine di spostare la punta dell'utensile sulla posizione definita dal sistema di coordinate del lavoro. Quindi, se non si presta attenzione quando si specifica il sistema di coordinate dell'utensile, il robot potrebbe spostarsi verso un punto non previsto.

Esempio:
Si stanno usando due utensili con offset TOOL1 e TOOL2.

```
PROGRAM SAMPLE
   TOOL1=TRANS(, , 10)
   TOOL2=TRANS(, , 30)
   TOOL = TOOL1
   MOVE A1
   TOOL = TOOL2
   MOVE A1
END
```

Questo programma prende due utensili diversi e posiziona le punte di questi utensili nella stessa posizione (punto A1). (Questo programma non presenta una routine di cambio degli utensili come sarebbe richiesto per il funzionamento reale.)

5.3.3 Spostamento rapido

Il linguaggio SCOL contempla un singolo movimento per iniziare quando il robot comincia a muoversi e per terminare quando il robot finisce di posizionarsi. Normalmente, il comando di un movimento corrisponde ad un movimento.

Tuttavia, si può anche dire al robot di muoversi continuamente in risposta a più comandi di movimento senza fermarsi per posizionarsi prima di dirigersi verso la destinazione successiva o di passare al punto più vicino in direzione della destinazione successiva. Questo viene definito spostamento rapido.

Lo spostamento rapido riduce i tempi di funzionamento dal momento che il robot può evitare gli angoli e non deve perdere tempo a posizionarsi.

Lo spostamento rapido non può essere eseguito tra il comando MOVES o MOVEC e altri comandi di movimento. (Tuttavia, lo spostamento rapido è disponibile tra il comando MOVES e il comando MOVEC.)

(1) Specificare lo spostamento rapido
Lo spostamento rapido viene attivato o disattivato con l'interruttore di sistema PASS.

ENABLE PASS – Attiva lo spostamento rapido.
DISABLE PASS – Disattiva lo spostamento rapido.

Con lo spostamento rapido, il robot cambia continuamente le velocità degli assi prestando attenzione a non superare nessuna velocità massima. Quando la quantità di movimento per singolo comando di movimento ha superato la percentuale specificata, il robot comincerà ad eseguire il successivo comando di movimento. Questa percentuale è specificata con la variabile di sistema PASS (che è diversa dall'interruttore di sistema PASS). Questa percentuale, che è formalmente chiamata parametro di spostamento rapido, può essere specificata come un valore intero tra 0 e 100. Qualunque valore inferiore a 50 verrà trattato come 50%.

Esempio:

```
PROGRAM SAMPLE
    MOVE A1          Spostarsi al punto A1.
    PASS = 80        Impostare il parametro di spostamento rapido su
                     80%.
    ENABLE PASS      Attivare lo spostamento rapido.
```

```
    MOVE A2          Quando viene completato l'80% del movimento
                     verso il punto A2, comincia quello verso il punto A3.
    MOVE A3          Quando viene completato l'80% del movimento
                     verso il punto A3, comincia quello verso il punto A4.
    DISABLE PASS     Disattivare lo spostamento rapido.
    MOVE A4          Spostarsi al punto A4.
END
```

Il parametro di spostamento rapido può essere cambiato mentre lo spostamento rapido è in azione.

Esempio:

```
PROGRAM SAMPLE
    MOVE A1          Spostarsi al punto A1.
    PASS = 80        Impostare il parametro di spostamento rapido su
                     80%.
    ENABLE PASS      Attivare lo spostamento rapido.
    MOVE A2          Quando viene completato l'80% del movimento
                     verso il punto A2, comincia quello verso il punto A3.
    MOVE A3 WITH PASS = 60
                     Quando viene completato il 60% del movimento
                     verso il punto A3, comincia quello verso il punto A4.
    MOVE A4          Quando viene completato l'80% del movimento
                     verso il punto A4, comincia quello verso il punto A5.
    PASS = 90        Imposta il parametro di spostamento rapido su 90%.
    MOVE A5          Quando viene completato il 90% del movimento
                     verso il punto A5, comincia quello verso il punto A6.
    DISABLE PASS     Disattivare lo spostamento rapido.
    MOVE A6          Spostarsi al punto A6.
END
```

(2) Comandi che interrompono lo spostamento rapido

I seguenti comandi interrompono lo spostamento rapido nel caso in cui questo fosse in azione in quel momento.

- Comando WAIT
- Comando INPUT
- Comando PRINT
- Comando STOP

Comando BREAK
Comando PAUSE

Inoltre, nel caso in cui DISABLE NOWAIT fosse attivo, i seguenti comandi interromperebbero lo spostamento rapido.

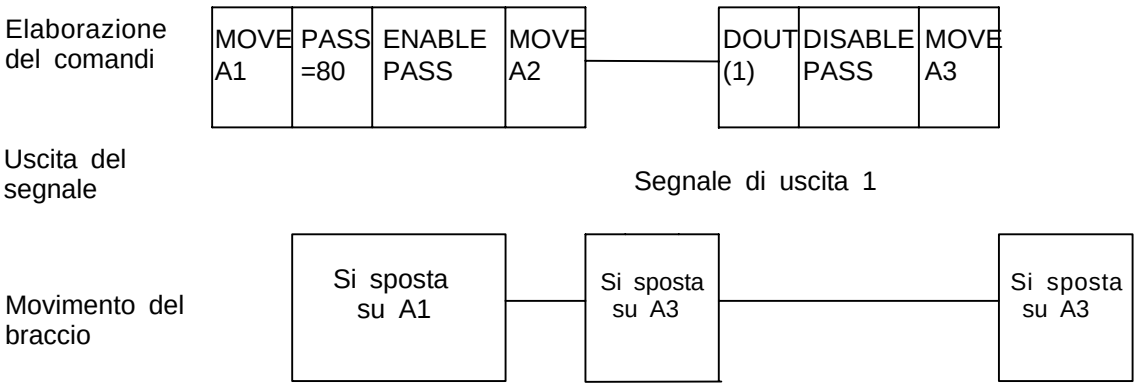
Comando DOUT
Comando RESET DOUT
Comando PULOUT
Comando DIN
Comando BCDIN
Comando BCDOUT

Inoltre, quando vi sono molti comandi tra i comandi di movimento o quando la quantità di movimento di un singolo movimento è bassa, lo spostamento rapido può essere arrestato. Quando lo spostamento rapido è specificato tra il comando MOVES o MOVEC e un altro comando di movimento, lo spostamento rapido sarà arrestato.

(3) Tempi dei segnali di uscita con lo spostamento rapido
Con il seguente esempio vengono illustrati i tempi dei segnali di uscita con lo spostamento rapido relativi al movimento del braccio del robot.

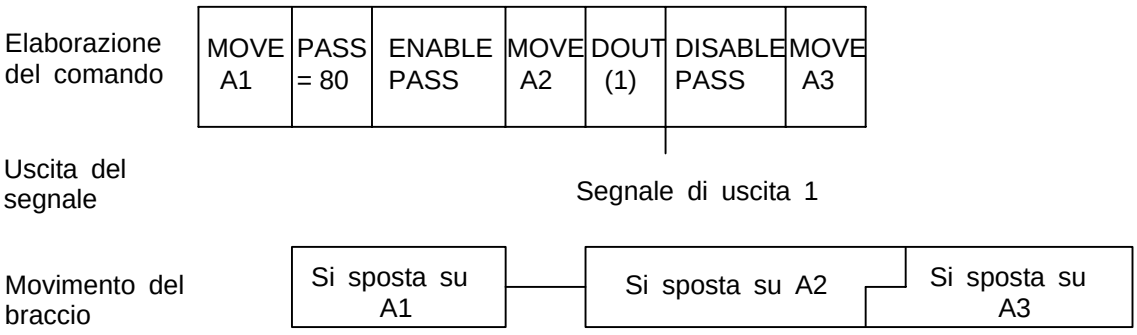
```
PROGRAM SAMPLE
    MOVE A1
    PASS = 80
    ENABLE PASS
    MOVE A2
    DOUT (1)
    DISABLE PASS
    MOVE A3
END
```

(a) Tempi con DISABLE NOWAIT



In questo caso, lo spostamento rapido verrà interrotto per poter elaborare il segnale di uscita.

(b) Tempi con ENABLE NOWAIT



In questo caso, il segnale di uscita viene elaborato mentre il braccio è in movimento.

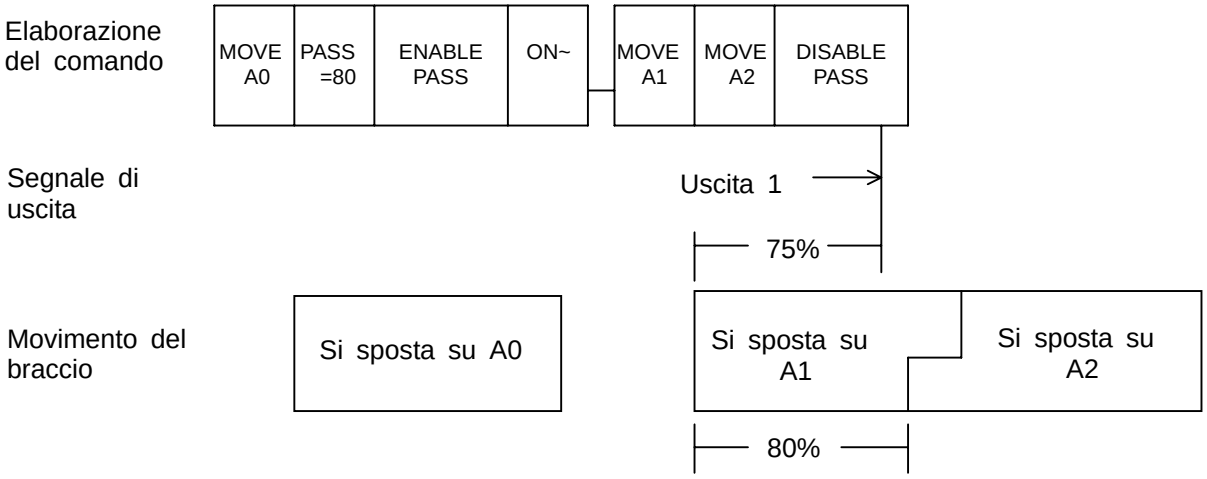
(4) In riferimento alle condizioni operative del robot con lo spostamento rapido Quando ci si riferisce alla quantità di movimento del robot con uno tra i comandi MOTION, MOTIONT, REMAIN o REMAINT con lo spostamento rapido, il risultato sarà un valore per un singolo comando di movimento.

Esempio:

```
PROGRAM SAMPLE
    MOVE A0
    PASS = 80
    ENABLE PASS
    ON MOTION > = 75 DO DOUT (1)
```

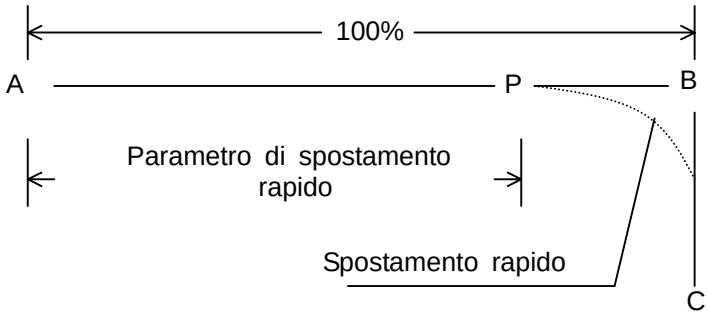
```
    MOVE A1
    MOVE A2
    DISABLE PASS
END
```

In questo caso, il Segnale 1 sarà emesso quando il robot si sarà spostato del 75% del valore specificato da A0 a A1.



(5) Parametri di controllo per lo spostamento rapido

Il parametro di spostamento rapido ha il seguente significato.



Nello schema sopra riportato, il robot comincia al punto A, passa nelle vicinanze del punto B e si sposta sul punto C. Nello spostamento rapido, il robot si sposta dal punto A verso il punto B fin quando non raggiunge il punto P. Quando il robot raggiunge il punto P, comincia a spostarsi verso il punto C. La posizione del punto P è definita come percentuale della distanza totale tra i punti A e B. In altre parole, il valore di P è il parametro di spostamento rapido. A volte questo parametro viene anche denominato tasso di passaggio.

Nell'esempio precedente, il parametro di spostamento rapido (rapporto di

passaggio) si ottiene così:

Rapporto di passaggio = ((Distanza da A a P) / (Distanza da A a B)) * 100 (%)
Il parametro di spostamento rapido deve rientrare nella gamma compresa tra 50 e 100%. Qualunque valore inferiore a 50% sarà computato come 50%.
A volte, lo spostamento rapido potrebbe non funzionare esattamente come descritto per le ragioni di seguito elencate.

(a) Restrizioni sulla massima accelerazione
A seconda della struttura del robot e delle parti in uso, si determina una velocità massima. Inoltre, l'accelerazione e la decelerazione del robot variano a seconda della riduzione della velocità del movimento del robot e dell'uso dei comandi ACCEL e DECEL scritti nel linguaggio del robot. Durante lo spostamento rapido, la velocità di ciascun asse viene calcolata in modo tale che queste accelerazioni non vengano superate.

(b) Restrizioni dal seguente movimento
Lo spostamento rapido sovrappone il movimento attuale al movimento successivo in una maniera tale che il movimento attuale termina quando il robot ha eseguito il 50% del movimento successivo.

Per le ragioni sopra descritte, i tempi secondo i quali lo spostamento rapido comincia non possono essere accelerati oltre una certa quantità. Ciò si applica anche ai casi in cui il parametro di spostamento rapido (rapporto di passaggio) è impostato su un valore più basso.

(6) Avviso nel caso in cui la direzione dello spostamento rapido diventi la stessa del vettore
Quando lo spostamento rapido diventa ad una direzione, a volte diventa più veloce della velocità che l'ingrediente di velocità ha aumentato e della velocità specificata.

5.3.4 Configurazione del robot

Con i robot SCRA, partendo dal centro di rotazione del robot secondo la cui configurazione il braccio si estende dritto, quando il secondo braccio si piega verso sinistra (ovvero quando il gomito sporge a destra), si parla di sistema destrorso, e quando il secondo braccio si piega verso destra (ovvero quando il gomito sporge a sinistra), si parla di sistema sinistrorso. Quindi il robot può assumere due posizioni: configurazione del sistema destrorso e configurazione del sistema sinistrorso su una posizione (X, Y) specificata nel sistema di coordinate del lavoro.

(1) Configurazione durante la programmazione e configurazione durante il funzionamento

Quando si programma una posizione sul robot, il robot ricorderà anche la sua configurazione in quel momento (come dati di posizione). Durante il normale funzionamento, il robot si sposterà secondo la configurazione che aveva al momento della programmazione.

(a) Specificare la configurazione per il movimento

La configurazione che il robot deve assumere durante il movimento è specificata dal comando CONFIG. Il comando CONFIG può essere usato o in modo indipendente o con un'istruzione WITH.

Quando il comando CONFIG viene usato con un'istruzione WITH, il comando CONFIG è valido solo per un singolo comando di movimento.
Quando un comando CONFIG viene usato da solo, tutti i comandi di movimento successivi vengono eseguiti con quella configurazione.
Se si vuole che il robot si muova con la stessa configurazione con la quale è stato programmato, è necessario impostare la configurazione del robot su indefinito, CONFIG=0 o CONFIG=FREE. L'impostazione iniziale del robot è FREE.

CONFIG = 1 o CONFIG = LEFTY impostano la configurazione del robot a sinistra e CONFIG = 2 o CONFIG = RIGHTY la impostano a destra.

Quando si imposta una configurazione del robot, il robot si muove secondo la configurazione specificata durante l'esecuzione del comando di movimento successivo.

Nel caso in cui si azionasse il robot con una configurazione diversa da quella con la quale è stato programmato, vi potrebbe essere una discrepanza tra il luogo verso il quale il robot si sarebbe dovuto spostare secondo la sua

programmazione e quello verso il quale si sposta. Quindi, è consigliabile cercare sempre di azionare il robot con la configurazione con la quale è stato programmato.

(b) Comandi per i quali la configurazione non può essere cambiata
Qualunque specifica di configurazione diventa invalida quando si esegue uno qualsiasi dei seguenti comandi di movimento.

1) Comandi di movimento interpolato
Con il comando per l'interpolazione lineare e l'interpolazione ad arco, dal momento che la configurazione del robot non può essere cambiata, si verificherà un errore. Ci sono due comandi di movimento interpolato, ovvero MOVES e MOVEC.

2) Comandi di controllo di un singolo asse
Con i comandi che muovono un singolo asse del robot, la configurazione del robot non può essere specificata. Ci sono due comandi di controllo di un singolo asse, ovvero MOVEA e MOVEI.

(2) Spostamento verso posizioni create nel programma
Quando si specificano direttamente dei valori di coordinate secondo le quali il robot si deve muovere (ad esempio, MOVE POINT (100, 100)), il robot si sposterà secondo la configurazione in vigore in quel momento. Questo è valido anche se la configurazione è indefinita.

Se si creano dati di posizione nel proprio programma, la configurazione del robot sarà la configurazione dei dati di posizione in quella variabile prima della sostituzione con i nuovi valori. Se la variabile con i dati di posizione non possiede nessuna configurazione specifica (ovvero se quella variabile subisce una sostituzione per la prima volta), la configurazione del robot sarà indefinita (libera).

5.3.5 Blocchi di dati

Questo paragrafo descrive come sono conservati i dati di posizione nei file del controller. Il programmatore non si deve preoccupare particolarmente di questo se sta programmando con il programmatore portatile. Piuttosto, questo paragrafo serve quando si crea un programma di linguaggio SCOL o dati di posizione con qualsiasi computer diverso dal controller robot.

Tutti i file di dati di posizione sono in codice ASCII.

(1) Blocchi di dati

I dati di posizione del robot sono archiviati nel controller in unità chiamate blocchi di dati. Il file nel controller contiene una molteplicità di programmi ed un solo blocco di dati. Un singolo file possiede sempre un singolo blocco di dati. I programmi dello stesso file condividono dati di posizione nel blocco di dati. Non ci si può riferire al blocco di dati di un file differente. Il blocco di dati archivia i dati di tipo coordinate e carico oltre a quelli di tipo posizione.

Un blocco di dati viene dichiarato in un file nel seguente modo:

```
DATA
(dichiarazioni di dati)
. . .
END
```

Un blocco di dati è dichiarato da DATA a END. Ogni singolo dato individuale viene dichiarato da DATA a END, uno dopo l'altro. Il blocco di dati è dichiarato alla fine del file. Il blocco di dati non può essere dichiarato in un programma.

(2) Dichiarazioni di dati

I dati di tipo posizione, i dati di tipo coordinate e i dati di tipo carico sono generalmente programmati sul robot con l'editor dei dati del controller. In tal caso, i dati vengono automaticamente inseriti nel blocco dei dati del file appropriato.

I dati definiti (creati) nel programma (e non dall'editor dei dati) non sono inseriti nel blocco dei dati. Piuttosto, questi dati sono temporaneamente archiviati nel controller (e non nel file) mentre il programma è in funzione.

I dati di un blocco di dati vengono dichiarati con il seguente formato.

<tipo di dati> <identificatore> = [<elemento>,]...

La designazione <tipo di dati> indica il tipo di dati che si stanno dichiarando. In questo esempio si dovrebbe scrivere POINT per i dati di tipo posizione, TRANS per i dati di tipo coordinate o PAYLOAD per i dati di tipo carico.

La designazione <identificatore> indica il nome dei dati. La designazione <elemento> indica il valore numerico di ciascun elemento in numeri reali. Qualunque designazione <elemento> omessa verrà considerata come 0.

(a) Dichiarare dati di posizione

I dati di posizione vengono dichiarati con il seguente formato.

POINT <identificatore> = X, Y, Z, C, T/<configurazione>

X, Y, Z, C e T sono valori di coordinate espressi in numeri reali. Le unità sono in millimetri o gradi.

<configurazione> è un numero intero da 0 a 2 che designa la configurazione del robot.

NONE: Indefinito (libero)
LEFTY: Sinistrorso
RIGHTY: Destrorso

Le costanti di sistema FREE (indefinito), LEFTY (sinistrorso) e RIGHTY (destrorso) possono anche essere usate per designare la configurazione del robot.

Esempi:

```
POINT A = 100, 200, 30, 45, 0
POINT A1 = 444.44, 333.33, , ,/RIGHTY
POINT ZERO =
```

(b) Dichiarare dati di tipo coordinate

I dati di tipo coordinate vengono dichiarati con il seguente formato.

TRANS <identificatore> = X, Y, Z, C

X, Y, Z e C sono valori di coordinate espressi in numeri reali. Le unità sono in millimetri o gradi.

Esempi:

```
TRANS WORK1 = 10, 20, 30, 45
TRANS TOOL2 = , , –20,
TRANS ZEROW =
```

(c) Dichiarare dati di carico

I dati di carico vengono dichiarati con il seguente formato.

PAYLOAD <identificatore> = <peso>, <offset del baricentro>

<peso> è la massa che agisce sulla punta della mano del robot espressa in numeri reali. Le unità sono in chilogrammi.

<offset del baricentro> è l'offset del baricentro che agisce sulla punta della mano del robot espressa in numeri reali. Le unità sono in millimetri.

Esempi:

```
PAYLOAD HAND1 = 4.8, 0.48
PAYLOAD HAND2 = 2, 0.004
PAYLOAD HAND0 =
```

(3) Specificare i sistemi di coordinate del lavoro

I dati di posizione sono specificati nel linguaggio SCOL come la posizione della punta dell'utensile definita in termini di sistema di coordinate del lavoro. Quindi, i dati di posizione contengono anche informazioni su quale sistema di lavoro è stato usato per definire tali dati. Per specificare esplicitamente un sistema di coordinate del lavoro, utilizzare un'istruzione WORK per attribuire un nome al sistema di coordinate del lavoro. Quindi è possibile utilizzare il sistema di coordinate del lavoro nel proprio programma. Le coordinate del lavoro dichiarate in questa maniera rimangono in vigore fino a quando non vengano sostituite da un'altra istruzione WORK.

Esempio:

```
DATA
    POINT A00 = 650, 0, 0, 0, 0
    POINT A01 = 400, 400, 0, 0 / RIGHTY
    TRANS WORK1 = 100, – 100, 0, 0
    WORK WORK1
    POINT A10 = 0, 0, 0, 0, 0, 0
    POINT A11 = 200, 0, 0, 0, 0, 0
    TRANS WORK2 = –100, –100, 0, 0, 0
    WORK WORK2
    POINT A20 = 246.8, 69.1, 23.5, 18.3, / RIGHTY
    POINT A21 = 0, 0, –30, 0, 0 / LEFTY
END
```

Nell'esempio precedente, i dati di posizione A10 e All sono definiti in termini di WORK1 e i dati di posizione A20 e A21 sono definiti in termini di WORK2. Inoltre, dal momento che non era in vigore nessuno specifico sistema di coordinate del lavoro quando sono stati specificati i dati di posizione A00 e A01, il sistema di coordinate del lavoro per questi punti è stabilito come {0, 0, 0, 0}.

5.3.6 Blocco dei dati globali

La variabile definita dal linguaggio SCOL contiene i dati globali e i dati temporanei. I dati globali ai quali si può fare riferimento da tutte le parti del programma sono descritti in questo paragrafo.

La variabile definita nel blocco di dati viene trattata come dati globali. I dati che possono essere definiti nel blocco dei dati sono limitati al tipo posizione, tipo coordinate e tipo carico.

La dichiarazione di dati globali può usare dati di tipo intero, di tipo numero reale e di tipo array.

(1) Blocco dei dati globali

I dati globali vengono dichiarati nel blocco dei dati globali e vengono trattati come parte del programma in maniera differente dal blocco dei dati. Il blocco dei dati globali è modificato dall'editor del programma. Il file del controller contiene una molteplicità di programmi ed un solo blocco di dati globali. Un singolo file possiede un singolo blocco di dati globali.

I dati globali possono essere condivisi nei programmi dello stesso file. Non è possibile fare riferimento a dati globali di diversi file. I dati interi, i dati di numeri reali e i dati array possono essere archiviati nel blocco dei dati globali.

Il blocco dei dati globali viene dichiarato nel file usando il seguente formato.

```
GLOBAL
(Dichiarazione dei dati)
. . .
END
```

Il blocco dei dati globali viene dichiarato nelle istruzioni GLOBAL ~ END.
I dati vengono dichiarati nelle istruzioni GLOBAL ~ END uno alla volta.
Il blocco dei dati globali è dichiarato all'inizio del file.
Il blocco dei dati globali non può essere dichiarato nel programma.

(2) Dichiarazione dei dati

Il blocco dei dati globali non può essere modificato dall'editor dei dati.
È modificato dall'editor del programma alla stessa stregua del programma.
I dati nel blocco dei dati globali sono dichiarati nel formato di istruzioni di sostituzione per la variabile nella stessa maniera del programma.

(a) Dichiarazione di dati interi
I dati interi vengono dichiarati come di seguito.

<Identificatore> = <costante intera>
Esempio: N1 = 1

Nota: Se si sostituisce la variabile globale di tipo intero con il numero reale, le cifre decimali vengono omesse. Prestare attenzione al riguardo.

(b) Dichiarazione di numero reale
I numeri reali vengono dichiarati come di seguito.

<Identificatore> = <costante numero reale>
Esempio: R1 = 1.0

(c) Dichiarazione di dati array
I dati array vengono dichiarati come di seguito.

DIM <identificatore> (I, j . . .) AS <tipo di variabile>

Esempio: Array tridimensionale di tipo intero a $2 \times 3 \times 4$ elementi
DIM IDAT (2, 3, 4) AS INT

Array bidimensionale di tipo numero reale a 4×3 elementi
DIM RDAT (10, 50) AS REAL

Array monodimensionale di tipo posizione a 5 elementi
DIM PDAT (5) AS POINT

Nota: Per il comando DIM, si specifica solo il tipo e numero di elementi di dati globali di tipo array e non si stabilisce il valore iniziale. Come per i dati globali normali, i valori iniziali di dati di tipo intero e numero reale dovrebbero essere specificati nei blocchi dei dati globali, e i dati di tipo posizione, di tipo coordinate e di tipo carico nei blocchi dei dati.

5.3.7 Velocità del movimento del robot

(1) Velocità in ogni modalità

Di seguito è illustrata la gamma di velocità del robot in ogni modalità.

Modalità		Gamma di velocità
Modalità di funzionamento automatico	PTP	0 ~ 100 %
	Interpolazione lineare e circolare	0 ~ 100 %
Modalità test	PTP	0 ~ 25%
	Interpolazione lineare e circolare	0 ~ 25 %

Nota: La velocità massima elencata nelle specifiche è considerata come il 100%. Con l'interpolazione lineare e circolare, il 100% è 1 m/s.

(2) Velocità per il funzionamento automatico

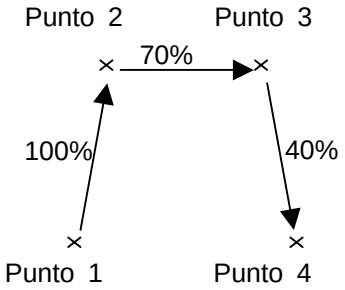
Le velocità per il funzionamento automatico e per il test hanno il seguente formato.

Velocità = {Impostazione della velocità in programma (da 1 a 100%) × velocità di controllo (da 1 a 100%); velocità limite}

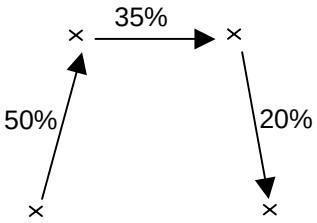
In questo caso,

Impostazione della velocità in programma: Impostazione specificata dal comando SPEED nel programma del robot.
Velocità di controllo: Cambia tutte le velocità con la stessa frazione.
Velocità limite: Riduce al valore limite qualsiasi velocità che sia al di sopra dello stesso.

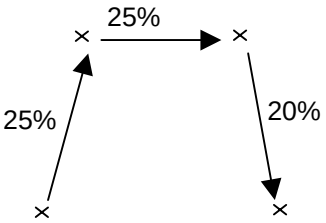
Impostazione del programma → controllo 50% → limite 25%



È possibile impostare la velocità per ogni operazione.



La velocità per tutti i movimenti è il 50% del valore di velocità specificato.



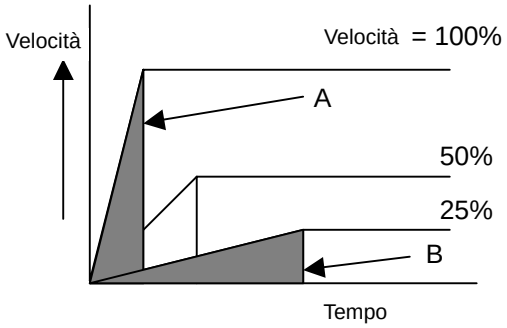
Dal momento che la velocità di movimento dal Punto 1 al Punto 3 supera il 25%, è ridotta al 25%.

5.3.8 Accelerazione del robot

L'accelerazione (e la decelerazione) del robot varierà a seconda dei seguenti fattori.

[1] Modalità operativa: Interpolazione lineare (MOVES), punto a punto (MOVE)
[2] Velocità: Comando SPEED, controllo
[3] Comandi di accelerazione: ACCEL, DECEL

(1) L'accelerazione massima in ciascuna modalità operativa è calcolata dalla forza del robot e dalla coppia del motore e impostata. Inoltre, i motori accelereranno e decelereranno in modo tale che il robot si muova in maniera scorrevole.

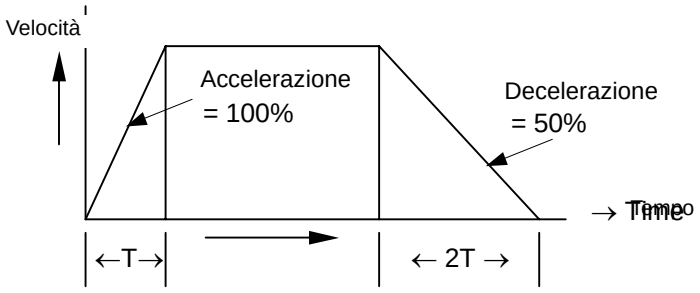


Cambio di velocità a seconda del controllo

Come mostrato nella figura qui sopra, il tempo di accelerazione è inversamente proporzionale all'impostazione di controllo della velocità. La distanza da percorrere durante l'accelerazione (area del triangolo A e B nella figura) è costante a prescindere dall'impostazione del controllo della velocità.

Quindi, il percorso del robot rimarrà più o meno lo stesso a prescindere dall'impostazione di controllo.

(2) L'accelerazione e la decelerazione sono specificati separatamente nel linguaggio SCOL. Questo risulta utile quando si gestiscono parti delicate, per le quali, ad esempio, si può decelerare il robot gradualmente prima di bloccarlo così da evitare che la mano e il pezzo di lavoro vibrino.



Appendice A Elenco di comandi

Comandi di controllo del movimento

MOVE<posizione>[istruzione WITH]	Movimento simultaneo
MOVES<posizione>[istruzione WITH]	Movimento di interpolazione lineare
MOVEC<posizione>,<posizione>[istruzione WITH]	Movimento di interpolazione circolare
MOVEA<asse>,<posizione assoluta>[istruzione WITH]	Movimento assoluto singolo asse
MOVEI<asse>,<posizione relativa>[istruzione WITH]	Movimento relativo singolo asse
MOVEJ<posizione>,<definizione di arco>	Movimento relativo singolo asse
READY	Si sposta all'origine delle coordinate della macchina
DELAY<time>	Resta in pausa per un tempo determinato
OPEN1,OPEN2	Apre la mano immediatamente
OPENI1,OPENI2	Apre la mano
CLOSE1,CLOSE2	Chiude la mano immediatamente
CLOSEI1,CLOSEI2	Chiude la mano
RESUME	Riprende il movimento interrotto

Comandi di controllo del programma

PROGRAM	<nome programma>	Contrassegna l'inizio di un programma
ON<condizione monitoraggio>	[{BREAK | PAUSE}]	Controlla le condizioni
DO<istruzione>		
IF<istruzione logica> THEN<istruzione> [ELSE<istruzione>]		Valuta le condizioni
WAIT<istruzione logica>		Attende un'operazione
IGNORE<condizione monitoraggio>		Cancella il monitoraggio
GOTO<etichetta>		Esegue la ramificazione senza condizioni

GOTO (<espressione>) <etichetta> [, <etichetta>]...	Esegue la ramificazione in base al valore di un'espressione
RCYCLE	Etichetta per l'impostazione del ciclo
RETURN	Torna al programma principale
FOR<variabile> = <espressione> TO <espressione> [STEP <espressione>]	Ripete un'operazione
NEXT[<variabile>]	
STOP	Arresta il programma
REMARK[<commento>]	Osservazioni e commenti
END	Contrassegna la fine di un programma
TASK (“nome programma”)	Avvia il programma dell'attività
KILL (<espressione>)	Termina il programma dell'attività
SWITCH	Cambia il programma dell'attività
LOADLIB<nome file>	Compilazione libreria a collegamento dinamico

Comandi di controllo I/O

DIN(<nome segnale>[,<nome segnale>]...)	Legge un segnale di ingresso
DOUT(<nome segnale>[,<nome segnale>]...)	Emette un segnale
PULOUT(<nome segnale>[,<nome segnale>]...)	Emette un segnale a impulso
RESET<condizione>[,<condizione>]	Ripristina il controller
BCDIN(<nome segnale>,<lunghezza segnale>)	Immette un segnale BCD
BCDOUT(<nome segnale>,<lunghezza segnale>,<espressione>)	Emette un segnale BCD
HEXIN(<nome segnale>,<lunghezza segnale>)	Immette i segnali con numerazione esadecimale
HEXOUT(<nome segnale>,<lunghezza segnale>,<espressione>)	Emette i segnali con numerazione esadecimale
PRINT[{COM0 | COM1 | TP},] {<stringa di caratteri> <espressione>}	Emette dati di comunicazione

[,{<stringa di caratteri><espressione>}]...[, CR]	
INPUT[{COM0 | COM1 | TP},] <variabile>,[<variabile>]...	Immette dati di comunicazione

Comandi di condizione del movimento

CONFIG=<espressione>	Specifica la configurazione
ACCUR=<espressione>	Specifica la precisione di posizione
ACCEL=<espressione>	Specifica l'accelerazione (durante l'accelerazione)
DECEL=<espressione>	Specifica la decelerazione (durante la decelerazione)
SPEED=<espressione>	Specifica la velocità
PASS=<espressione>	Parametro di spostamento rapido
TORQUE={<espressione>, <espressione>, <espressione>, <espressione>, <espressione>}	Coppia su ogni albero
GAIN={<espressione>, <espressione>, <espressione>, <espressione>, <espressione>}	Guadagno su ogni albero
SETGAIN=(<intero>, <intero>, <intero>, <intero>, <intero>)	Guadagno su ogni asse nel movimento sincrono
ENABLE<interruttore>[,<interruttore>]...	Accensione dell'interruttore di sistema
DISABLE<interruttore>[,<interruttore>]...	Spegnimento dell'interruttore di sistema
PAYLOAD={<peso>,<offset del baricentro>}	Imposta dati di tipo carico
FREELOAD	Cancella dati di tipo carico

Comando di pallettizzazione

INITPLT (<numero pallet>,<i>,<j>,<k>)	Inizializza un pallet
MOVEPLT (<numero pallet>, <numero elemento>, <X>, <Y>,<Z>,<C>)	Sposta un pallet su una specifica posizione

Comandi calcolatrice

SIN(<espressione>)	Seno
COS(<espressione>)	Coseno
TAN(<espressione>)	Tangente

ASIN(<espressione>)	Arcoseno
ACOS(<espressione>)	Arcocoseno
ATAN(<espressione>)	Arcotangente
ATAN2(<espressione>,<espressione>)	Arcotangente
SQRT(<espressione>)	Radice quadrata
ABS(<espressione>)	Valore assoluto

SGN(<espressione>)	Estrae segno
INT(<espressione>)	Cambia il numero in un numero intero
REAL(<espressione>)	Cambia il numero in un numero reale
LN(<espressione>)	Logaritmo naturale
<espressione> MOD <espressione>	Resto
LOG10 (<espressione>)	Logaritmo comune
EXP (<espressione>)	Elevato a potenza e
<espressione logica> AND <espressione logica>	Prodotto logico
<espressione logica> OR <espressione logica>	Somma logica
NOT <espressione logica>	Negazione
HERE	Posizione attuale
DEST	Posizione di destinazione
POINT (<espressione>, <espressione>, <espressione>, <espressione>, <espressione>, <configurazione>)	Crea dati di tipo posizione
TRANS (<espressione>, <espressione>, <espressione>, <espressione>)	Crea dati di tipo coordinate

Comandi di condizione del movimento

MOTION	Quantità di movimento eseguito
MOTIONT	Tempo impiegato per un movimento
REMAIN	Quantità di movimento rimanente da eseguire
REMAINT	Tempo rimanente per un movimento
TIMER	Timer
MODE	Modalità operativa del sistema
TOOL	Sistema di coordinate utensile
BASE	Sistema di coordinate di base
WORK	Sistema di coordinate lavoro

Appendice B Elenco di parole riservate

ABS	ACCEL	ACCUR	ACOS
AND	AS	ASIN	ATAN
ATAN2	BASE	BCDIN	BCDOUT
BREAK	CLOSE1	CLOSE2	CLOSEI1
CLOSEI2	COARSE	COM0	COM1
CONFIG	CONT	COS	CR
CYCLE	DATA	DECEL	DELAY
DEST	DIM	DIN	DISABLE
DO	DOUT	ELSE	ENABLE
END	EXP	FINE	FOR
FREE	FREELOAD	GAIN	BLOBAL
GOTO	HERE	HEXIN	HEXOUT
IF	IGNORE	INITPLT	INPUT
INT	KILL	LATCH	LATCHTRG1~8
LATCHSIG1~8	LATCHPSN1~8	LEFTY	LOADLIB
LN	LOG10	MAXTASK	MOD
MODE	MOTION	MOTIONT	MOVE
MOVEA	MOVEC	MOVEI	MOVEJ
MOVEPLT	MOVES	MOVESYNC	NEXT
NOT	NOWAIT	OFF	OFFGAIN
ON	ONGAIN	OPEN1	OPEN2
OPENI1	OPENI2	OR	PAI
PASS	PAUSE	PAYLOAD	PLCDATAR1~8
PLCDATAW1~8	POINT	PRINT	PROGRAM
PULOUT	RCYCLE	READY	REAL
REMAIN	REMAINT	REMARK	RESET
RESTORE	RESUME	RETURN	RIGHTY
SAVEEND	SEGMENT	SETGAIN	SGN
SIN	SMOOTH	SPEED	SQRT
STEP	STOP	SWITCH	TAN
TASK	THEN	TID	TIMER
TO	TP	TOOL	TORQUE
TRANS	WAIT	WITH	WORK

Appendice C Contenuti del file di libreria (SCOL.LIB)

I contenuti del file di libreria incluso come standard nel sistema sono illustrati di seguito. I dettagli possono variare leggermente a seconda del cliente.

Dopo aver cambiato il file di libreria, assicurarsi di eseguire nuovamente il comando SELECT. In caso contrario, il cambiamento apportato non si rifletterà sul programma attualmente selezionato.

```
    ‘(C) COPYRIGHT 2008 by TOSHIBA MACHINE CO., LTD.
    ‘ALL RIGHTS RESERVED.
    ‘TS3000 SCOL SUBPROGRAM LIBRARY

PROGRAM READY
    MOVEA 3, 0
    MOVEA 4, 0
    MOVEA 2, 0
    MOVEA 1, 0
    MOVEA 5, 0
END

PROGRAM OPEN1                    ’ OPEN HAND-1
    WAIT MOTION > = 100
    DOUT (203, –204)
END

PROGRAM CLOSE1                   ’ CLOSE HAND-1
    WAIT MOTION > = 100
    DOUT (–203, 204)
END

PROGRAM OPENI1                   ’ OPEN HAND-1 IMMEDIATE
    DOUT (203, –204)
END

PROGRAM CLOSEI1                  ’ CLOSE HAND-1 IMMEDIATE
    DOUT (–203, 204)
END
```

```
PROGRAM OPEN2                                   ' OPEN HAND-2
    WAIT MOTION > = 100
    DOUT (201, –202)
END

PROGRAM CLOSE2                                  ' CLOSE HAND-2
    WAIT MOTION > = 100
    DOUT (–201, 202)
END

PROGRAM OPENI2                                  ' OPEN HAND-2 IMMEDIATE
    DOUT (201, –202)
END

PROGRAM CLOSEI2                                 ' CLOSE HAND-2 IMMEDIATE
    DOUT (–201, 202)
END

PROGRAM FREELOAD                                ' FREE PAYLOAD
    PAYLOAD = (0, 0)
END

PROGRAM SETGAIN (J_0, J_1, J_2, J_3, J_4)       ' SET GAIN VARIABLE
    WAIT MOTION>=100
    GAIN = (J_0, J_1, J_2, J_3, J_4)
    MOVE HERE WITH PASS=100
    MOVE HERE
END

PROGRAM ONGAIN (J_1, J_2, J_3, J_4, J_5)
    J_6 = 0
    J_7 = 0
    J_8 = 0
    J_9 = 0
    J_0 = 0
    IF J_1 == 0 THEN J_6 = GAIN.1 ELSE J_6 = 1
```

```
    IF J_2 == 0 THEN J_7 = GAIN.2 ELSE J_7 = 1
    IF J_3 == 0 THEN J_8 = GAIN.3 ELSE J_8 = 1
    IF J_4 == 0 THEN J_9 = GAIN.4 ELSE J_9 = 1
    IF J_5 == 0 THEN J_0 = GAIN.5 ELSE J_0 = 1
    WAIT MOTION >= 100
    GAIN = (J_6, J_7, J_8, J_9, J_0)
    MOVE HERE WITH PASS = 100
    MOVE HERE
    RETURN
END

PROGRAM OFFGAIN (J_1,J_2,J_3,J_4,J_5)
   J_6 = 0
   J_7 = 0
   J_8 = 0
   J_9 = 0
   J_0 = 0
   IF J_1 == 0 THEN J_6 = GAIN.1 ELSE J_6 = 0
   IF J_2 == 0 THEN J_7 = GAIN.2 ELSE J_7 = 0
   IF J_3 == 0 THEN J_8 = GAIN.3 ELSE J_8 = 0
   IF J_4 == 0 THEN J_9 = GAIN.4 ELSE J_9 = 0
   IF J_5 == 0 THEN J_0 = GAIN.5 ELSE J_0 = 0
   WAIT MOTION > = 100
   GAIN = {J_6, J_7, J_8, J_9, J_0}
   MOVE HERE WITH PASS = 100
   MOVE HERE
   RETURN
END
```

Appendice D Domini e gamme di funzioni di calcolo

Funzione	Dominio di argomenti X, Y	Gamma del risultato Z
SIN (X)	(*)	$-1 \le Z \le 1$
COS (X)	(*)	$-1 \le Z \le 1$
TAN (X)	(*)	(*)
ASIN (X)	$-1 \le X \le 1$	$-90° \le Z \le 90°$
ACOS (X)	$-1 \le X \le 1$	$0 \le Z \le 180°$
ATAN (X)	(*)	$-90° < Z < 90°$
ATAN2 (X, Y)	$Y \neq 0$	$-180° < Z < 180°$
SQRT (X)	$X \ge 0$	$Z \ge 0$
ABS (X)	(*)	$Z \ge 0$
SGN (X)	(*)	Z = –1, 0, 1
INT (X)	(*)	(*)
REAL (X)	(*)	(*)
LN (X)	$X > 0$	(*)
X MOD Y	$Y \neq 0$	(*)
LOG10 (X)	$X > 0$	(*)
EXP (X)	(*)	$Z > 0$
Commenti	(*) si riferisce a qualsiasi numero all'interno della gamma che può essere gestito dal controller.	

Appendice E Come leggere i simboli

Di seguito vengono illustrati i significati dei tasti e dei simboli usati nel robot (i caratteri alfanumerici sono stati omessi).

[F1] ~ [F5]	:	Tasti di funzione da F1 a F5
[Esc]	:	Tasto Escape
[INS]	:	Tasto Insert
[DEL]	:	Tasto Delete
[BS]	:	Tasto Backspace
[{]	:	Parentesi graffa aperta
[}]	:	Parentesi graffa chiusa
[[]	:	Parentesi quadra aperta
[]]	:	Parentesi quadra chiusa
[Error]	:	Tasto di indicazione dell'errore
[Utility]	:	Tasto di utility
[!]	:	Punto esclamativo
[;]	:	Punto e virgola
[:]	:	Due punti
[’]	:	Apostrofo
[%]	:	Percentuale
[^]	:	Accento circonflesso
[&]	:	E commerciale
[”]	:	Virgolette (virgolette doppie)
[(]	:	Parentesi aperta
[)]	:	Parentesi chiusa
[Alt]	:	Tasto Alt (tasto alternativo)
[+]	:	Più
[–]	:	Meno
[/]	:	Barra
[*]	:	Asterisco
[ ]	:	Spazio
[<]	:	Inferiore a
[>]	:	Superiore a

[,]	:	Virgola
[.]	:	Punto
[?]	:	Punto interrogativo
[=]	:	Uguale
[EXE]	:	Tasto di esecuzione
[↑]	:	Tasto cursore (Su)
[↓]	:	Tasto cursore (Giù)
[←]	:	Tasto cursore (Sinistro)
[→]	:	Tasto cursore (Destro)

Appendice F Elenco di errori di compilazione

Nella seguente tabella vengono illustrati i messaggi degli errori di compilazione visualizzati sul programmatore portatile.

Errore Nr.	Descrizioni
200	Il sistema non è pronto per l'esecuzione.
201	Non è possibile eseguire la manutenzione della memoria di lavoro.
202	Il comando non è valido.
205	La costante del valore numerico non è valida.
206	La costante della stringa di caratteri non è valida.
207	È stato trovato un carattere che non può essere usato.
208	È stato trovato un errore nell'espressione di sostituzione.
209	È stato trovato un errore nell'espressione del formato del programma.
210	È stato trovato un errore nel formato del programma.
211	La variabile GLOBAL viene usata per l'etichetta GOTO.
212	Non è possibile inizializzare una variabile di vettore nell'area GLOBAL.
213	L'istruzione PROGRAM non si trova all'inizio della riga.
214	La posizione del comando RETURN non è valida.
215	L'istruzione PROGRAM non concorda con l'istruzione END.
216	Il comando PROGRAM è stato dichiarato nell'area DATA.
217	Il comando PROGRAM è stato dichiarato nell'area GLOBAL.
218	L'istruzione GLOBAL non si trova all'inizio della riga.
219	L'istruzione GLOBAL non concorda con l'istruzione END.
220	Il comando GLOBAL è stato dichiarato nell'area PROGRAM.
221	Il comando GLOBAL è stato dichiarato nell'area DATA.
222	Il numero di aree di monitoraggio delle condizioni (ON~DO~) supera 50.
223	L'etichetta [GOTO] è definita ripetutamente.
224	Il comando DATA è stato dichiarato nell'area PROGRAM.
225	Il comando DATA è stato dichiarato nell'area GLOBAL.
226	Questo comando non può essere dichiarato in aree diverse da quella GLOBAL.
227	Il numero delle dimensioni di array non è valido.
228	L'istruzione IF, l'istruzione THEN e l'istruzione ELSE non concordano l'una con l'altra.
229	L'istruzione FOR non concorda con l'istruzione NEXT.
230	Il numero di annidamenti delle istruzioni FOR~NEXT supera 127.
231	È stata comandata la multidefinizione della parola riservata.

Errore Nr.	Descrizioni
232	La condizione di monitoraggio non è valida.
233	L'espressione non è valida.
234	L'operatore non è valido.
235	L'etichetta RCYCLE può essere usata solo nella funzione MAIN.
236	Non è possibile usare nessuna variabile di vettore.
237	Questo comando non può essere usato per la variabile di vettore.
238	Sono stati specificati troppi elementi.
239	Non è possibile usare l'istruzione I/O per un argomento di funzione.
240	Le parentesi non sono state specificate in modo valido.
242	L'etichetta RCYCLE non può essere usata solo per la funzione MAIN.
243	Non è consentito alcun comando di salto all'area di sequenza FOR.
244	L'etichetta non si trova all'inizio della riga.
245	L'etichetta specificata è assente.
246	Non è disponibile nessun dato PROGRAM.
247	Non è possibile usare nessuna disuguaglianza per l'istruzione THEN o l'istruzione ELSE.
248	È stato trovato un errore nella parola riservata.
249	L'argomento reale e l'argomento temporaneo della funzione non sono identici.
250	Non è possibile specificare più di dieci argomenti di funzione.
252	La variabile GLOBAL non può essere usata come nome funzione.
253	La parola riservata non può essere usata come nome funzione.
254	La dichiarazione funzionale non è valida.
255	Il nome funzione è già stato dichiarato.
256	La funzione specificata è assente.
257	Il nome della variabile GLOBAL non è ancora stato definito.
258	L'etichetta GOTO viene usata come nome di variabile.
259	Questo comando non è stato dichiarato nell'area PROGRAM.
260	La parola riservata specificata non può essere usata per l'area GLOBAL o DATA.
261	Non è possibile sostituire o ridichiarare né la variabile GLOBAL né la variabile DATA.
262	La variabile o la costante specificata non può essere usata.
263	Non è possibile usare né un operatore logico né una disuguaglianza.

Errore Nr.	Descrizioni
264	Il tipo di variabile usato non è comune.
265	Sotto dichiarazione. Non è presente nessuna istruzione END.
266	Questo comando non può essere usato in una posizione diversa dall'inizio del blocco GLOBAL.
267	Il numero di variabili di backup è troppo elevato.
268	Il numero massimo totale di variabili array dichiarabili è 11.000.
269	Il numero massimo di punti di programmazione POINT specificabili è 1.500.
270	Il numero massimo di punti di programmazione diversi da POINT specificabili è 500.
271	Il numero di segnali è troppo elevato.
272	Non è consentita una doppia definizione di PASS e SMOOTH.

* Per maggiori dettagli sull'errore nr. 201 leggere le descrizioni nelle pagine successive.

Qui di seguito vengono presentate informazioni dettagliate sull'errore di compilazione nr. 201.

Nr.	Contenuti degli errori	Valore massimo	Descrizioni dettagliate
1	Buffer di conversione del codice token	75500	
2	Variabile GLOBAL	500	Il numero massimo di variabili GLOBAL definito è 500.
3	Variabili AUTO	600	Il numero massimo di variabili AUTO definito è 600.
4	Funzione	50	Il numero massimo di funzioni è 50.
5	Etichetta GOTO	1000	Il numero massimo di etichette GOTO è 1.000.
6	Programmazione delle coordinate WORK	50	Il numero massimo di coordinate WORK è 50.
7	Buffer di salvataggio dei nomi delle variabili	10000	
8	Nr. di righe del programma	6000	Il numero massimo totale di GLOBAL, programmi e blocchi di dati è 6.000 (incluse le righe vuote).
9	Tabella di informazioni sulle variabili non definite	100	Non usata.
10	Variabile di array	100	Il numero massimo di variabili di array definito è 100.
11	Dichiarazione variabile GLOBAL di tipo intero	100	Il numero massimo di variabili GLOBAL di tipo intero definito è 100.
12	Dichiarazione variabile GLOBAL di tipo numero reale	100	Il numero massimo di variabili GLOBAL di tipo numero reale definito è 100.
13	Dichiarazione variabile GLOBAL di tipo carico	100	Il numero massimo di variabili GLOBAL di tipo carico definito è 100.
14	Dichiarazione variabile GLOBAL di tipo coordinate	100	Il numero massimo di variabili GLOBAL di tipo coordinate definito è 100.
15	Dichiarazione variabile GLOBAL di tipo posizione	1500	Il numero massimo di variabili GLOBAL di tipo posizione definito è 1.500.

Nr.	Contenuti degli errori	Valore massimo	Descrizioni dettagliate
16	Dichiarazione variabile AUTO di tipo intero	500	Il numero massimo di variabili AUTO di tipo intero definito è 500.
17	Dichiarazione variabile AUTO di tipo numero reale	500	Il numero massimo di variabili AUTO di tipo numero reale definito è 500.
18	Dichiarazione variabile AUTO di tipo carico	100	Il numero massimo di variabili AUTO di tipo carico definito è 100.
19	Dichiarazione variabile AUTO di tipo coordinate	100	Il numero massimo di variabili AUTO di tipo coordinate definito è 100.
20	Dichiarazione variabile AUTO di tipo posizione	1000	Il numero massimo di variabili AUTO di tipo posizione definito è 1.000.
21	Dichiarazione variabile AUTO indefinita	100	
22	Informazioni sulla destinazione alla quale viene inviata la funzione	300	Il numero massimo di richiamate di funzione è 300. (Una (1) funzione viene richiamata in media sei (6) volte.)
23	Informazioni sulla sorgente dalla quale viene richiamata la funzione.	500	Limitata dal nr. 22.
24	Informazioni su un argomento di funzione	100	Il numero massimo di argomenti di funzione è 100. (Una (1) funzione può avere in media due (2) argomenti.)
25	Informazioni sulla sorgente dalla quale viene richiamato il comando GOTO	1000	Il numero massimo di comandi GOTO è 1.000.
26	Costante AUTO di tipo intero	2000	Il numero massimo di costanti di tipo intero usate nel blocco del programma è 2.000.
27	Costante AUTO di tipo numero reale	2000	Il numero massimo di costanti di tipo numero reale usate nel blocco del programma è 2.000.
28	Costante AUTO di tipo carico	200	Il numero massimo di costanti di tipo carico usate nel blocco del programma è 100 (= 200 ÷ 2).

Nr.	Contenuti degli errori	Valore massimo	Descrizioni dettagliate
29	Costante AUTO di tipo coordinate	100	Il numero massimo di costanti di tipo coordinate usate nel blocco del programma è 25 (= 100 ÷ 4).
30	Costante AUTO di tipo posizione	2000	Il numero massimo di costanti di tipo posizione usate nel blocco del programma è 333 (= 2000 ÷ 6).
31	Informazioni sulla stringa di caratteri AUTO	1000	Il numero massimo di costanti di tipo posizione usate nel programma è 1.000.
32	Costante GLOBAL di tipo intero	100	Il numero massimo di costanti di tipo intero usate in GLOBAL e nel blocco dei dati è 100.
33	Costante GLOBAL di tipo numero reale	100	Il numero massimo di costanti di tipo numero reale usate in GLOBAL e nel blocco dei dati è 100.
34	Costante GLOBAL di tipo carico	100	Il numero massimo di costanti di tipo carico usate in GLOBAL e nel blocco dei dati è 50 (= 100 ÷ 2).
35	Costante GLOBAL di tipo coordinate	100	Il numero massimo di costanti di tipo coordinate usate in GLOBAL e nel blocco dei dati è 25 (= 100 ÷ 4).
36	Costante GLOBAL di tipo posizione	10000	Il numero massimo di costanti di tipo posizione usate in GLOBAL e nel blocco dei dati è 1666 (= 10000 ÷ 6).
37	Nr. di variabili GLOBAL di tipo intero usate	1000	Il numero massimo totale di variabili GLOBAL di tipo intero usate è 1.000.
38	Nr. di variabili GLOBAL di tipo numero reale usate	200	Il numero massimo totale di variabili GLOBAL di tipo numero reale usate è 200.
39	Nr. di variabili GLOBAL di tipo carico usate	100	Il numero massimo totale di variabili GLOBAL di tipo carico usate è 100.

Nr.	Contenuti degli errori	Valore massimo	Descrizioni dettagliate
40	Nr. di variabili GLOBAL di tipo coordinate usate	100	Il numero massimo totale di variabili GLOBAL di tipo coordinate usate è 100.
41	Nr. di variabili GLOBAL di tipo posizione usate	3000	Il numero massimo totale di variabili GLOBAL di tipo posizione usate è 3.000.
42	Nr. di variabili AUTO di tipo intero usate	1000	Il numero massimo totale di variabili AUTO di tipo intero usate è 1.000.
43	Nr. di variabili AUTO di tipo numero reale usate	1000	Il numero massimo totale di variabili GLOBAL di tipo numero reale usate è 1.000.
44	Nr. di variabili AUTO di tipo carico usate	100	Il numero massimo totale di variabili AUTO di tipo carico usate è 100.
45	Nr. di variabili AUTO di tipo coordinate usate	100	Il numero massimo totale di variabili AUTO di tipo coordinate usate è 100.
46	Nr. di variabili AUTO di tipo posizione usate	2000	Il numero massimo totale di variabili AUTO di tipo posizione usate è 2.000.
47	Nr. di variabili AUTO indefinite usate	100	Non usata.
48	Nr. di indici sorgente GOTO	2000	Limitata dal nr. 25.
49	Nr. di indici sorgente di funzione	1000	Limitata dal nr. 22.
50	Nr. di indici argomento di funzione	200	Limitata dal nr. 24.
51	Nr. di variabili di array usate	200	Il numero massimo di variabili di array definito è 200.
52	Nr. di comandi RESTORE	100	Il numero massimo di comandi RESTORE usati è 100.
53	Nr. di numeri di variabili di array salvati	17000	Il numero massimo totale di elementi di variabili di array inizializzate è 17.000. (Nell'array di tipo posizione di dimensioni 2 × 3, è 36 (= 2 × 3 × 6).)

Nr.	Contenuti degli errori	Valore massimo	Descrizioni dettagliate
100	Nr. di indici	5000	Il numero totale di variabili, costanti, funzioni, etichette, ecc. usate è 5.000.
101	Nr. di dati numerici	25000	Il numero totale di variabili, costanti, funzioni, etichette, ecc. definite è 25.000.
102	Nr. di codici creati	399800	
200	Area di informazioni sull'esecuzione di interpretazione		

* Se c'è il file della libreria (SOCL.LIB), viene aggiunto anche il numero di dati usati lì.

Per informazioni sulle restrizioni, leggere le restrizioni sul programma SCOL descritte a seguire.

Di seguito sono elencate le restrizioni imposte sul programma SCOL.

Elemento	Nr. massimo per file	Commenti
Nr. di righe del programma	5,500	
Funzione	49	
Argomento di funzione	10	
Nr. di etichette GOTO	999	Tuttavia, il numero massimo di etichette dichiarate e di etichette GOTO che possono essere specificate in una (1) funzione è 599. (La stessa etichetta GOTO viene contata come una (1) anche se esiste una pluralità di etichette GOTO identiche.)
Variabile GLOBAL di tipo intero	99	
Variabile GLOBAL di tipo numero reale	99	
Variabile GLOBAL di tipo carico	48	
Variabile GLOBAL di tipo coordinate	23	
Variabile GLOBAL di tipo posizione	499	
Variabile AUTO di tipo intero	499	
Variabile AUTO di tipo numero reale	499	
Variabile AUTO di tipo carico	99	
Variabile AUTO di tipo coordinate	99	Tuttavia, è possibile impostare fino a 24 costanti nello stesso file (ovvero programma).
Variabile AUTO di tipo posizione	999	Tuttavia, il numero massimo di variabili impostabili in una (1) funzione è 599. È possibile impostare fino a 333 costanti nello stesso file (ovvero programma).
Informazioni sulla variabile AUTO e sulle etichette	599 (per funzione)	Questo è il numero totale di variabili AUTO ed etichette GOTO incluse in una (1) funzione e non è il valore limite specificato in un (1) file.
Variabile di array	99	
Numero totale di elementi di variabili di array	11,000	

Elemento	Nr. massimo per file	Commenti
Nr. di punti di programmazione di dati di tipo posizione di variabili di array (POINT)	1,500	
Nr. di punti di programmazione di dati diversi da quelli di tipo posizione di variabili di array (POINT)	500	
Nr. di annidamenti di FOR~NEXT	127	
Nr. di monitoraggi delle condizioni (ON~DO~) specificati simultaneamente	10	
Nr. di monitoraggi delle condizioni (ON~DO~) dichiarati	50	

* Anche il file SCOL.LIB è contato come un (1) file.

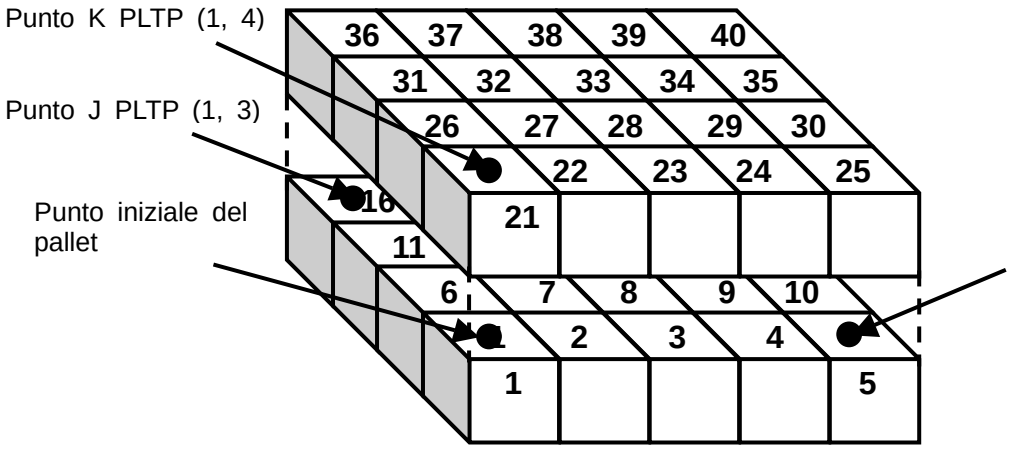
Appendice G Libreria a collegamento dinamico

Appendice G–1 Libreria di pallettizzazione

Nome libreria	PALLET. LIB
Scopo	Libreria di comandi di pallettizzazione. È possibile una pallettizzazione fino a tre (3) dimensioni di (i × j × k) programmando il punto iniziale del pallet, il punto i, il punto j e il punto k.
Comando	INITPLT (<Numero pallet>, <i>, <j>, <k>) Il pallet specificato dal numero di pallet viene inizializzato come pallet a tre (3) dimensioni di "i × j × k". i : Numero di elementi tra il punto iniziale del pallet e il punto I j : Numero di elementi tra il punto iniziale del pallet e il punto J k : Numero di elementi tra il punto iniziale del pallet e il punto K MOVEPLT (<Numero pallet>, <Numero elemento>, X, Y, Z, C) Il robot si sposta sulla posizione specificata dal numero di pallet e il numero di elemento e include offset X, Y, Z e C. I valori di offset X, Y, Z e C non possono essere omessi. (A meno che non sia eseguito l'offset, specificare zero (0).)

[Descrizione dei termini]

Es. Pallet nr. 1, pallet tridimensionale di "5 × 4 × 2":



Numero pallet:	Il numero di pallet viene assegnato a turno, a partire dal numero "1" per i pallet usati nel programma appropriato.
Punto di programmazione:	I quattro (4) punti citati (ovvero punto iniziale, punto I, punto J e punto K) sono i punti di programmazione di questo pallet. Il nome dei punti di programmazione è predeterminato come "PLTP (<Numero pallet>, 1 ~ 4)."
Numero di elemento:	Questo numero è assegnato automaticamente agli elementi del pallet. Per il pallet di "5 × 4 × 2" dell'esempio precedente, vengono assegnati numeri 1 ~ 40 per i rispettivi elementi. Il comando di pallettizzazione permette al robot di muoversi sulla posizione desiderata designando il numero di pallet e il numero di elemento.

Compilazione libreria

Per utilizzare il "PALLET LIB", sono necessari i seguenti comandi [1] and [2].

[1] Nell'area GLOBAL del programma utente, dovrebbe essere dichiarata la compilazione libreria.

LOADLIB PALLET.LIB
↑
Dichiarazione compilazione libreria

[2] Nell'area GLOBAL del programma utente, dovrebbe essere dichiarata la variabile globale usata nella libreria. Il nome variabile è predeterminata come "PLTP".

DIM PLTP (<Numero pallet>, 7) AS POINT

Il numero di pallet dovrebbe essere qualsiasi valore maggiore di "1" e il suo valore massimo cambia con il numero di punti di programmazione e numero di array specificati nel programma.

Il numero "7" è una costante ed è usato per mantenere l'area della variabile usata nella libreria.

Questa variabile globale viene usata per trasferire il numero di punti di programmazione e i valori calcolati al e dal PALLET LIB.

```
GLOBAL
    LOADLIB PALLET.LIB          [1] Dichiarazione compilazione
                                    libreria.
    DIM PLTP (2,7) AS POINT     [2] Dichiarazione variabile
                                    globale.
END
PROGRAM MAIN
    :
    (Omitted)
    :
END
```

Analisi e consigli

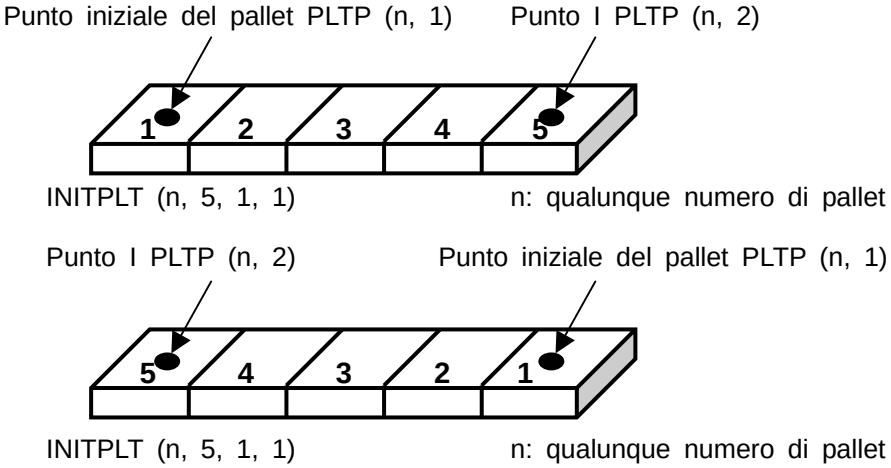
[1] Pallet
Il pallet dovrebbe essere impostato orizzontalmente nel piano X–Y.
(Non dovrebbe essere inclinato.)

[2] Programmazione e dati effettivi
Viene effettuata la programmazione di quattro (4) punti PLTP (n, 1) ~ PLTP (n, 4). (n: numero di pallet 1 ~ n)
Punto iniziale del pallet:
Tutte le coordinate di X, Y, Z, C e T possono essere usate come dati di programmazione. La posizione del movimento viene calcolata aggiungendo un valore di spostamento a questi dati PLTP (n, 1).
Punto I PLTP (n, 2), punto J PLTP (n, 3):
Solo le coordinate X e Y possono essere specificate come dati di programmazione. Vengono usate per ottenere un valore di spostamento in direzione X e Y.
Punto K PLTP (n, 4):
Solo la coordinata Z può essere specificata come dato di programmazione. Viene usata per ottenere un valore di spostamento in direzione Z.
Per il pallet ad una (1) dimensione, è possibile omettere la programmazione del punto J PLTP (n, 3) e del punto K PLTP (n, 4).
Per il pallet a due (2) dimensioni, è possibile omettere la programmazione del punto K PLTP (n, 4).
Il nome del punto di programmazione non può essere cambiato.

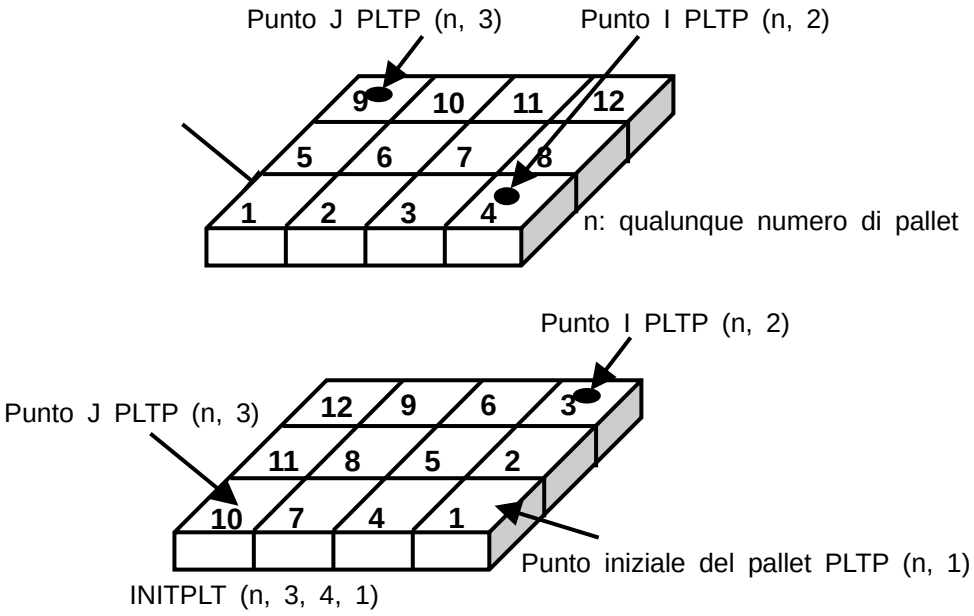
[3] Quando "PALLET.LIB" viene letto dal comando LOADLIB, i nomi delle variabili usati in "PALLET.LIB" (INITPLT****, MOVEPLT**** ; * qualunque numero) non possono essere usati nel programma utente come nomi delle variabili o nomi dei punti di programmazione.

[4] Metodo di programmazione e numero di elemento del pallet
Il numero di elemento è assegnato automaticamente dal comando INITPLT. Anche se il pallet è lo stesso, il numero di elemento differisce dalla sequenza di programmazione.

- Pallet a una (1) dimensione

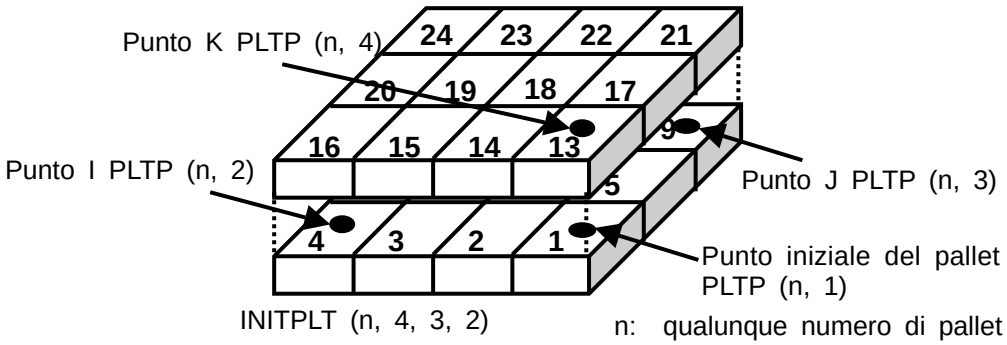


- Pallet a due (2) dimensioni



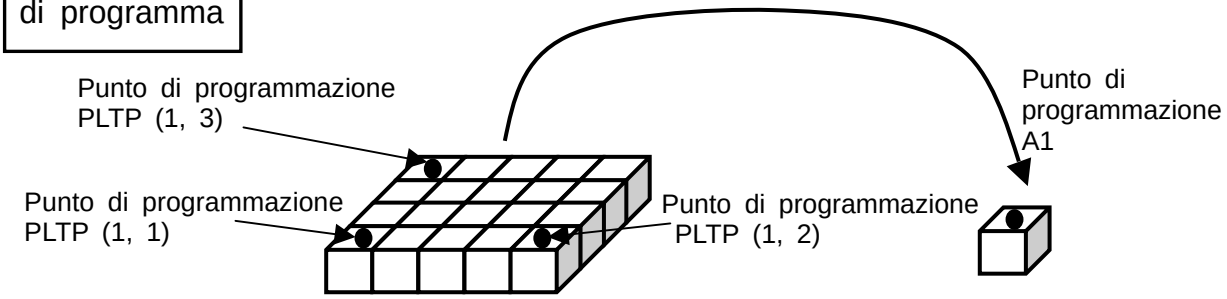
n: qualunque numero di pallet

• Pallet a tre (3) dimensioni



Esempio di programma

[1] Quando le parti vengono fornite dal pallet al punto A1:



```
GROBAL
    LOADLIB PALET.LIB
    DIM PLTP(1,7) AS POINT
END

PROGRAM PALLET
    INITPLT(1,5,4,1)
    OPEN1
    FOR I=1 TO 20 STEP 1
        MOVEPLT(1,I,0,0,50,0)
        MOVEPLT(1,I,0,0,0,0)
     CLOSE1
        MOVEPLT(1,I,0,0,50,0)
        MOVE A1+POINT(0,0,50)
        MOVE A1
        OPEN1
        MOVE A1+POINT(0,0,50)
    NEXT I
END

DATA
    POINT A1          = 650.000, –0.010, 187.140, 2.457,
0.000 / LEFTY
    POINT PLTP(1,1) = 203.346, 390.635, 94.252, 30.261,
0.000 / LEFTY
    POINT PLTP(1,2) = 357.548, 503.825, 94.252, 30.261,
0.000 / LEFTY
    POINT PLTP(1,3) = 337.299, 207.424, 94.252, 30.261,
0.000 / LEFTY
    POINT PLTP(1,4) = 337.299, 207.424, 94.252, 30.261,
0.000 / LEFTY
END
```

Contenuti di PALLET.LIB

Di seguito vengono illustrati i contenuti di PALLET.LIB allegati di norma al sistema.

```
'*************************************************************************************
'*      TS3000 Dynamic Link Library                                                  *
'*                                                                                   *
'*      file name : PALET.LIB                                                        *
'*      function   : PALLETIZE                                                       *
'*      command    : INITPLT(PALLET_NO,I,J,K)                                        *
'*                 : MOVEOLT(PALLET_NO,POZITION_NO,X,Y,Z,C   *
'*                                                                                   *
'*      Copyright(C) 2008 by TOSHIBA MACHINE CO.,LTD                                 *
'*-----------------------------------------------------------------------------------*
'*   08/08/28New                                                                     *
'*                                                                                   *
'*************************************************************************************

PROGRAM INITPLT (INITPLTNO,INITPLTI,INITPLTJ,INITPLTK)
'
      INITPLT1P = PLTP(INITPLTNO,1)
      INITPLT2P = PLTP(INITPLTNO,2)
      INITPLT3P = PLTP(INITPLTNO,3)
      INITPLT4P = PLTP(INITPLTNO,4)

      PLTP(INITPLTNO,5) = POINT(INITPLTI,INITPLTJ,INITPLTK)
INITPLT010:
      IF INITPLTI < 1 THEN GOTO INITPLTERR
      IF INITPLTI < 2 THEN GOTO INITPLT015
      INITPLT5I = INITPLTI – 1
      INITPLTXX = (INITPLT2P.X – INITPLT1P.X) / INITPLT5I
      INITPLTXY = (INITPLT2P.Y – INITPLT1P.Y) / INITPLT5I
      GOTO INITPLT020
```

```
INITPLT015:
    INITPLTXX = 0
    INITPLTXY = 0
INITPLT020:
    IF INITPLTJ < 1 THEN GOTO INITPLTERR
    IF INITPLTJ < 2 THEN GOTO INITPLT025
    INITPLT5J = INITPLTJ ─ 1
    INITPLTYX = (INITPLT3P.X – INITPLT1P.X) / INITPLT5J
    INITPLTYY = (INITPLT3P.Y – INITPLT1P.Y) / INITPLT5J
    GOTO INITPLT030
INITPLT025:
    INITPLTYX = 0
    INITPLTYY = 0
INITPLT030:
    IF INITPLTK < 1 THEN GOTO INITPLTERR
    IF INITPLTK < 2 THEN GOTO INITPLT035
    INITPLT5K = INITPLTK – 1
    INITPLTZZ   = (INITPLT4P.Z – INITPLT1P.Z) / INITPLT5K
    GOTO INITPLT040
INITPLT035:
    INITPLTZZ = 0
INITPLT040:
    PLTP(INITPLTNO,6) =
POINT(INITPLTXX,INITPLTXY,INITPLTZZ)
    PLTP(INITPLTNO,7) =
POINT(INITPLTYX,INITPLTYY,INITPLTZZ)
    GOTO INITPLTEND
INITPLTERR:
    PRINT "ERR !!   ELEMENT IS TOO SMALL."
    STOP
INITPLTEND:

END
PROGRAM MOVEPLT
(MOVEPLTNO,MOVEPLTPSN,MOVEPLTX,MOVEPLTY,MOVEPLTZ,MO
VEPLTC)
```

```
        MOVEPLTI = 0
        MOVEPLTJ = 0
        MOVEPLTK = 0
        MOVEPLTPS1 = MOVEPLTPSN –1
        MOVEPLT1P = PLTP(MOVEPLTNO,1)
        MOVEPLT5P = PLTP(MOVEPLTNO,5)
        MOVEPLT6P = PLTP(MOVEPLTNO,6)
        MOVEPLT7P = PLTP(MOVEPLTNO,7)
        MOVEPLTA = MOVEPLT5P.X * MOVEPLT5P.Y
        MOVEPLTB = MOVEPLTPS1 MOD MOVEPLTA
        MOVEPLTMAX = MOVEPLTA * MOVEPLT5P.Z
        IF 1 > MOVEPLTPSN THEN GOTO MOVEPLTER2
        IF MOVEPLTMAX < MOVEPLTPSN THEN GOTO MOVEPLTER3
        MOVEPLTI = MOVEPLTB MOD    MOVEPLT5P.X
        MOVEPLTJ = INT(MOVEPLTB / MOVEPLT5P.X)
        MOVEPLTK = INT(MOVEPLTPS1 / MOVEPLTA )
        MOVEPLTXXX = MOVEPLTI * MOVEPLT6P.X    + MOVEPLTJ *
MOVEPLT7P.X + MOVEPLTX
        MOVEPLTYYY = MOVEPLTI * MOVEPLT6P.Y    + MOVEPLTJ *
MOVEPLT7P.Y + MOVEPLTY
        MOVEPLTZZZ = MOVEPLTK * MOVEPLT6P.Z    + MOVEPLTZ
        MOVE MOVEPLT1P+
POINT(MOVEPLTXXX,MOVEPLTYYY,MOVEPLTZZZ,MOVEPLTC,0)
        GOTO MOVEPLTEND
MOVEPLTER2:
        PRINT "ERR !!    ELEMENT NO. IS TOO SMALL."
        STOP
MOVEPLTER3:
        PRINT "ERR !!    ELEMENT NO. IS TOO LARGE."
        STOP
MOVEPLTEND:
END
```

Appendice H Arresto della pre-lettura eseguito dal linguaggio SCOL del programma

Di seguito vengono elencati i comandi che eseguono l'arresto della pre-lettura.

- PRINT
- WAIT
- ON to DO
- IGNORE
- DOUT
- DIN
- XIN (Opzione di convogliatore)
- BCDIN
- BCDOUT
- PULOUT
- MOVE
- MOVEA
- MOVEI
- MOVES
- MOVEC
- MOVEJ
- DELAY
- LATCH
- SYNC (Opzione di convogliatore)
- UNSYNC (Opzione di convogliatore)
- STOP
- RETURN
- END